

Programación Java

DAM / DAW

Sergi García Barea — CC BY-SA 4.0

Contents

-  [Programació Java — DAM/DAW Curs complet d'iniciació a la programació en Java. Bilingüe \(valencià / castellà\), amb cercador, mode fosc i disseny premium.](#)
 -  [Unitats](#)
- [Unidad 1: Introducción a Java](#)
 - [L'Ordinador T'Obeeix: Entorns de Desenvolupament](#)
 - [Què és això del JDK, JRE i JVM? \(La Trilogia del Cafè\)](#)
 - [Muntant el Xiringuito: Instal·lació](#)
 - [El Teu Primer Programa: Hola Món \(o com parlar-li a la màquina\)](#)
 - [El Depurador: Ets un Detectiu](#)
 - [No Hi Ha Preguntes Tontes!](#)
 - [Els Comentaris: Notes Apegaloses Digital](#)
 - [Més Exemples de Codi](#)
 - [Resum \(el que importa de veritat\)](#)
 - [Exercicis Proposats](#)
- [Unidad 2: Variables, Tipos de Datos y Operadores](#)
 - [Variables: Les Caixes On Viuen Les Teues Dades](#)
 - [Operadors: El Gimnàs de les Dades](#)
 - [No Hi Ha Preguntes Tontes!](#)
 - [Més Exemples de Codi](#)
 - [Resum \(el que importa de veritat\)](#)
 - [Exercicis Proposats](#)
- [Unidad 3: Estructuras de Control y Excepciones](#)
 - [if, else i switch: L'Art de Decidir](#)
 - [Bucles: Com Fer Que l'Ordinador Es Repetisca](#)
 - [Excepcions: Quan el Teu Programa Ensopega](#)
 - [Exercicis Proposats](#)
 - [Resum](#)
- [Unidad 4: Algorítmica I — Fundamentos](#)
 - [Què és un Algorisme? \(L'Art de Donar Instruccions Precises\)](#)
 - [Cerca Lineal: El Cercador de Sabates Perdudes](#)
 - [Cerca Binària: El Cercador Jedi](#)
 - [Ordenació Bombolla: Simple però Torta](#)

- [Ordenació per Inserció: Com Ordenar Cartes en la Mà](#)
- [Complexitat Algorísmica: Big O per a Mortals](#)
- [★ BE THE CODE, MY FRIEND: Implementa Cerca Binària des de Zero](#)
- [No Hi Ha Preguntes Tontes!](#)
- [Resum de la Unitat](#)
- [Unidad 5: Algorítmica II — Técnicas Avanzadas](#)
 - [1. Recursivitat: una funció que es crida a si mateixa \(sense tornar-se boja\)](#)
 - [2. Exemples clàssics \(els clàssics per alguna cosa ho són\)](#)
 - [3. Dividix i venceràs \(Divide & Conquer\)](#)
 - [4. Quicksort — el ràpid \(quan li eix bé\)](#)
 - [5. Mergesort — el fiable \(sempre complix el que promet\)](#)
 - [6. Comparació: quan use cada un?](#)
 - [7. ★ BE THE CODE, MY FRIEND: implementa Quicksort des de zero](#)
 - [8. No Hi Ha Preguntes Tontes!](#)
 - [Resum de la unitat](#)
- [Unidad 4: POO - Clases y Objetos](#)
 - [POO: Els Teus Objectes Cobren Vida](#)
 - [Classes i Objectes: Tallagalletas vs Galetes](#)
 - [Resum](#)
 - [Exercicis Proposats](#)
- [Unidad 5: Visibilidad, Encapsulación y Static](#)
 - [Visibilitat: L'Art de No Ensenyar-ho Tot](#)
 - [Encapsulació: El Pilar que Sosté la POO](#)
 - [Static: El Que Pertany a la Classe \(No a l'Objecte\)](#)
 - [Resum](#)
 - [Exercicis Proposats](#)
- [Unidad 8: Herencia, Polimorfismo e Interfaces](#)
 - [Herència: Quan els Teus fills Segueixen els Teus Passos \(Però Millor\)](#)
 - [Polimorfisme: El Camaleó de la POO](#)
 - [La Classe Object: El Besavi de Tot](#)
 - [Classes Abstractes: El Esbós Que No Pots Usar Directament](#)
 - [Interfícies: El Contracte Que El Teu Codi Firmar](#)
 - [Interfícies Funcionals: Una Sola Missió](#)

- [Jerarquia de Classes i Composició](#)
- [Resum Global](#)
- [Unidad 8: Arrays y Colecciones](#)
 - [Arrays: L'Aparcament de Dades](#)
 - [Col·leccions: Quan un Array Es Queda Xicotet](#)
 - [Exercicis Proposats](#)
- [Unidad 9: Genéricos y Mapas](#)
 - [Genèrics: El Que Ho Va Canviar Tot](#)
 - [Mapes: La Guia Telefònica](#)
 - [Exercicis Proposats](#)
-  [Unitat 11: Consola, Fitxers i Expressions Regulars](#)
 - [Comunicació per Consola](#)
 - [Fitxers](#)
 - [Expressions Regulars](#)
 - [Exercicis Proposats](#)
-  [Unitat 12: Connexió a Bases de Dades amb JDBC](#)
 - [Què és JDBC?](#)
 - [Connexió a SQLite](#)
 - [CRUD amb JDBC](#)
 - [PreparedStatement i SQL Injection](#)
 - [Patró DAO](#)
 - [Transaccions](#)
 -  [BE THE CODE, MY FRIEND: Implementa un DAO Complet Des de Zero](#)
 - [¡No Hay Preguntas Tontas!](#)
 - [Exercicis Proposats](#)
 - [Bones Pràctiques: El Decàleg del JDBC](#)
 - [Resum Exprés](#)
-  [Unitat 13: Servir i Consumir APIs amb Web](#)
 - [Hola, Món Web](#)
 - [1. El Protocol HTTP en 30 Segons](#)
 - [2. Servidor Web Mínim amb HttpServer](#)
 - [3. Servint HTML](#)
 - [4. Paràmetres GET: El Client Pregunta](#)

- [5. Formularis POST: El Client Envia Dades](#)
- [6. Tornant JSON \(Com una API de Veritat\)](#)
- [7. Mini Projecte: Gestor de Tasques \(API REST\)](#)
- [8. Consumir APIs Externes amb HttpClient](#)
- [9. Frameworks Moderns](#)
- [? No Hi Ha Preguntes Tontes!](#)
- [Resum](#)
- [Exercicis Proposats](#)
- [Butlletí 1 - Inicial Result: Introducció](#)
 - [Exercici 1: Què imprimeix este programa?](#)
 - [Exercici 2: Troba els 3 errors](#)
 - [Exercici 3: Completa el mètode](#)
 - [Exercici 4: Escriu el teu primer programa](#)
 - [Exercici 5: Què fa este programa?](#)
 - [Exercici 6: AceptaElReto.com — 116 ¡Hola mundo!](#)
 - [Exercici 7: CodeWars — Multiply](#)
- [Butlletí 1 - Inicial: Introducció](#)
 - [Exercici 1: Desordena això](#)
 - [Exercici 2: Què imprimeix?](#)
 - [Exercici 3: Caçador d'errors](#)
 - [Exercici 4: La teua fitxa personal](#)
 - [Exercici 5: Completa el programa](#)
 - [Exercici 6: Aparella conceptes](#)
 - [Exercici 7: CodeWars — Square\(n\) Sum](#)
 - [Exercici 8: El detectiu d'errors](#)
 - [Exercici 9: La teua biografia](#)
- [Butlletí 1 - Result: Introducció](#)
 - [★ Exercici 1: El programa que saluda tres vegades](#)
 - [★ Exercici 2: Arguments en l'arena](#)
 - [★★ Exercici 3: Comentari o no comentari](#)
 - [★★ Exercici 4: La festa d'acceptaelreto.com](#)
 - [★★ Exercici 5: Calculadora sense Scanner](#)
 - [★★★ Exercici 6: Javadoc de la teua vida](#)

-  [Exercici 7: El primer depurador](#)
-  [Exercici 8: CodeWars — Return Negative](#)
-  [Referències](#)
- [Butlletí 1 - Intermedi: Introducció](#)
 -  [Exercici 1: ASCII art amb prints](#)
 -  [Exercici 2: Sense executar — seqüències d'escapament](#)
 -  [Exercici 3: El rellotge de mil·lisegons](#)
 -  [Exercici 4: Comptador d'arguments](#)
 -  [Exercici 5: L'edat còsmica](#)
 -  [Exercici 6: CodeWars — Grasshopper - Summation](#)
 -  [Exercici 7: AceptaElReto — 119 Futbolistes](#)
 -  [Referències](#)
- [Butlletí 01 — Extres: Introducció a Java](#)
- [Butlletí 2 - Inicial Result: Variables i Operadors](#)
 - [Exercici 1: Declara el tipus correcte](#)
 - [Exercici 2: Què imprimeix?](#)
 - [Exercici 3: Troba l'error](#)
 - [Exercici 4: L'intercanvi](#)
 - [Exercici 5: Escriu aquest programa](#)
 - [Exercici 6: AceptaElReto 149 — San Fermín](#)
 - [Exercici 7: CodeWars — Even or Odd](#)
- [Butlletí 2 - Inicial: Variables i Operadors](#)
 - [Exercici 1: Conversor de temperatures](#)
 - [Exercici 2: Què imprimeix?](#)
 - [Exercici 3: Calculadora de descomptes](#)
 - [Exercici 4: Precedència de malson](#)
 - [Exercici 5: El tipus perfecte](#)
 - [Exercici 6: L'intercanvi màgic](#)
 - [Exercici 7: CodeWars — Will you make it?](#)
- [Butlletí 2 - Intermedi Result: Variables i Operadors](#)
 -  [Exercici 1: Parell o senar?](#)
 -  [Exercici 2: Constant del mal](#)
 -  [Exercici 3: Dau trucat](#)

- [!\[\]\(7e19807c61da14f515588e95cd49886c_img.jpg\) !\[\]\(8ff9e60a4b0560d7ec99179ef4779d9e_img.jpg\) Exercici 4: Any de traspàs](#)
- [!\[\]\(ab9b69bf5753a01c76b30af859454360_img.jpg\) !\[\]\(c5af66b13c724ca428497900cdbbc9b3_img.jpg\) Exercici 5: Nom al revés](#)
- [!\[\]\(1fde827780c8f912fd3ae9174d52d155_img.jpg\) !\[\]\(49ab9fdb6ddb6816bcb8ccc012d5cebd_img.jpg\) !\[\]\(a10cf212d457430b842f8ac59c63db70_img.jpg\) Exercici 6: AceptaElReto 152 — Números de parells](#)
- [!\[\]\(e8a826213cf8b53a8c13f5432344afc9_img.jpg\) !\[\]\(7ffe3c6e7552aa3eb962276cd7a9a979_img.jpg\) !\[\]\(28e94a65fe1d8cf887928bbaaa2c7303_img.jpg\) Exercici 7: L'enigma del ++](#)
- [!\[\]\(7db790dc622e1ac5f1c44afb7a5212a6_img.jpg\) !\[\]\(86147531a4f05b1215989ff8ab43fe6d_img.jpg\) !\[\]\(c3492017d65b370ec6b463430fff1ce7_img.jpg\) Exercici 8: AceptaElReto 140 — Suma de dígit](#)
- [!\[\]\(eadeaa5506f71c8d915378340dd044f1_img.jpg\) Referències](#)
- [Butlletí 2 - Intermedi: Variables i Operadors](#)
 - [!\[\]\(2e7c96d436a2266b49a49932113a1657_img.jpg\) Exercici 1: Calculadora de propines](#)
 - [!\[\]\(6a8a243cf3443d7797a7e525dc6a1efc_img.jpg\) Exercici 2: Conversor dòlar-euro](#)
 - [!\[\]\(33e2662dd35315fbb8bde6de2141f6aa_img.jpg\) !\[\]\(56890bcfd6a4f9f79fd5acc5be8e52b2_img.jpg\) Exercici 3: Què imprimeix? — el casting traïdor](#)
 - [!\[\]\(fd7e0a3996f31269d6928e9995a1b87e_img.jpg\) !\[\]\(a1cf103b9c5f9b28e1bde5f1a6e89e23_img.jpg\) Exercici 4: Interés compost \(sense bucle\)](#)
 - [!\[\]\(3376c19c9b30a763743ecfcb079fddcd_img.jpg\) !\[\]\(d09a86bc59f80cb7523dc82f971d2e57_img.jpg\) !\[\]\(df26f226b94da77f641ade2d1b2f5c50_img.jpg\) Exercici 5: L'enigma del post-increment](#)
 - [!\[\]\(cb8575ff682ca8aa5b833067765ddef1_img.jpg\) !\[\]\(1b79c0b496f3b8bcd939680eaf19bcab_img.jpg\) !\[\]\(828d909f844bc87d60e101352abee488_img.jpg\) Exercici 6: CodeWars — Convert boolean values to strings 'Yes' or 'No'](#)
 - [!\[\]\(c67736552050e2baa44bd853740e7502_img.jpg\) !\[\]\(af52c7a4d481e6428d0fb7841b675e03_img.jpg\) !\[\]\(9b7fb70cd3e4be229efbfff0ac230a00_img.jpg\) Exercici 7: AceptaElReto — 114 Últim dígit del factorial](#)
 - [!\[\]\(0d3f1a4e974645225ecf037c1b87a135_img.jpg\) Referències](#)
- [Butlletí 02 — Extres: Variables, Tipus de Dades i Operadors](#)
- [Butlletí 3 - Inicial Result: Estructures de Control](#)
 - [Exercici 1: Què imprimeix este if-else?](#)
 - [Exercici 2: Completa el switch](#)
 - [Exercici 3: Troba l'error \(bucle infinit\)](#)
 - [Exercici 4: Escriu este programa \(while\)](#)
 - [Exercici 5: El for de tota la vida](#)
 - [Exercici 6: AceptaElReto 200 — Aburrimiento en las aulas](#)
 - [Exercici 7: CodeWars — Century From Year](#)
- [Butlletí 3 - Inicial: Estructures de Control](#)
 - [Exercici 1: Què imprimeix este if?](#)
 - [Exercici 2: Troba l'error en el for](#)
 - [Exercici 3: Completa el programa \(do-while\)](#)
 - [Exercici 4: Compte arrere](#)
 - [Exercici 5: Menú amb switch](#)
 - [Exercici 6: Taula de multiplicar del 7](#)
 - [Exercici 7: CodeWars — Grasshopper - Grade book](#)
- [Butlletí 3 - Intermedi Result: Estructures de Control](#)

- [!\[\]\(effbd7993c63c039a58fd3395789cf3f_img.jpg\) Exercici 1: El classificador de notes sarcàstic](#)
- [!\[\]\(144980d038f2541d7b588a8a9132bd70_img.jpg\) Exercici 2: Taula de multiplicar](#)
- [!\[\]\(c4ce2d477989700c971cf3d240ad9283_img.jpg\) !\[\]\(5013555a72072875cb154b597e002a46_img.jpg\) Exercici 3: Divisió segura amb try-catch](#)
- [!\[\]\(bf2038c114ec21ea58ad011774351c98_img.jpg\) !\[\]\(1ad0c3425edfa4762c2f20e33e3e5bbf_img.jpg\) Exercici 4: El triangle constructor](#)
- [!\[\]\(74d2fc5645add84f8511beb934060048_img.jpg\) !\[\]\(ddc5533caa0187e636e3d3234e0983a3_img.jpg\) Exercici 5: Fibonacci amb for](#)
- [!\[\]\(7c207f8f59385c6dd11f9d9bdc7a0d1d_img.jpg\) !\[\]\(57a1bf910af99362b80b3ac4f2eecbac_img.jpg\) !\[\]\(6b114000ab07dda576e2920e2dc838fa_img.jpg\) Exercici 6: AceptaElReto 340 — Juegos de naipes](#)
- [!\[\]\(9129a6a4a4b11facb5cf665660eef788_img.jpg\) !\[\]\(7cbd4924d31aa4c46df484d5c5bf4696_img.jpg\) !\[\]\(b3461332979d340882fda628798720af_img.jpg\) Exercici 7: Excepció personalitzada](#)
- [!\[\]\(5dc380853f1779b9d7a8d6fbbcdbb357_img.jpg\) !\[\]\(2894a5ead456ff0145f35b01b4a6c7df_img.jpg\) !\[\]\(6b9cc5709aff55a0cbbb7dd78fcf3ba1_img.jpg\) Exercici 8: AceptaElReto 100 — Kaprekar](#)
- [!\[\]\(cd4202444435d9e729cdf801e3403cf4_img.jpg\) Referències](#)
- [Butlletí 3 - Intermedi: Estructures de Control](#)
 - [!\[\]\(a595a22b25819b91e6cd221ac336245d_img.jpg\) Exercici 1: Nombres parells i senars](#)
 - [!\[\]\(912cec22f21865fe086f1aaa377b4c97_img.jpg\) Exercici 2: Suma acumulativa](#)
 - [!\[\]\(d66d6cd7d0eca6e4b13ae9e87c7f7f49_img.jpg\) !\[\]\(2e58555536753c4dae915c3689ae8744_img.jpg\) Exercici 3: Nombre perfecte](#)
 - [!\[\]\(704be90457dfbf7435f703bdc5293a9e_img.jpg\) !\[\]\(9a8314206d1a19a24da8a71532ac6fea_img.jpg\) Exercici 4: Menú amb validació](#)
 - [!\[\]\(d0819331239cfe31193900cc06944bf3_img.jpg\) !\[\]\(4909cf475d986aa46b298b5fdca4ee07_img.jpg\) !\[\]\(a7d5e2c96349c1d63a82cd31458462fc_img.jpg\) Exercici 5: Garbell d'Eratòstenes \(nombres primers\)](#)
 - [!\[\]\(e28368bd9a276ed4dec00f0d27ffd39f_img.jpg\) !\[\]\(8085f10a37332aa3c492c2f85ed04dd1_img.jpg\) !\[\]\(1e310b45e9af4e0a614ad2f402ca1d06_img.jpg\) Exercici 6: CodeWars — Vowel Count](#)
 - [!\[\]\(1383aab67436378c92b79e100ee645ad_img.jpg\) !\[\]\(6dab23411f791f74b147780420cb4980_img.jpg\) !\[\]\(985b64aafa01f3489b890b3563f99253_img.jpg\) Exercici 7: AceptaElReto — 151 ¿Es matriz identidad?](#)
 - [!\[\]\(46db6ce894f8c180d34486b3df9bdb7e_img.jpg\) Referències](#)
- [Butlletí 03 — Extres: Estructures de Control i Excepcions](#)
- [Butlletí 4 - Inicial Result: Algorítmica I](#)
 - [!\[\]\(4ca48801391a3a9a97659307204af6f0_img.jpg\) Exercici 1: Cerca lineal](#)
 - [!\[\]\(cf8249ef15b74deec09abf5291634357_img.jpg\) Exercici 2: Cerca binària](#)
 - [!\[\]\(ae8ab053a5c7f1bfc1804a673873eac4_img.jpg\) !\[\]\(efc69f5f5eeb0d90daf9ca3c17658807_img.jpg\) Exercici 3: Bombolla](#)
 - [!\[\]\(ee7cf7662b15cdf945754de15a83a6b8_img.jpg\) !\[\]\(596d4a1b17ee45434ed8f5de077912c5_img.jpg\) Exercici 4: Inserció](#)
 - [!\[\]\(ac52b2317a81509e2f8496833885a734_img.jpg\) !\[\]\(54db7f82fa6165ad3d9083e4758247ed_img.jpg\) Exercici 5: Comptar passos bombolla](#)
 - [!\[\]\(1ecdcf3cd287c17c2930696cf8365967_img.jpg\) !\[\]\(0dbe36c4fa4289778441db5b75e0f5f6_img.jpg\) !\[\]\(62c0fdaec744fcc4002b168d1d6bee9b_img.jpg\) Exercici 6: Buscar en matriu ordenada](#)
- [Butlletí 4 - Inicial: Algorítmica I](#)
 - [!\[\]\(e8b587030bc589b6dfd5501d76c30031_img.jpg\) Exercici 1: El més gran i el més petit](#)
 - [!\[\]\(01eec8c2e8080dcdfdad078a9d3d0aab_img.jpg\) Exercici 2: Suma total](#)
 - [!\[\]\(22ac3953b85701c5e3d6d91896b7652a_img.jpg\) !\[\]\(723dade54ed500940b67f5d80e2a192c_img.jpg\) Exercici 3: Cercador elemental](#)
 - [!\[\]\(e0e6e81f6cc17f31c4928a946b5e0ccd_img.jpg\) !\[\]\(940d96f1b93cb5129ba0107e81f2764d_img.jpg\) Exercici 4: Invertir array](#)
 - [!\[\]\(77bbe13878d9aef282903a3867dd5425_img.jpg\) !\[\]\(02db0e3277ae5bdaf9a37ce93f97978d_img.jpg\) Exercici 5: Mitjana de doubles](#)
 - [!\[\]\(5abcc53bf97db101ba8864e49380226f_img.jpg\) !\[\]\(2272cc4c3b0ed62da966aaf43b3210fb_img.jpg\) !\[\]\(01af68df1aec289628b5bf9a72d5e2cb_img.jpg\) Exercici 6: CodeWars — Sum without highest and lowest number](#)

- [!\[\]\(83eb2aa26b610eb6a9dca7cf4702d681_img.jpg\) Exercici 7: CodeWars — You're a square!](#)
- [Butlletí 4 - Intermedi Resolt: Algorítmica I](#)
 - [!\[\]\(94dfacbf937cdd7da4837a6fcd8fc785_img.jpg\) Exercici 1: Parells amb suma objectiu](#)
 - [!\[\]\(dae8c3c5fa7c80febd6526a5e8a853bf_img.jpg\) Exercici 2: Eliminar duplicats \(in-place\)](#)
 - [!\[\]\(8f38ab9775d1331a4e1fd6648d0a83f1_img.jpg\) Exercici 3: Major suma de subarray \(Kadane\)](#)
 - [!\[\]\(5e48b3241d711ef916255d822ab3415f_img.jpg\) Exercici 4: Rotar array k voltes](#)
 - [!\[\]\(c4c3604751fde0df44855086d7798e30_img.jpg\) Exercici 5: Comptar ocurrences \(binària optimitzada\)](#)
- [Butlletí 4 - Intermedi: Algorítmica I](#)
 - [!\[\]\(a16ee0329f478adb49fd7876490e96fe_img.jpg\) Exercici 1: Moure zeros al final](#)
 - [!\[\]\(a8a494e883c31c6f4a74584cc935a790_img.jpg\) Exercici 2: El número que falta](#)
 - [!\[\]\(8a50a875cd94cd57f6a7348a4d34b45f_img.jpg\) Exercici 3: Primer caràcter no repetit](#)
 - [!\[\]\(7ddc0498d10972f0f157498fccf65370_img.jpg\) Exercici 4: Anagrames](#)
 - [!\[\]\(1481f3a2b8d8d64dbbf1eb2242b57620_img.jpg\) Exercici 5: Ordenar 0s, 1s i 2s \(Dutch National Flag\)](#)
 - [!\[\]\(1d12b5523dd7503f9f6b056602d4b0bb_img.jpg\) Exercici 6: CodeWars — Disemvowel Trolls](#)
 - [!\[\]\(a3cf9896eece75d50b7dee194989c9bc_img.jpg\) Exercici 7: AceptaElReto — 219 La loteria](#)
 - [!\[\]\(26147612c3694ee09d020cac0faae7d9_img.jpg\) Referències](#)
- [Butlletí 04 — Extres: Algorísmica I: Fonaments](#)
- [Boletín 5 - Inicial Resuelto: Algorítmica II](#)
 - [!\[\]\(46af714d66844046465576855b12c56a_img.jpg\) Ejercicio 1: Factorial recursivo](#)
 - [!\[\]\(e123f1fe9eaf288ea9a688d32daa4ea4_img.jpg\) Ejercicio 2: Fibonacci recursivo](#)
 - [!\[\]\(780c9e9bdcf3e80b97a0a12adfab01dc_img.jpg\) Ejercicio 3: Suma recursiva de array](#)
 - [!\[\]\(dd4f403c0f3886141896591186a0332a_img.jpg\) Ejercicio 4: Invertir cadena](#)
 - [!\[\]\(eba5717f9532900568c94b25862a3240_img.jpg\) Ejercicio 5: Fibonacci con memoización](#)
 - [!\[\]\(8787ffe874068f6bc6ac6dfc8c73a8ea_img.jpg\) Ejercicio 6: Quicksort](#)
- [Boletí 5 - Inicial: Algorísmica II](#)
 - [Exercici 1: Potència recursiva](#)
 - [Exercici 2: Comptar dígit recursiu](#)
 - [Exercici 3: Què imprimeix?](#)
 - [Exercici 4: Palíndrom recursiu](#)
 - [Exercici 5: Troba l'error](#)
 - [Exercici 6: Sumatori recursiu](#)
 - [Exercici 7: CodeWars — Reversed Strings](#)
 - [Exercici 8: AceptaElReto — 165 Número hyperpar](#)

- [Boletín 5 - Intermedio Resuelto: Algorítmica II](#)
 - [★ ★ Ejercicio 1: Torres de Hanoi](#)
 - [★ ★ Ejercicio 2: Mergesort](#)
 - [★ ★ ★ Ejercicio 3: Búsqueda binaria recursiva](#)
 - [★ ★ ★ Ejercicio 4: Potencia rápida recursiva \(\$O\(\log n\)\$ \)](#)
 - [★ ★ ★ Ejercicio 5: Subconjuntos \(backtracking\)](#)
- [Boletí 5 - Intermedi: Algorísmica II: Tècniques Avançades](#)
 - [★ Exercici 1: MCD per Euclides recursiu](#)
 - [★ Exercici 2: Comptar ocurrencies recursiu](#)
 - [★ ★ Exercici 3: Invertir nombre recursiu](#)
 - [★ ★ Exercici 4: Combinacions d'un array \(backtracking\)](#)
 - [★ ★ ★ Exercici 5: N-Reines \(backtracking\)](#)
 - [★ ★ ★ Exercici 6: CodeWars — Tribonacci Sequence](#)
 - [★ ★ ★ Exercici 7: AceptaElReto — 140 Suma de dígit](#)
 - [!\[\]\(2dc8cdc0c918df88cde61039ecf68682_img.jpg\) Referències](#)
- [Butlletí 05 — Extres: Algorísmica II: Tècniques Avançades](#)
- [Boletín 4 - Inicial Resuelto: POO - Clases y Objetos](#)
 - [Ejercicio 1: Completa la clase Perro](#)
 - [Ejercicio 2: ¿Qué imprime?](#)
 - [Ejercicio 3: Encuentra el error](#)
 - [Ejercicio 4: Escribe la clase Persona](#)
 - [Ejercicio 5: ¿Constructor por defecto?](#)
 - [Ejercicio 6: AceptaElReto 291 — Números afortunados](#)
 - [Ejercicio 7: CodeWars — Grasshopper - Summation](#)
- [Boletí 4 - Inicial: POO: Classes i Objectes](#)
 - [Exercici 1: Completa la classe Alumne](#)
 - [Exercici 2: Què imprimeix?](#)
 - [Exercici 3: Troba l'error](#)
 - [Exercici 4: Escriu la classe Pel·lícula](#)
 - [Exercici 5: Constructor buit i setters](#)
 - [Exercici 6: AceptaElReto — 417 Nombres binomials](#)
 - [Exercici 7: CodeWars — Return the day](#)
- [Boletín 4 - Resuelto: POO - Clases y Objetos](#)

- [!\[\]\(fd4127b9e2af37bd6ea0fa06afa8e6d8_img.jpg\) Ejercicio 1: Clase Rectángulo](#)
- [!\[\]\(3278d6283d12f18012b5aa7d40747611_img.jpg\) Ejercicio 2: Clase CuentaBancaria](#)
- [!\[\]\(bb96b32142ec45f72f12316beae3ef61_img.jpg\) !\[\]\(a75d0d7d1ac0ea815a0c027a77b137d5_img.jpg\) Ejercicio 3: Clase Hora con validación](#)
- [!\[\]\(adab168ecea4e2ae40993afba3fcd26e_img.jpg\) !\[\]\(e95ea1a0e297340f96c3d715d76d8c23_img.jpg\) Ejercicio 4: equals\(\) en Persona](#)
- [!\[\]\(0e0cdd76cf8fb3c858658faf72d7f324_img.jpg\) !\[\]\(c7bdebfc2a52eab4f6683b0c1c0c28d2_img.jpg\) Ejercicio 5: Clase Punto y distancia euclídea](#)
- [!\[\]\(8a5c092736ceaf7cfeed23b17ea5e6ff_img.jpg\) !\[\]\(dd896e8effa212be4a7431ac3e5bf510_img.jpg\) !\[\]\(1136f3a14b3ba5842784adf33e611db1_img.jpg\) Ejercicio 6: AceptaElReto 417 — Números binomiales](#)
- [!\[\]\(d536ae100fbef949d5488b8fe23458f5_img.jpg\) !\[\]\(ca07eb06779edef206c13bdfc2c41b40_img.jpg\) !\[\]\(7e7914364faa22e4d6649c877c7e723a_img.jpg\) Ejercicio 7: AceptaElReto 458 — El espejo](#)
- [!\[\]\(8ca478e471a5a40f192eb324324796da_img.jpg\) !\[\]\(1ef1c1e934c0d3468708b1b65c673a8e_img.jpg\) !\[\]\(2feef24d2b62f527f17ec4a26c99892b_img.jpg\) Ejercicio 8: Simulación de batalla Pokémon \(lite\)](#)
- [!\[\]\(62fbabcb8f920179a05cac5e6f77b9ec_img.jpg\) Referencias](#)
- [Boletí 6 - Intermedi: POO: Classes i Objectes](#)
 - [!\[\]\(ea93f456ce60033911637cde1e339ebc_img.jpg\) Exercici 1: Classe Llibre](#)
 - [!\[\]\(5ee090fbd54350d5f2535fbbb0cd8843_img.jpg\) Exercici 2: Classe Termòmetre](#)
 - [!\[\]\(4dd804f7f115c0312bcea2f969779146_img.jpg\) !\[\]\(0917fa5071e46a535e4715939ddc0471_img.jpg\) Exercici 3: Classe Data amb validació](#)
 - [!\[\]\(27fec434af9954d953dfa4171448e1b3_img.jpg\) !\[\]\(d145641c384340f63dd70fecc7aabb25_img.jpg\) Exercici 4: Classe Relotge amb avanç](#)
 - [!\[\]\(243bbedc054f34725b139b217207eb1a_img.jpg\) !\[\]\(f3c6d79887bd9f21070de580cbc98211_img.jpg\) !\[\]\(2d1ece5ae055d1ae0a9970ac957b3b56_img.jpg\) Exercici 5: Classe Matriu 2D](#)
 - [!\[\]\(5b120c922ef28e4fc683ca8b94422cdb_img.jpg\) !\[\]\(3ecc71fc952378d9f4ffcad55785a56c_img.jpg\) !\[\]\(b72ce2783b5377c43149ef95e1a2e921_img.jpg\) Exercici 6: CodeWars — Build Tower](#)
 - [!\[\]\(411232913beeb58fd0033db165ed1b36_img.jpg\) !\[\]\(e83c8c58bd2fde02f1aaa2a40b47a814_img.jpg\) !\[\]\(0c02fd0d294a73ffdb4e0a069fbf4bfa_img.jpg\) Exercici 7: AceptaElReto — 246 Buscant el pin](#)
 - [!\[\]\(eb1dbccbedb5ea0b264d2e100bde2403_img.jpg\) Referències](#)
- [Butlletí 06 — Extres: POO: Classes i Objectes](#)
- [Boletín 5 - Inicial Resuelto: Visibilidad, Encapsulación y Static](#)
 - [Ejercicio 1: Encuentra el error de visibilidad](#)
 - [Ejercicio 2: Completa los getters y setters](#)
 - [Ejercicio 3: ¿Qué imprime? Static vs instancia](#)
 - [Ejercicio 4: Escribe la clase utilitaria](#)
 - [Ejercicio 5: Escribe la clase Config](#)
 - [Ejercicio 6: AceptaElReto 106 — Código de barras](#)
 - [Ejercicio 7: CodeWars — Convert boolean to string](#)
- [Boletí 5 - Inicial: Visibilitat, Encapsulació i Static](#)
 - [Exercici 1: Troba l'error de visibilitat \(la nevera\)](#)
 - [Exercici 2: Completa els getters i setters de la classe Producte](#)
 - [Exercici 3: Què imprimeix? Static vs instància II](#)
 - [Exercici 4: Escriu la classe utilitària MathUtils](#)
 - [Exercici 5: Escriu la classe AppConfig](#)

- [Exercici 6: AceptaElReto — 117 Trobar el major](#)
- [Exercici 7: CodeWars — Opposite number](#)
- [Boletín 5 - Resuelto: Visibilidad, Encapsulación y Static](#)
 - [★ Ejercicio 1: Clase Empleado con validación](#)
 - [★ Ejercicio 2: Círculo encapsulado](#)
 - [★★ Ejercicio 3: JavaBean Alumno](#)
 - [★★ Ejercicio 4: Contador de objetos con static](#)
 - [★★ Ejercicio 5: Conversor de unidades \(estáticos\)](#)
 - [★★★ Ejercicio 6: AceptaElReto 364 — Spiderman](#)
 - [★★★ Ejercicio 7: AceptaElReto 462 — Tres dedos](#)
 - [★★★ Ejercicio 8: Simulación estática](#)
 - [📖 Referencias](#)
- [Boletí 5 - Intermedi: Visibilitat, Encapsulació i Static](#)
 - [★ Exercici 1: Classe CompteEstalvis encapsulada](#)
 - [★ Exercici 2: Classe Persona amb validació estricta](#)
 - [★★ Exercici 3: Classe TicketCompra amb ID autoincremental](#)
 - [★★ Exercici 4: Classe Biblioteca amb comptador estàtic](#)
 - [★★★ Exercici 5: Classe CalculadoraEstadística utilitària](#)
 - [★★★ Exercici 6: CodeWars — Find the odd int](#)
 - [★★★ Exercici 7: AceptaElReto — 120 Nombre de parells i senars](#)
 - [📖 Referències](#)
- [Butlletí 07 — Extres: Visibilitat, Encapsulació i Static](#)
- [Boletín 6 - Inicial Resuelto: Herencia y Polimorfismo](#)
 - [Ejercicio 1: ¿Qué imprime? — La llamada a la familia](#)
 - [Ejercicio 2: Encuentra el error — extends mal usado](#)
 - [Ejercicio 3: Completa el código — the instanceof](#)
 - [Ejercicio 4: Escribe este programa — la jerarquía de animales](#)
 - [Ejercicio 5: ¿Qué imprime? — Referencias polimórficas](#)
 - [Ejercicio 6: Encuentra el error — polimorfismo y casting](#)
 - [Ejercicio 7: Escribe este programa — el zoo polimórfico](#)
 - [🔗 Referencias para seguir practicando](#)
- [Boletí 6 - Inicial: Herència i Polimorfisme](#)
 - [Exercici 1: Què imprimeix? — La família musical](#)

- [Exercici 2: Troba l'error — extends mal usat II](#)
- [Exercici 3: Completa el codi — instanceof amb downcasting](#)
- [Exercici 4: Escriu este programa — l'herència de vehicles](#)
- [Exercici 5: Què imprimeix? — Polimorfisme amb referències](#)
- [Exercici 6: Troba l'error — ClassCastException](#)
- [Exercici 7: Escriu este programa — la granja polimòrfica](#)
- [🔗 Referències per a seguir practicant](#)
- [Boletín 6 - Resuelto: Herencia y Polimorfismo](#)
 - [★ Ejercicio 1: La orquesta polimórfica](#)
 - [★ Ejercicio 2: Herencia de constructores — la cadena alimenticia](#)
 - [★ Ejercicio 3: El problema del jarrón \(Fragile Base Class\)](#)
 - [★ ★ Ejercicio 4: Batalla de personajes](#)
 - [★ ★ Ejercicio 5: La calculadora de figuras](#)
 - [★ ★ Ejercicio 6: Downcasting seguro](#)
 - [★ ★ ★ Ejercicio 7 \(ProgramaMe\): El simulador de ecosistema](#)
 - [★ ★ ★ Ejercicio 8 \(ProgramaMe\): Vehículos con combustible](#)
 - [🔗 Referencias para seguir practicando](#)
- [Boletí 6 - Intermedi: Herència, Polimorfisme i Interfícies](#)
 - [★ Exercici 1: Interfície FiguraGeometrica](#)
 - [★ Exercici 2: Jerarquia d'Empleats](#)
 - [★ ★ Exercici 3: Sistema de pagaments amb interfície](#)
 - [★ ★ Exercici 4: Interfícies múltiples: Volador i Nedador](#)
 - [★ ★ ★ Exercici 5: Sistema de notificaciones polimòrfic](#)
 - [★ ★ ★ Exercici 6: CodeWars — Is this a triangle?](#)
 - [★ ★ ★ Exercici 7: AceptaElReto — 154 L'ascensor](#)
 - [📚 Referències](#)
- [Butlletí 08 — Extres: Herència, Polimorfisme i Interfícies](#)
- [Boletín 8 - Inicial Resuelto: Arrays y Colecciones](#)
 - [Ejercicio 1: ¿Qué imprime? — Array básico](#)
 - [Ejercicio 2: Encuentra el error — ArrayIndexOutOfBoundsException](#)
 - [Ejercicio 3: Completa el código — for-each](#)
 - [Ejercicio 4: Escribe este programa — Array de 10 enteros al revés](#)
 - [Ejercicio 5: ¿Qué imprime? — ArrayList misterioso](#)

- [Ejercicio 6: Encuentra el error — colección sin genérico](#)
- [Ejercicio 7: Escribe este programa — HashSet sin duplicados](#)
- [🔗 Referencias para seguir practicando](#)
- [Boletín 8 - Inicial: Arrays i Col·leccions](#)
 - [Ejercicio 1: ¿Qué imprime? — Array de booleanos](#)
 - [Ejercicio 2: Encuentra el error — NullPointerException](#)
 - [Ejercicio 3: Completa el código — for básico para buscar el mayor](#)
 - [Ejercicio 4: Escribe este programa — contar números pares](#)
 - [Ejercicio 5: ¿Qué imprime? — ArrayList remove por índice vs valor](#)
 - [Ejercicio 6: Encuentra el error — length vs length\(\)](#)
 - [Ejercicio 7: Escribe este programa — búsqueda lineal](#)
 - [🔗 Referències per seguir practicant](#)
- [Boletín 8 - Resuelto: Arrays y Colecciones](#)
 - [★ Ejercicio 1: El inversor de arrays](#)
 - [★ Ejercicio 2: ArrayList de números pares](#)
 - [★ Ejercicio 3: Estadísticas con array](#)
 - [★ ★ Ejercicio 4: Buscaminas simplificado](#)
 - [★ ★ Ejercicio 5: Lista de la compra con ArrayList](#)
 - [★ ★ Ejercicio 6: HashSet y TreeSet — eliminando duplicados](#)
 - [★ ★ ★ Ejercicio 7 \(ProgramaMe\): El juego de la memoria](#)
 - [★ ★ ★ Ejercicio 8 \(ProgramaMe\): Estadísticas de texto con colecciones](#)
 - [🔗 Referencias para seguir practicando](#)
- [Boletín 8 - Intermedi: Arrays i Col·leccions](#)
 - [★ Ejercicio 1: La fusión de arrays ordenados](#)
 - [★ Ejercicio 2: Rotación circular a la derecha](#)
 - [★ Ejercicio 3: Suma de diagonales \(matriz cuadrada\)](#)
 - [★ ★ Ejercicio 4: Cola del supermercado con LinkedList](#)
 - [★ ★ Ejercicio 5: Intersección y unión de conjuntos](#)
 - [★ ★ Ejercicio 6: Eliminar duplicados manteniendo orden](#)
 - [★ ★ ★ Ejercicio 7 \(CodeWars\): Find the odd int](#)
 - [★ ★ ★ Ejercicio 8 \(AceptaElReto\): La lotería de la peña](#)
 - [📺 Referències](#)
- [Butlletí 09 — Extres: Arrays i Col·leccions](#)

- Boletín 9 - Inicial Resuelto: Genéricos y Mapas
 - Ejercicio 1: Completa el código — clase genérica
 - Ejercicio 2: ¿Qué imprime? — HashMap básico
 - Ejercicio 3: Encuentra el error — tipo primitivo en genérico
 - Ejercicio 4: Escribe este programa — mini agenda HashMap
 - Ejercicio 5: Completa el código — getOrDefault
 - Ejercicio 6: ¿Qué imprime? — método genérico
 - Ejercicio 7: Encuentra el error — la clave mutable
 -  Referencias para seguir practicando
- Boletín 9 - Inicial: Genèrics i Mapes
 - Ejercicio 1: Completa el código — clase con dos tipos genéricos
 - Ejercicio 2: ¿Qué imprime? — HashMap con merge
 - Ejercicio 3: Encuentra el error — TreeSet sin Comparable
 - Ejercicio 4: Escribe este programa — contador de palabras con HashMap
 - Ejercicio 5: ¿Qué imprime? — método genérico con límite
 - Ejercicio 6: Encuentra el error — clave duplicada el primero se pierde
 - Ejercicio 7: Escribe este programa — TreeMap con valores por defecto
 -  Referències per seguir practicant
- Boletín 9 - Resuelto: Genéricos y Mapas
 -  Ejercicio 1: Clase genérica Pareja
 -  Ejercicio 2: Contador de palabras con HashMap
 -  Ejercicio 3: Método genérico maximo
 -   Ejercicio 4: Agenda completa con menú
 -   Ejercicio 5: TreeMap — ordenando por clave
 -    Ejercicio 6 (ProgramaMe): Wildcards — el método de comparación
 -    Ejercicio 7 (ProgramaMe): Sistema de votaciones
 -    Ejercicio 8 (ProgramaMe): Cache simple con genéricos
 -  Referencias para seguir practicando
- Boletín 9 - Intermedi: Genèrics i Mapes
 -  Ejercicio 1: Pila genérica
 -  Ejercicio 2: HashMap inverso
 -  Ejercicio 3: Método genérico — filtrar por condición
 -   Ejercicio 4: Caché LRU con LinkedHashMap

- [!\[\]\(effbd7993c63c039a58fd3395789cf3f_img.jpg\) !\[\]\(144980d038f2541d7b588a8a9132bd70_img.jpg\) Ejercicio 5: TreeMap — frecuencia de letras](#)
- [!\[\]\(c4ce2d477989700c971cf3d240ad9283_img.jpg\) !\[\]\(5013555a72072875cb154b597e002a46_img.jpg\) Ejercicio 6 \(Wildcards\): Suma de números genérica](#)
- [!\[\]\(bf2038c114ec21ea58ad011774351c98_img.jpg\) !\[\]\(1ad0c3425edfa4762c2f20e33e3e5bbf_img.jpg\) !\[\]\(74d2fc5645add84f8511beb934060048_img.jpg\) Ejercicio 7 \(ProgramaMe\): Sistema de inventario genérico](#)
- [!\[\]\(ddc5533caa0187e636e3d3234e0983a3_img.jpg\) !\[\]\(7c207f8f59385c6dd11f9d9bdc7a0d1d_img.jpg\) !\[\]\(57a1bf910af99362b80b3ac4f2eecbac_img.jpg\) Ejercicio 8 \(CodeWars + AceptaElReto\)](#)
- [!\[\]\(6b114000ab07dda576e2920e2dc838fa_img.jpg\) Referències](#)
- [Butlletí 10 — Extres: Genèrics i Mapes](#)
- [Boletín 10 - Inicial Resuelto: Consola y Ficheros](#)
 - [Ejercicio 1: ¿Qué imprime? — printf\(\)](#)
 - [Ejercicio 2: Encuentra el error — nextInt\(\) + nextLine\(\)](#)
 - [Ejercicio 3: Completa el código — leer archivo](#)
 - [Ejercicio 4: Escribe este programa — Hola mundo archivo](#)
 - [Ejercicio 5: ¿Qué imprime? — el contador de líneas](#)
 - [Ejercicio 6: Encuentra el error — archivo no cerrado](#)
 - [Ejercicio 7: Escribe este programa — Scanner que suma números](#)
 - [!\[\]\(9129a6a4a4b11facb5cf665660eef788_img.jpg\) Referencias para seguir practicando](#)
- [Boletín 10 - Inicial: Consola, Fitxers i Regex](#)
 - [Ejercicio 1: ¿Qué imprime? — printf con conversiones](#)
 - [Ejercicio 2: Encuentra el error — IOException sin capturar](#)
 - [Ejercicio 3: Completa el código — try-with-resources](#)
 - [Ejercicio 4: Escribe este programa — guardar array en archivo](#)
 - [Ejercicio 5: ¿Qué imprime? — Scanner next vs nextLine](#)
 - [Ejercicio 6: Encuentra el error — File.createNewFile sin comprobar](#)
 - [Ejercicio 7: Escribe este programa — menú con Scanner y switch](#)
 - [!\[\]\(7cbd4924d31aa4c46df484d5c5bf4696_img.jpg\) Referències per seguir practicant](#)
- [Boletín 10 - Resuelto: Consola y Ficheros](#)
 - [!\[\]\(b3461332979d340882fda628798720af_img.jpg\) Ejercicio 1: La entrevista de trabajo](#)
 - [!\[\]\(5dc380853f1779b9d7a8d6fbbcdbb357_img.jpg\) Ejercicio 2: Calculadora de propinas](#)
 - [!\[\]\(2894a5ead456ff0145f35b01b4a6c7df_img.jpg\) Ejercicio 3: El diario personal \(append\)](#)
 - [!\[\]\(6b9cc5709aff55a0cbbb7dd78fcf3ba1_img.jpg\) !\[\]\(cd4202444435d9e729cdf801e3403cf4_img.jpg\) Ejercicio 4: El contador de líneas, palabras y caracteres](#)
 - [!\[\]\(a595a22b25819b91e6cd221ac336245d_img.jpg\) !\[\]\(912cec22f21865fe086f1aaa377b4c97_img.jpg\) Ejercicio 5: El gestor de contactos \(archivo\)](#)
 - [!\[\]\(d66d6cd7d0eca6e4b13ae9e87c7f7f49_img.jpg\) !\[\]\(2e58555536753c4dae915c3689ae8744_img.jpg\) Ejercicio 6: Lectura con NIO \(Files y Paths\)](#)
 - [!\[\]\(704be90457dfbf7435f703bdc5293a9e_img.jpg\) !\[\]\(9a8314206d1a19a24da8a71532ac6fea_img.jpg\) !\[\]\(d0819331239cfe31193900cc06944bf3_img.jpg\) Ejercicio 7 \(ProgramaMe\): Cifrado César con archivos](#)
 - [!\[\]\(4909cf475d986aa46b298b5fdca4ee07_img.jpg\) !\[\]\(a7d5e2c96349c1d63a82cd31458462fc_img.jpg\) !\[\]\(e28368bd9a276ed4dec00f0d27ffd39f_img.jpg\) Ejercicio 8 \(ProgramaMe\): Serialización de objetos](#)

- [!\[\]\(83eb2aa26b610eb6a9dca7cf4702d681_img.jpg\) Referencias para seguir practicando](#)
- [Boletín 10 - Intermedi: Consola, Fitxers i Regex](#)
 - [★ Ejercicio 1: Formateador de ticket de compra](#)
 - [★ Ejercicio 2: Buscador de archivos por extensión](#)
 - [★ Ejercicio 3: Lector de CSV con Scanner](#)
 - [★ ★ Ejercicio 4: Filtro de líneas por palabra clave](#)
 - [★ ★ Ejercicio 5: Separador de líneas pares e impares](#)
 - [★ ★ Ejercicio 6: Split con regex — analizador de frases](#)
 - [★ ★ ★ Ejercicio 7 \(ProgramaMe\): Validador de datos con regex](#)
 - [★ ★ ★ Ejercicio 8 \(CodeWars + AceptaElReto\)](#)
 - [!\[\]\(94dfacbf937cdd7da4837a6fcd8fc785_img.jpg\) Referències](#)
- [Butlletí 11 — Extres: Consola, Fitxers i Expressions Regulars](#)
- [Boletín 14 - Inicial Resuelto: JDBC - Conexión y Consultas](#)
 - [Ejercicio 1: Los 5 pasos](#)
 - [Ejercicio 2: Completa la conexión](#)
 - [Ejercicio 3: ¿Qué imprime?](#)
 - [Ejercicio 4: Encuentra el error](#)
 - [Ejercicio 5: INSERT con PreparedStatement](#)
 - [Ejercicio 6: ¿Qué hace executeUpdate\(\)?](#)
 - [Ejercicio 7: Diferencia entre executeQuery y executeUpdate](#)
- [Boletín 14 - Inicial: Connexió a BD amb JDBC](#)
 - [Ejercicio 1: ¿Qué necesitas para usar JDBC?](#)
 - [Ejercicio 2: Completa el código — try-with-resources](#)
 - [Ejercicio 3: ¿Qué imprime? — ResultSet vacío](#)
 - [Ejercicio 4: Encuentra el error — SQLException sin manejo](#)
 - [Ejercicio 5: Escribe este programa — mostrar nombres de columnas](#)
 - [Ejercicio 6: ¿Qué imprime? — executeQuery en UPDATE](#)
 - [Ejercicio 7: Encuentra el error — PreparedStatement con índices incorrectos](#)
 - [Ejercicio 8: Completa el código — cerrar recursos](#)
 - [!\[\]\(dae8c3c5fa7c80febd6526a5e8a853bf_img.jpg\) Referències per seguir practicant](#)
- [Boletín 14 - Resuelto: JDBC - Conexión y Consultas](#)
 - [★ Ejercicio 2: INSERT con PreparedStatement](#)
 - [★ Ejercicio 3: UPDATE con PreparedStatement](#)

- [!\[\]\(2dc8cdc0c918df88cde61039ecf68682_img.jpg\) Ejercicio 4: DELETE con PreparedStatement](#)
- [!\[\]\(793119bf0d613bd9b598fb8668922511_img.jpg\) !\[\]\(0a4819029e810ca9d2aba79260b63a4d_img.jpg\) Ejercicio 5: Búsqueda por nombre con LIKE](#)
- [!\[\]\(5b78a2fafd05db5e14d20573d68ef9b3_img.jpg\) !\[\]\(25fe2c0d7244c22c84de6bda963b471d_img.jpg\) Ejercicio 6: Contar alumnos por curso](#)
- [!\[\]\(d4bd0dc972749ad3ba477eac47688a0b_img.jpg\) !\[\]\(5eab3de5002abb449199a3fc43c9f414_img.jpg\) Ejercicio 7: Transacciones básicas](#)
- [!\[\]\(3f2384a64e2c0ffe3eae9a8107dd00c7_img.jpg\) !\[\]\(0a4ab723df2c815236fb0c30cb14280f_img.jpg\) !\[\]\(a5e6025d913df625081ab04ab57538d0_img.jpg\) Ejercicio 8: CRUD completo con menú](#)
- [!\[\]\(933db2af0bc51ccc9956c85daceec771_img.jpg\) !\[\]\(aa37c4d902b06a4c60c80e22dbdcae63_img.jpg\) !\[\]\(305017e3492be9834d83526ded9a5546_img.jpg\) Ejercicio 9: Paginación con LIMIT y OFFSET](#)
- [!\[\]\(ce93526cc7275dc99a5e28e42c2dec45_img.jpg\) !\[\]\(3d8621e1356074fde3f4532b15fe8afa_img.jpg\) !\[\]\(871f3967dd06baf62bc0e930d9243f73_img.jpg\) Ejercicio 10: Exportar a CSV con metadatos](#)
- [Boletín 14 - Intermedi: Connexió a BD amb JDBC](#)
 - [!\[\]\(37c6dc81049e8a4c6dd88ffa288e134a_img.jpg\) Ejercicio 1: Conexión desde archivo de propiedades](#)
 - [!\[\]\(4d2dd53ee1a5a6f5efa820e745814047_img.jpg\) Ejercicio 2: INSERT con clave autogenerada](#)
 - [!\[\]\(632644865a98825d4ab6066f88459a98_img.jpg\) Ejercicio 3: UPDATE condicional](#)
 - [!\[\]\(c5f5eef4efa8c0ad34aae60427d6ac5b_img.jpg\) !\[\]\(d82eccc468be357cae6806191127e55d_img.jpg\) Ejercicio 4: INNER JOIN con PreparedStatement](#)
 - [!\[\]\(29682f0a9343f6406dd572d30664137a_img.jpg\) !\[\]\(9072e497dc9469cd8fef36b3e671b928_img.jpg\) Ejercicio 5: Fechas en JDBC](#)
 - [!\[\]\(d5eb365f0ca476d7473041e988d395dc_img.jpg\) !\[\]\(87e611d5e1961433c877458822e2b766_img.jpg\) Ejercicio 6: Batch INSERT — 100 alumnos de prueba](#)
 - [!\[\]\(8f5ae82295611d92d64aa46df97d9f7a_img.jpg\) !\[\]\(523047843acc9a1e0403ccd04a69c0cc_img.jpg\) !\[\]\(68537e80d498309418ddde7ee99e37be_img.jpg\) Ejercicio 7 \(ProgramaMe\): Patrón DAO](#)
 - [!\[\]\(eb37c866e2cac2929b775e1ab51b8671_img.jpg\) !\[\]\(ef46f420fb9fc925f0c471e0545a81a3_img.jpg\) !\[\]\(b4a763b912a4c8c20df5568990dccca7_img.jpg\) Ejercicio 8 \(CodeWars + AceptaElReto\)](#)
 - [!\[\]\(1e9974895b7911d081cc354cb27e4686_img.jpg\) Referències](#)
- [Butlletí 12 — Extres: Connexió a Bases de Dades amb JDBC](#)
- [Boletín 11 - Inicial resuelto: Interfaz Web](#)
 - [Ejercicio 1: Hola Mundo Web](#)
 - [Ejercicio 2: Hora del servidor](#)
 - [Ejercicio 3: HTML bonito](#)
 - [Ejercicio 4: Saludo personalizado](#)
 - [Ejercicio 5: Contador de visitas](#)
 - [Ejercicio 6: Tabla HTML](#)
- [Butlletí 13 - Inicial: Servir i Consumir APIs amb Web](#)
 - [Exercici 1: Servidor de l'hora](#)
 - [Exercici 2: Pàgina que diu el teu nom](#)
 - [Exercici 3: Comptador de visites global](#)
 - [Exercici 4: Generador d'excuses per a lliuraments tardans](#)
 - [Exercici 5: Taula de multiplicar personalitzada](#)
 - [Exercici 6: Pàgina d'estat del servidor](#)
 - [Exercici 7: Convertidor d'euros a pessetes \(sí, pessetes\)](#)

-  [Referències](#)
- [Boletín 11 - Resuelto: Interfaz Web](#)
 -  [Ejercicio 1: Servidor de archivos estáticos](#)
 -  [Ejercicio 2: Formulario de contacto](#)
 -   [Ejercicio 3: API JSON de productos](#)
 -   [Ejercicio 4: Chat en memoria](#)
 -    [Ejercicio 5: Mini framework MVC](#)
 -    [Ejercicio 6: Subida de archivos](#)
- [Butlletí 13 - Intermedi: Servir i Consumir APIs amb Web](#)
 -  [Exercici 1: API de frases motivacionals](#)
 -  [Exercici 2: Traductor xungo \(però funcional\)](#)
 -   [Exercici 3: API REST de tasques amb prioritat](#)
 -   [Exercici 4: El temps que NO fa](#)
 -   [Exercici 5: Catàleg de pel·lícules amb filtres](#)
 -    [Exercici 6: Middleware de logging](#)
 -    [Exercici 7: Server-Sent Events — El rellotge del servidor](#)
 -  [Exercici 8: Pedra, paper, tisora online](#)
 -  [Referències](#)
- [Butlletí 13 — Extres: Servir i Consumir APIs amb Web](#)



Programació Java — DAM/DAW

Curs complet d'iniciació a la programació en Java. Bilingüe (valencià / castellà), amb cercador, mode fosc i disseny premium.



Sergi Garcia Barea — CC BY-SA 4.0

↓ Prefereixes estudiar sense connexió? Descarrega el curs complet

PDF (Castellà)

PDF (Valencià)

EPUB (Castellà)

EPUB (Valencià)



Unitats

UNITAT 1

RA1

Introducció a Java

Primer contacte amb Java: instal·la el JDK, escriu el teu primer programa, coneix el mètode `main`, els comentaris i els arguments de línia de comandos.

Inicial

✓ Inicial res.

Intermedi

Intermedi res.

★ Extres

UNITAT 2

RA2

Variables, Tipus i Operadors

Declara variables, usa tipus primitius, operadors aritmètics i lògics, conversions de tipus i llig dades per teclat amb `Scanner`.

Inicial

✓ Inicial res.

Intermedi

Intermedi res.

★ Extres

UNITAT 3

RA3

Estructures de Control i Excepcions

Domina `if / else`, `switch`, bucles `while` i `for`, i gestiona excepcions amb `try / catch` perquè el teu programa no es trenque.

Inicial

✓ Inicial res.

Intermedi

Intermedi res.

★ Extres

UNITAT 4

RA2, RA6

Algorísmica I: Fonaments

Aprén a pensar com un programador: dividix problemes en parts, usa pseudocodi, diagrames de flux i crea funcions reutilitzables.

Inicial

✓ Inicial res.

Intermedi

Intermedi res.

★ Extres

UNITAT 5

RA2, RA6

Algorísmica II: Tècniques Avançades

Algorismes d'ordenació, cerca binària, recursivitat i tècniques dividix i venceràs per a resoldre problemes més complexos.

 Inicial  Inicial res.  Intermedi

 Intermedi res.  Extres

UNITAT 6

RA2, RA4

POO: Classes i Objectes

Programació Orientada a Objectes: crea classes, instància objectes, definix atributs i mètodes, i entén la màgia dels constructors.

 Inicial  Inicial res.  Intermedi

 Intermedi res.  Extres

UNITAT 7

RA4

Visibilitat, Encapsulació i Static

Controla qui veu què: modificadors d'accés (`public`, `private`, `protected`), encapsulació amb getters/setters, membres `static` i constants.

 Inicial  Inicial res.  Intermedi

 Intermedi res.  Extres

UNITAT 8

RA4, RA7

Herència, Polimorfisme i Interfícies

Herència, polimorfisme, classes abstractes i interfícies: la base del disseny flexible i reutilitzable en Java. Aprén a sobreesciure mètodes i a usar `super`.

 Inicial  Inicial res.  Intermedi

 Intermedi res.  Extres

UNITAT 9

RA6

Arrays i Col·leccions

Arrays unidimensionals i multidimensionals, la classe `Arrays`, `ArrayList`, `LinkedList` i com triar la col·lecció adequada per a cada problema.

 Inicial  Inicial res.  Intermedi

 Intermedi res.  Extres

UNITAT 10

RA6

Genèrics i Mapes

Genèrics per a classes i mètodes segurs de tipus, la interfície `Map` i les seues implementacions `HashMap`, `TreeMap`, i com iterar sobre elles.

 Inicial  Inicial res.  Intermedi

 Intermedi res.  Extres

UNITAT 11

RA5

Consola, Fitxers i Expressions Regulars

Entrada/eixida per consola, lectura i escriptura de fitxers de text i binaris, serialització d'objectes i expressions regulars per a buscar patrons.

 Inicial  Inicial res.  Intermedi

 Intermedi res.  Extres

UNITAT 12

RA9

Connexió a Bases de Dades amb JDBC

Connecta Java amb bases de dades relacionals usant JDBC: `Connection`, `Statement`, consultes, insercions, actualitzacions i transaccions segures.

 Inicial  Inicial res.  Intermedi

 Intermedi res.  Extres

Servir i Consumir APIs amb Web

Crea un servidor HTTP amb Java `HttpServer`, servix pàgines HTML/JS, gestiona peticions JSON, implementa una API REST completa des de zero.

● Inicial

✓ Inicial res.

📄 Intermedi

👉 Intermedi res.

★ Extres

📄 Llicència



CC BY-SA 4.0 — *Sergi Garcia Barea*

[Creative Commons Attribution-ShareAlike 4.0 International](https://creativecommons.org/licenses/by-sa/4.0/)

Unidad 1: Introducción a Java

 Objectius d'aprenentatge

- Instal·lar i configurar el JDK
- Escriure i compilar el primer programa Java
- Utilitzar el depurador de l'IDE
- Diferenciar JDK, JRE i JVM
- Escriure comentaris i documentació bàsica

L'Ordinador T'Obeeix: Entorns de Desenvolupament

Sabies que el teu ordinador és bàsicament un cadell molt llest però amb zero iniciativa? No fa res fins que li dones ordres precises. I per a això necessitem un **entorn de desenrotllament**.

Què és això del JDK, JRE i JVM? (La Trilogia del Cafè)

Java funciona com una cafeteria d'especialitat:

- **JVM (Java Virtual Machine):** És la màquina de cafè. Té la seua pròpia recepta (bytecode) i funciona igual en qualsevol lloc. Viatja amb el teu programa a totes parts.
- **JRE (Java Runtime Environment):** És la cafeteria sencera. Té la màquina, els gots, el sucre... tot el necessari per a *executar* cafè ya fet.
- **JDK (Java Development Kit):** És el kit complet per a muntar la teua pròpia cafeteria. Té la màquina, els grans de cafè verd, el molinet, el manual de barista... TOT per a *crear* programes.

```
// Imagina que esto es un grano de café verde:
public class Cafe {
    public static void main(String[] args) {
        System.out.println("☕ ¡Café listo!");
    }
}
```

El JDK compila açò a bytecode (cafè molt), el JRE ho passa per la JVM i... tachán! cafè a la teua pantalla.

 Nota:

Dada freak: La creació de Java en Sun Microsystems (1995) es va inspirar en la màquina de cafè de l'oficina. Per això el logo és una tassa humejant. No m'ho invente.

 Advertència:

No confongues JDK amb JRE. El JDK és el *ganivet del chef*, el JRE és el *plat servit*. Si només vols executar programes, et basta el JRE. Si vols *crear-los*, necessites el JDK.

Muntant el Xiringuito: Instal·lació

Instal·laràs el JDK. No t'espantes, és més fàcil que muntar un moble d'Ikea i no et sobran caragols.

```
> java -version
openjdk version "21" 2026-01-01
> javac -version
javac 21
```

Si veus alguna cosa pareguda, enhorabona! Tens poders de compilació.

 Consell:

Si uses [Eclipse Temurin](#) t'estalviaràs maldecaps. És com el JDK oficial però sense fums rars.

El Teu Primer Programa: Hola Món (o com parlar-li a la màquina)

Programar és com parlar-li a un extraterrestre molt literal. Si li dius “saluda”, no ho fa. Has de dir-li *com* saludar, *quan* i *per què*.

```
public class HolaMundo {
    public static void main(String[] args) {
        System.out.println("¡Hola, Mundo! Llevo años esperando a que me crearas.");
    }
}
```

Anem a disseccionar açò com si fóra una granota en biologia:

- `public class HolaMundo`: Declares una classe. Pensa en això com “Escolta, Java, vaig a crear una cosa que es diu HolaMundo”.
- `public static void main(String[] args)`: Este és el “botó d’inici”. Quan executes el programa, Java busca esta línia i diu “Per ací es comença!”.
- `System.out.println(...)`: És la veu del programa. Li dius que cride alguna cosa per la consola.

★ BE THE CODE, MY FRIEND

👁️ **Don Tip:** Segueix el codi línia a línia com si fores la JVM. Cada instrucció s’executa en orde.

Exercici 1: Tu ets el Compilador

Vas a ser Java per un moment. Pren paper i boli (o mentalment). Et donen este codi:

```
public class Computadora {  
    public static void main(String[] args) {  
        int x = 5;  
        int y = 10;  
        int z = x + y;  
        System.out.println("El resultado es: " + z);  
    }  
}
```

Pregunta: Sense executar-ho, què imprimeix? Segueix els passos com si fores la JVM:

1. Trobes la classe Computadora.
2. Busques el mètode `main` - ahí està.
3. Crees un espai anomenat `x` i fiques un 5.
4. Crees `y` i fiques un 10.
5. Crees `z`, sumes `x` i `y` (15), ho guardes.
6. Crides per pantalla “El resultat és: 15”.

Si la teua resposta va ser diferent, torna a començar. L’ordinador és tonto però precis: no interpreta, *executa*. Cada línia, en orde, sense saltar-se’n cap.

★ BE THE CODE, MY FRIEND

Exercici 2: El Mètode que no Troba

Observa este codi. S'executarà correctament? Per què?

```
public class Saludos {  
    public static void main(String[] args) {  
        System.out.println("¡Hola desde el método main!");  
    }  
  
    public static void saludo() {  
        System.out.println("Esto nunca se ejecuta...");  
    }  
}
```

Resposta: Sí s'executa, però sols imprimeix la primera línia. El mètode saludo() existix però com mai el crides des de main, es queda ahí fent el vague. Java sols executa el que està dins del main (a no ser que explícitament crides a uns altres mètodes). El mètode saludo() és com un actor que té el guió après però mai eix a l'escenari.

★ BE THE CODE, MY FRIEND

Exercici 3: L'Error del Novat

Què està malament ací? Assenyala tots els errors que trobes:

```
Public class Calculadora  
    public static void main(string[] args) {  
        System.out.println("Suma: " + 5 + 3)  
        SYSTEM.OUT.PRINTLN("Resta: " + (5 - 3));  
    }  
}
```

Resposta: Hi ha 4 errors! (1) Public hauria de ser public (minúscula). (2) Falta { després de Calculadora. (3) string[] args hauria de ser String[] args (la S majúscula importa). (4) Falta ; al final de la primera línia del println. Java és molt puntillós, com un professor de llengua amb les comes.

El Depurador: Ets un Detectiu

Un programa rar. La variable `edad` eix 25 quan hauria d'eixir 18. Què fas? Li pegues a l'ordinador? No. Uses el **depurador**.

El depurador és com tindre visió de raigs X per al teu codi:

- **Breakpoint (punt de ruptura)**: Li dius a Java “para ACÍ, vull vore què està passant”.
- **Step Over (F8)**: “Executa esta línia però no em contenen els detalls”.
- **Step Into (F7)**: “Executa esta línia i porta'm dins d'eixa funció, vull espia”.
- **Watch (inspecció)**: “Ensenya'm el valor de la variable `edad` ARA MATEIX”.

```
public class DetectivesDeCodigo {  
    public static void main(String[] args) {  
        int sospechoso = 0;  
        for (int i = 0; i < 10; i++) {  
            sospechoso += i; // Pon un breakpoint aquí  
        }  
        System.out.println("El culpable es: " + sospechoso);  
    }  
}
```

Posa un breakpoint en la línia del `sospechoso += i`, executa en mode depuració, i mira com `sospechoso` canvia en cada volta. És com vore una sèrie de crims en càmera lenta!

No Hi Ha Preguntes Tontes!

? No Hi Ha Preguntes Tontes!

Q: Per què `public static void main(String[] args)`? Pareix un encanteri de Harry Potter.

A: Perquè sí. Val, no és bona resposta. `public` és perquè Java puga trobar el mètode des de fora. `static` és perquè puga cridar-lo sense necessitat de crear un objecte (arribarem a això). `void` significa que no torna res. `main` és el nom que Java busca al arrancar. `String[] args` és una butxaca on pots ficar arguments al executar el programa. I sí, pareix un encanteri de Harry Potter.

Q: Puc cridar a la meua classe `Holamundo` en minúscula?

A: Pots, però Java et mirarà malament. Les classes comencen en majúscula per convenció. No és obligatori, però si no ho fas, uns altres programadors pensaran que eres un psicòpata.

Q: Si m'equivoque en un punt i coma, l'ordinador explota?

A: No, però el compilador et llançarà un error críptic i tu passaràs 20 minuts buscant un ; perdut. Benvingut a la programació.

Q: Puc tindre dos classes amb el mateix nom en el mateix archiu?

A: No, i Java es posarà molt borde. Cada archiu .java pot tindre sols una classe public, i eixa classe ha de dir-se exactament igual que l'archiu. Si el teu archiu es diu HolaMundo.java, no pots tindre dins una classe public class AdiosMundo. Pots tindre classes no públiques, però cada una en el seu propi archiu és més net.

Q: Java i JavaScript són cosins?

A: No, ni tan sols són del mateix planeta. Java és a JavaScript com un gos és a un gosset calent. El nom va ser una estratègia de màrqueting de Netscape per a muntar-se en el boom de Java.

Els Comentaris: Notes Apegaloses Digitalis

Els comentaris són missatges que et deixes a tu mateix (o a uns atres). L'ordinador els ignora completament.

```
// Comentario de una línea: "Aquí va la magia"

/*
    Comentario de varias líneas:
    "Si esto funciona, no lo toques.
    Si no funciona, no lo toques tampoco.
    Ya llamaremos a alguien."
*/

/**
 * Comentario Javadoc (el elegante):
 * Sirve para generar documentación automática.
 * @param argumentos la lista de argumentos de la línea de comandos
 * @return nada, esto es void, ¿no te enteras?
 */
```

Consell:

Comenta el *per què*, no el *què*. `int i = 0; // Declare i amb valor 0 és com posar "Obro la porta" en una porta.` El codi ja diu això. En canvi `int i = 0; // Comencem des de 0 perquè`

l'usuari no ha polsat res això sí és útil.

Més Exemples de Codi

```
public class MiPrimerPrograma {  
    public static void main(String[] args) {  
        // Esto es mi primer programa  
        System.out.println("¡Holaaaa, mundo!");  
        System.out.println("Estoy aprendiendo Java");  
        System.out.println("Y me está gustando (de momento)");  
    }  
}
```

```
public class UsoDeArgumentos {  
    public static void main(String[] args) {  
        System.out.println("Has escrito " + args.length + " palabras:");  
        for (int i = 0; i < args.length; i++) {  
            System.out.println("Palabra " + (i + 1) + ": " + args[i]);  
        }  
    }  
}
```

Si executes: `java UsoDeArgumentos Java mola molt`, veuràs:

```
Has escrito 3 palabras:  
Palabra 1: Java  
Palabra 2: mola  
Palabra 3: mucho
```

✖ EL LÍO

El teu cap ha deixat este codi fet un desastre. Les línies estan barrejades. Eres capaç d'ordenar-les perquè siga un programa Java vàlid i que imprimisca “La suma és: 8”?

```
System.out.println("La suma es: " + (a + b));  
int a = 5;  
public class CalculoLioso {  
    System.out.println("Calculando...");  
    public static void main(String[] args) {  
        int b = 3;
```

Pista: busca primer on comença la classe i el mètode main.

👁️ **Don Tip:** La classe és el contenidor, el main és la porta d'entrada, les instruccions van dins del main.

Resum (el que importa de veritat)

- El JDK compila, el JRE executa, la JVM transporta. Com Amazon però amb cafè.
- L'IDE és la teua navalla suïssa: editor, compilador, depurador, tot en un.
- `public static void main(String[] args)` és la porta d'entrada.
- El depurador et permet espiar el teu codi en càmera lenta.
- Els comentaris són per a humans, no per a màquines.

Exercicis Proposats

1. **Hola, qui eres?** Escriu un programa que mostre el teu nom, la teua edat i la teua ciutat favorita en tres línies separades.

💡 **Consell:** Els exercicis que usen `if` i bucles els veurem oficialment en la Unitat 3. Si et sents aventurer, intenta'ls — són resolubles amb el que saps + una mica d'intuïció. Si preferixes anar pas a pas, torna a ells després de la Unitat 3. No hi ha pressa!

2. **El detectiu incansable** Programa un bucle simple que sume números del 1 al 100. Posa un breakpoint i observa com canvia la variable acumuladora.
3. **Error buscamines** Escriu intencionadament un programa que oblide un punt i coma. Compila i anota el missatge d'error. Després arregla-ho.
4. **Javadoc de la teua vida** Crea una classe `SobreMi` amb un mètode `main` i afegix comentaris Javadoc a la classe explicant qui eres i per què estàs aprenent Java.
5. **Arguments secrets** Escriu un programa que imprimisca tots els arguments que rep des de la línia de comandes (usa `args`). Executa-ho amb `java MiPrograma hola món açò és una`

prova.

6. **Mini-calculadora a pelo** Sense usar Scanner, declara dos números `int` directament en el codi, suma'ls, resta'ls, multiplica'ls i dividix'ls. Mostra els resultats.
 7. **Fred o calor?** Declara una variable `int temperatura = 30`. Usa un `if` perquè imprimisca “Fa calor” si és major de 25 i “Fa fresc” si és menor o igual.
 8. **El depurador xafarder** Crea un programa amb tres variables (`a`, `b`, `c`). Posa un breakpoint i executa pas a pas, anotant com canvien els valors. Coincidix amb el que esperaves?
-

RAs trabajados en esta unidad:

- **RA1** - Entornos de desarrollo
-



Sergi Garcia Barea — CC BY-SA 4.0

Unidad 2: Variables, Tipos de Datos y Operadores

 Objectius d'aprenentatge

- Declarar i usar variables de tipus primitius
- Comprendre la diferència entre tipus primitius i String
- Utilitzar operadors aritmètics, relacionals i lògics
- Aplicar casting i conversions entre tipus
- Generar nombres aleatoris amb Math.random()

Variables: Les Caixes On Viuen Les Teues Dades

Imagina't que la memòria del teu ordinador és un magatzem gegant ple de prestatgeries. Cada prestatgeria té caixes. Les **variables** són eixes caixes, i cada caixa té una etiqueta per a que sàpies què hi ha dins.

Les Caixes (Declaració de Variables)

Per a crear una caixa li dius a Java:

```
tipo nombreDeLaCaja = valorQueMetoDentro;
```

Exemples reals:

```
int edad = 25;           // Caja etiquetada "edad" con un 25 dentro
double precio = 19.99;   // Caja con decimales
String nombre = "María"; // Caja mágica que guarda texto
boolean hambre = true;   // Caja de verdadero/falso (ahora mismo: true)
```

Consell:

Les variables es diuen així perquè... ¡varien! Pots canviar el seu contingut. `int edad = 25;` i després `edad = 26;` el dia del teu cumple. L'etiqueta és la mateixa, el contingut canvia.

Les Regles de Nomenclatura (o com no ficar-la)

- Poden tindre lletres, números, _ i \$. *No* espais ni ñ's ni coses rares.
- No poden començar amb número. 1numero és il·legal. numero1 és legal. Així de tiquismiquis és Java.
- Les majúscules importen: edad, Edad i EDAD són tres caixes distintes. Com si etiquetares “Zapatos”, “zapatos” i “ZAPATOS”.
- No uses paraules reservades: int, class, if, while... són de Java, no teues.
- Usa **camelCase**: miVariableEjemplo. Com un camell, amb gepa enmig.

Els 8 Primitius: Caixes de Mides Distintes

Java té 8 tipus primitius. Pensa en ells com caixes de distintes mides al teu magatzem:

Tipus	Mida	El que cap	Analogia
byte	8 bits	-128 a 127	Caixa de llumins
short	16 bits	-32.768 a 32.767	Caixa de sabates
int	32 bits	-2.147M a 2.147M	Caixa de mudança (la que més usaràs)
long	64 bits	-9 quadrilions a +9 quadrilions	Contenedor de vaixell
float	32 bits	Decimals de precisió simple	Got d'aigua
double	64 bits	Decimals de precisió doble	Cubell d'aigua
char	16 bits	Un sol caràcter Unicode	Una lletra en una caixa de sabates
boolean	1 bit	true o false	Interruptor de llum

```
byte nivel = 100;
short poblacion = 30000;
int habitantes = 1500000;           // El más usado
long distancia = 384400000L;       // La L al final es obligatoria
float precio = 12.99f;             // La f al final es obligatoria
double pi = 3.14159265359;
char letra = 'A';                  // Comillas SIMPLES para char
boolean esJavaDivertido = true;    // Esto es opinable
```

Nota:

Usa `int` per a quasi tot el numèric enter. Només usa `long` si vas a contar estrelles. Usa `double` per a decimals a menys que estalviar memòria siga el teu fetitxe.

String: La Caixa Màgica (No és primitiu, però ho pareix)

`String` no és primitiu, és una **classe**. Però es comporta tan natural que pareix primitiu. Es com un amic que encaixa tan bé en el teu grup que jures que és família.

```
String saludo = "Hola, DAM";
String nombre = new String("Ana"); // También se puede crear así
```

Advertència:

Els `String` són **inmutables**. Una volta creats, no es poden canviar. Quan fas `texto = texto + " más"`, en realitat estàs tirant el vell i creant-ne un de nou. Es com si cada volta que volgueres posar un cartell nou cremares l'anterior.

Constants: Caixes amb Superglue

Les constants es declaren amb `final`. Una volta que fiques alguna cosa ahí, no ixes ni amb palanca.

```
final double IVA = 0.21;
final int MAXIMO_INTENTOS = 3;
final String NOMBRE_APP = "Gestión DAM";

IVA = 0.10; // ERROR: ¡Has roto Java! (de compilación, no te preocupes)
```

Per convenció, les constants s'escriuen EN MAJÚSCULES_AMB_GUIONS_BAIXOS. Com si estigueren cridant “¡SOC IMMUTABLE!”.

Casting: Prem que Cap

Hi ha dos tipus de conversions:

- **Implícita (Widening):** De caixa menuda a gran. Java ho fa sol. `int` → `long` → `double`. Com canviar d'un pis a una mansió. No preguntes, simplement et mudes.
- **Explícita (Narrowing):** De gran a menuda. Java t'obliga a posar (tipus) davant. Com ficar una maleta XXL al maleter d'un Smart: has d'empènyer (tipus) i resar.

```
// Implícita: ensanchando
int num = 100;
long numLong = num;           // Cabe, no problema
double numDouble = num;       // También

// Explícita: estrechando
double precio = 19.99;
int entero = (int) precio;     // Pierdes los .99 → sale 19
System.out.println(entero);    // 19 – los céntimos desaparecen en el olvido
```

Math.random(): El Casino de Java

`Math.random()` retorna un nombre aleatori entre 0.0 i 1.0 (l'1.0 no està inclòs, com quan et toca la loteria però no).

```
double aleatorio = Math.random();           // Entre 0.0 y 0.999999...
int dado = (int) (Math.random() * 10);      // Entre 0 y 9
int dadoReal = (int) (Math.random() * 6) + 1; // Entre 1 y 6 (como un dado)
```

💡 Consell:

Per a un nombre entre min i max: `(int)(Math.random() * (max - min + 1)) + min`. Exemple del 5 al 10: `(int)(Math.random() * 6) + 5`.

★ BE THE CODE, MY FRIEND

💡 **Don Tip:** El *casting* explícit amb (tipus) pot truncar valors. Comprova sempre si el valor cap abans de forçar-lo.

Exercici 1: El Guarda de Magatzem

Eres el guarda d'un magatzem de dades. Et donen estes instruccions:

```
int a = 10;
double b = a;
int c = (int) b;
byte d = (byte) c;
System.out.println(d);
```

Segueix el procés pas a pas:

1. `int a = 10;` — Fiques un 10 en una caixa `int`.
2. `double b = a;` — Agafes el 10 i el fiques en una caixa `double`. Conversió implícita.
3. `int c = (int) b;` — Agafes el 10.0 del `double`. Necessites `(int)` perquè passar de `double` a `int` requereix apretar.
4. `byte d = (byte) c;` — Agafes el 10 del `int` i el fiques en un `byte`. Cap, però forces amb `(byte)`.
5. Imprimeix: **10**.

Ara prova este:

```
int grande = 300;
byte pequeno = (byte) grande;
System.out.println(pequeno);
```

Què ix? (Pista: en un `byte` només caben -128 a 127. Sobren 172. En binari, es truncaren els bits sobrants). Resultat: **44**. Es com intentar ficar un elefant en un Mini Cooper i que isca un gos salchicha.

★ BE THE CODE, MY FRIEND

💡 **Don Tip:** `==` compara referències (són el mateix objecte?), `.equals()` compara contingut (tenen el mateix text?).

Exercici 2: Què Imprimeix Este Embolic de Strings?

Sense executar, digues QUÈ imprimeix exactament este codi:


```
String a = "Hello";
String b = "Hello";
String c = new String("Hello");
System.out.println(a == b);
System.out.println(a == c);
```

Resposta: true i false. a i b apunten al mateix objecte en el “pool de Strings” (Java reutilitza literals iguals). Però c es va crear amb `new String(...)`, així que és un objecte nou. `==` compara referències, no contingut. Per a comparar contingut usa `.equals()`: `a.equals(c)` retornaria true. ¡Trampa típica d’examen!

★ BE THE CODE, MY FRIEND

👁️ **Don Tip:** Els operadors `++` pre i post tenen prioritats diferents. Pre: primer canvia, després usa. Post: primer usa, després canvia.

Exercici 3: Increment Misteriós

Què imprimeix este codi?

```
int a = 5;
int b = a * 2 + ++a;
System.out.println("a = " + a);
System.out.println("b = " + b);
```

Resposta: `a = 6`, `b = 16`. `++a` s’avalua primer (unari), a passa a 6. Després `a * 2 + 6` → `5 * 2 + 6` → 16.

Si haguera sigut `a * 2 + a++`, seria distint: `5 * 2 + 5 = 15` (primer usa `a=5`, després incrementa a 6).

Mètodes Útils de String (perquè els necessitaràs)

```
String texto = "  Programación DAM  ";
texto.length();           // 18
texto.trim();             // "Programación DAM" (sin espacios)
texto.toUpperCase();       // "  PROGRAMACIÓN DAM  "
texto.toLowerCase();       // "  programación dam  "
texto.contains("DAM");     // true
texto.startsWith("  ");   // true
texto.endsWith("AM ");    // true
texto.indexOf("DAM");      // 14 ¿dónde empieza "DAM"?
texto.substring(2, 13);    // "Programación"
texto.replace("DAM", "DAW"); // "  Programación DAW  "
```

Operadors: El Gimnàs de les Dades

Les variables estan molt bé, però no serveixen de res si no fas coses amb elles. Els **operadors** són les màquines de pesos del teu gimnàs de dades: sumen, resten, comparen i transformen.

Operadors Aritmètics: El Dia al Gym

Operador	Exercici	Exemple
+	Press de banca	$5 + 3 = 8$
-	Curl de bíceps	$5 - 3 = 2$
*	Sentadilla	$5 * 3 = 15$
/	Pes mort	$10 / 3 = 3$ (enters) o $10.0 / 3 = 3.333...$
%	L'odiat abdominal	$10 \% 3 = 1$ (el reste de 10/3)

```
int a = 10;
int b = 3;
double c = 10.0;

System.out.println(a / b);           // 3 (divisió entera)
System.out.println(a % b);           // 1 (el resto)
System.out.println(c / b);           // 3.333... (divisió real)
System.out.println((double) a / b);  // 3.333... (obligas decimal)
```

La divisió entera mata. Si tens `int alumnos = 17; int grupos = 5;` i fas `alumnos / grupos`, Java diu que cada grup té **3** alumnes. Per a Java, 17 dividit entre 5 són 3. Punt.

Precedència: Qui Va Primer?

```
int resultado = 2 + 3 * 4;           // 14 – la multiplicación se cuele
int conParentesis = (2 + 3) * 4;    // 20 – los paréntesis tienen pase VIP
```

La llei del menjador:

1. **Parèntesis ()** — Passe VIP, van els primers.
2. **Multiplicació, divisió i mòdul * / %** — Els populars.
3. **Suma i resta + -** — Els normals, els últims.

Operadors d'Assignació Composta: La Drecera Peresosa

```
int x = 10;
x += 5;    // x = 15 (x = x + 5, pero más cool)
x -= 3;    // x = 12
x *= 2;    // x = 24
x /= 4;    // x = 6
x %= 3;    // x = 0
```

Es com si en lloc d'anar a la cuina a per un got d'aigua, tingueres una aixeta al sofà.

++ i --: Flexions per a Variables

```
int a = 5;
int b = a++; // b = 5, a = 6 (POST: "usa y luego sube")
int c = ++a; // a = 7, c = 7 (PRE: "sube y luego usa")
```

 **Consell:**

Regla d'or: Si uses `++` o `--` *dins* d'una expressió complicada, estaràs escrivint codi que ni tu entendràs en una setmana. Usa'ls sols, en la seua pròpia línia.

 **BE THE CODE, MY FRIEND**

 **Don Tip:** Desglossa l'expressió pas a pas. Quin valor té `x` en cada moment?

Exercici 4: L'Acròbata de les Variables

Sense executar, calcula quant val tot aquí:

```
int x = 3;
int y = x++ + ++x;
System.out.println("x = " + x + ", y = " + y);
```

Pas a pas:

1. $x = 3$
2. $x++$ — POST: usa x (3), després incrementa x a 4. El valor de $x++$ és **3**.
3. $++x$ — PRE: x val 4 ara. Incrementa x a **5**, després val **5**.
4. $y = 3 + 5 = 8$
5. Resultat: $x = 5$, $y = 8$.

Als programadors professionals també els costa. Per això quasi ningú escriu codi així en producció. Però en els exàmens... ¡ai, apareix!

Operadors Relacionals: El Jutge de la Discussió

```
int edad = 18;
boolean puedeVotar = edad >= 18;           // true
boolean tieneDescuento = edad < 12 || edad > 65; // false
boolean noEsEl = edad != 18;              // false
```

Operadors Lògics: El Club Nocturn

- **&& (AND)**: Tens més de 18 I tens entrada? Les dos s'han de complir.
- **|| (OR)**: Tens més de 18 O eres el amo? Basta una.
- **! (NOT)**: NO tens menys de 18?

```

boolean mayorEdad = true;
boolean tieneEntrada = false;

boolean entra = mayorEdad && tieneEntrada;    // false
boolean entraVip = mayorEdad || tieneEntrada; // true

int x = 5;
boolean resultado = (x > 10) && (++x > 0); // false, y x sigue siendo 5

```

¡Curtcircuit! Amb &&, si el primer és false, Java ni es molesta en mirar el segon. Amb ||, si el primer és true, igual.

L'Operador Ternari: El Bouncer del Club

```

String mensaje = (edad >= 18) ? "Pasa, joven" : "Vuelve cuando crezcas";

int nota = 7;
String resultado = nota >= 5 ? "Aprobado" : "Suspenso";

```

L'estructura és: condició ? valorSiTrue : valorSiFalse.

EL LÍO

El corrector automàtic de l'institut ha escopit este codi ple d'errors. Identifica i corregeix els 5 errors que té:

```

public class LioVariables {
    public static void main(String[] args) {
        int a = 10.5;
        double b = "Hola";
        String c = true;
        int d = a + c;
        System.out.println("Resultado: " + d)
    }
}

```

Pista: cada variable ha de tindre el tipus correcte per al seu valor.

👁️ **Don Tip:** Repassa els tipus primitius: int només admet enters, double admet decimals, String va amb cometes dobles.

No Hi Ha Preguntes Tontes!

? No Hi Ha Preguntes Tontes!

Q: Per què long porta L i float porta f al final?

A: Perquè si poses `long x = 3000000000`; sense la L, Java pensa que és un `int` i es queixa. El `float` necessita f perquè per defecte els decimals són `double`.

Q: Què passa si dividisc un `int` entre un altre `int` i esper decimals?

A: Java et donarà un enter. $5 / 2 = 2$, no 2.5. Per a decimals, almenys un ha de ser `double`: $5 / 2.0 = 2.5$ o `(double)5 / 2 = 2.5`.

Q: Quina és la diferència entre `=` i `==`? Sempre m'embolique.

A: `=` és *assignar*: “agafa este valor i fica'l en esta caixa”. `==` és *comparar*: “són iguals?”. Confondre'ls és l'error més clàssic. Es com confondre “posa la taula” amb “està posada la taula?”.

Q: I el `%` per a què servix en la vida real?

A: Per a saber si un nombre és parell (`numero % 2 == 0`), per a cicles, per a jocs. Sense ell no tindries hores en un rellotge ni res cíclic.

Q: Precedència, associativitat... he de memoritzar-ho tot?

A: No. La regla d'or: *quan tingues dubtes, posa parèntesis*. `resultado = (a + b) * (c - d)` és molt més llegible. Els parèntesis no dolen.

Més Exemples de Codi

```

public class CalculadoraBasica {
    public static void main(String[] args) {
        int a = 15;
        int b = 4;

        System.out.println("a + b = " + (a + b));
        System.out.println("a - b = " + (a - b));
        System.out.println("a * b = " + (a * b));
        System.out.println("a / b = " + (a / b) + " (divisi3n entera)");
        System.out.println("a / b real = " + ((double) a / b));
        System.out.println("a % b = " + (a % b) + " (resto)");
    }
}

```

```

public class JuegoDeDados {
    public static void main(String[] args) {
        int dado1 = (int) (Math.random() * 6) + 1;
        int dado2 = (int) (Math.random() * 6) + 1;
        int suma = dado1 + dado2;

        System.out.println("Dado 1: " + dado1);
        System.out.println("Dado 2: " + dado2);
        System.out.println("Suma: " + suma);

        boolean esPar = suma % 2 == 0;
        String mensaje = esPar ? "Suma par - ganas" : "Suma impar - pierdes";
        System.out.println(mensaje);
    }
}

```

Resum (el que importa de veritat)

- Les variables s3n caixes etiquetades en la mem3ria.
- 8 tipus primitius: byte < short < int < long < float < double < char < boolean.
- final 3s superglue: constant que no canvia.
- Casting impl3cit = ficar caixa menuda en gran. Expl3cit = a la inversa amb p3rdues.
- String no 3s primitiu, 3s una classe. Per3 es comporta.
- Math.random() per a jugar a la loteria.

- + - * / % són els bàsics. El % et dona el reste.
- ++ i -- són flexions. Pre (primer puja) vs Post (primer usa).
- && i || tenen curtcircuit: si la primera ja decidix, no miren la segona.
- El ternari ? : és un if-else en una línia.
- La precedència se soluciona amb parèntesis. Sempre.

Exercicis Proposats

1. **Caixes variades** Declara una variable de cada tipus primitiu, assigna-li un valor coherent i imprimeix el resultat.
2. **El casting assassí** Declara un double amb valor 9.99. Converteix-lo a int explícitament. Quin valor es perd?
3. **Nom al revés** Demana a l'usuari el seu nom i mostra: longitud, majúscules, primera i última lletra.
4. **Dau trucat** Genera 10 nombres aleatoris entre 1 i 6. Compta quants 6 han eixit.
5. **Constant del mal** Declara final double PRECIO_BASE = 100; i final double IVA = 0.21. Calcula el preu final i intenta modificar IVA després.
6. **Quant mesura?** Calcula l'àrea d'un cercle amb Math.PI. Radi = 7.5.
7. **Endevina el número** La màquina tria un número a l'atzar de l'1 al 100. L'usuari introdueix un número i el programa diu si és major, menor o igual.
8. **L'intercanvi** Declara int a = 5; int b = 10;. Intercanvia els seus valors usant una tercera variable temporal.
9. **Àrea i perímetre** Calcula l'àrea i el perímetre d'un rectangle amb base 7.5 i altura 3.2.
10. **Parell o senar?** Demana un número i determina si és parell o senar usant %.
11. **Any de traspàs** Demana un any. Determina si és de traspàs: divisible entre 4 i (no entre 100 O sí entre 400).
12. **Ternari en acció** Pregunta l'edat a l'usuari. Usa el ternari per a mostrar "Major d'edat" o "Menor d'edat".
13. **L'enigma del ++** Sense executar, determina el resultat de: int a = 2; int b = a++ * 3 + --a;
14. **Curtcircuit** Declara int x = 0;. Fes boolean test = (5 < 3) && (++x == 1);. Imprimeix x. S'incrementà?
15. **Conversió Celsius ↔ Fahrenheit** Demana graus Celsius. Convertix a Fahrenheit ($^{\circ}\text{F} = ^{\circ}\text{C} \times 9/5 + 32$).

16. **Els desbordats** Declara `int max = Integer.MAX_VALUE`; i suma-li 1. Què passa?

RAs treballats en esta unitat:

- **RA2** - Escriu i prova programes senzills
-



Sergi Garcia Barea — CC BY-SA 4.0

Unidad 3: Estructuras de Control y Excepciones

Objectius d'aprenentatge

- Utilitzar estructures de selecció: if/else if/else i switch
- Emprar bucles while, do-while i for
- Comprendre el flux amb break i continue
- Gestionar excepcions amb try-catch-finally
- Llançar excepcions pròpies amb throw

if, else i switch: L'Art de Decidir

El teu programa és com un robot maldestre però obedient. Sense estructures de selecció, faria TOT el que li dius, en ordre, sempre. Necessitem que PRENGA DECISIONS.

if: El Porter de la Discoteca

Imagina que if és un porter de discoteca. Avalua la teua cara (la condició) i decidix si passes o no.

```
if (tieneEdad >= 18) {  
    System.out.println("Puedes pasar, disfruta de la música!");  
}
```

Si la condició és true, entres. Si és false, et quedes al carrer.

if-else: El Porter amb Pla B

A voltes el porter té una alternativa: “No passes, però ves al bar del costat”.

```
if (tieneEdad >= 18) {  
    System.out.println("Adelante, el techno te espera");  
} else {  
    System.out.println("Lo siento, vuelve cuando crezcas");  
}
```

if-else if-else: El Bouncer amb Múltiples Llistes

```
int nota = 85;

if (nota >= 90) {
    System.out.println("Sobresaliente – eres el orgullo de la familia");
} else if (nota >= 70) {
    System.out.println("Notable – no está mal");
} else if (nota >= 50) {
    System.out.println("Aprobado – por los pelos");
} else {
    System.out.println("Suspenso – tus padres quieren hablar contigo");
}
```

⚠ **Advertència:** Error mortal: posar ; després de la condició. `if (edad >= 18);` ← el ; buit fa que el bloc de després s'execute SEMPRE.

★ BE THE CODE, MY FRIEND: El Detectiu de l'Edat

👁 **Don Tip:** Recorre les condicions de dalt a baix. La primera que siga true executa el seu bloc i es salta la resta.

El Detectiu de l'Edat

Què imprimix açò?

```
int edad = 17;
boolean conPadres = true;

if (edad >= 18) {
    System.out.println("Entrada libre");
} else if (conPadres) {
    System.out.println("Pasas con tus viejos");
} else {
    System.out.println("A casa, campeón");
}
```

Solució: `edad >= 18` → false. `conPadres` → true. Imprimix “Pasas con tus viejos”. Açò es diu “seguiment de codi” i és el teu superpoder.

? No Hi Ha Preguntes Tontes!

P: Puc tindre un `if` sense claus?

R: Tècnicament sí, però executarà **NOMÉS** la primera línia després del `if`. Sempre posa claus. Sempre.

P: Què passa si pose `=` en lloc de `==`?

R: En Java no compila perquè `if (edad = 18)` retorna un `int`, no un `boolean`. El compilador et salva!

P: L'ordre dels `else if` importa?

R: I tant! Java avalua de dalt a baix i es queda amb el **PRIMER** que complix. Avalua sempre de més específic a menys específic.

P: Puc posar cent `else if` seguits?

R: Pots, però si tens més de 3-4 condicions, planteja't usar `switch`.

L'Operador Ternari: El Ninja d'una Sola Línia

```
int edad = 20;  
String mensaje = (edad >= 18) ? "Mayor de edad" : "Menor de edad";
```

💡 **Consell:** El ternari niuat és com les nines russes: pareix xulo fins que has de depurar-lo a les 3 de la matinada.

switch: La Màquina Expenedora de Codi

`if-else` és un porter. `switch` és una màquina expenedora: fiques un número, obtens el teu producte.

```
int dia = 3;
String nombreDia;

switch (dia) {
    case 1:
        nombreDia = "Lunes – la resaca del finde";
        break;
    case 2:
        nombreDia = "Martes – todavía duele";
        break;
    case 3:
        nombreDia = "Miércoles – mitad de semana!";
        break;
    case 4:
        nombreDia = "Jueves – ya casi";
        break;
    case 5:
        nombreDia = "Viernes – se acerca la gloria";
        break;
    case 6:
        nombreDia = "Sábado – libertad!";
        break;
    case 7:
        nombreDia = "Domingo – la depresión pre-lunes";
        break;
    default:
        nombreDia = "Eso no es un día, inventado";
}
```

⚠ **Advertència:** Oblidar el break causa *fall-through*: el codi “es cau” al següent case. El 99% de les voltes és un error.

💡 **Consell:** Des de Java 14 pots usar switch com a expressió amb fletxetes ->. No necessita break:

```
String tipo = switch (dia) {
    case 1, 2, 3, 4, 5 -> "Laborable – a currar";
    case 6, 7 -> "Festivo – a dormir";
    default -> "No válido";
};
```

★ BE THE CODE, MY FRIEND: Fall-Through

👁️ **Don Tip:** Sense break, el codi 'es cau' al següent case. Segueix-lo sense saltar res fins que trobes un break.

Fall-Through

Què imprimix este codi?

```
int x = 2;
switch (x) {
    case 1:
        System.out.println("Uno");
    case 2:
        System.out.println("Dos");
    case 3:
        System.out.println("Tres");
        break;
    default:
        System.out.println("Otro");
}
```

Solució: Com no hi ha break en case 2, s'executa "Dos" i es cau a "Tres" (ahí el break frena). Mai arriba a default. Si $x = 1$, imprimix "Uno", "Dos" i "Tres".

✂ EL LÍO

L'assistent de programació ha mesclat les línies d'este programa. Ordena les línies perquè el programa funcione correctament i mostre si un número és positiu, negatiu o zero:

```
} else if (numero < 0) {
public class Clasificador {
System.out.println("El número és positiu");
System.out.println("El número és zero");
int numero = -7;
System.out.println("El número és negatiu");
if (numero > 0) {
} else {
public static void main(String[] args) {
}
```

Pista: no oblidés les claus d'obertura i tancament de la classe i el main.

💡 **Don Tip:** Primer estructura la classe i el main, després col·loca el if-else if-else dins del main.

Bucles: Com Fer Que l'Ordinador Es Repetisca

Els humans ens cansem de repetir coses. Els ordinadors, no. Els bucles són la forma de dir-li “fes açò 500 voltes” sense escriure-ho 500 voltes.

while: El Bucle “Ja Hem Arribat?”

Imagina un xiquet en un viatge en cotxe que pregunta “ja hem arribat?” una i altra volta mentre la condició no es complisca.

```
int kmRecorridos = 0;

while (kmRecorridos < 100) {
    System.out.println("¿Ya llegamos? Llevamos " + kmRecorridos + " km");
    kmRecorridos++;
}
System.out.println("¡Por fin hemos llegado!");
```

📄 **Nota:** while primer comprova la condició. Si és false des del principi, NO executa el bloc ni una volta.

⚠️ **Advertència: Bucle infinit:** oblidar actualitzar la variable de control és com tindre el xiquet al cotxe per a sempre.

```
int i = 1;
while (i <= 5) {
    System.out.println("¡Atrapado en el tiempo!");
    // Falta i++ → ESTO NUNCA TERMINA
}
```

do-while: “Almenys Intentat-ho”

Primer fa, després pregunta. Garantix que el bloc s'execute almenys una volta.

```
int opcion;
do {
    System.out.println("=== MENÚ ===");
    System.out.println("1. Comer");
    System.out.println("2. Dormir");
    System.out.println("3. Programar");
    System.out.println("0. Salir");
    opcion = sc.nextInt();
} while (opcion != 0);
```

 **Consell:** Usa do-while quan necessites que alguna cosa passe almenys una volta: menús, confirmacions, preguntes existencials.

for: “Ho Faré Exactament N Voltes”

El bucle for és l'alemany dels bucles: disciplinat, sap exactament quantes voltes repetirà.

```
for (int i = 1; i <= 5; i++) {
    System.out.println("Vuelta " + i + " de 5");
}
```

El for posa tres coses separades per ;:

1. **Inicialització:** `int i = 1` — “comença ací”
2. **Condicció:** `i <= 5` — “seguix mentre siga cert”
3. **Actualització:** `i++` — “com avances”

```
// Recorrer un array
int[] numeros = {10, 20, 30, 40, 50};

for (int i = 0; i < numeros.length; i++) {
    System.out.println("Elemento " + i + ": " + numeros[i]);
}
```

 **Advertència:** Els arrays comencen en 0. Si poses `i <= numeros.length`, explota amb `ArrayIndexOutOfBoundsException`. És el clàssic *off-by-one error*.

for-each: “Vull Vore Tots els Caramels”


```
String[] nombres = {"Ana", "Luis", "Eva"};

for (String nombre : nombres) {
    System.out.println("Hola " + nombre + ", te he puesto en la lista");
}
```

 **Nota:** El for-each és de NOMÉS LECTURA. No pots modificar l'array mentre el recorres.

★ BE THE CODE, MY FRIEND: El Puzzle dels Bucles Niuats

 **Don Tip:** El bucle extern controla les files, l'intern les columnes. Quantes iteracions fa cada u?

El Puzzle dels Bucles Niuats

Què imprimix açò?

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

Solució:

```
*
* *
* * *
```

Has dibuixat un triangle amb bucles! Eres bàsicament un artista digital.

break i continue: El Comandament a Distància dels Bucles

```
// break: "En cuanto vea un 5, me piro"
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        break; // ¡ZAS, fuera!
    }
    System.out.println(i);
}
// Salida: 1, 2, 3, 4

// continue: "Los pares no me molan, siguiente"
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) {
        continue; // "paso de este"
    }
    System.out.println(i);
}
// Salida: 1, 3, 5, 7, 9 (solo impares)
```

? No Hi Ha Preguntes Tontes!

P: I si m'oblido de posar i++ en un for?

R: Bucle infinit. En for és més difícil oblidar-ho perquè està en la tercera casella, però si ho borres... enhorabona, has creat el primer bucle sense fi.

P: Quan use while i quan for?

R: Si saps quantes voltes (contar, recórrer array), usa for. Si depens d'una condició (seguir demanant fins que l'usuari es canse), usa while.

P: El for-each és més lent?

R: En arrays i col·leccions com ArrayList, no. I sempre és més LLEGIBLE.

Taula de Supervivència: Quin Bucle Usar?

Bucle	Quan usar-lo
for	Saps el nombre exacte de voltes
while	No saps quantes, només quan parar

Bucle	Quan usar-lo
do-while	Necessites que passe almenys una volta
for-each	Vols vore tots els elements sense índexs

EL RING: if/else vs switch

Dos estructures de control s'enfronten al ring. Qui guanya?

if/else: «Jo soc el tot-terreny. Puc avaluar qualsevol condició: rangs, combinacions lògiques, objectes... No tinc límits!»

switch: «Sí, però per a comparar un mateix valor contra moltes opcions, soc més ràpid i més llegible. Mira'm: un sol switch (dia) i 7 case. Amb els teus if encadenats sembles una escala de caragol.»

if/else: «¿Ràpid? En rendiment modern és igual. I a més, què passa amb els rangs? Intenta fer case > 18: en switch. No pots.»

switch: «Per a això estan els if. Cada u a la seua. Jo per a menús, dies de la setmana, estats d'una màquina. Tu per a decisions complexes. Per què barallem?»

if/else: «Tens raó. Al final, ens necessitem mútuament.»

💡 **Don Tip:** Usa switch quan compares una variable contra valors fixos concrets. Usa if quan tingueres rangs, condicions booleanes compostes o lògica més complexa.

Excepcions: Quan el Teu Programa Ensopega

Els programes fallen. És un fet de la vida com la mort, els impostos i que el café es refrede. Les excepcions són la forma que té Java de gestionar les fallades amb dignitat.

Què és una Excepció?

Una excepció és un esdeveniment anòmal. Una cosa que no hauria de passar, però passa.

```
int resultado = 10 / 0;    // ArithmeticException
int[] array = {1, 2, 3};
int valor = array[5];     // ArrayIndexOutOfBoundsException
int num = sc.nextInt();   // InputMismatchException si escribes "hola"
```

 **Nota:** El missatge roig gegant que escup Java es diu *stack trace* i és la caixa negra del teu avió: diu exactament on i com es va estavellar tot.

La Jerarquia del Caos

Les excepcions es dividixen en **checked** (el compilador t'obliga a capturar-les, com `IOException`) i **unchecked** (`RuntimeException`, com `NullPointerException`). Els **Error** són coses greus (`OutOfMemoryError`) — no intentes capturar-los. És com tancar una presa amb un xiclet.

try-catch: Caure amb Xarxa

```
try {
    System.out.print("Dime un número: ");
    int numero = sc.nextInt();
    System.out.println("Tu número: " + numero);
} catch (InputMismatchException e) {
    System.out.println("¡Te dije un NÚMERO!");
}
```

Pots tindre diversos catch per a diferents desastres:

```
try {
    int[] numeros = new int[3];
    numeros[0] = 10;
    numeros[1] = 0;
    int division = numeros[0] / numeros[2];
    int valor = numeros[5];
} catch (ArithmeticException e) {
    System.out.println("Error matemático: " + e.getMessage());
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Te pasaste de índice: " + e.getMessage());
} catch (Exception e) {
    System.out.println("Algo raro pasó: " + e.getMessage());
}
```



Consell: Posat els catch més específics PRIMER. Si poses catch (Exception e) al principi, els altres mai s'executen.

finally: “No Importa Què, Renta’t les Dents”

El bloc finally s'executa SEMPRES, hi haja o no excepció.

```
Scanner sc = null;
try {
    sc = new Scanner(System.in);
    int num = sc.nextInt();
} catch (InputMismatchException e) {
    System.out.println("Error de entrada");
} finally {
    System.out.println("Cerrando recursos... (como un adulto responsable)");
    if (sc != null) sc.close();
}
```



Nota: finally s'executa fins i tot si hi ha un return dins de try. Des de Java 7, try-with-resources tanca automàticament.

throw: Llançar Excepcions a Propòsit

```
public static void validarEdad(int edad) {
    if (edad < 0) {
        throw new IllegalArgumentException(
            "Edad negativa? Te has colado");
    }
    if (edad > 150) {
        throw new IllegalArgumentException(
            edad + " años? O eres inmortal o me tomas el pelo");
    }
    System.out.println("Edad válida: " + edad);
}
```



BE THE CODE, MY FRIEND: El Detector de Problemes

💡 **Don Tip:** Quan salta una excepció, el flux salta directament al catch. El que va després en el try no s'executa.

Què imprimix este programa?

```
public class PruebaExcepciones {
    public static void main(String[] args) {
        try {
            System.out.println("1. Entrando en peligro");
            int[] datos = {10, 20};
            System.out.println("2. Array creado");
            System.out.println("3. " + datos[2]);
            System.out.println("4. Esto no se imprime");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("5. ¡Capturado! Índice fuera de rango");
        } finally {
            System.out.println("6. FINALLY: Siempre");
        }
        System.out.println("7. El programa sigue como si nada");
    }
}
```

Solució: 1 → 2 → datos[2] (índex 2 no existix en array de 2) → 5 → 6 → 7. La línia 4 mai s'imprimix. És com el cosí que promet vindre al sopar de Nadal.

? No Hi Ha Preguntes Tontes!

P: I si no pose try-catch? El programa explota?

R: Sí. Java escup un *stack trace* roig. En aplicacions de veritat és inacceptable. En els teus exercicis, passa més del que creus.

P: Quan cree la meua pròpia excepció?

R: Quan la que necessites no existix. Per exemple, SaldoInsuficienteException. Crea una classe que hereta de Exception o RuntimeException.

P: finally s'executa fins i tot si hi ha System.exit()?

R: No! És el botó nuclear. Ni finally pot evitar-ho.

Excepcions Pròpies

```
class SaldoInsuficienteException extends Exception {  
    public SaldoInsuficienteException(String mensaje) { super(mensaje); }  
}
```

Usa-la amb throw i throws com qualsevol excepció checked.

Exercicis Proposats

Selecció

1. **Eres major?** — Demana l'edat a l'usuari i digues-li si pot votar, conduir o hauria d'estar dormint la migdiada.
2. **La calculadora d'insults** — Demana dos números i un operador amb switch. Captura divisió per zero.
3. **El classificador de notes sarcàstic** — Demana una nota (0-100) i classifícala amb if.
4. **Dies del mes** — Demana un mes (1-12) i mostra els dies amb switch.
5. **Parell o impar amb ternari** — Demana un número i usa el ternari en una línia.
6. **El validador ruïnes** — Demana tres números i determina el major amb if niuats.

Bucles

1. **El comptador venjatiu** — while de l'1 al 100. Que es queixi cada 10 números.
2. **Sumatori amb trauma** — Demana números fins a 0. Suma i digues alguna cosa als negatius.
3. **Factorial** — Calcula el factorial amb for.
4. **Endevina el número** — El programa tria un de l'1 al 100. Usa do-while.
5. **El triangle constructor** — Dibuixa un triangle d'asteriscos amb bucles niuats.
6. **Fibonacci** — Mostra els primers N números de Fibonacci.

Excepcions

1. **Divisió segura** — Dividix dos números. Captura ArithmeticException i InputMismatchException.
2. **L'indexador imprudent** — Array de 5 enters. Demana índex. Captura ArrayIndexOutOfBoundsException.

3. **Conversió kamikaze** — Convertix cadena a enter. Captura `NumberFormatException`.
4. **try-with-resources** — Llig un fitxer que no existix. Captura `FileNotFoundException`.
5. **Excepció personalitzada** — Crea `EdatInvalidaException`. Llança-la si edat < 0 o > 150 .
6. **finally: la prova definitiva** — Amb return dins de try, demostra que finally s'executa abans.

Resum

Concepte	Quan usar-lo
if/else	Decisions amb 2-3 camins possibles
switch	Molts camins basats en un mateix valor
while	Repetir mentre es complisca una condició (0+ voltes)
do-while	Repetir almenys una volta, després comprovar
for	Repetir un nombre conegut de voltes
try/catch	Capturar i gestionar errors sense trencar el programa
finally	Codi que s'executa sempre, hi haja o no error

Bones pràctiques:

- Preferix switch sobre múltiples if-else encadenats
- Els bucles for són més llegibles que while quan saps el nombre d'iteracions
- Captura excepcions específiques, no Exception genèrica
- Usa try-with-resources per a recursos que cal tancar
- No uses excepcions per a control de flux normal

RAs treballats en esta unitat:

- **RA3** - Estructures de control



Unidad 4: Algorítmica I — Fundamentos

Objectius d'aprenentatge

- Entendre què és un algorisme i com dissenyar-lo
- Implementar cerca lineal i binària en Java
- Implementar ordenació bombolla i inserció
- Analitzar la complexitat algorísmica amb notació Big O
- Triar l'algorisme adequat segons el context

Què és un Algorisme? (L'Art de Donar Instruccions Precises)

Un **algorisme** és una seqüència de passos finita, ordenada i sense ambigüitats que resol un problema. Bàsicament, és una recepta de cuina, però per al teu ordinador.

Quan cuines una truita de creïlles, segueixes un algorisme mental:

1. Pelar les creïlles
2. Tallar-les en rodanxes fines
3. Fregir-les en oli abundant
4. Batir els ous
5. Barrejar-ho tot i quallar

Si un pas és ambigu (“posa sal al gust”), cada persona ho interpreta diferent. Un algorisme de veritat no deixa espai a la interpretació: cada pas ha de ser **precís** i **determinista**.

Nota:

La paraula “algorisme” ve del matemàtic persa **Al-Juarismi** (segle IX). Va escriure un llibre sobre com fer càlculs amb nombres indis. Segles després, els informàtics li vam robar la paraula.

Algorismes en Programació

En informàtica, els algorismes més clàssics es dividixen en dos grans famílies:

- **Cerca:** trobar un element dins d'un conjunt de dades.
- **Ordenació:** posar un conjunt de dades en un ordre determinat (numèric, alfabètic, etc.).

I d'això va esta unitat: de buscar i ordenar com un professional.

```
// El algoritmo más simple que existe: sumar dos números
public class AlgoritmoSimple {
    public static void main(String[] args) {
        int a = 5;
        int b = 3;
        int resultado = a + b; // Esto ES un algoritmo: pasos precisos que producen un
resultado
        System.out.println("5 + 3 = " + resultado);
    }
}
```

⚠ Advertència:

No tot codi és un algorisme. Un algorisme és la *idea*. El codi és la *materialització*. Pots implementar el mateix algorisme en Java, Python o ensamblador. L'essència és la mateixa.

Propietats d'un Algorisme

Perquè un algorisme siga considerat com a tal, ha de complir:

1. **Finit**: ha de terminar en algun moment. Si s'executa per sempre, no és un algorisme, és un malson.
2. **Precís**: cada pas ha d'estar definit sense ambigüitat.
3. **Entrada**: ha de rebre zero o més valors d'entrada.
4. **Eixida**: ha de produir almenys un valor d'eixida.
5. **Eficaç**: ha de resoldre el problema en temps finit.

Cerca Lineal: El Cercador de Sabates Perdudes

Imagina que perds una sabatilla en la teua habitació. Què fas? Mires davall del llit, darrere de la porta, en l'armari... bàsicament **revises cada lloc fins a trobar-la**.

Doncs això és la **cerca lineal**: recorres un array element per element fins a trobar el que busques. És simple, directa, i funciona inclús si les dades estan desordenades.

```

public class BusquedaLineal {

    public static int buscar(int[] array, int objetivo) {
        for (int i = 0; i < array.length; i++) {
            if (array[i] == objetivo) {
                return i; // ¡Encontrado! Devuelve la posición
            }
        }
        return -1; // No está en el array
    }

    public static void main(String[] args) {
        int[] numeros = {34, 12, 56, 78, 23, 9, 45, 67};

        int resultado = buscar(numeros, 23);
        if (resultado != -1) {
            System.out.println("¡Encontrado el 23 en la posición " + resultado + "!");
        } else {
            System.out.println("El 23 no está en el array.");
        }

        resultado = buscar(numeros, 99);
        if (resultado == -1) {
            System.out.println("El 99 no está. Como unas zapatillas que nunca
aparecen.");
        }
    }
}

```

Anàlisi: Com de ràpid és?

En el **millor cas**, l'element està en la primera posició → trobes en 1 pas.

En el **pitjor cas**, l'element està al final o no existix → recorres els n elements sencers.

Diem que la seua **complexitat és $O(n)$** — lineal. Si l'array té 10 elements, tardes ~10 passos; si en té 10.000, tardes ~10.000 passos. Creix al mateix ritme que les dades.

💡 Consell:

La cerca lineal és com buscar en la teua nevera: si és xicoteta, tant li fa el mètode. Però si tens un magatzem de 10.000 ítems, necessites alguna cosa millor.

Cerca Binària: El Cercador Jedi

La **cerca binària** és com buscar una paraula en el diccionari. No obris per la pàgina 1 i passes d'una en una. Obris per la meitat, veus si la paraula està abans o després, i descartes mitja tona de paper en cada intent.

Requisit imprescindible: l'array ha d'estar ordenat. Si no, este mètode no funciona.

```
public class BusquedaBinaria {

    public static int buscar(int[] array, int objetivo) {
        int izquierda = 0;
        int derecha = array.length - 1;

        while (izquierda <= derecha) {
            int medio = izquierda + (derecha - izquierda) / 2; // Mitad del segmento

            if (array[medio] == objetivo) {
                return medio; // ¡Bingo!
            }

            if (array[medio] < objetivo) {
                izquierda = medio + 1; // Descartamos la mitad izquierda
            } else {
                derecha = medio - 1; // Descartamos la mitad derecha
            }
        }
        return -1; // No encontrado
    }

    public static void main(String[] args) {
        // OJO: tiene que estar ORDENADO
        int[] numeros = {2, 5, 8, 12, 19, 24, 31, 37, 42, 50, 58, 63};

        int resultado = buscar(numeros, 31);
        System.out.println("31 encontrado en posición: " + resultado); // 6

        resultado = buscar(numeros, 3);
        System.out.println("3 encontrado en: " + resultado); // -1
    }
}
```

Per què $\text{izquierda} + (\text{derecha} - \text{izquierda}) / 2$ en lloc de $(\text{izquierda} + \text{derecha}) / 2$?

Perquè si l'array és molt gran (prop de `Integer.MAX_VALUE` elements), `izquierda + derecha` pot desbordar-se. La fórmula alternativa evita eixe problema. És un clàssic bug que va aparèixer inclús en la biblioteca de Java original.

⚠ Advertència:

Si executes cerca binària sobre un array **no ordenat**, els resultats són brossa. No hi ha error de compilació, no hi ha excepció. Simplement obtens la resposta equivocada. Com buscar “berenar” en un diccionari que té les paraules a l’atzar.

Anàlisi: $O(\log n)$

En cada pas, la cerca binària **descartada la meitat** de l'array restant.

- Array de 16 elements → 4 passos màxims
- Array de 32 elements → 5 passos
- Array de 1024 elements → 10 passos
- Array de 1.000.000 elements → 20 passos

Això és **$O(\log n)$** , o complexitat logarítmica. Creix molt a poc a poc inclús amb dades enormes. És la diferència entre preguntar a 1000 persones una per una vs. preguntar “està a l’esquerra o a la dreta?” i descartar-ne 500 de colp.

```
public class DemoComplejidad {
    public static void main(String[] args) {
        // Simulación visual de pasos necesarios
        System.out.println("n\tLineal\tBinaria");
        System.out.println("-----");
        for (int n = 10; n <= 1000000; n *= 10) {
            int pasosLineal = n;
            int pasosBinaria = (int)(Math.log(n) / Math.log(2));
            System.out.println(n + "\t" + pasosLineal + "\t\t" + pasosBinaria);
        }
    }
}
```

Eixida esperada:

n	Lineal	Binària
10	10	4
100	100	7
1000	1000	10
10000	10000	14
100000	100000	17
1000000	1000000	20

Amb un milió d'elements, la cerca lineal necessita un milió de passos. La binària només **20**. Llig això una altra volta. **20**.

Nota:

$\log_2(n)$ en informàtica s'assumix sempre en base 2. No confongues amb el logaritme decimal de tota la vida. Quan un informàtic diu “log n”, pensa en “meitat, meitat, meitat...”.

Ordenació Bombolla: Simple però Torta

L'**ordenació bombolla** (bubble sort) és l'algorisme d'ordenació més senzill d'entendre... i el més lent dels que valen la pena. Funciona així:

Va recorrent l'array. Si l'element actual és major que el següent, intercanvia'ls. Repetix fins que no hi haja intercanvis.

Els elements grans “pugen com bombolles” cap al final de l'array. D'ací el nom.

```

public class Burbuja {

    public static void ordenar(int[] array) {
        int n = array.length;
        boolean huboIntercambio;

        for (int i = 0; i < n - 1; i++) {
            huboIntercambio = false;

            for (int j = 0; j < n - 1 - i; j++) {
                if (array[j] > array[j + 1]) {
                    // Intercambio
                    int temp = array[j];
                    array[j] = array[j + 1];
                    array[j + 1] = temp;
                    huboIntercambio = true;
                }
            }

            // Si no hubo intercambio, el array ya está ordenado
            if (!huboIntercambio) break;
        }
    }

    public static void main(String[] args) {
        int[] datos = {64, 34, 25, 12, 22, 11, 90};

        System.out.print("Antes: ");
        for (int n : datos) System.out.print(n + " ");

        ordenar(datos);

        System.out.print("\nDespués: ");
        for (int n : datos) System.out.print(n + " ");
        // 11 12 22 25 34 64 90
    }
}

```

Per què és tan lent?

Dos bucles aniuats. Per a un array de n elements:

- Primer bucle: n voltes
- Segon bucle: $\sim n$ voltes (en realitat $n-i-1$, però a grans trets n)

Total: $\sim n \times n = n^2$ operacions. Complexitat $O(n^2)$.

Per a 10 elements $\rightarrow$ 100 operacions (bé). Per a 1000 elements $\rightarrow$ 1.000.000 d'operacions (comença a doldre). Per a 1.000.000 d'elements $\rightarrow$ 1.000.000.000.000 operacions (el teu ordinador demana la jubilació).

Consell:

Bombolla només s'usa en dos situacions: (1) estàs aprenent, (2) saps que l'array tindrà < 50 elements. Per a tot lo demás, hi ha millors alternatives.

L'Optimització del Flag

Fixat't en la variable `huboIntercambio`. Si en una passada completa no intercanviem res, és que l'array ja està ordenat i podem parar. Esta optimització no millora el pitjor cas (array invertit), però ajuda en arrays quasi ordenats.

Ordenació per Inserció: Com Ordenar Cartes en la Mà

L'**ordenació per inserció** (insertion sort) és com ordenar les cartes que et repartixen al pòker. Agafes una carta nova i la col·loques en el seu lloc dins de les que ja tens ordenades en la mà.


```

public class Insercion {

    public static void ordenar(int[] array) {
        for (int i = 1; i < array.length; i++) {
            int clave = array[i]; // La carta que vamos a colocar
            int j = i - 1;

            // Desplazar elementos mayores hacia la derecha
            while (j >= 0 && array[j] > clave) {
                array[j + 1] = array[j];
                j--;
            }
            array[j + 1] = clave; // Colocar la carta en su sitio
        }
    }

    public static void main(String[] args) {
        int[] datos = {9, 5, 1, 4, 3};

        System.out.print("Antes: ");
        for (int n : datos) System.out.print(n + " ");

        ordenar(datos);

        System.out.print("\nDespués: ");
        for (int n : datos) System.out.print(n + " ");
        // 1 3 4 5 9
    }
}

```

Com funciona pas a pas?

Donat {9, 5, 1, 4, 3}:

```

Paso 0: [9] | 5 1 4 3 → la "mano" empieza con el 9
Paso 1: [5 9] | 1 4 3 → 5 se coloca a la izquierda del 9
Paso 2: [1 5 9] | 4 3 → 1 se coloca al principio
Paso 3: [1 4 5 9] | 3 → 4 se cuela entre el 1 y el 5
Paso 4: [1 3 4 5 9] → 3 se coloca entre el 1 y el 4

```

Anàlisi i Quan és bona?

També és $O(n^2)$ en el pitjor cas (array invertit). Però:

- **Millor cas (array quasi ordenat):** $O(n)$. Només fa una passada. És rapidíssim.
- És **estable**: manté l'ordre relatiu d'elements iguals.
- **No necessita memòria extra** (ordena in-place).
- És **més ràpid que bombolla** en la pràctica, encara que tots dos siguin $O(n^2)$.

💡 Consell:

L'ordenació per inserció és la reina de les dades **quasi ordenades**. Si saps que el teu array té 100 elements i ja està “quasi bé” (només uns pocs fora de lloc), inserció et va a sorprendre. S'usa com a pas final en algorismes avançats (TimSort, usat per Java i Python).

```
public class Comparacion {
    public static void main(String[] args) {
        // Demostración: inserción vs burbuja con array casi ordenado
        int[] casiOrdenado = {1, 2, 3, 4, 6, 5, 7, 8, 9, 10};
        // Solo el 6 y el 5 están intercambiados

        int[] burbuja = casiOrdenado.clone();
        int[] insercion = casiOrdenado.clone();

        // Con burbuja da varias pasadas aunque casi esté ordenado
        // Con inserción resuelve en un suspiro
        System.out.println("Inserción es ideal cuando tienes que añadir");
        System.out.println("un elemento a un array ya ordenado.");
    }
}
```

Complexitat Algorísmica: Big O per a Mortals

La notació **Big O** descriu com creix el temps d'execució d'un algorisme quan creix la quantitat de dades d'entrada. No et dona el temps exacte en segons (això depèn del maquinari), et dona la tendència.

Les Complexitats Més Comunes

Notació	Nom	Exemple	Per a n=1000
$O(1)$	Constant	Accedir a un array per índex	1 operació
$O(\log n)$	Logarítmica	Cerca binària	~10 ops
$O(n)$	Lineal	Cerca lineal	1000 ops
$O(n \log n)$	Quasi lineal	Ordenació ràpida (ho veuràs més avant)	~10.000 ops
$O(n^2)$	Quadràtica	Bombolla, inserció	1.000.000 ops
$O(2^n)$	Exponencial	Fibonacci recursiu sense optimitzar	¡inviabile!

```

public class EjemplosComplejidad {

    // O(1) – CONSTANTE: siempre igual, no importa el tamaño
    public static int obtenerPrimero(int[] array) {
        return array[0]; // Un solo paso, siempre
    }

    // O(n) – LINEAL: crece proporcional a n
    public static int sumar(int[] array) {
        int suma = 0;
        for (int num : array) {
            suma += num; // n pasos
        }
        return suma;
    }

    // O(n²) – CUADRÁTICA: dos bucles anidados
    public static void imprimirPares(int[] array) {
        for (int i = 0; i < array.length; i++) {
            for (int j = 0; j < array.length; j++) {
                System.out.println(array[i] + ", " + array[j]); // n × n pasos
            }
        }
    }

    public static void main(String[] args) {
        int[] datos = {10, 20, 30, 40, 50};

        System.out.println("O(1): " + obtenerPrimero(datos));
        System.out.println("O(n): " + sumar(datos));
        System.out.println("O(n²): mira la consola llenándose de pares...");
        imprimirPares(datos);
    }
}

```

Regles Pràctiques de Big O

1. **Ignora constants:** $O(2n)$ és el mateix que $O(n)$. El 2 no importa quan n tendix a infinit.
2. **Queda't amb el terme dominant:** $O(n^2 + n) \rightarrow O(n^2)$. El n^2 es menja el n quan n creix.
3. **Els bucles anuats multipliquen:** un bucle dins d'un altre $\rightarrow n \times n \rightarrow O(n^2)$.

4. Els bucles seqüencials sumen: un bucle i després un altre $\rightarrow O(n + n) \rightarrow O(2n) \rightarrow O(n)$.

 Nota:

“Big O” descriu el **pitjor cas** (o cota superior). Si dius que cerca lineal és $O(n)$, estàs dient “com a molt, tardarà el mateix que recórrer tot l’array”. Existixen també Big Omega Ω (millor cas) i Big Theta Θ (cas mitjà), però amb Big O en tens suficient per a començar.

```
public class ReglasBigO {
    public static void main(String[] args) {
        // REGLA 1: Las constantes no importan
        //  $O(2n) \rightarrow O(n)$ 
        //  $O(100n) \rightarrow O(n)$ 

        // REGLA 2: El término dominante se queda
        //  $O(n^2 + 5n + 1) \rightarrow O(n^2)$ 
        //  $O(n + \log n) \rightarrow O(n)$ 
        //  $O(n! + n^2) \rightarrow O(n!)$ 

        // REGLA 3: Bucles anidados  $\rightarrow$  multiplica
        // for(i...) { for(j...) }  $\rightarrow O(n * m) \rightarrow O(n^2)$ 

        // REGLA 4: Bucles secuenciales  $\rightarrow$  suma
        // for(i...) { } for(i...) { }
        //  $O(n + n) \rightarrow O(2n) \rightarrow O(n)$ 

        System.out.println("Big O no es magia, es simplificar.");
        System.out.println("Pregúntate: ¿qué pasa cuando n se hace MUY grande?");
    }
}
```

★ BE THE CODE, MY FRIEND: Implementa Cerca Binària des de Zero

👁️ **Don Tip:** La cerca binària reduïx el problema a la meitat en cada pas. Centra’t en els límits izquierda i derecha — l’error més comú és no actualitzar-los bé.

Este exercici és un clàssic en entrevistes tècniques de Google, Amazon i companyia. Si algun dia vols treballar en una FAANG, la cerca binària la tens que saber escriure dormit, borratxo i amb una mà lligada a l’esquena.

Instruccions:

Sense mirar el codi de dalt, implementa el mètode `busquedaBinaria` que rep un array ordenat d'enters i un objectiu, i torna la posició de l'objectiu o -1 si no existix.

```
/**
 * Implementa este método:
 *
 * public static int busquedaBinaria(int[] array, int objetivo)
 *
 * Requisitos:
 * - No uses Arrays.binarySearch() – el sentido es programarlo
 * - No mires el código de la unidad – fuerza tu cerebro a recordar
 * - Piénsalo así: coge el medio, compara, descarta la mitad, repite
 */
public class RetoBinaria {

    public static int busquedaBinaria(int[] array, int objetivo) {
        // 🧠 TU CÓDIGO AQUÍ
        return -1;
    }

    public static void main(String[] args) {
        int[] pruebas = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19};

        System.out.println("Probando búsqueda binaria...");
        System.out.println("El 7 está en posición: " + busquedaBinaria(pruebas, 7));
// 3
        System.out.println("El 13 está en posición: " + busquedaBinaria(pruebas, 13));
// 6
        System.out.println("El 8 está en posición: " + busquedaBinaria(pruebas, 8));
// -1
        System.out.println("El 19 está en posición: " + busquedaBinaria(pruebas, 19));
// 9
        System.out.println("El 1 está en posición: " + busquedaBinaria(pruebas, 1));
// 0
        System.out.println("Primer elemento fuera: " + busquedaBinaria(new int[]{},
5)); // -1
    }
}
```

Pistes (si t'encalles):

- Necessites dos punters: izquierda i derecha.
- Calcula el medio com $\text{izquierda} + (\text{derecha} - \text{izquierda}) / 2$.
- Si `array[medio] == objetivo`, torna medio.
- Si `array[medio] < objetivo`, mou izquierda a `medio + 1`.
- Si `array[medio] > objetivo`, mou derecha a `medio - 1`.
- El bucle termina quan `izquierda > derecha`.
- Array buit → directament -1.

💡 Consell:

L'error més comú en cerca binària (inclús en programadors amb 10 anys d'experiència) és el **off-by-one**: `<= 0 <? medio + 1 o medio?` Dibuixa l'array en un paper amb 3 elements i simula els casos. El paper i boli són les teues millors ferramentes de debugging.

Solució (intenta-ho abans de mirar)

```
public static int busquedaBinaria(int[] array, int objetivo) {
    if (array == null || array.length == 0) return -1;

    int izquierda = 0;
    int derecha = array.length - 1;

    while (izquierda <= derecha) {
        int medio = izquierda + (derecha - izquierda) / 2;

        if (array[medio] == objetivo) {
            return medio;
        } else if (array[medio] < objetivo) {
            izquierda = medio + 1;
        } else {
            derecha = medio - 1;
        }
    }
    return -1;
}
```

El departament de qualitat ha rebut este algoritme d'ordenació. Alguna cosa fa olor de cremat. Identifica els errors i explica per què no funciona:

```
public class BurbujaLlosa {  
    public static void ordenar(int[] arr) {  
        for (int i = 0; i < arr.length; i++) {  
            for (int j = 0; j < arr.length; j++) {  
                if (arr[j] > arr[j + 1]) {  
                    int temp = arr[j];  
                    arr[j] = arr[j + 1];  
                    arr[j + 1] = temp;  
                }  
            }  
        }  
    }  
}
```

Pista: hi ha un error d'índexs i un altre de rendiment.

💡 **Don Tip:** Quan un bucle accedix a `arr[j + 1]`, assegura't que `j + 1` no isca de l'array. També pensa: recorres menys elements en cada passada?

No Hi Ha Preguntes Tontes!

Q: Per què no usem sempre cerca binària si és tan ràpida?

A: Perquè requereix que les dades estiguen **ordenades**. Ordenar també costa temps. Si has d'ordenar cada volta que busques, perds l'avantatge. La cerca binària és ideal quan ordenes una vegada i busques moltes voltes.

Q: Big O servix per a alguna cosa en el món real o és només teoria d'examen?

A: Servix i molt. Quan la teua app web comença a anar lenta amb 1000 usuaris, la diferència entre $O(n)$ i $O(n^2)$ és la diferència entre “prendre un café mentre carrega” i “jubilar-te abans que acabe”. Les empreses tech (Google, Amazon, Netflix) maten per eficiència. Un algorisme roïn en producció costa diners reals en servidors.

Q: Hi ha algorismes d'ordenació més ràpids que $O(n^2)$?

A: Sí, i els veuràs en la següent unitat. QuickSort, MergeSort i HeapSort són de l'ordre **$O(n \log n)$** , que és molt més ràpid que $O(n^2)$. Per exemple, per a 1.000.000 d'elements, $O(n \log n)$ són

~20 milions d'operacions; $O(n^2)$ són 1 bilió.

Q: Puc barrejar tipus de cerca i ordenació?

A: Clar. És molt comú ordenar amb QuickSort i després buscar amb binària. O si l'array és molt xicotet (< 50), usar cerca lineal encara que estiga ordenat, perquè la sobrecàrrega de la binària no compensa.

Q: Val, però... quan use cada algorisme d'ordenació?

A: Regla pràctica:

- Array xicotet (< 50 elements) → qualseval val, usa inserció per simplicitat.
- Array quasi ordenat → **inserció** arrasa.
- Array gran → no uses bombolla ni inserció. Espera a la pròxima unitat.
- Array enorme amb memòria limitada → **HeapSort**.
- Array enorme i necessites estabilitat → **MergeSort**.
- Array enorme i velocitat bruta → **QuickSort**.

Q: Què significa que un algorisme siga “estable”?

A: Un algorisme d'ordenació és estable si manté l'ordre relatiu d'elements amb el mateix valor. Per exemple, si tens dos usuaris amb edat 25 (“Ana” abans que “Luis”), un algorisme estable els deixa en eixe ordre. Inserció i MergeSort són estables; Bombolla es pot fer estable o no segons la implementació; QuickSort no ho és per defecte.

Q: I tot això servix per a aprovar el mòdul?

A: Servix per a aprovar el mòdul, per a les entrevistes tècniques, per a escriure codi que no done pena, i perquè quan algú diga “és $O(n^2)$ ” tu sàpques exactament per què és roïn. Així que sí, posa atenció.

EL ACERTIJO

Tens 9 monedes aparentment idèntiques. Una d'elles és falsa i pesa menys que les altres. Disposes d'una balança de dos platets. Quin és el nombre mínim de pesades que necessites per a trobar la moneda falsa?

Pista: no les peses una per una. Usa divideix i venceràs.

👁️ **Don Tip:** Si dividixes 9 monedes en 3 grups de 3, amb una sola pesada pots descartar 6 monedes.

Resum de la Unitat

Algorisme	Complexitat	Quan usar-lo?
Cerca lineal	$O(n)$	Arrays menuts o desordenats
Cerca binària	$O(\log n)$	Arrays grans ORDENATS
Bombolla	$O(n^2)$	Només per a aprendre o arrays molt menuts
Inserció	$O(n^2) / O(n)$	Arrays menuts o quasi ordenats

Conceptes clau:

- Un algorisme és una seqüència finita, precisa i no ambigua de passos.
- La notació Big O descriu la taxa de creixement del temps d'execució.
- $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$
- Ordenar abans de cercar pot valdre la pena si fas moltes cerques.
- El millor algorisme depén del context: grandària, ordre inicial, requisits de memòria i estabilitat.

⚠️ Advertència:

Això és només el principi. La següent unitat puja el nivell: QuickSort, MergeSort, cerca amb hash, i complexitat en casos reals. Però si domines estos fonaments, lo demás és costa avall. Algorísmica és com anar en bici: al principi pareix impossible, després no se t'oblida mai.



Sergi Garcia Barea — CC BY-SA 4.0

Unidad 5: Algorítmica II — Técnicas Avanzadas

Benvingut, valent explorador de la pila de llamades. Fins ara has sobreviscut a arrays, bucles, objectes i condicions. Has dominat l'art de fer que l'ordinador faça el que li dius. Però el bosc s'enfosqueix. Ha arribat l'hora d'enfrontar-te als algorismes que **canvien la teua forma de pensar** (i de pas, et fan quedar genial en les entrevistes tècniques de Google, que mai se sap).

En esta unitat deixarem arrere els bucles obvis i ens endinsem en la **recursivitat** i **dividix i venceràs**. No tingues por. Respira fondo. I recorda: per a entendre la recursivitat, primer has d'entendre la recursivitat.

Anem allà.

1. Recursivitat: una funció que es crida a si mateixa (sense tornar-se boja)

La recursivitat és quan una funció s'invoca a si mateixa. Sí, sona a bucle infinit i a mal de cap. Però amb una **condició de parada** ben posada, és una de les ferramentes més elegants i potents que existixen en programació.

Pensa-ho així: en lloc de resoldre el problema sencer de colp, resols una xicoteta part i li passes la resta a... tu mateix. Com quan neteges la teua habitació: agarras una cosa del sòl i després tornes a cridar a la mateixa funció "limpiarHabitacion" amb el que queda. Eventualment no queda res, i has acabat.

Les dos cares de la moneda recursiva

[!NOTE] Tota funció recursiva necessita **dos parts** imprescindibles. Si et falta una, estàs mort:

- **Cas base:** la condició que deté la recursió. Sense això, el teu programa s'executa fins que la JVM es cansa i et llança un `StackOverflowError`.
- **Cas recursiu:** la crida a si mateixa, normalment amb una versió més xicoteta del mateix problema.

L'estructura general és sempre la mateixa:

```

public static tipo funcion(parametros) {
    if (/* condicion de parada */) {
        return /* valor base */;
    } else {
        // hacer algo
        return funcion(/* version mas pequena */);
    }
}

```

La pila de llamadas (call stack) — l'origen de tota la diversió

Cada volta que una funció es crida, Java reserva un trocet de memòria en el **stack** (la pila). Eixe trocet es diu **stack frame** i guarda: els paràmetres de la funció, les variables locals, i la direcció de tornada per a quan la funció acabe.

Si crides a una funció 5 voltes, tens 5 frames en el stack. Si la crides 10.000 voltes... bum. `StackOverflowError`.

```

public class StackExplorer {

    static int prof = 0;

    static void recursivo() {
        prof++;
        System.out.println("Llamada numero: " + prof);
        recursivo(); // ¡no hay caso base!
    }

    public static void main(String[] args) {
        try {
            recursivo();
        } catch (StackOverflowError e) {
            System.out.println("Stack explotó en la llamada: " + prof);
        }
    }
}

```

Prova-ho. Veuràs que el número varia segons la teua màquina i la JVM. Normalment entre 10.000 i 20.000 crides. No és infinit. Res ho és.

[!WARNING] Sense cas base, no hi ha pietat. El stack té un límit i Java no et va a salvar. Cada frame ocupa espai, i quan el got es desborda, la JVM diu “fins aquí hem arribat”.

Recursivitat vs iteració: el duel

Aspecte	Recursivitat	Iteració (bucles)
Llegibilitat	Molt elegant per a problemes jeràrquics	Més verbosa però clara
Memòria	Gasta stack (cada crida = frame nou)	Només variables locals
Velocitat	Més lenta (overhead de crides)	Més ràpida
Stack overflow	Risc real si profunditat és alta	No aplica
Casos ideals	Arbres, grafs, backtracking, dividix i venceràs	Recorreguts lineals, processament simple

La regla d'or: **usa recursivitat quan el problema siga inherentment recursiu** (arbres, expressions anidades, algorismes d'ordenació avançats). Per a la resta, un for de tota la vida.

2. Exemples clàssics (els clàssics per alguna cosa ho són)

Anem a vore tres exemples que et perseguiran la resta de la teua carrera. Acostuma-t'hi. Apareixen en exàmens, entrevistes, i conversacions d'ascensor amb altres programadors.

2.1. Factorial — el “Hello World” de la recursivitat

```

public class Factorial {

    static int fact(int n) {
        if (n <= 1) return 1;           // caso base
        return n * fact(n - 1);        // caso recursivo
    }

    public static void main(String[] args) {
        System.out.println("fact(5) = " + fact(5)); // 120
        System.out.println("fact(0) = " + fact(0)); // 1
        System.out.println("fact(7) = " + fact(7)); // 5040
    }
}

```

Què passa quan cridem a fact(4)? Pas a pas:

```

fact(4) → 4 * fact(3)
        → 4 * (3 * fact(2))
        → 4 * (3 * (2 * fact(1)))
        → 4 * (3 * (2 * 1))
        → 4 * (3 * 2)
        → 4 * 6
        → 24

```

Primer “baixa” fins al cas base, després “puja” fent les multiplicacions. Com un ascensor que baixa al soterrani i torna a pujar obrint portes.

2.2. Fibonacci (versió ingènua — no faces això a casa)

```
public class FiboNaive {

    static long fibo(int n) {
        if (n <= 1) return n;
        return fibo(n - 1) + fibo(n - 2);
    }

    public static void main(String[] args) {
        long inicio = System.currentTimeMillis();
        System.out.println("fibo(40) = " + fibo(40));
        long fin = System.currentTimeMillis();
        System.out.println("Tiempo: " + (fin - inicio) + " ms");
    }
}
```

Executa això. Seran uns segons d'infern. El problema? `fibo(40)` genera un **arbre de crides monstruós**. Cada crida es bifurca en dos, que es bifurquen en dos, etc. El número total de crides és aproximadament $O(2^n)$.

```

          fibo(5)
        /      \
      fibo(4)   fibo(3)
     /  \    /  \
  fibo(3) fibo(2) fibo(2) fibo(1)
  /  \  /  \  /  \
fibo(2) fibo(1) ... ..
 /    \
fibo(1) fibo(0)
```

Compta les voltes que es crida a `fib(1)`. Exacte, massa. És com preguntar-li al teu company una i altra volta la mateixa pregunta esperant una resposta diferent. Això, senyor meu, és bogeria.

[!TIP] Este és l'exemple perfecte per a entendre per què **l'eficiència importa**. Un algorisme que es veu bonic en codi pot ser un desastre en temps d'execució. Fibonacci naive és el rei dels algorismes bonics però inútils per a números grans.

2.3. Fibonacci optimitzat (amb memoització — ara sí)

```

public class FiboOptimizado {

    static long[] memo;

    static long fibo(int n) {
        if (n <= 1) return n;
        if (memo[n] != 0) return memo[n];          // ya lo calculamos antes
        memo[n] = fibo(n - 1) + fibo(n - 2);
        return memo[n];
    }

    public static void main(String[] args) {
        int n = 92;
        memo = new long[n + 1];
        long inicio = System.currentTimeMillis();
        System.out.println("fibo(" + n + ") = " + fibo(n)); // instantáneo
        long fin = System.currentTimeMillis();
        System.out.println("Tiempo: " + (fin - inicio) + " ms");
    }
}

```

Què canvia? Guardem els resultats en un array (`memo[]`) per a no recalcular. Si ja hem calculat `fib(10)`, el tornem directament. Cada fibonacci es calcula **una sola volta**.

La millora és brutal: de **$O(2^n)$ a $O(n)$** . Eixa és la major pujada de nivell des que vas passar de pedra a pokéball.

[!NOTE] Això es diu **memoització** (sí, sense la “r”). És la base de la programació dinàmica, que veuràs en unitats més avançades. Consistix en: “si ja ho vaig calcular, no ho torne a calcular, ho guarde en un mapa o array i ho reutilice”.

2.4. Suma d'un array


```

public class SumArray {

    static int sumar(int[] arr, int i) {
        if (i == arr.length) return 0;           // caso base: no hay más elementos
        return arr[i] + sumar(arr, i + 1);       // caso recursivo
    }

    public static void main(String[] args) {
        int[] nums = {3, 8, 2, 10, 5};
        System.out.println("Suma: " + sumar(nums, 0)); // 28

        int[] vacio = {};
        System.out.println("Suma vacia: " + sumar(vacio, 0)); // 0
    }
}

```

Este exemple és quasi idèntic al factorial, però recorrent un array. L'índex `i` avança fins a arribar a `arr.length`. En eixe moment, tornem 0 i comencem a sumar cap amunt.

[!TIP] Pensat en la recursivitat com si pelares una ceba. Llevat una capa (un element), i el que queda és el mateix problema però més xicotet. Eventualment no queda ceba.

3. Dividix i venceràs (Divide & Conquer)

L'estratègia és tan antiga com Juli Cèsar, però aplicada a algorismes continua sent igual d'efectiva. El principi és senzill:

1. **Dividir** el problema en subproblemes més xicotets i manejables.
2. **Conquistar** cada subproblema recursivament (crides recursives).
3. **Combinar** les solucions dels subproblemes per a obtindre la solució del problema original.

```

Input gran
  |
  ├──Dividir──> Subproblema A    -> Conquistar (recursiu) -> Combinar -> Output
  └──Dividir──> Subproblema B    -> Conquistar (recursiu) ─┘

```

Els dos reis indiscutibles d'esta tècnica són els algorismes d'ordenació més famosos del món: **Quicksort** i **Mergesort**. Mereixen secció pròpia. I corbata.

4. Quicksort — el ràpid (quan li eix bé)

Creat per Tony Hoare en 1959. Sí, té més anys que els teus pares. I continua sent l'algorisme d'ordenació més usat del món. Per alguna cosa serà.

Com funciona

1. Triem un **pivot** (un element de l'array).
2. Col·loquem tots els elements **menors** que el pivot a la seua esquerra, i els **majors** a la seua dreta. Això es diu **particionar**.
3. Apliquem el mateix procés recursivament a les dos meitats (esquerra i dreta del pivot).

Quan l'array té 0 o 1 elements... ja està ordenat. Cas base.

Implementació

```

public class Quicksort {

    static void qs(int[] arr, int izq, int der) {
        if (izq >= der) return;    // caso base: 0 o 1 elementos

        int pivote = arr[(izq + der) / 2];    // elegimos el del medio
        int i = izq, j = der;

        // partición
        while (i <= j) {
            while (arr[i] < pivote) i++;
            while (arr[j] > pivote) j--;
            if (i <= j) {
                int tmp = arr[i];
                arr[i] = arr[j];
                arr[j] = tmp;
                i++;
                j--;
            }
        }

        // llamadas recursivas a cada mitad
        qs(arr, izq, j);
        qs(arr, i, der);
    }

    public static void main(String[] args) {
        int[] datos = {9, 3, 7, 1, 8, 2, 6, 4, 5};
        qs(datos, 0, datos.length - 1);
        System.out.println(java.util.Arrays.toString(datos));
    }
}

```

Execució pas a pas amb {3, 1, 4, 1, 5, 9, 2, 6}:

Array inicial: [3, 1, 4, 1, 5, 9, 2, 6]

Pivote = 5 (posición media)

Posiciones $i=0$, $j=7$

1. i avança hasta encontrar 9 (≥ 5), j retrocede hasta encontrar 2 (≤ 5)
2. Intercambiamos 9 y 2 $\rightarrow$ [3, 1, 4, 1, 5, 2, 9, 6]
3. $i=5$, $j=4 \rightarrow i > j$, terminamos partición
4. Llamada recursiva izquierda: [3, 1, 4, 1, 5, 2]
5. Llamada recursiva derecha: [9, 6]
- ...

[!TIP] El secret de Quicksort està en la partició. Si aconseguixes que els elements es repartisquen més o menys equilibradament, l'algorisme vola. Si no... prepara els $O(n^2)$.

Elecció del pivot

Estratègia	Avantatge	Desavantatge
Primer element	Simple	Pèssim si array ja ordenat
Últim element	Simple	Pèssim si array ja ordenat
Element central	Millor equilibri	Segueix tenint casos dolents
Mediana de tres (primer, mig, últim)	Molt robust	Un poc més de càlcul
Aleatori	Evita el cas pitjor en la pràctica	Aleatorietat no determinista

Complexitat

- **Cas mitjà:** $O(n \log n)$ — quasi sempre.
- **Millor cas:** $O(n \log n)$ — quan el pivot dividix sempre en meitats iguals.
- **Pitjor cas:** $O(n^2)$ — quan l'array ja està ordenat i tries el primer o últim pivot.

[!WARNING] Si tries sempre el primer element com a pivot i l'array ja està ordenat, Quicksort es torna més lent que una tortuga amb ressaca. $O(n^2)$. Literalment pitjor que un for anadat cutre.

És estable?

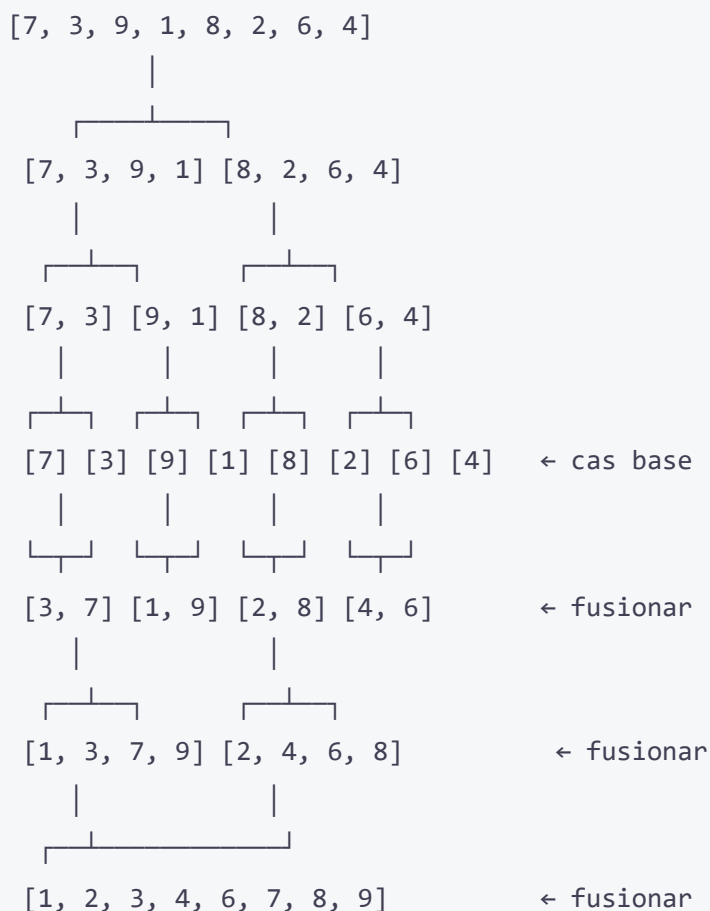
No. Durant la partició, dos elements iguals poden intercanviar-se de posició. Si necessites estabilitat (ordre original d'elements iguals), millor usa Mergesort.

5. Mergesort — el fiable (sempre complix el que promet)

Creat per John von Neumann en 1945. Sí, el mateix de l'arquitectura d'ordinadors. El tio no parava.

Com funciona

1. **Dividir** l'array en dos meitats (per la meitat exactament, no cal triar pivot).
2. **Ordenar** cada meitat recursivament.
3. **Fusionar** (merge) les dos meitats ordenades en un únic array ordenat.



Implementació

```

public class Mergesort {

    static void ms(int[] arr, int izq, int der) {
        if (izq >= der) return; // caso base

        int mid = (izq + der) / 2;
        ms(arr, izq, mid);           // ordenar mitad izquierda
        ms(arr, mid + 1, der);       // ordenar mitad derecha
        fusionar(arr, izq, mid, der); // combinar las dos mitades
    }

    static void fusionar(int[] arr, int izq, int mid, int der) {
        int[] tmp = new int[der - izq + 1];
        int i = izq, j = mid + 1, k = 0;

        // Comparar elementos de las dos mitades
        while (i <= mid && j <= der)
            tmp[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];

        // Copiar lo que quede de la mitad izquierda
        while (i <= mid) tmp[k++] = arr[i++];

        // Copiar lo que quede de la mitad derecha
        while (j <= der) tmp[k++] = arr[j++];

        // Copiar el array temporal al original
        System.arraycopy(tmp, 0, arr, izq, tmp.length);
    }

    public static void main(String[] args) {
        int[] datos = {9, 3, 7, 1, 8, 2, 6, 4, 5};
        ms(datos, 0, datos.length - 1);
        System.out.println(java.util.Arrays.toString(datos));
    }
}

```

Complexitat

- **Sempre:** $O(n \log n)$. No importa com estiga l'array. Mergesort no té cas dolent.
- **Memòria:** $O(n)$ extra per l'array temporal tmp. Eixe és el seu punt feble.

És estable?

Sí. Quan dos elements són iguals, el de l'esquerra va primer en la fusió. Això manté l'ordre original d'elements iguals, una cosa que Quicksort no pot garantir.

[!NOTE] Estabilitat no és un concepte teòric avorrit. És important quan ordenes per múltiples criteris. Per exemple, si ordenes una llista d'alumnes per nota i després per nom, vols que els que tenen la mateixa nota mantinguen l'ordre alfabètic. Mergesort t'ho dona. Quicksort et fa plorar.

6. Comparació: quan use cada un?

No hi ha un “millor algorisme” universal. Tot depend del context. Ací tens una guia pràctica per a prendre decisions:

Situació	Tria
Array xicotet (< 50 elements)	Dona igual, usa <code>Arrays.sort()</code>
Array gran amb dades a l'atzar	Quicksort (anirà genial en mitjana)
Array gran, quasi ordenat o amb molts duplicats	Mergesort o Quicksort amb mediana de tres
Necessites estabilitat (ordre relatiu)	Mergesort
La memòria és justa (sistema encastrat, mòbil)	Quicksort ($O(\log n)$ vs $O(n)$)
No et paguen per pensar	<code>Arrays.sort()</code> i a seguir amb la teua vida
L'array és enorme i les dades estan en disc	Mergesort extern (s'usa en bases de dades)

En Java, `Arrays.sort()` per a tipus primitius usa **Dual-Pivot Quicksort** (una versió millorada amb dos pivots). Per a objectes usa **TimSort** (una mescla de Mergesort i Insertion Sort). Per alguna cosa Will Smith no canta sobre algorismes d'ordenació.

Rendiment en la pràctica

Algorisme	n=10	n=100	n=1.000	n=10.000	n=100.000	n=1.000.000
Quicksort	~0 ms	~0 ms	~0 ms	~1 ms	~15 ms	~120 ms
Mergesort	~0 ms	~0 ms	~0 ms	~2 ms	~20 ms	~150 ms
Burbuja (per a plorar)	~0 ms	~1 ms	~100 ms	~10.000 ms	no esperes	

[!WARNING] Mai, baix cap concepte, uses Burbuja (Bubble Sort) en producció. És $O(n^2)$, lent, i els teus companys t'odiaràn. És com usar un caragol per a repartir pizzes. Existix, però no hauria.

7. ★ BE THE CODE, MY FRIEND: implementa Quicksort des de zero

🧠 **Don Tip:** Dividix i venceràs: tria un pivot, partix l'array en menors i majors, i repetix recursivament. Si domines eixe patró, Quicksort és teu.

Tanca esta pàgina. Obri un editor de text en blanc. No miris ni una línia del que has llegit fins ara.

Escriu Quicksort des de zero. La firma del mètode ha de ser:

```
static void quicksort(int[] arr, int inicio, int fin)
```

Quan acabes, dona-li un ull al teu codi i compara'l amb el d'esta unitat.

Nivells d'assoliment:

- ★ Ho tens, però has hagut de mirar el codi una volta. Aprovat raspadet.
- ★★ T'ha eixit a la primera i funciona. Eres una màquina.
- ★★★ T'ha eixit a la primera, sense errors d'off-by-one, i a més has triat mediana de tres com a pivot. No necessites este curs. Vés a donar una xarrada TED.

[!TIP] Pista mental gratuïta: necessites tres coses — triar un pivot, partir l'array en dos zones (menors i majors), i cridar recursivament a cada costat. Si memoritzes eixe patró, Quicksort

8. No Hi Ha Preguntes Tontes!

La recursivitat és més lenta que un bucle?

Quasi sempre, sí. Cada crida recursiva afig overhead: crear un stack frame, guardar variables, calcular direcció de retorn, etc. Per a operacions simples com sumar un array, un for sempre guanya. **Però** hi ha problemes que s'expressen de forma molt més natural amb recursivitat: arbres, expressió de parèntesis anidats, recorregut de laberints, backtracking. La regla és: usa bucles per a lo simple, recursivitat per a lo complex.

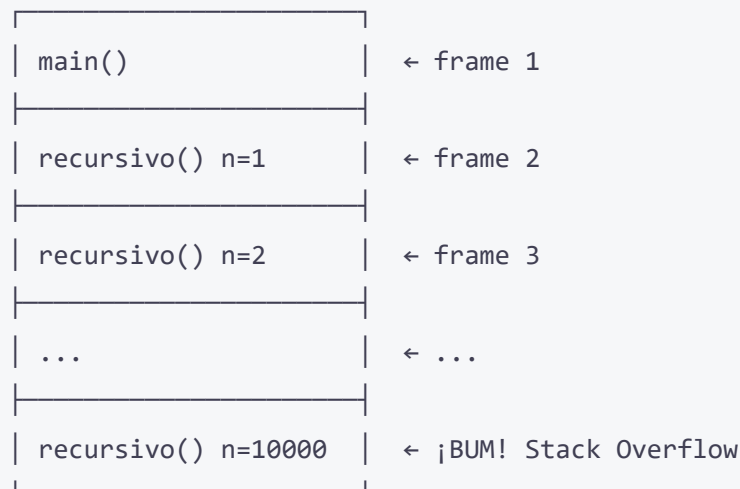
Què és exactament un StackOverflowError?

El **call stack** és una regió de memòria on la JVM guarda informació de cada crida a funció. Cada crida ocupa un espai (stack frame) que conté: paràmetres, variables locals, i la direcció de retorn.

Quan fas massa crides recursives (o una recursió infinita), el stack s'ompli. La següent crida ya no cap. La JVM diu “no puc més” i et llança un `StackOverflowError`.

És com omplir un got d'aigua: cada gota és una crida. Arriba un punt en què una gota més ho desborda tot.

Stack memòria:



Què és tail recursion? Java l'optimitza?

Tail recursion (recursió de cua) és quan la crida recursiva és la **última operació** que realitza la funció. No hi ha cap operació pendent després de la crida.

```
// Sin tail recursion – hay una multiplicación pendiente
static int factNormal(int n) {
    if (n <= 1) return 1;
    return n * factNormal(n - 1); // la multiplicación espera a que vuelva
}

// Con tail recursion – la llamada es lo último
static int factTail(int n, int acum) {
    if (n <= 1) return acum;
    return factTail(n - 1, n * acum); // nada pendiente después
}
```

En llenguatges funcionals com Scala, Haskell o Scheme, el compilador detecta tail recursion i la convertix en un bucle internament (no gasta stack). **Java no fa això**. Ni ara ni en el futur previsible. Si crides a `factTail(10000, 1)` en Java, explota igual.

[!NOTE] Encara que Java no optimitze tail recursion, escriure funcions amb tail recursion és un bon hàbit: clarifica la teua intenció, és més fàcil de llegir, i si algun dia uses un llenguatge que sí ho faça, ya ho domines.

Puc usar recursivitat sense saber-ho?

Sí, constantment. Cada volta que crides a `Arrays.sort()`, estàs usant Quicksort o Mergesort recursiu per davall. Quan recorres un sistema de fitxers amb `File.listFiles()`, la JVM pot usar recursivitat internament. La recursivitat no és una característica exòtica: és el pa de cada dia de la computació.

I si la meua recursivitat explota el stack? Què faig?

Tens diverses opcions:

1. **Augmentar la grandària del stack** de la JVM: `java -Xss2m MiPrograma` (a 2 MB).
2. **Convertir a iteratiu** (usa una pila explícita amb un `Stack<T>` o `Deque<T>`).
3. **Usar una tècnica diferent** que no requerisca tanta profunditat.
4. **Acceptar la derrota** i buscar un altre treball. (No, és broma. Fes l'opció 1 o 2).

Dividir i venceràs només servix per a ordenar arrays?

Per a res. És un dels paradigmes més universals de la computació. Ací tens aplicacions famoses:

- **Cerca binària:** partix un array ordenat per la meitat i busca en el costat correcte. $O(\log n)$.
- **Multiplicació de matrius (Strassen):** dividix matrius grans en submatrius. $O(n^{2.807})$ en lloc de $O(n^3)$.
- **Transformada Ràpida de Fourier (FFT):** dividix senyals digitals en freqüències. Base del JPEG, MP3, WiFi.
- **Convex Hull:** troba el polígon convex mínim que conté un conjunt de punts.
- **Algorisme de Karatsuba:** multiplica números grans més ràpid que el mètode tradicional.

Una volta entens Dividir i Venceràs, comences a vore patrons per tot arreu. És com quan aprens una paraula nova i de sobte la veus per tot arreu.

L'ENIGMA

Estàs en un laberint amb 3 interruptors. Cadascun controla una bombeta en una habitació tancada. Només pots entrar a l'habitació UNA vegada. Com esbrines quin interruptor encén quina bombeta?

Pista: les bombetes no sols s'encenen, també es calfen.

💡 **Don Tip:** Si una bombeta ha estat encesa molt de temps, estarà calenta encara que l'apagues. Utilitza-ho al teu favor.

Resum de la unitat

Concepte	Clau
Recursivitat	Cas base + cas recursiu. Sense cas base = <code>StackOverflowError</code>
Factorial	$O(n)$. L'exemple canònic de recursivitat lineal
Fibonacci naive	$O(2^n)$. No ho uses per a res seriós
Fibonacci optimitzat	$O(n)$ amb memoització. La diferència entre lent i ràpid

Concepte	Clau
Dividir i venceràs	Dividir, conquerir, combinar. Paradigma universal
Quicksort	$O(n \log n)$ mitjana, $O(n^2)$ pitjor, inestable. El més ràpid en la pràctica
Mergesort	$O(n \log n)$ sempre, estable, $O(n)$ memòria extra. El més fiable
Partició	El cor de Quicksort. Com repartir elements al voltant del pivot
Fusió	El cor de Mergesort. Com combinar dos arrays ordenats
Estabilitat	Mergesort la manté, Quicksort no. Important per a ordenacions multi-criteri
Tail recursion	Quan la crida recursiva és l'últim. Java no l'optimitza
Stack overflow	El límit del call stack. Cada crida costa memòria

[!NOTE] Esta unitat cobrix els RA2 (ús d'estructures de control avançades incloent recursivitat) i RA6 (aplicació d'algorismes d'ordenació, anàlisi de complexitat i elecció justificada de l'algorisme segons el context). Efectivament, la programació no és només escriure codi que funcione: és pensar en **com** fer-ho eficient, triar la ferramenta adequada, i entendre què passa davall del capó.

En la següent unitat explorarem **estructures de dades avançades**: arbres, grafs i taules hash. Prepara't per a dibuixar fletxes i quadradets. Molts quadradets.



Sergi Garcia Barea — CC BY-SA 4.0

Unidad 4: POO – Clases y Objetos

Objectius d'aprenentatge

- Crear classes, objectes i constructors
- Diferenciar entre classe i objecte
- Usar Scanner per a entrada per consola
- Entendre paquets i imports
- Sobreescrivre toString() i equals()
- Usar this i sobrecàrrega de constructors

POO: Els Teus Objectes Cobren Vida

Fins ara hem escrit programes lineals: això, després això, després això. Però el món real no funciona així. Al món real tens *coses*: un gos, un cotxe, un professor de programació amb ulleres de pasta. Cada cosa té **atributs** (color, edat, nombre de ganes de corregir exàmens) i **comportaments** (lladrar, accelerar, posar faltes d'ortografia).

La Programació Orientada a Objectes (POO) és simplement això: escriure codi com funciona el món real.

Classes i Objectes: Tallagalletas vs Galetes

Una **classe** és un *tallagalletas* (un motle). Un **objecte** és la *galleta* que fas amb eixe motle.

Pots tindre un sol tallagalletas amb forma d'estrella i fer milions de galetes estrella. Totes tindran la mateixa forma, però cada una pot tindre diferent quantitat de pepitas de xocolate.

```
// Esta es la clase (el cortapastas)
public class Galleta {
    String sabor;
    boolean tieneChocolate;

    public void comer() {
        System.out.println("¡Nam, galleta sabor " + sabor);
    }
}
```

Per a crear galetes (objectes) a partir del motle:

```
Galleta g1 = new Galleta();    // Primera galleta
g1.sabor = "Chocolate";
g1.tieneChocolate = true;

Galleta g2 = new Galleta();    // Segunda galleta (mismo molde)
g2.sabor = "Vainilla";
g2.tieneChocolate = false;

g1.comer(); // "Ñam, galleta sabor Chocolate"
g2.comer(); // "Ñam, galleta sabor Vainilla"
```

Dos galetes diferents (objectes diferents) amb el mateix tallagalletas (classe). Cada una amb els seus propis valors.

Scanner: L'Intèrpret Amigable

Fins ara les dades estaven *escrites en el codi*. Però... i si volem que l'usuari meta dades? Ahí apareix Scanner, el nostre intèrpret personal.

```
import java.util.Scanner;

public class CharlaConElUsuario {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in); // Creamos un intérprete

        System.out.print("¿Cómo te llamas? ");
        String nombre = sc.nextLine();        // Lee una línea entera

        System.out.print("¿Cuántos años tienes? ");
        int edad = sc.nextInt();              // Lee un número entero

        System.out.print("¿Cuánto mides (en metros)? ");
        double altura = sc.nextDouble();      // Lee un decimal

        System.out.println("Hola " + nombre + ", tienes " + edad + " años y mides " +
            altura + " m.");

        sc.close(); // Cierra el intérprete (buena educación)
    }
}
```

💡 **Consell:** Scanner és de java.util. Si no poses import java.util.Scanner; dalt de tot, Java et mirarà amb cara de “no sé qui és eixe tal Scanner”. Els imports són com presentar als teus amics abans de parlar d’ells.

⚠️ **Advertència:** Quan barreges nextInt() i nextLine() poden passar coses rares. nextInt() llig el número però *deixa el salt de línia sense llegir*. Després nextLine() llig eixe salt de línia i sembla que es salta la pregunta. Solució: posa un sc.nextLine(); extra després de nextInt() per a consumir eixa broseta.

Constructors: La Festa de Benvinguda

Quan crees un objecte, s’executa un **constructor**, que és un mètode especial que prepara l’objecte. Com la festa de benvinguda a un nou empleat.

```

public class Persona {
    String nombre;
    int edad;

    // Constructor sin parámetros ("vale, te pongo valores por defecto")
    public Persona() {
        this.nombre = "Desconocido";
        this.edad = 0;
    }

    // Constructor con parámetros ("te paso los datos, tú inicialízalos")
    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    public void presentarse() {
        System.out.println("Hola, soy " + this.nombre + " y tengo " + this.edad + "
años.");
    }
}

```

```

Persona p1 = new Persona();           // "Desconocido", 0
Persona p2 = new Persona("Ana", 25);  // "Ana", 25
p2.presentarse();                     // "Hola, soy Ana y tengo 25 años."

```

★ BE THE CODE, MY FRIEND: El Creador de Persones

👁️ **Don Tip:** Segueix cada new com si creares un objecte en la memòria. Després segueix les crides a mètodes pas a pas.

Vas a ser la JVM. Et donen:


```

public class Coche {
    String marca;
    int velocidad;

    public Coche(String marca) {
        this.marca = marca;
        this.velocidad = 0;
    }

    public void acelerar(int incremento) {
        this.velocidad += incremento;
    }

    public void frenar(int decremento) {
        this.velocidad -= decremento;
        if (this.velocidad < 0) this.velocidad = 0;
    }

    public static void main(String[] args) {
        Coche c = new Coche("Seat");
        c.acelerar(50);
        c.acelerar(30);
        c.frenar(20);
        System.out.println(c.marca + " va a " + c.velocidad + " km/h");
    }
}

```

Traça mental:

1. Es crea un objecte Coche amb marca "Seat". La seua velocitat comença en 0.
2. `c.acelerar(50)` → `this.velocidad += 50` → velocitat = 50.
3. `c.acelerar(30)` → `this.velocidad += 30` → velocitat = 80.
4. `c.frenar(20)` → `this.velocidad -= 20` → velocitat = 60.
5. Imprimeix: **"Seat va a 60 km/h"**.

Variante: I si frenem amb 100 en lloc de 20? `c.frenar(100)`: velocitat passaria a -20, però el `if` la posa a 0. Es com si el cotxe tinguera frens de veritat: no pots anar a velocitat negativa.

Packages: Els Barris de la Ciutat Java

Java té MILERS de classes. Per a no tornar-se boig, les organitza en **paquets** (packages). Pensau en ells com barris d'una ciutat:

- `java.lang` — El centre històric. `String`, `Math`, `System`. S'importa sol.
- `java.util` — El barri de les eines. `Scanner`, `ArrayList`, `Date`.
- `java.io` — La zona portuària. Per a llegir i escriure fitxers.
- `java.net` — L'aeroport. Per a comunicacions en xarxa.
- `javax.swing` — El barri dels arquitectes. Interfícies gràfiques.

Per a usar una classe d'un altre barri, has de *importar-la*:

```
import java.util.Scanner;           // "Quiero usar el Scanner del barrio util"
import java.util.*;                 // "Tráeme TODO lo que haya en java.util"
```

Les classes de `java.lang` no necessiten import. És com ta casa: no necessites demanar permís per a entrar en la teua pròpia cuina.

? ¡No Hay Preguntas Tontas!

Q: Vale, `new` crea objetos. Pero, ¿qué pinta `new`? ¿Es un operador?

A: `new` és el *constructor d'objectes* de Java. Reserva memòria, crida al constructor i retorna l'adreça de l'objecte. Sense `new`, no hi ha objecte. És com la clau que encén el cotxe: sense ella, no arranques.

Q: ¿Y por qué `String nombre = "Ana";` no lleva `new`?

A: Perquè Java és un amor amb `String`. És tan comú que et deixa crear strings amb cometes directament (literals). És un *drecera*. Darrere del teló, Java ho tracta quasi com si tinguera `new`. Però `Scanner sc = "System.in";` no funciona. `Scanner` no és especial, `String` sí.

Q: ¿Cuál es la diferencia entre `equals()` y `==` para comparar objetos?

A: `==` compara si dos variables apunten al *mateix objecte* (mateixa adreça). `equals()` compara si el *contingut* és el mateix. Amb `String`:

```
String a = "Hola";  
String b = "Hola";  
String c = new String("Hola");  
  
System.out.println(a == b);    // true (Java reutiliza literales iguales)  
System.out.println(a == c);    // false (c es otro objeto)  
System.out.println(a.equals(c)); // true (el contenido es el mismo)
```

Q: ¿Puedo tener un método main en cualquier clase?

A: Sí. Cada classe pot tindre el seu propi main. És com tindre diverses portes d'entrada a una casa. Quan executes `java NombreClase`, Java busca el main d'eixa classe concreta. Els altres main d'unes altres classes... ahí estan, dormint.

Q: ¿Qué pasa si no pongo constructor y luego hago `new Persona("Ana")`?

A: Java et dirà: “Ei, no existeix un constructor `Persona(String)`. Et vaig a deixar tirat amb un error de compilació.” Si no poses CAP constructor, Java et dona un de buit sense paràmetres. Però si poses ALGUN constructor, el buit NO es crea automàticament.

Constructors: Les Instruccions del Forn

Un constructor és un mètode especial amb el MATEIX nom que la classe i que NO retorna res (ni void). És com programar el forn: “temperature: 180, mode: turbo”.

Constructor per Defecte (El Forn per Defecte)

Si no escrius cap constructor, Java et regala un de gratuït. Inicialitza tot a `null`, `0` o `false`. Com un forn fred.

Constructor Parametritzat (El Forn amb Programa)

Tu li dius com vols la galeta.

```
public class Galleta {
    String forma;
    boolean tieneChocolate;
    int temperatura;

    public Galleta(String forma, boolean tieneChocolate, int temperatura) {
        this.forma = forma;
        this.tieneChocolate = tieneChocolate;
        this.temperatura = temperatura;
    }
}
```

Sobrecàrrega de Constructors: Diversos Forns, Una Cuina

Pots tindre varios constructors amb diferents paràmetres. Com una rentadora: programa curt, programa llarg, programa de “això és una camisa de seda, que no s'emrecorde”.

```
public class Galleta {
    String forma;
    boolean tieneChocolate;

    public Galleta() {
        this("redonda", false); // Llama al otro constructor
    }

    public Galleta(String forma, boolean tieneChocolate) {
        this.forma = forma;
        this.tieneChocolate = tieneChocolate;
    }
}
```

La Paraula Clau this: Jo, Mi, Me, Amb Mi

this és l'objecte dient “SENT, PARLE DE MI, NO D'UN ALTRE”. Servix per a:

1. Desambiguar: quan el paràmetre es diu igual que l'atribut.

```
public class Persona {
    String nombre;

    public Persona(String nombre) {
        this.nombre = nombre; // "this.nombre" es el de arriba, "nombre" es el
parámetro
    }
}
```

2. Cridar a un altre constructor: `this(...)`.

3. Passar-te a tu mateix com a argument.

```
public class Coche {
    String marca;
    int velocidad;

    public Coche(String marca) {
        this.marca = marca;
    }

    public void acelerar(int inc) {
        this.velocidad += inc;
        System.out.println(this.marca + " va a " + this.velocidad + " km/h");
    }
}
```



Nota: Si no poses `this` quan hi ha ambigüitat, Java es confon. És com si en una conversa dius “nom” i hi ha dos persones anomenades “nom”. Et miraran rar.

★ BE THE CODE, MY FRIEND: La Caixa Misteriosa



Don Tip: Fixa't en quin objecte es crida cada mètode. La variable de referència determina quins mètodes pots invocar.

Tu eres Java. Et donen este codi:

```

public class Caja {
    int ancho;
    int alto;
    int profundo;

    public Caja(int ancho, int alto, int profundo) {
        this.ancho = ancho;
        this.alto = alto;
        this.profundo = profundo;
    }

    public int volumen() {
        return ancho * alto * profundo;
    }
}

public class Main {
    public static void main(String[] args) {
        Caja c1 = new Caja(2, 3, 4);
        Caja c2 = new Caja(5, 1, 2);
        System.out.println(c1.volumen());
    }
}

```

- Quants objectes hi ha en memòria al final de main?
- Quant imprimeix el System.out.println?
- Què passa si a c1 li canvies ancho = 10? c2 es veu afectat?

Solució: 2 objectes, 24, NO (cada objecte té la seua pròpia còpia). Les variables d'instància són independents per a cada objecte.

★ BE THE CODE, MY FRIEND: Quin Constructor Es Crida?

👁 **Don Tip:** Comprova el número i tipus d'arguments. Java tria el constructor que millor coincideix.

Sense executar, què imprimeix este codi?

```

public class Pedido {
    String producto;
    int cantidad;

    public Pedido() {
        this("Sin producto", 0);
        System.out.println("Constructor vacío");
    }

    public Pedido(String producto, int cantidad) {
        this.producto = producto;
        this.cantidad = cantidad;
        System.out.println("Constructor con parámetros");
    }

    public static void main(String[] args) {
        Pedido p = new Pedido();
        System.out.println(p.producto + " x" + p.cantidad);
    }
}

```

Solució: Es crida a Pedido() que fa this("Sin producto", 0), el que primer executa Pedido(String, int) (imprimeix "Constructor con parámetros"), després torna i termina Pedido() (imprimeix "Constructor vacío"). Després imprimeix "Sin producto x0". El this(...) sempre va primer.

? ¡No Hay Preguntas Tontas!

Q: Vale, ¿y por qué no puedo hacer simplemente `int galleta1 = 42;` en vez de todo este rollo de clases?

A: Perquè un `int` només guarda un número. Una classe guarda NÚMEROS + TEXT + MÈTODES + tot el que vulgues. És com comparar un posagots amb un dron. El dron pot fer mil coses, el posagots... posa gots.

Q: ¿Cuántos objetos puedo crear? ¿Hay límite?

A: Fins que et quedes sense memòria (i aleshores Java llança `OutOfMemoryError` i el teu programa mor). Però anem, amb 8 GB de RAM pots crear milions d'objectes xicotets. No et preocupes.

Q: this es una palabra reservada, ¿no? ¿Puedo usarla fuera de una clase?

A: No. this fora d'una classe és com demanar una pizza en una ferreteria. No té sentit. Només existix dins del context d'un objecte.

Q: ¿Por qué hace falta escribir new? ¿No podría Java crear el objeto solito?

A: No, perquè new és el “permís de construcció”. Sense new, només declares una variable (com `Coche c;`), però no hi ha cotxe al garatge, només una plaça de pàrquing buida. Fins que no faces new, l'objecte no existix.

Mètodes `toString()` i `equals()`: La Carta de Presentació

`toString()` és com el teu objecte es presenta en públic. Per defecte Java imprimeix algo com `Persona@3e3abc` (la classe i una adreça de memòria). No molt útil.

```
public class Persona {
    private String nombre;
    private int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    @Override
    public String toString() {
        return nombre + " (" + edad + " años)";
    }
}
```

`equals()` és per a preguntar: “este objecte ÉS IGUAL a este altre?” (pel seu contingut, no perquè siguin el mateix lloc en memòria).


```
public class Persona {  
    private String dni;  
  
    @Override  
    public boolean equals(Object o) {  
        if (this == o) return true;  
        if (o == null || getClass() != o.getClass()) return false;  
        Persona otra = (Persona) o;  
        return dni != null && dni.equals(otra.dni);  
    }  
  
    @Override  
    public int hashCode() {  
        return Objects.hash(dni);  
    }  
}
```

💡 **Consell:** Sobreescriu toString() sempre. El teu jo del futur (i el teu professor) t'ho agrairà. equals() i hashCode() van junts com el pernil i el formatge. Si sobreescrus un, sobreescriu l'altre.

Exemple Complet: Cotxe (Amb Accelerador de Veritat)

```

public class Coche {
    private String marca;
    private String modelo;
    private int velocidad;

    public Coche(String marca, String modelo) {
        this.marca = marca;
        this.modelo = modelo;
        this.velocidad = 0;
    }

    public void acelerar(int kmh) {
        this.velocidad += kmh;
        System.out.println(marca + " " + modelo + ": " + velocidad + " km/h");
    }

    public void frenar(int kmh) {
        this.velocidad = Math.max(0, this.velocidad - kmh);
    }

    @Override
    public String toString() {
        return marca + " " + modelo + " - " + velocidad + " km/h";
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Coche c = (Coche) o;
        return marca.equals(c.marca) && modelo.equals(c.modelo);
    }

    @Override
    public int hashCode() {
        return Objects.hash(marca, modelo);
    }
}

```

Bones Pràctiques (O Com No Ficarla)

- Atributs **privats sempre** (ja veurem per què en el següent tema. Spoiler: no volem que ningú toque les vostres parts privades).
- `this` quan hi haja ambigüitat. Si no n'hi ha, pots ometre-ho (però posar-ho no dol).
- `toString()` sempre. És el teu amic.
- `equals()` i `hashCode()` si compares objectes per valor.
- Inicialitza-ho tot en el constructor. No deixes atributs ballant.

EL RING: Classe vs Objecte

Dos conceptes discuteixen acaloradament. Qui té raó?

Classe: «Jo soc el motle, el plàmol, la idea platònica. Sense mi no existiries. Jo definisc quins atributs i mètodes tenen els objectes. Soc la creadora!»

Objecte: «Sí, però jo soc qui realment fa coses. Tu ets només un fitxer `.java` al disc. Jo ocupe memòria, tinc estat, puc canviar els meus atributs. Sense mi el teu codi no serveix per a res.»

Classe: «Ah sí? I quants de tu existeixen? Pots tindre milers d'objectes creats a partir de mi. Jo soc únic, tu eres una còpia. Soc original, eres reproduïble!»

Objecte: «Exacte. Perquè tu eres el plàmol, però jo soc l'edifici construït. Ningú viu en un plàmol. Quan executes el programa, el que treballa soc jo.»

Classe: «Val, ens necessitem. Sense classe no hi ha objecte. Sense objecte, la classe és només teoria.»

Objecte: «Tracte fet.»

💡 **Don Tip:** La classe defineix el QUÈ (atributs) i el CÒM (mètodes). L'objecte és el QUIÉ (la instància concreta que executa i té valors propis).

Resum

- Classe = motle / tallagalletas. Objecte = galeta / resultat.
- `new` crea objectes. Crida al constructor.
- Scanner llig del teclat com un intèrpret amigable.
- Paquets = barris. `import` = demanar permís per a entrar.
- `String` és especial: es pot crear amb cometes sense `new`.

- `this` desambigua i permet cridar a altres constructors.
 - Sobrecàrrega: varios constructors amb diferents paràmetres.
 - `toString()` i `equals()` es sobreescrueixen sempre.
-

Exercicis Proposats

Introducció a POO

1. **Salutació personalitzada** Usa `Scanner` per a preguntar nom, edat i ciutat. Mostra: “Hola, soc [nom], tinc [edat] anys i visc en [ciutat].”
2. **La frase mutant** Demana una frase a l'usuari i mostra: nombre de caràcters, en majúscules, en minúscules, primera paraula (abans de l'espai), i reemplaça totes les vocals per 'e'.
3. **Suma de strings** Demana dos números com a text (`String`). Converteix-los a enters amb `Integer.parseInt()` i mostra la seua suma. Si no són números vàlids, quin error ix?
4. **Daus virtuals** Genera 5 números aleatoris entre 1 i 100. Mostra el major i el menor (pots usar `Math.max()` i `Math.min()` o fer-ho a mà).
5. **Lletra, número o símbol?** Demana un caràcter a l'usuari. Usa mètodes de `Character` (`isDigit()`, `isLetter()`...) per a dir-li què és.
6. **Classe Gos** Crea una classe `Perro` amb atributs `nombre` (`String`) i `edad` (`int`). Un mètode `ladrar()` que imprimisca “Guau, soc [nombre]”. Crea dos gossos diferents i fes que cada un lladre.
7. **Calculadora IMC** Demana pes (kg) i altura (m). Calcula l'IMC = $\text{pes} / (\text{altura} * \text{altura})$. Mostra el resultat amb 2 decimals. Usa `Math.round()` o imprimeix amb format.
8. **Comptador d'objectes** Crea una classe `Contador` amb un atribut `static int totalObjetos`. En el constructor, incrementa `totalObjetos`. En el main, crea 5 objectes `Contador` i al final imprimeix `Contador.totalObjetos`. Ix 5?

Classes i Objectes

1. **Classe Rectangle**: Crea `Rectangulo` amb `ancho` i `alto` (`double`). Constructor parametritzat, getters/setters, `calcularArea()` i `calcularPerimetro()`. Sobreescrueix `toString()`.

2. **Classe CompteBancari:** CuentaBancaria amb titular, saldo i numeroCuenta. Dos constructors: només titular (saldo 0) o titular + saldo. Mètodes ingresar(double) i retirar(double) (valida saldo suficient). toString().
3. **Classe Hora:** Hora amb hora (0-23), minuto (0-59), segundo (0-59). Constructor amb validació. incrementarSegundo() que maneja desbordaments.
4. **Sobrecàrrega de constructors:** Classe Email amb destinatario, asunto, cuerpo. Tres constructors: tot, només destinatari+assumpte (cos buit), només destinatari (assumpte i cos per defecte).
5. **equals() en Persona:** Afig fechaNacimiento (LocalDate) a Persona. Sobreescriu equals() usant nombre + fechaNacimiento.
6. **Classe Punt:** Punto amb x i y (int). Constructor, getters, distancia(Punto otro) (distància euclídea), toString(). Calcula perímetre d'un triangle donats 3 punts.
7. **Classe Fracció:** Fraccion amb numerador i denominador (int). Mètodes: sumar, restar, multiplicar, dividir, simplificar(). toString() com "numerador/denominador".
8. **Classe Joc:** Juego amb nombre, genero, precio, edadMinima. Mètode esAptoPara(int edad). Crea varios jocs i mostra quins són aptes per a algú de 12 anys.

RAs treballats en esta unitat:

- **RA2** - Programes senzills
- **RA4** - Classes



Sergi Garcia Barea — CC BY-SA 4.0

Unidad 5: Visibilidad, Encapsulación y Static

Objectius d'aprenentatge

- Aplicar els 4 nivells de visibilitat
- Encapsular atributs amb getters i setters
- Validar dades en setters
- Entendre el patró JavaBeans
- Usar membres estàtics (variables, mètodes, constants)
- Crear classes utilitàries amb constructor privat

Visibilitat: L'Art de No Ensenyar-ho Tot

El Gran Problema (O Com la Gent Toca Les Teues Coses)

Imagina que vius en una casa de vidre. Qualsevol pot veure-ho tot: la teua roba interior, la teua col·lecció de cromos de Pokémon, eixa caixa de galetes buida que guardes per si de cas. Incòmode, oi?

Doncs el mateix passa amb els teus objectes. Si tot és públic, qualsevol des de qualsevol lloc pot fer:

```
Persona p = new Persona();  
p.edad = -666; // ¡Edad negativa! Esto no tiene sentido.  
p.saldo = 999999; // Multiplicate por 0 el dinero.
```

I el teu objecte queda fet un desastre. Necessitem **control d'accés**. Necessitem... **VISIBILITAT**.

Els 4 Nivells de Visibilitat: De la Tanca a la Caixa Forta

Modificador	Es veu des de			És com...
public	Tothom,	absolutament		Una tanca publicitària a Times Square
protected	Mateix	paquet	+	Els secrets de família: ho saben els teus cosins i els teus fills
	subclasses (fills)			

Modificador	Es veu des de	És com...
<i>package-private</i> (default)	Mateix paquet (veïnat)	El xafardeig del barri
<code>private</code>	Només la classe	El teu diari secret amb cadenat

“El `private` és com el teu calaix dels calcetins desaparellats: existix, però no cal que ningú més ho veja.”

public: La Tanca Publicitària

Tothom ho veu. Des de qualsevol classe, qualsevol paquet. És com posar el teu número de telèfon en una pancarta.

```
public class VallaPublicitaria {  
    public String mensaje; // "CÓMPRAME, SOY UNA CLASE"  
  
    public void mostrar() {  
        System.out.println(mensaje);  
    }  
}
```

Usa-ho per al que VULGUES que altres usen. No per als teus atributs (a menys que t'agrade el caos).

private: El Diari amb Cadenat

Només la classe ve els seus propis `private`. Ni sa mare, ni el seu millor amic, ni el gos.

```
public class DiarioSecreto {
    private String contenido; // Nadie fuera de esta clase puede leerlo

    public void escribir(String mensaje) {
        this.contenido = "Querido diario: " + mensaje;
    }

    public String leer() {
        return contenido; // Solo yo puedo decidir qué mostrar
    }
}
```

⚠ **Advertència:** Mai, MAI, facés un atribut public. És com deixar la porta de casa teua oberta amb un cartell: “Passeu i toqueu-ho tot”.

protected: Els Secrets de Família

És com les històries vergonyoses de la família. Els teus cosins (mateix paquet) i els teus fills (subclasses) poden accedir. Però un desconegut d'un altre paquet... no.

package-private (default): El Xafardeig del Barri

Si NO poses cap modificador, Java assumix “package-private”. Ho veuen les classes del mateix paquet. Com el grup de WhatsApp del veïnat.

Taula Comparativa: Qui Veu Què?

```
package barrio;

public class Casa {
    public String direccion; // Lo sabe todo el mundo
    protected String telefono; // Lo sabe la familia
    int numeroHabitaciones; // Lo saben los vecinos (package-private)
    private String contrasenaWifi; // SOLO YO
}
```

Des del mateix paquet (barrio):


```

public class Vecino {
    public void espiar() {
        Casa c = new Casa();
        System.out.println(c.direccion);           // OK: public
        System.out.println(c.telefono);           // OK: protected (mismo paquete)
        System.out.println(c.numeroHabitaciones); // OK: package-private
        // System.out.println(c.contrasenaWifi); // ERROR: private
    }
}

```

Des d'un altre paquet, sent subclasse:

```

package otraCiudad;
import barrio.Casa;

public class CasaHeredada extends Casa {
    public void espiar() {
        System.out.println(direccion);           // OK: public
        System.out.println(telefono);           // OK: protected (soy subclase)
        // System.out.println(numeroHabitaciones); // ERROR: package-private
        // System.out.println(contrasenaWifi);    // ERROR: private
    }
}

```

★ BE THE CODE, MY FRIEND: El Banc

💡 **Don Tip:** Els getters exposen dades, els setters les validen. Si un setter permet saldos negatius, el banc fa fallida.

Eres una classe Banco. Tens estos membres:

```
public class Banco {
    public String nombreBanco;
    protected String direccionSucursal;
    String listaClientes;
    private double saldoCaja;

    public void mostrarInfo() {
        System.out.println(nombreBanco);
        System.out.println(direccionSucursal);
        System.out.println(listaClientes);
        System.out.println(saldoCaja);
    }
}
```

Pregunta: Pot una classe Sucursal en un altre paquet vore listaClientes? I direccionSucursal? Pot main() d'una classe en el mateix paquet vore saldoCaja?

Solució: no (package-private, altre paquet no ho veu), sí si és subclasse (protected), no (private). La pròpia classe sempre pot vore-ho tot (per això el mètode mostrarInfo() funciona).

? No Hi Ha Preguntes Tontes!

Q: I si no pose res? Package-private és el mateix que “default”?

A: Sí, es diuen “default” o “package-private”. És el nivell que Java assumix quan no escrius public, private o protected. No és que existisca una paraula clau default per a visibilitat (eixa paraula és per a una altra cosa).

Q: Per què hauria de fer private un atribut i després crear getters i setters públics? És més treball!

A: Perquè així CONTROLES el que entra i ix. Pots validar: “Edat no pot ser negativa”. Pots canviar la implementació interna sense que ningú se n'assabente. És com tindre un porter en la teua discoteca: deixes entrar a qui vols i tires als que van borratxos.

Q: El meu professor va dir que protected és “perquè les subclasses ho vegem”. I què més dona?

A: Dona molt. Pensat en una classe Vehiculo amb protected int velocidadMaxima. La subclasse Coche pot usar-lo. Però una classe Taller del mateix paquet també pot. Si vols

que NOMÉS les subclasses ho vegen i NO els veïns de paquet... males notícies: `protected` no discrimina entre “subclasse d’un altre paquet” i “mateix paquet”. A les dos els ho permet.

Q: I els mètodes? També tenen visibilitat?

A: Per descomptat! Tot té visibilitat. Pots tindre un mètode `private` que només s’use internament, com `private void calcularImpuesto()`. Ningú fora de la classe necessita saber com calcules els impostos (ni tu mateix vols saber-ho).

Q: Si faig tot públic total és més ràpid d’escriure, no?

A: És més ràpid d’escriure i més lent de depurar. Quan algú (o tu) meta un valor impossible en un atribut públic, passaràs hores buscant qui ho va canviar. Amb encapsulació, l’error es detecta a l’instant en el setter.

Encapsulació: El Pilar que Sosté la POO

Encapsulació = **privacitat + control**. És la idea que:

1. Els teus atributs són `private`.
2. Controles l’accés amb getters i setters `public`.
3. Dins dels setters, **VALIDES**.

Exemple de vida (o mort):

```

public class CuentaBancaria {
    private double saldo; // Nadie toca el saldo directamente

    public void ingresar(double cantidad) {
        if (cantidad > 0) {
            this.saldo += cantidad;
        }
    }

    public void retirar(double cantidad) {
        if (cantidad > 0 && cantidad <= saldo) {
            this.saldo -= cantidad;
        } else {
            System.out.println("No tienes tanto dinero, amigo");
        }
    }

    public double getSaldo() {
        return saldo; // Solo lectura
    }
}

```

⚠ **Advertència:** Si fas els atributs public, estàs renunciant a l'encapsulació. És com portar la cartera oberta al metro. Tard o d'hora algú ficarà mà.

Avantatges de l'Encapsulació (O Per Què No Dormiràs Pitjor)

- **Control:** Valides i filtres. Res d'edats negatives.
- **Mantenibilitat:** Canvis internament i el codi client ni s'assabenta.
- **Seguretat:** Ningú deixa el teu objecte en un estat inconsistent.
- **Baix acoblament:** Cada classe va a la seua. No es fiquen unes en els assumptes d'unes altres.

La Convenció JavaBeans: El Protocol

JavaBeans és una convenció (no obligatòria, però sí sensata) que diu:

1. Classe pública.
2. Constructor sense arguments.

3. Atributs privats.
4. Getters i setters públics.
5. Implementa `Serializable` (opcional).

Convenció de noms:

Tipus	Getter	Setter
<code>String nombre</code>	<code>getNombre()</code>	<code>setNombre(String n)</code>
<code>boolean activo</code>	<code>isActive()</code>	<code>setActivo(boolean a)</code>
<code>int cantidad</code>	<code>getCantidad()</code>	<code>setCantidad(int c)</code>

Bones Pràctiques (El Decàleg del Programador Paranoic)

- `private` per a atributs. Sempre. Per defecte. Sense discussió.
- Getter només si cal (atributs immutables no necessiten setter).
- Valida en els setters. No confies en ningú.
- `protected` per a mètodes que les subclasses necessiten. No abuses.
- Comença amb l'accés més restrictiu i obri'l només si és necessari.

Static: El Que Pertany a la Classe (No a l'Objecte)

La Gran Diferència: El Grup de WhatsApp vs Els Missatges Privats

Imagina que eres part d'una classe de 30 alumnes. Tens:

- **El grup de WhatsApp de la classe** (static): tots veuen el mateix missatge. Si algú escriu "demà hi ha examen", els 30 ho veuen. És compartit.
- **Els teus missatges privats** (instància): només tu els veus. Cada alumne té els seus. No es barregen.

Doncs en Java és igual:

- **Variables de classe** (static): una sola còpia per a tots els objectes. Tots compartixen el mateix valor.

- **Variables d'instància** (sense static): cada objecte té la seua pròpia còpia. Cadascuna independent.

“Static és el grup de WhatsApp de la classe. Instància són els DMs que ningú més veu.”

Els mètodes **estàtics** (amb static) pertanyen a la *classe*, no als objectes:

```
public class UtilidadesMatematicas {  
    public static int sumar(int a, int b) {  
        return a + b;  
    }  
  
    public static double media(double a, double b) {  
        return (a + b) / 2;  
    }  
}  
  
// Llamada: ni necesito crear un objeto, llamo a la clase directamente  
int resultado = UtilidadesMatematicas.sumar(5, 3);  
double med = UtilidadesMatematicas.media(10, 20);
```

La diferència pràctica: els mètodes d'instància (sense static) necessiten un objecte:

```
String texto = "Hola";  
int longitud = texto.length(); // length() NO és static. Necessite l'objecte texto.
```

 **Nota:** Math.random(), Math.sqrt(), Integer.parseInt()... tots són estàtics. No necessites crear un Math `m = new Math()`; Seria com comprar una nevera per a tindre un imant. Usa Math.random() directament.

Atributs Estàtics: El Cartell del Col·legi

Un atribut static és com el cartell de “Queden 10 minuts per al pati” al passadís. Està ahí, ho veu tot el món, i només n'hi ha un.

```

public class Estudiante {
    private static int totalEstudiantes = 0; // Variable de clase
    private String nombre;                  // Variable de instancia
    private int id;

    public Estudiante(String nombre) {
        this.nombre = nombre;
        this.id = ++totalEstudiantes; // Incrementa el contador global
    }

    public static int getTotalEstudiantes() {
        return totalEstudiantes;
    }

    public int getId() {
        return id;
    }
}

```

En memòria es veu així:

CLASSE: Estudiante

totalEstudiantes = 3	← static: UN per a tota la classe
----------------------	-----------------------------------

OBJECTE e1

nombre
id = 1

OBJECTE e2

nombre
id = 2

OBJECTE e3

nombre
id = 3

Cada objecte té el seu nombre i id (el seu DNI), però tots compartixen totalEstudiantes.



Consell: Usa NombreClase.miembroEstatico per a accedir. Estudiante.getTotalEstudiantes(). No uses objeto.getTotalEstudiantes(), encara que funcione, és confús.

Mètodes Estàtics: El Telèfon de la Classe

Els mètodes `static` es diuen usant la classe, no un objecte. És com el número de telèfon d'atenció al client d'una empresa: NO necessites parlar amb un empleat concret, crides al número general.

Característiques dels mètodes estàtics:

- **NO poden accedir a atributs d'instància** (no saben de quin objecte parlen).
- **NO tenen `this`** (no hi ha “jo” perquè no hi ha objecte).
- Només poden cridar a altres mètodes estàtics (directament).

```
public class Calculadora {  
    public static int sumar(int a, int b) {  
        return a + b;  
    }  
  
    public static double dividir(int a, int b) {  
        if (b == 0) throw new ArithmeticException("¡No dividirás por cero!");  
        return (double) a / b;  
    }  
}  
  
// Uso: SIN crear ningún objeto  
public class Main {  
    public static void main(String[] args) {  
        int resultado = Calculadora.sumar(10, 5);  
        System.out.println(resultado); // 15  
    }  
}
```

L'Exemple Definitiu: La Classe Math

`java.lang.Math` és la classe utilitària per excel·lència. TOTS els seus mètodes són estàtics. No pots (ni vols) fer `new Math()`.


```
double max = Math.max(10, 20);           // 20
double min = Math.min(10, 20);           // 10
double raiz = Math.sqrt(25);              // 5.0
double potencia = Math.pow(2, 10);        // 1024.0
double absoluto = Math.abs(-7);           // 7
double random = Math.random();            // Aleatori [0.0, 1.0)
double pi = Math.PI;                     // 3.141592653589793
```



Nota: Math té el constructor **privat**. Ningú pot instanciar-la. És com una estàtua: per a admirar-la, no per a fer-li clons.

Classes Utilitàries: El “No Necessite Parella” de Java

Una classe utilitària és una classe que **NOMÉS** té membres estàtics. Són com l'amic que està solter i feliç: no necessita instanciar-se per a ser útil.

Per a evitar que algú intente crear un objecte, li posem el constructor private:

```
public class StringUtils {
    private StringUtils() {} // Nadie puede hacer new StringUtils()

    public static boolean esVacio(String str) {
        return str == null || str.trim().isEmpty();
    }

    public static String invertir(String str) {
        return str == null ? null : new StringBuilder(str).reverse().toString();
    }

    public static String capitalizar(String str) {
        if (esVacio(str)) return str;
        return str.substring(0, 1).toUpperCase() + str.substring(1).toLowerCase();
    }
}
```

Constants: El Que Mai Canvia (Com l'Amor de Ta Mare)

`static final` és “una constant de classe”. Per convenció en MAJÚSCULES.

```
public class Config {
    public static final String NOMBRE_APP = "Gestión DAM";
    public static final String VERSION = "2.1.0";
    public static final int MAX_USUARIOS = 100;
    public static final double IVA = 0.21;
}
```

Usa-les així: Config.IVA, Config.MAX_USUARIOS. Mai canviïn. Són més fermes que els teus propòsits d'Any Nou.

★ BE THE CODE, MY FRIEND: Els Gats Estàtics

☞ **Don Tip:** Allò static pertany a la classe, no a l'objecte. Tots els objectes compartixen el mateix valor.

Mira este codi i respon SENSE EXECUTAR:

```
public class Gato {
    public static int totalGatos = 0;
    public String nombre;

    public Gato(String nombre) {
        this.nombre = nombre;
        totalGatos++;
    }

    public static void decirTotal() {
        System.out.println("Hay " + totalGatos + " gatos");
        // System.out.println(nombre); // ¿Esto funciona?
    }
}
```

- Funciona System.out.println(nombre); dins del mètode decirTotal()?
- Si crees 3 gats i després fas Gato.decirTotal(), què imprimeix?
- I si crees 5 gats més? Què imprimeix ara?

Solució: NO funciona (és una variable d'instància), imprimeix 3, imprimeix 8. El mètode decirTotal() no sap quin “nombre” demanar-li a l'objecte.

★ BE THE CODE, MY FRIEND: Static vs Instància

💡 **Don Tip:** Un mètode static no pot accedir a variables d'instància perquè no té this. Pensa: ¿pertany a l'objecte o a la classe?

Què imprimeix este codi?

```
public class PruebaStatic {
    int x = 1;
    static int y = 1;

    public void incrementarX() { x++; }
    public static void incrementarY() { y++; }

    public static void main(String[] args) {
        PruebaStatic a = new PruebaStatic();
        PruebaStatic b = new PruebaStatic();

        a.incrementarX();
        b.incrementarX();
        a.incrementarX();

        PruebaStatic.incrementarY();
        PruebaStatic.incrementarY();
        b.incrementarY(); // también vale aunque sea confuso

        System.out.println("a.x = " + a.x);
        System.out.println("b.x = " + b.x);
        System.out.println("y = " + PruebaStatic.y);
    }
}
```

Solució: a.x = 3 (es va incrementar 2 voltes en a), b.x = 2 (b.incrementarX() una volta), y = 3 (es va incrementar 3 voltes: dos des de classe, una des d'objecte b). Cada objecte té el seu propi x, però y és compartit.

? No Hi Ha Preguntes Tontes!

Q: Puc cridar a un mètode estàtic des d'un objecte? Com `miObjeto.metodoEstatico()`?

A: Tècnicament Sí. Java t'ho permet. Però és com cridar a ta mare pel cognom. Funciona, però queda estrany. La convenció és usar la classe: `Clase.metodoEstatico()`. Alguns IDEs et marquen una warning.

Q: Aleshores `main` és estàtic perquè...

A: Perquè quan comença el programa, NO hi ha cap objecte encara. Algú ha d'arrancar la festa. `main` és el primer en arribar. Ha de ser estàtic per a poder executar-se sense que ningú haja creat un objecte abans.

Q: Els mètodes estàtics són més ràpids?

A: Lleugerament. No necessites la referència a l'objecte. Però la diferència és tan xicoteta que en el 99.9% dels casos no ho notaràs. No t'obsessiones amb la velocitat dels estàtics. Preocupa't que el teu codi tinga sentit.

Q: Puc tindre un atribut estàtic que siga un objecte de la seua pròpia classe?

A: Sí! És el patró **Singleton**. Tens un `private static MiClase instancia = new MiClase();` i un mètode `getInstance()`. És com tindre una única pedra filosofal. Però això és un altre tema...

Q: Puc posar-li `static` a tot i estalviar-me crear objectes?

A: Pots, però aleshores no estàs fent POO, estàs fent "Programació Estàtica a la bruta". Java t'ho permet, però és com usar un tornavís per a clavar un clau: pots, però per a això existix el martell. Usa `static` per al que és de la classe, no per a tot.

Diferència Ràpida (Perquè no et llies)

Variable d'instància	Variable estàtica
Pertany a l'objecte	Pertany a la classe
Necessites <code>new</code>	Uses <code>NombreClase.variable</code>
Cada objecte té la seua	Una còpia per a tots
<code>this</code> disponible	No hi ha <code>this</code>
Llig/escriu en l'objecte	Llig/escriu en la classe

Bones Pràctiques

- Usa `static` per a mètodes que NO depenguen de l'estat de l'objecte (com `Math.sqrt()`).
- Usa `static` per a constants (`static final`).

- Usa `static` per a comptadors compartits.
- NO abuses de `static`. No convertixques tot en estàtic “perquè és més fàcil”. Perdràs els beneficis de la POO.
- Constructor privat en classes utilitàries (perquè ningú les instanciï).

✖ L'EMBOLIC

El següent codi hauria d'encapsular correctament una classe `CuentaBancaria`, però té diversos errors de visibilitat i lògica. Troba'ls:

```
public class CuentaBancaria {  
    public double saldo;  
    private String titular;  
  
    public CuentaBancaria(String titular, double saldoInicial) {  
        titular = titular;  
        saldo = saldoInicial;  
    }  
  
    public double getSaldo() {  
        return saldo;  
    }  
  
    private void setSaldo(double saldo) {  
        if (saldo >= 0) {  
            this.saldo = saldo;  
        }  
    }  
}
```

Preguntes:

1. ¿L'atribut `saldo` està ben encapsulat?
2. ¿Quin error hi ha al constructor amb `titular = titular`?
3. ¿Per què `setSaldo` és `private`? Com retira diners l'usuari llavors?

💡 **Don Tip:** `this` resol ambigüitats. Si paràmetre i atribut es diuen igual, sense `this` t'assignes el paràmetre a si mateix.

Resum

- Visibilitat: `public` > `protected` > `package-private` > `private`.
 - Encapsulació: atributs `private`, getters/setters `public` amb validació.
 - JavaBeans: classe pública, constructor sense args, atributs privats, getters/setters.
 - `static` = del grup (classe). Sense `static` = de cada un (objecte).
 - Mètodes estàtics no accedixen a atributs d'instància ni tenen `this`.
 - `static final` = constant de classe.
 - Classes utilitàries tenen constructor privat i només membres estàtics.
-

Exercicis Proposats

Visibilitat

1. **Classe Empleat amb validació:** `Empleado` amb `nombre`, `salarioBase` (`double`) i `departamento` privats. El salari no pot ser negatiu. El setter ha de llançar `IllegalArgumentException` si és invàlid. Mètode `calcularSalarioAnual()`.
2. **Classe Cercle encapsulat:** `Circulo` amb `radio` privat. Setter valida que el radi siga positiu. `getArea()` i `getPerimetro()`. Intenta accedir a `radio` des d'una altra classe. Què passa?
3. **JavaBean Alumne:** `Alumno` amb `nombre`, `edad`, `curso`, `notaMedia` (`double`) i `matriculado` (`boolean`). Constructor sense args: nom buit, `matriculado` false. Nota mitjana entre 0 i 10.
4. **Classe Immutable Hora:** `Hora` sense setters. Només constructor amb validació. Getters i `toHoraString()`. Ningú pot canviar l'hora després de crear-la.
5. **Protected i herència:** Crea `Animal` en paquet `zoologico` amb `protected String nombre`. Crea `Perro` en un altre paquet. Qui veu `nombre`?
6. **Compte Bancari encapsulat:** `CuentaBancaria` amb `saldo` privat. Només `ingresar(double)` i `retirar(double)` modifiquen el saldo. Validacions.
7. **Getter sense Setter:** Classe `Configuracion` amb `static final String VERSION = "1.0"` (públic). Atributs privats `maxIntentos` i `timeout`. Getter per a tots dos, setter només per a `maxIntentos`. Per què?

8. **Refactorització:** Et donen Coche amb atributs públics marca, modelo, anio. Refactoritza: fes-los privats, afegir getters/setters validant que l'any estiga entre 1886 i any actual + 1.

Static

1. **Comptador d'objectes:** Classe Usuario amb comptador estàtic d'objectes creats, id autoincremental, `static final String DOMINIO_EMAIL = "@dam.com"` i mètode `generarEmail()` que torne `nombre + DOMINIO_EMAIL`.
2. **Classe utilitària OperacionsArray:** Classe OperacionesArray amb constructor privat i mètodes: `sumar(int[])`, `media(double[])`, `maximo(int[])`, `minimo(int[])`, `estaOrdenado(int[])`, `buscar(int[], int)`. Usa sense instanciar.
3. **Conversor d'unitats:** Classe Conversor amb constants `KM_A_MILLAS = 0.621371`, `LIBRA_A_KG = 0.453592`, etc. Mètodes: `kmAMillas`, `millasAKm`, `celsiusAFahrenheit`, `fahrenheitACelsius`, `librasAKg`, `kgALibras`.
4. **Simulació d'aleatoris:** Classe Aleatorio amb mètodes: `entero(int min, int max)`, `decimal(double min, double max)`, `booleano()`, `colorHex()`, `elemento(String[])`.
5. **Classe Config amb constants:** Classe Config amb `MAX_INTENTOS_LOGIN = 3`, `TIMEOUT_SEGUNDOS = 300`, `RUTA_LOG = "./logs/app.log"`. Atribut privat `contadorAccesos` amb `incrementarAcceso()` i `getAccesos()`.
6. **Validador de dades:** Classe utilitària Validador amb: `esEmailValido(String)`, `esTelefonoValido(String)` (9 dígit), `esDNIValido(String)` (8 dígit + lletra), `esFechaValida(int, int, int)`.
7. **Estadístiques de notes:** Programa que donades notes `double[]` calcule amb mètodes estàtics: mitjana, màxima, mínima, nombre d'aprovat (≥ 5) i desviació típica. Classe Estadisticas.
8. **Joc de daus:** Simula llançar dos daus 1000 voltes amb `Math.random()`. Classe Dado amb mètode `lanzar()` (1-6). Compta quantes voltes ix cada suma (2-12). Classe Simulacion amb estructura estàtica.

RAs treballats en esta unitat:

- **RA4** - Classes (visibilitat, encapsulació)

- **RA7** - Estaticitat
-



Sergi Garcia Barea — CC BY-SA 4.0

Unidad 8: Herencia, Polimorfismo e Interfaces

Objectius d'aprenentatge

- Heredar d'una classe amb `extends`
- Usar `super` per a accedir a membres de la superclasse
- Sobreescriure mètodes amb `@Override`
- Diferenciar IS-A vs HAS-A
- Entendre el polimorfisme dinàmic (dynamic binding)
- Usar `instanceof` i downcasting correctament
- Aplicar polimorfisme amb col·leccions i paràmetres
- Dissenyar classes abstractes amb mètodes abstractes i concrets
- Implementar interfícies amb `implements`
- Diferenciar quan usar `abstract class` vs `interface`
- Usar mètodes `default` en interfícies
- Conèixer les interfícies funcionals (`Consumer`, `Supplier`, `Predicate`, `Function`)
- Sobreescriure `toString()`, `equals()` i `hashCode()` de la classe `Object`
- Construir jerarquies de classes completes

Herència: Quan els Teus fills Segueixen els Teus Passos (Però Millor)

Recordes quan vas heretar el nas de l'àvia o el geni per a enfadar als professors? Doncs en Java passa el mateix, però amb menys drama i més codi reutilitzable.

L'herència és el mecanisme pel qual una classe *filla* (subclasse) obté tots els membres d'una classe *pare* (superclasse). I pot afegir els seus propis o *millorar* els existents.

`extends`: “Sóc com tu, però amb millores”

Useu `extends` per a dir-li a Java que una classe és filla d'una altra:

```

public class Animal {
    protected String nombre;
    protected int edad;
    public void hacerSonido() {
        System.out.println("Algún sonido genérico...");
    }
}

public class Perro extends Animal {
    public void hacerSonido() {
        System.out.println("¡Guau guau!");
    }
    public void moverCola() {
        System.out.println("*mueve la cola felizmente*");
    }
}

```

Perro ara té nombre, edad, hacerSonido() (millorat) i moverCola(). Cortesia de l'herència.



Consell: Codi reutilitzat, neurones estalviades. L'herència existix perquè NO hages de copiar i enganxar el mateix codi en 15 classes.

super: Cridant a mamà/papà perquè t'ajuden

De vegades volem *estendre* el mètode del pare, no reemplaçar-lo. Ahí entra super:

```

public class Gato extends Animal {
    @Override
    public void hacerSonido() {
        super.hacerSonido();
        System.out.println("¡MIAU!");
    }
}

```

super és com cridar “¡MAAAAAMÁ!” en el supermercat. Li dius a Java: “executa la versió del meu pare, i després jo faig la meua”.



Advertència: super només servix per a invocar constructors i mètodes de la superclasse. No pots passar-lo com a paràmetre ni assignar-lo a una variable.

@Override: “Papa, jo ho faig millor”

@Override li diu al compilador: “assegura’t que realment estic sobreescrivint un mètode del pare”:

```
public class Pez extends Animal {  
    @Override  
    public void hacerSonido() { } // ✓ existe en Animal  
    @Override  
    public void nadar() { }      // X ERROR: no existe en Animal  
}
```

 **Nota:** Posem sempre @Override. No és obligatori, però és com el cinturó de seguretat: no passa res si no ho fas... fins que passa.

La regla d'or: IS-A vs HAS-A

- **IS-A** (és-un): Relació d'herència. Perro IS-A Animal.
- **HAS-A** (té-un): Relació de composició. Coche HAS-A Motor.

```
public class Coche extends Vehiculo { } // IS-A ✓  
public class Coche { private Motor m; } // HAS-A ✓
```

¿Who Wants to Be a Millionaire? — Edició Java: Quina és la relació correcta? a) Cliente extends Persona b) Cliente has-a Persona c) Coche extends Rueda **Resposta:** La a. Cliente IS-A Persona. Coche NO és una Rueda, té rodes (HAS-A).

Jerarquia de classes: L'arbre genealògic

```
public class Animal { }  
public class Mamifero extends Animal { }  
public class Canino extends Mamifero { }  
public class Perro extends Canino { }
```

Perro hereta de Canino, que hereta de Mamifero, que hereta de Animal.



protected: El membre que només la família veu

```
public class Animal {  
    private String secreto;    // Solo Animal  
    protected String familia; // Animal y subclases  
    public String nombre;     // Todos  
}
```

★ BE THE CODE, MY FRIEND:

👁 Don Tip: super sempre crida al mètode del pare. Si no l'uses, estàs sobreescrivint completament.

```
public class Animal {  
    private String idSecreto = "X-123";  
    protected String nombre = "Animal";  
    public int edad = 5;  
}  
  
public class Perro extends Animal {  
    public void mostrar() {  
        System.out.println(idSecreto); // ¿compila?  
        System.out.println(nombre);    // ¿compila?  
        System.out.println(edad);      // ¿compila?  
    }  
}
```

Resposta: idSecreto NO (private), nombre sí (protected), edad sí (public).

Què imprimix?

```

class Abuelo { void decir() { System.out.println("Abuelo"); } }
class Padre extends Abuelo { void decir() { System.out.println("Padre"); } }
class Hijo extends Padre { void decir() { System.out.println("Hijo"); } }
public class Test {
    public static void main(String[] args) { new Hijo().decir(); }
}

```

Resposta: “Hijo”. Java busca el mètode des de la classe més específica cap amunt.

Què imprimix amb herència encadenada i super?

```

class Vehiculo {
    void describir() { System.out.println("Soy un vehículo"); }
}
class Coche extends Vehiculo {
    void describir() { System.out.println("Soy un coche"); }
    void describirCompleto() { super.describir(); this.describir(); }
}
class Deportivo extends Coche {
    void describir() { System.out.println("Soy un coche deportivo"); }
}
public class Test {
    public static void main(String[] args) {
        Deportivo d = new Deportivo();
        d.describirCompleto();
        d.describir();
    }
}

```

Resposta: “Soy un vehículo”, “Soy un coche deportivo”, “Soy un coche deportivo”.
 super.describir() va a Vehiculo, this.describir() es resol en runtime com Deportivo.

? No hi ha Preguntes Tontes!

Q: Puc heredar de diverses classes ahora? **A:** No. Java no permet herència múltiple (el *problema del diamant*). Però existixen les interfícies per a això.

Q: Classe final? Mètode final? **A:** Classe final no pot tindre filles (String). Mètode final no pot sobreescriure's.

Q: Atribut amb el mateix nom en subclasse i superclasse? **A:** El de la subclasse *oculta* el de la superclasse. Però els atributs NO són polimòrfics com els mètodes. Amb referència de la superclasse, veus el de la superclasse.

Q: Els constructors s'hereden? **A:** No. Però la subclasse crida al constructor del pare amb `super()`. Si no ho poses, Java posa `super()` automàticament.

El problema de la classe base fràgil

Tens Jarrón amb `romper()`. JarrónChino ho sobreescriu. Algú afegix `caerAlSuelo()` a Jarrón que crida a `romper()`. De sobte JarrónChino es comporta inesperadament. És el *fragile base class problem*.

Q: Llavors l'herència és dolenta? **A:** No! És una ferramenta. Ben usada és perfecta per a relacions IS-A clares. El problema és usar-la quan una simple composició bastaria.

EL LÍO

El següent codi d'herència està malament. Troba els errors:

```

public class Vehiculo {
    private String marca;

    public Vehiculo(String marca) {
        this.marca = marca;
    }

    public void arrancar() {
        System.out.println("Vehículo arrancado");
    }
}

public class Coche extends Vehiculo {
    private int puertas;

    public Coche(String marca, int puertas) {
        this.puertas = puertas;
    }

    @Override
    public void arrancar() {
        System.out.println("Coche arrancado con " + puertas + " puertas");
    }
}

```

Quin error impeditx que Coche compile? Per què el constructor de Coche falla?

💡 **Don Tip:** Si el pare té un constructor amb paràmetres, el fill ha de cridar-lo amb `super()`. Si no, el compilador intenta cridar el constructor buit del pare... que no existix.

Resum visual de l'herència

Concepte	Traducció Java
Ta mare et dona els seus gens	<code>class Filla extends Mare</code>
Li demanes ajuda al teu pare	<code>super.metodo()</code>
“Jo ho faig millor”	<code>@Override</code>
El secret que només sap la família	<code>protected</code>

Exercicis Proposats - Herència

1. **La família Simpson:** Classe base `IntegranteFamilia` amb nombre i edat. Homero, Marge, Bart, Lisa que hereden. Cada una amb un mètode únic (`comerRosquilla()`, `decirAyeCaramba()`).
 2. **Vehicles terrestres:** `Vehiculo` → `Coche`, `Moto`, `Bicicleta`. Tots amb `mover()`.
 3. **El problema del gerro:** Base amb `a()` i `b()` (`b` crida a `a`). Derivada sobreescriu `a()`. Crida a `b()`. Què ocorre?
 4. **Biblioteca de mitjans:** `Medio` → `Libro`, `Pelicula`, `Disco`. Tots amb `mostrarInfo()`.
 5. **El joc dels animals:** Jerarquia d'animals amb almenys 4 nivells.
-

Polimorfisme: El Camaleó de la POO

Polimorfisme (*poly* = moltes, *morphé* = formes): un mateix mètode es comporta diferent segons l'objecte que l'invoque. Com un comandament universal on el mateix botó "PLAY" funciona en Netflix, Spotify i la teua torradora.

El mateix mètode, diferents comportaments

```
public class Animal {
    public void hacerSonido() { System.out.println("..."); }
}
public class Perro extends Animal {
    @Override
    public void hacerSonido() { System.out.println("¡Guau!"); }
}
public class Gato extends Animal {
    @Override
    public void hacerSonido() { System.out.println("¡Miau!"); }
}
```

Màgia:


```
Animal a;  
a = new Perro(); a.hacerSonido(); // ¡Guau!  
a = new Gato(); a.hacerSonido(); // ¡Miau!
```

La variable `a` és tipus `Animal`, però apunta a objectes diferents. S'executa el mètode de l'objecte real.

Dynamic Binding

El compilador verifica que `Animal` tinga `hacerSonido()`. Però *quina* implementació s'executa es decidix en runtime. Això és **dynamic binding** o **late binding**.

Compilador: “`Animal` té `hacerSonido()`? Sí. Endavant.” **Runtime:** “L'objecte és un `Perro`. Execute el de `Perro`.”

Referències polimòrfiques

```
Animal miMascota = new Perro(); // ✓  
Animal tuMascota = new Gato(); // ✓
```

Però la variable només “veu” els mètodes del tipus de la referència:

```
Animal a = new Perro();  
a.hacerSonido(); // ✓  
a.moverCola(); // ✗ Animal no tiene moverCola()
```

Polimorfisme amb col·leccions

```
ArrayList<Animal> animales = new ArrayList<>();  
animales.add(new Perro());  
animales.add(new Gato());  
animales.add(new Vaca());  
  
for (Animal a : animales) {  
    a.hacerSonido(); // Cada uno el suyo  
}  
// ¡Guau! / ¡Miau! / ¡Muuu!
```

Una sola llista, un sol bucle. Sense polimorfisme necessaries quatre llistes.

Polimorfisme amb paràmetres

```
public class Veterinario {
    public void vacunar(Animal a) {
        System.out.print("Vacunando a: ");
        a.hacerSonido();
    }
}

Veterinario vet = new Veterinario();
vet.vacunar(new Perro()); // Vacunando a: ¡Guau!
vet.vacunar(new Gato());  // Vacunando a: ¡Miau!
```

I pots anar més lluny:

```
public class Zoologico {
    private ArrayList<Animal> animales = new ArrayList<>();
    public void agregarAnimal(Animal a) { animales.add(a); }
    public void hacerDesfile() {
        for (Animal a : animales) {
            System.out.print(a.getClass().getSimpleName() + " dice: ");
            a.hacerSonido();
        }
    }
}
```

Afegir un Gato o un Perro no requerix canviar una línia del Zoológico.

instanceof: “Qui eres realment?”

```
Animal a = new Perro();
if (a instanceof Perro) {
    System.out.println("¡Es un perro!");
}
```



Consell: Usa-ho amb moderació. Si uses instanceof constantment, alguna cosa estàs fent mal. El polimorfisme hauria de resoldre-ho sense preguntar.

Downcasting: Perillós però de vegades necessari

```
Animal a = new Perro();
Perro p = (Perro) a; // Downcasting
p.moverCola();        // ✓

Animal a2 = new Gato();
Perro p2 = (Perro) a2; // ClassCastException en runtime
```

Sempre amb instanceof primer:

```
if (a instanceof Perro) {
    Perro p = (Perro) a;
    p.moverCola();
}
```

★ BE THE CODE, MY FRIEND:

👁️ **Don Tip:** El compilador mira el tipus de la variable, la JVM mira el tipus real de l'objecte. Ahí està la màgia del polimorfisme.

```
class A { void saluda() { System.out.println("Hola desde A"); } }
class B extends A { void saluda() { System.out.println("Hola desde B"); } }
class C extends B { void saluda() { System.out.println("Hola desde C"); } }
public class Test {
    public static void main(String[] args) {
        A ref1 = new B(); A ref2 = new C(); B ref3 = new C();
        ref1.saluda(); ref2.saluda(); ref3.saluda();
    }
}
```

Solució: Hola desde B, Hola desde C, Hola desde C. Només importa el tipus real de l'objecte.

```

class Vehiculo { void mover() { System.out.println("Vehículo se mueve"); } }
class Coche extends Vehiculo {
    void mover() { System.out.println("Coche acelera"); }
    void abrirPuertas() { System.out.println("Puertas abiertas"); }
}
public class Test {
    public static void main(String[] args) {
        Vehiculo v = new Coche();
        v.mover();
        // v.abrirPuertas(); <- ¿compila?
    }
}

```

Solució: “Coche acelera”. La línia comentada NO compila: el compilador només mira el tipus de la referència.

```

interface Volable { void volar(); }
class Pajaro implements Volable {
    public void volar() { System.out.println("Vuela"); }
}
class Aguila extends Pajaro {
    public void volar() { System.out.println("Vuela alto"); }
}
public class Test {
    public static void main(String[] args) {
        Volable v = new Aguila();
        Pajaro p = new Aguila();
        Object o = new Aguila();
        v.volar(); p.volar(); // ¿y o?
    }
}

```

Solució: Els tres imprimixen “Vuela alto”. El tipus de la referència no importa.

? No hi ha Preguntes Tontes!

Q: Per què no puc fer `Perro p = new Animal();` **A:** Perquè `Animal` podria no tindre els mètodes de `Perro`. Què passa si crides a `moverCola()` i `Animal` no té cua?

Q: Per a què servix declarar variables del tipus més genèric? **A:** Per a ser flexible. `ArrayList<Animal>` accepta qualsevol subclasse. `ArrayList<Perro>` només Perros.

Q: El polimorfisme funciona amb atributs? **A:** No. Els atributs no són polimòrfics. Els mètodes sí; els atributs, no.

EL RING: extends vs implements

Dues paraules clau discuteixen sobre qui és més important.

extends: «Jo soc l'herència pura. Codi reutilitzat, una jerarquia clara. Gos extend Animal. Cotxe extend Vehicle. ¡Soc la base de la POO!»

implements: «Sí, però amb mi no hi ha límits. Una classe pot implementar diverses interfícies. Amb extends només tens un pare. Jo et permeto ser varies coses a la volta: Serializable, Comparable, Cloneable...»

extends: «Les meues classes poden tindre codi ja fet. Tu només declares mètodes buits. ¡Jo aporte implementació!»

implements: «Des de Java 8 tinc mètodes default i static. Mira: `default void log()` ja funciona. A més, soc més flexible: no imposo una jerarquia rígida.»

extends: «Val, però sense mi les interfícies no tindrien sentit. Una interfície no pot instanciar-se sola.»

implements: «I sense mi tindries herència múltiple, que és un caos. Mira el problema del diamant en C++.»

extends: «Ens necessitem.»

implements: «Sí. extends per a la jerarquia, implements per als contractes.»

💡 **Don Tip:** Usa extends per a reutilitzar codi (IS-A). Usa implements per a definir capacitats (contractes). Es poden combinar: `class Gos extends Animal implements Mascota, Jugable.`

Resum visual del polimorfisme

Concepte	Traducció
Un mètode, mil formes	Override + dynamic binding
Variable genèrica → objecte específic	<code>Animal a = new Perro()</code>
Decidix en execució	Dynamic binding

Concepte	Traducció
Preguntar qui eres	<code>instanceof</code>
Convertir a la força	Downcasting (Perro) a

Exercicis Proposats - Polimorfisme

- Orquestra polimòrfica:** Instrumento amb `tocar()`. Guitarra, Piano, Bateria, Flauta.
- Calculadora de figures:** Figura amb `calcularArea()`. Circulo, Rectangulo, Triangulo. Usa `ArrayList<Figura>`.
- Downcasting segur:** Empleado → Programador (`escribirCodigo`), Diseñador (`disenar`). Recorre'ls amb `instanceof`.
- Batalla de personatges:** Personaje → Guerrero, Mago, Arquero. Mètodes: `atacar()`, `recibirDano()`.

La Classe Object: El Besavi de Tot

Totes les classes hereden d'`Object`. Sempre.

```
public class MiClase { } // = public class MiClase extends Object { }
```

Mètodes que tota classe hereta:

Mètode	Què fa?	Sobreescriure'l?
<code>toString()</code>	Representació textual	Quasi sempre
<code>equals(Object)</code>	Compara per valor	Quan tinga sentit
<code>hashCode()</code>	Codi hash	Amb equals
<code>getClass()</code>	Classe de l'objecte	No

`toString()`: La targeta de presentació

Per defecte: `MiClase@1a2b3c`. Sobreescriu-lo:

```
public class Perro {
    private String nombre;
    private int edad;
    @Override
    public String toString() {
        return "Perro{nombre='" + nombre + "', edad=" + edad + "}";
    }
}
System.out.println(new Perro("Firulais", 3));
// Perro{nombre='Firulais', edad=3}
```

`equals()`: Mateix objecte o iguals?

Per defecte compara *referències*. Per a comparar per *valor*:

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    Perro perro = (Perro) o;
    return edad == perro.edad && Objects.equals(nombre, perro.nombre);
}
```

`hashCode()`: El codi de barres

Si dos objectes són iguals segons `equals()`, han de tindre el mateix `hashCode()`:

```
@Override
public int hashCode() { return Objects.hash(nombre, edad); }
```

 **Advertència:** Si sobreescrus `equals()`, HAS de sobreescriure `hashCode()`. Si no, `HashSet/HashMap` es comporten impredeciblement.

BE THE CODE, MY FRIEND:

 **Don Tip:** Els mètodes abstractes són contractes: la subclasse HA d'implementar-los. Si no, no compila.

```
Empleado e1 = new Programador("Ana", "001", 2500, "Java");
Empleado e2 = new Programador("Ana", "001", 2500, "Java");
Empleado e3 = new Gerente("Ana", "002", 3000, 500);
System.out.println(e1);
System.out.println(e1.equals(e2));
System.out.println(e1.equals(e3));
System.out.println(e1.hashCode() == e2.hashCode());
```

Solució: Programador: Ana (ID: 001), true (mateix id), false (distinta classe), true.

```
class Perro extends Animal { Perro(String n) { super(n); } }
class Gato extends Animal { Gato(String n) { super(n); } }
System.out.println(new Perro("Firulais").equals(new Gato("Firulais")));
```

Resposta: false. getClass() torna classes diferents.

? No hi ha Preguntes Tontes!

Q: Sobrescriu equals() sempre? **A:** No. Només quan necessites comparar per valor lògic. Dos Persona amb el mateix DNI sí, dos Scanner no.

Q: I clone()? **A:** Complicat. Per defecte còpia superficial. Millor un constructor de còpia.

Classes Abstractes: El Esbós Que No Pots Usar Directament

Imagina “A la venda: Esbós de cadira”. No pots seure’t en un esbós. Doncs això són les classes abstractes: plànols incomplets perquè *altres* els completen.

Què és una classe abstracta?

Una classe que NO pot instanciar-se. Pot tindre mètodes abstractes (sense implementació) i concrets (amb implementació):


```
public abstract class Animal {
    protected String nombre;
    public abstract void hacerSonido();
    public void dormir() {
        System.out.println(nombre + " está durmiendo... Zzz");
    }
}
```

You MUST: Implementar els mètodes abstractes

Si una classe concreta estén una abstracta, està OBLIGADA a implementar tots els mètodes abstractes:

```
public class Perro extends Animal {
    @Override
    public void hacerSonido() { System.out.println("¡Guau!"); }
}

public abstract class Pajaro extends Animal { } // No obligada: también abstracta
```

abstract vs concrete

Classe abstracta	Classe concreta
No pots crear objectes directament	Pots crear objectes
Pot tindre mètodes abstractes	Tots implementats
Concepte general	Alguna cosa específica
abstract class	Només class

Exemple: Figures geomètriques

```

public abstract class Figura {
    protected String color;
    public Figura(String color) { this.color = color; }
    public abstract double calcularArea();
    public abstract double calcularPerimetro();
    public void mostrarColor() { System.out.println("Color: " + color); }
}

public class Circulo extends Figura {
    private double radio;
    public Circulo(String color, double radio) { super(color); this.radio = radio; }
    @Override public double calcularArea() { return Math.PI * radio * radio; }
    @Override public double calcularPerimetro() { return 2 * Math.PI * radio; }
}

public class Rectangulo extends Figura {
    private double ancho, alto;
    public Rectangulo(String color, double ancho, double alto) { super(color);
this.ancho = ancho; this.alto = alto; }
    @Override public double calcularArea() { return ancho * alto; }
    @Override public double calcularPerimetro() { return 2 * (ancho + alto); }
}

```

Constructors en classes abstractes

Sí, les abstractes poden tindre constructors. Es criden amb `super()`:

```

public abstract class Animal {
    protected String nombre;
    public Animal(String nombre) {
        this.nombre = nombre;
        System.out.println("Constructor de Animal");
    }
}

public class Perro extends Animal {
    public Perro(String nombre) { super(nombre); }
}

```

Template Method: Les abstractes en acció

Definixes l'esquelet d'un algoritme i deixes que les subclasses ompliquen els detalls:

```

public abstract class Bebida {
    public final void preparar() {
        hervirAgua();
        prepararIngrediente();
        servirEnTaza();
        añadirExtras();
    }
    private void hervirAgua() { System.out.println("Hirviendo agua..."); }
    private void servirEnTaza() { System.out.println("Sirviendo en taza..."); }
    protected abstract void prepararIngrediente();
    protected abstract void añadirExtras();
}
public class Te extends Bebida {
    @Override protected void prepararIngrediente() { System.out.println("Poniendo la
bolsita de té..."); }
    @Override protected void añadirExtras() { System.out.println("Añadiendo limón..."); }
}
}

```

★ BE THE CODE, MY FRIEND:

👁 **Don Tip:** implements és un contracte. La classe es compromet a tindre tots els mètodes de la interfície.

```

public abstract class A { public abstract void metodo(); }
public class B extends A { }

```

Resposta: NO compila. B ha d'implementar els mètodes abstractes de A.

```

public abstract class A { public abstract void metodo(); }
public class B extends A { public void metodo() { } }
A a = new A(); // ¿Error?
B b = new B(); // ¿Error?

```

Resposta: new A() ERROR (abstracta), new B() OK.

```
abstract class Animal { abstract void hablar(); }
abstract class Mamifero extends Animal { void hablar() { System.out.println("Mamífero raro"); } }
class Gato extends Mamifero { void hablar() { System.out.println("Miau"); } }
class GatoPersa extends Gato { }
public class Test {
    public static void main(String[] args) { Animal a = new GatoPersa(); a.hablar();
}
}
```

Resposta: “Miau”. Dynamic binding.

? No hi ha Preguntes Tontes!

Q: Per què existixen les classes abstractes? **A:** 1) Definir un contracte obligatori. 2) Compartir codi. 3) Modelar conceptes sense instància directa (“Figura”).

Q: Puc tindre una classe abstracta sense mètodes abstractes? **A:** Sí. Per a impedir que s’instancie directament.

Exercicis Proposats – Classes Abstractes

- Selecció abstracta:** Empleado amb nombre, salarioBase, abstracte calcularSalario(), concret mostrarInfo(). Programador i Vendedor.
- Vehicles abstractes:** Vehiculo amb mover() abstracte, detener() concret. Coche, Bicicleta, Avion.
- Figures 3D:** Afig calcularVolumen() abstracte a Figura. Implementa Esfera, Cubo.

Interfícies: El Contracte Que El Teu Codi Firmar

Has firmat un contracte? “El treballador es compromet a: programar en Java, no dormir-se en les reunions...” però no diu COM. Una **interfície** en Java és això: un contracte.

```
public interface Programable {
    void programar();
    void tomarCafe();
    void irReunion(String hora);
}
```

Qualsevol classe que firme el contracte (amb implements) HA d'implementar tots eixos mètodes.

Declarant i Implementant

```
public interface Reproducible {
    void reproducir();
    void pausar();
    void detener();
    int obtenerDuracion();
}

public class Cancion implements Reproducible {
    private String titulo;
    public Cancion(String titulo) { this.titulo = titulo; }
    @Override public void reproducir() { System.out.println(" 🎵 Reproduciendo: " + titulo); }
    @Override public void pausar() { System.out.println(" ⏸ Canción pausada"); }
    @Override public void detener() { System.out.println(" 🛑 Canción detenida"); }
    @Override public int obtenerDuracion() { return 240; }
}
```

Polimorfisme amb Interfícies

```
public class Reproductor {
    public static void main(String[] args) {
        List<Reproducible> lista = new ArrayList<>();
        lista.add(new Cancion("Bohemian Rhapsody"));
        lista.add(new Pelicula("Inception"));
        for (Reproducible r : lista) {
            r.reproducir(); // No sabe si es canción o película
        }
    }
}
```

Múltiples Interfícies

Una classe només estén UNA classe però pot implementar VARIES interfícies:

```
public interface Nadador { void nadar(); }
public interface Corredor { void correr(); }
public class Triatleta implements Nadador, Corredor {
    public void nadar() { System.out.println("🏊 Nadando 1.5 km"); }
    public void correr() { System.out.println("🏃 Corriendo 10 km"); }
}
```

default Methods: Pegats Sense Trencar Res

Abans de Java 8, afegir un mètode a una interfície trencava totes les implementacions. Arribaren els mètodes default:

```
public interface Volable {
    void volar();
    default void despegar() { System.out.println("✈ Despegando..."); }
}
public class Avion implements Volable {
    public void volar() { System.out.println("✈ Volando a 900 km/h"); }
    // despegar() ya viene implementada
}
```

★ BE THE CODE, MY FRIEND:

💡 **Don Tip:** toString(), equals() i hashCode() venen d'Object. Sobreescriure'ls bé evita bugs raríssims.

```
public interface Cantante { void cantar(); }
public interface Bailarin { void bailar(); }
public class Artista implements Cantante, Bailarin {
    public void cantar() { System.out.println("canta"); }
    // ¡FALTA bailar()!
}
```

Resposta: NO compila. Implementes TOTS els mètodes.

```
interface Guerrero { default void atacar() { System.out.println("Ataca con
espada"); } }
interface Mago { default void atacar() { System.out.println("Lanza hechizo"); } }
class Personaje implements Guerrero, Mago {
    public void atacar() {
        Guerrero.super.atacar();
        Mago.super.atacar();
        System.out.println("¡Y usa ambas!");
    }
}
```

Resposta: “Ataca con espada”, “Lanza hechizo”, “¡Y usa ambas!”. Conflicte resolt.

? No hi ha Preguntes Tontes!

Q: Per què no usar simplement una classe abstracta? **A:** Una classe només hereda d’UNA classe, però implementa VARIES interfícies. Les interfícies no tenen estat.

Q: Puc instanciar una interfície? **A:** No. Com intentar casar-te amb el concepte d’“amor”.

Q: Dos interfícies amb mètode default del mateix nom? **A:** Conflicte. Obliga a sobreescriure i usar `Interfaz.super.metodo()`.

Q: Atributs en interfícies? **A:** Sí, però són `public static final`. Constants, no atributs d’instància.

Resum: Abstract Class vs Interface

Aspecte	Abstract Class	Interface
Mètodes amb codi	Sí	Sí (default)
Atributs	Qualsevol	<code>public static final</code>
Herència múltiple	No (extends un)	Sí (implements varis)
Constructors	Sí	No

Exercicis Proposats – Interfícies

1. **Interface Encendible:** Encendible amb encender(), apagar(), default estaEncendido(). Television i Lampara.
2. **Múltiples interfícies:** Cantante i Bailarin. Artista implementa ambdues. Demostra polimorfisme.
3. **Sistema de notificaciones:** Notificable amb enviar(String) i getDestinatario(). EmailNotificacion i SMSNotificacion.

Interfícies Functionals: Una Sola Missió

Una interfície funcional té UN SOL mètode abstracte. Java les usa amb lambdes (->):

Interfície	Mètode	Rep	Torna
Consumer<T>	accept(T)	Un valor	void
Supplier<T>	get()	Res	Un valor
Predicate<T>	test(T)	Un valor	boolean
Function<T,R>	apply(T)	Un valor	Un altre valor

```
Consumer<String> imprimir = s -> System.out.println(s);
imprimir.accept("Hola");

Supplier<Double> aleatorio = () -> Math.random();
System.out.println(aleatorio.get());

Predicate<Integer> esPar = n -> n % 2 == 0;
System.out.println(esPar.test(4)); // true

Function<String, Integer> longitud = s -> s.length();
System.out.println(longitud.apply("Java")); // 4
```

Són la base de la programació funcional en Java.

Jerarquia de Classes i Composició

Composició sobre herència: “Tindre vs Ser”

Herència és “IS-A”. Composició és “HAS-A”. Cada volta més gent preferix composició.

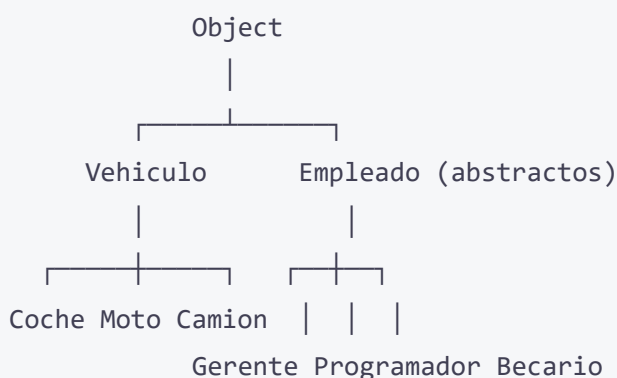
```
// HERENCIA: un coche ES UN vehículo
public class Coche extends Vehiculo { }

// COMPOSICIÓN: un coche TIENE UN motor
public class Coche {
    private Motor motor;
    private Rueda[] ruedas;
}
```

Per què preferir composició?

1. **Menys acoblament:** Canviar la classe pare no trenca les filles.
2. **Més flexible:** Pots canviar les parts en temps d'execució.
3. **Evites jerarquies profundes:** 5 nivells d'herència = 5 nivells de dolor.

La jerarquia completa



Cada fletxa = extends. Object està sempre dalt, encara que no ho escrigues.

? No hi ha Preguntes Tontes!

Q: Fins a quina profunditat hauria de tindre la meua jerarquia? **A:** Com a molt, 2-3 nivells. Jerarquies molt profundes són difícils de mantindre.

Exercicis Proposats – Jerarquies

1. **Jerarquia d'animals completa:** Almenys 3 nivells, amb `toString()`, `equals()` i `hashCode()`. Classe base abstracta.
2. **Sistema de biblioteca:** `ItemBiblioteca` (abstracta) amb `prestar()`, `devolver()`, `toString()` i `equals()` per id. Libro i DVD.
3. **L'empresa completa:** `Departamento` amb nom, cap (Gerente) i llista d'empleats. Mètodes per a agregar, calcular nòmina i llistar.

🌟 L'ENIGMA

Tens tres classes: A, B i C. B extends A, C extends B. Les tres tenen un mètode void `parlar()` que imprimix el seu nom. Al main tens:

```
A obj = new C();
((B) obj).hablar();
((A) obj).hablar();
obj.hablar();
```

Què imprimix cada línia? Per què?

💡 **Don Tip:** El polimorfisme dinàmic significa que el mètode executat depén del tipus REAL de l'objecte, no del tipus de la variable. Pase el que pase amb els casts, l'objecte continua sent un C.

Resum Global

Els 4 pilars de la POO en Java:

Pilar	Què és	Com es fa en Java
Herència	Una classe obté atributs/mètodes d'una altra	<code>extends</code>
Polimorfisme	Un mateix mètode es comporta diferent segons l'objecte	<code>@Override</code> + dynamic binding
Abstracció	Ocultar detalls, mostrar només l'essencial	Classes abstractes (<code>abstract</code>)
Interfícies	Contracte que les classes han de complir	<code>interface</code> + <code>implements</code>

Herència o interfície?

- Usa **herència** quan hi ha una relació “és-un” clara (Cotxe és-un Vehicle)
 - Usa **interfícies** quan hi ha un comportament comú sense relació jeràrquica (Ànec implements Volable, Cantable)
 - Preferix **composició** sobre herència per a reutilització flexible
-

RAs treballats en esta unitat:

- **RA4** - Classes (herència, polimorfisme, interfícies, abstractes, Object)
 - **RA6** - Estructures de dades (interfícies funcionals)
 - **RA7** - POO Avançat (herència, polimorfisme, dynamic binding, interfícies, classes abstractes, jerarquies)
-



Sergi Garcia Barea — CC BY-SA 4.0

Unidad 8: Arrays y Colecciones

 Objectius d'aprenentatge

- Declarar i recórrer arrays unidimensionals i multidimensionals
- Usar el bucle for-each per a llegir col·leccions
- Elegir la col·lecció adequada segons el problema
- Utilitzar iteradors per a recórrer i modificar col·leccions
- Conèixer les diferències entre ArrayList, LinkedList, HashSet i TreeSet

Arrays: L'Aparcament de Dades

El Problema: Tens 100 Gats i un Sol Nom

Imagina que tens 100 gats i necessites guardar els seus noms. Podries fer això:

```
String gato1 = "Bigotes";  
String gato2 = "Garfield";  
String gato3 = "Misifú";  
// ... 97 l neas despu s ...  
String gato100 = "Calcetines";
```

Per  aleshores arriba el gat 101 i el teu programa es cau. O pitjor: vols saber quants gats comencen amb "M" i has d'escriure 100 if. L'esquena ja et fa mal nom s de pensar-ho.

 **Advert ncia:** Si alguna volta escrius gato1, gato2, gato3, ... gatoN al teu codi, en algun lloc un programador s nior plora. Els arrays existeixen exactament per a aix .

L'Array: El Teu Primer Aparcament

Un array  s com un p rquing de diverses plantes. Cada pla a t  un n mero ( ndex) i a cada pla a nom s caben cotxes del mateix tipus (b , i les seues subclasses).

```
String[] gatos = new String[100];  
// Has creat un parking amb 100 places per a Strings
```

La primera pla a  s la 0, no la 1. Aix  confon a tothom al principi. Accepta-ho.

💡 **Consell:** Pensar en els índexs com a distàncies des de la primera posició. La primera casa està a 0 passos de tu, no a 1.

Com Ficar Coses al Pàrquing

```
String[] gatos = new String[3];
gatos[0] = "Bigotes";
gatos[1] = "Garfield";
gatos[2] = "Misifú";
gatos[3] = "Calcetines"; // ¡BOOM!
```

Què passa en l'última línia? T'estavellaràs.

¡BOOM! L'ArrayIndexOutOfBoundsException

```
int[] numeros = new int[5];
numeros[0] = 10;
numeros[1] = 20;
numeros[2] = 30;
numeros[3] = 40;
numeros[4] = 50;
numeros[5] = 60; // Index 5 out of bounds for length 5
```

⚠️ **Advertència:** L'array és un objecte (està al heap), però la referència està a la pila (stack). Quan passes un array a un mètode, passes la referència, no les dades. Compte! Si modifiques l'array dins del mètode, els canvis afecten a l'original.

El Duo Inseparable: for + Array

Els arrays i els bucles for són com el pa i la mantega. Mai no veuràs l'un sense l'altre.

```
String[] gatos = {"Bigotes", "Garfield", "Misifú", "Calcetines"};

for (int i = 0; i < gatos.length; i++) {
    System.out.println("Gato " + i + ": " + gatos[i]);
}
```

Fixa't: `gatos.length` NO porta parèntesis. No és un mètode, és un atribut. Els Strings usen `length()`. Els arrays usen `length`. És un parany mortal en els exàmens.

★ BE THE CODE, MY FRIEND:

💡 **Don Tip:** Els arrays tenen mida fixa. Una vegada creats, no pots afegir ni traure elements.

```
public class BeTheArray {  
    public static void main(String[] args) {  
        int[] arr = new int[4];  
        arr[0] = 2;  
        arr[1] = 4;  
        arr[2] = 6;  
        arr[3] = 8;  
  
        for (int i = 0; i < arr.length; i++) {  
            arr[i] = arr[i] * 2;  
        }  
  
        System.out.println(arr[2]);  
    }  
}
```

★ BE THE CODE, MY FRIEND

💡 **Don Tip:** Les variables de tipus array guarden una referència, no les dades. Assignar un array a un altre no copia les dades.

Què imprimeix?

- (A) 6
- (B) 8
- (C) 12
- (D) 16

Resposta: (C) 12. L'array original és {2,4,6,8}. Després del bucle, cada element es multiplica per 2: {4,8,12,16}. arr[2] = 12.

★ BE THE CODE, MY FRIEND: L'Array Rebel

💡 **Don Tip:** Quan passes un array a un mètode, passes la referència. Si el modifiques dins, canvia fora.

```
public class BeTheArrayRevelde {  
    public static void main(String[] args) {  
        int[] nums = {1, 2, 3, 4, 5};  
        for (int i = 0; i < nums.length; i++) {  
            nums[i] = nums[i] * nums[i];  
        }  
        System.out.println(nums[2]);  
    }  
}
```

★ BE THE CODE, MY FRIEND

💡 **Don Tip:** Els arrays es passen per referència. Les variables primitives es passen per valor.

Què imprimeix?

- (A) 3
- (B) 6
- (C) 9
- (D) 25

Resposta: (C) 9. S'eleva cada número al quadrat: {1,4,9,16,25}. `nums[2] = 9`.

for-each: La Variant Peresosa

Si no necessites l'índex (només vols llegir els valors), hi ha una sintaxi més curta:

```
String[] gatos = {"Bigotes", "Garfield", "Misifú"};  
  
for (String gato : gatos) {  
    System.out.println("Miau: " + gato);  
}
```

Es llig: “per a cada String gat en gats, fes això”.



Nota: El for-each és de només lectura. No pots modificar l'array original dins del bucle. Bé, pots intentar-ho, però el canvi es perd en l'èter.

Arrays Multidimensionals: El Pàrquing de Diverses Plantes

Necessites una taula amb files i columnes? Usa un array bidimensional.

```
int[][] tabla = new int[3][4]; // 3 files, 4 columnes
tabla[0][0] = 1; // fila 0, columna 0
tabla[1][2] = 5; // fila 1, columna 2
// ...
```

Java també permet “arrays d’arrays” irregulars (cada fila amb diferent nombre de columnes):

```
int[][] irregular = new int[3][];
irregular[0] = new int[2];
irregular[1] = new int[5];
irregular[2] = new int[3];
```

Per a què serveix? Triangles, piràmides, o simplement dades que no formen un rectangle perfecte.

Recórrer un Array 2D

```
int[][] matriz = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};

for (int i = 0; i < matriz.length; i++) {
    for (int j = 0; j < matriz[i].length; j++) {
        System.out.print(matriz[i][j] + " ");
    }
    System.out.println();
}
```

💡 **Consell:** Nomena els índexs d’arrays multidimensionals com `fila` i `col` o `i` i `j`. NO uses `x` i `y` tret que realment treballes amb coordenades. El teu jo del futur t’ho agrairà.

La Classe Arrays: La Teua Navalla Suïssa

Java proporciona la classe `java.util.Arrays` amb mètodes utilitat:


```
import java.util.Arrays;

int[] numeros = {5, 2, 8, 1, 9};

Arrays.sort(numeros);           // {1, 2, 5, 8, 9}
int pos = Arrays.binarySearch(numeros, 5); // 2
int[] copia = Arrays.copyOf(numeros, 3);   // {1, 2, 5}
String texto = Arrays.toString(numeros);   // "[1, 2, 5, 8, 9]"
boolean igual = Arrays.equals(a, b);       // compara dos arrays
Arrays.fill(numeros, 0);                  // tot a zeros
```

⚠ **Advertència:** `array1.equals(array2)` NO compara els elements. Compara si són el MATEIX objecte en memòria. Usa SEMPRE `Arrays.equals()` per a comparar contingut. El teu cap t'ho agrairà.

★ BE THE CODE, MY FRIEND: La Cerca Binària

👁 **Don Tip:** `Arrays.binarySearch()` requereix l'array ORDENAT. Si no, el resultat és imprevisible.

```
import java.util.Arrays;

public class BeTheSort {
    public static void main(String[] args) {
        int[] datos = {42, 17, 8, 99, 3};
        Arrays.sort(datos);

        int indice = Arrays.binarySearch(datos, 42);
        System.out.println(indice);
    }
}
```

★ BE THE CODE, MY FRIEND

👁 **Don Tip:** La cerca binària només funciona en arrays ordenats. No ho oblidés.

Què imprimeix?

- (A) 0
- (B) 3
- (C) 4

- (D) 99

Resposta: (B) 3. Després d'ordenar: {3, 8, 17, 42, 99}. 42 està a l'índex 3.

Arrays i Mètodes: Passant el Testimoni

```
public static void main(String[] args) {  
    int[] edades = {10, 20, 30};  
    modificar(edades);  
    System.out.println(edades[0]); // 99  
}  
  
public static void modificar(int[] arr) {  
    arr[0] = 99; // Modifica l'original perquè és la mateixa referència  
}
```

Això funciona perquè Java passa la referència de l'objecte array per valor. L'array no es copia, només es copia l'adreça de memòria on està.

? No Hi Ha Preguntes Tontes!

? No Hi Ha Preguntes Tontes! P: Per què el primer índex és 0 i no 1? Això no és natural! **R:** Perquè en la memòria, l'índex és el desplaçament des de la direcció base de l'array. El primer element està en la direcció base + 0. Java no va inventar això per a fastidiar-te; ve de les caveres del llenguatge C.

P: Puc tindre un array de mida 0? **R:** Sí. `int[] buit = new int[0];`. No pots ficar res, però no dona error. Útil quan no saps si tindràs dades.

P: Passa alguna cosa si cree un array d'objectes però no els inicialitze? **R:** Els arrays d'objectes s'inicialitzen amb `null`. Si intentes cridar un mètode en un element `null`, obtens `NullPointerException`. No és divertit.

P: Quina diferència hi ha entre `null` i "buit"? **R:** Un array buit (`new int[0]`) existeix però no té elements. Una referència `null` significa que no hi ha cap array. És com la diferència entre un pàrquing buit i un pàrquing que no s'ha construït.

P: Els arrays poden canviar de mida? **R:** No. Són immutables en mida. Si necessites que cresca, crea'n un de nou i copia les dades amb `System.arraycopy()` o `Arrays.copyOf()`.

? No Hi Ha Preguntes Tontes! (Arrays i Memòria)

? **No Hi Ha Preguntes Tontes!** P: Els arrays de primitius guarden els valors al heap o al stack? R: L'objecte array (amb tots els seus elements) està al heap. La variable que el referència està al stack. L'array és un objecte, encara que siga de int.

P: Es pot canviar la mida d'un array després de crear-lo? R: No, els arrays tenen mida fixa. Pensar en ells com a places de pàrquing ja construïdes. Si necessites més, toca construir (crear) un nou pàrquing.

P: Què passa si use `Arrays.sort()` en un array de Strings? R: Els ordena alfabèticament (segons l'ordre natural de String, que és lexicogràfic). Compte amb majúscules: "Zebra" va abans que "abc" perquè les majúscules tenen menor valor Unicode.

🔴 EL RING: Array vs ArrayList

Dues formes d'emmagatzemar dades s'enfronten.

Array: «Jo soc l'original. Ràpid, eficient, directe. Accés $O(1)$ a qualsevol posició. Soc la base de tot!»

ArrayList: «Sí, però tens mida fixa. Una vegada et crees amb 10 posicions, no pots tindre'n 11. Jo creix i encongisc sota demanda. Soc flexible.»

Array: «Però jo soc més ràpid en accés i més lleuger en memòria. ArrayList utilitza un array per dins i afegeix overhead.»

ArrayList: «Cert, però els meus mètodes `add()`, `remove()`, `contains()` em fan molt més còmode. Quantes línies de codi necessites per a afegir un element a un array? Jo una: `lista.add(42)`.»

Array: «Per a dades primitives soc més eficient. `int[]` ocupa menys que `ArrayList<Integer>` per l'autoboxing.»

ArrayList: «Val, per a tipus primitius i rendiment extrem, utilitza arrays. Per a tot lo demás, utilitza'm a mi. Tregua?»

Array: «Tregua.»

👁️ **Don Tip:** Regla pràctica: saps quants elements necessites i no canviarà? Usa array. No ho saps o canviarà? Usa ArrayList.

Resum Ràpid

```
int[] arr = new int[10];           // Parking per a 10 enters
int[] arr2 = {1, 2, 3};           // Inicialització directa
arr.length;                       // Mida (sense parèntesis)
arr[0];                           // Primer element
arr[arr.length - 1];             // Últim element
int[][] mat = new int[3][4];      // Parking de 3 plantes, 4 places
Arrays.sort(arr);                 // Ordenar
Arrays.toString(arr);             // Imprimir bonic
Arrays.equals(a, b);              // Comparar contingut
Arrays.copyOf(arr, n);             // Copiar els primers n
Arrays.binarySearch(arr, val);    // Buscar (requereix ordenació)
```

Col·leccions: Quan un Array Es Queda Xicotet

El Problema: El Teu Array S'ha Quedat Sense Places

Has creat un array de 10 places. Han arribat 11 gats. Què fas?

```
String[] gatos = new String[10]; // 10 places, 11 gats... mal assumpte
```

Amb un array clàssic hauries de crear-ne un de nou, copiar-ho tot, i després afegir el que falta. És com aparcar al carrer perquè el pàrquing està ple.

```
String[] gatosMasGrande = new String[gatos.length + 1];
System.arraycopy(gatos, 0, gatosMasGrande, 0, gatos.length);
gatosMasGrande[gatosMasGrande.length - 1] = "Bigotes Jr.";
gatos = gatosMasGrande; // Ara apunta al nou array
```

Funciona, però és tediós. I si has de borrar un element al mig, és pitjor. Necessites alguna cosa que cresca i s'enculla sola.

El Java Collections Framework (JCF)

Per a això està el JCF: una família de classes i interfícies a `java.util` que manegen grups d'objectes com si foren de goma.

? No Hi Ha Preguntes Tontes!

? No Hi Ha Preguntes Tontes! P: Per què no usar sempre ArrayList i oblidar-me dels arrays?

R: Els arrays són més ràpids i consumeixen menys memòria. Per a un milió d'elements, la diferència es nota. Usa'ls quan sàpies la mida per endavant.

P: Què significa <String>? **R:** És un *generic*. Li diu a la col·lecció: "Només accepte Strings". Si intentes ficar un int, et falla en compilació, no en execució. És la teua xarxa de seguretat.

ArrayList: El Pàrquing que Creix Sol

ArrayList és com un array, però amb superpoders: es redimensiona automàticament quan et passes de capacitat.

```
import java.util.ArrayList;

ArrayList<String> gatos = new ArrayList<>();
gatos.add("Bigotes");      // [Bigotes]
gatos.add("Garfield");     // [Bigotes, Garfield]
gatos.add("Misifú");       // [Bigotes, Garfield, Misifú]
gatos.remove(1);           // [Bigotes, Misifú] - adeu, Garfield
gatos.get(0);              // "Bigotes"
gatos.size();              // 2 (NO length, es size())
gatos.contains("Misifú");  // true
gatos.indexOf("Bigotes");  // 0
```

⚠ Advertència: ArrayList usa size(), no length. Array usa length, no size(). String usa length(), no length ni size(). Cadascú té la seua pròpia forma de preguntar quant mesura. És un parany en el 90% dels exàmens.

ArrayList NO guarda primitius

No pots fer ArrayList<int>. Els genèrics només funcionen amb objectes. Usa les classes wrapper:

```
ArrayList<Integer> numeros = new ArrayList<>();
numeros.add(42);           // autoboxing: int → Integer
int n = numeros.get(0);    // unboxing: Integer → int
```

Des de Java 5, l'autoboxing/unboxing és automàtic, però per dins continua havent-hi objectes Integer.

★ BE THE CODE, MY FRIEND: L'ArrayList Misteriós

💡 **Don Tip:** ArrayList creix sol. No necessites definir mida, però cada operació té el seu cost.

```
import java.util.ArrayList;

public class BeTheList {
    public static void main(String[] args) {
        ArrayList<Integer> lista = new ArrayList<>();
        lista.add(10);
        lista.add(20);
        lista.add(30);
        lista.add(1, 15);
        lista.remove(Integer.valueOf(20));

        for (Integer n : lista) {
            System.out.print(n + " ");
        }
    }
}
```

★ BE THE CODE, MY FRIEND

💡 **Don Tip:** ArrayList utilitza arrays per dins. Quan s'ompli, en crea un de nou i copia.

Què imprimeix?

- (A) 10 20 30
- (B) 10 15 30
- (C) 10 15 20 30
- (D) 10 15

Resposta: (B) 10 15 30. S'afeg 15 a l'índex 1 → {10,15,20,30}. Després se borra l'objecte Integer(20) → {10,15,30}.

★ BE THE CODE, MY FRIEND: Suma d'ArrayList

💡 **Don Tip:** Recórrer un ArrayList amb for-each és més llegible que amb índexs.

```
import java.util.ArrayList;

public class BeTheSum {
    public static void main(String[] args) {
        ArrayList<Integer> nums = new ArrayList<>();
        for (int i = 1; i <= 5; i++) {
            nums.add(i * 10);
        }
        int total = 0;
        for (int n : nums) {
            if (n % 20 == 0) {
                total += n;
            }
        }
        System.out.println(total);
    }
}
```

★ BE THE CODE, MY FRIEND

💡 **Don Tip:** El for-each és de només lectura. No pots modificar la col·lecció mentre la recorres.

Què imprimeix?

- (A) 30
- (B) 60
- (C) 90
- (D) 150

Resposta: (B) 60. La llista és {10, 20, 30, 40, 50}. Els múltiples de 20 són 20 i 40. $20 + 40 = 60$.

LinkedList: La Conga Line

LinkedList és una llista enllaçada. Cada element sap qui està davant i darrere, com en una conga. És lenta si busques per índex, però rapidíssima per a afegir/borrar al principi o al final.

```
import java.util.LinkedList;

LinkedList<String> cola = new LinkedList<>();
cola.addLast("Persona 1"); // al final
cola.addLast("Persona 2");
cola.addFirst("Colat"); // es cola al principi
String primero = cola.removeFirst(); // "Colat" - se'n va
```

💡 **Consell:** Usa `LinkedList` quan necessites una cua (FIFO) o una pila (LIFO). Per a accés aleatori freqüent, usa `ArrayList`. `LinkedList` busca element per element. `ArrayList` va directe a l'índex.

HashSet: El Porter que No Deixa Duplicats

`HashSet` és com una discoteca: no deixa entrar a ningú que ja estiga dins. No importa l'ordre, només l'exclusivitat.

```
import java.util.HashSet;

HashSet<String> invitados = new HashSet<>();
invitados.add("Ana");
invitados.add("Bob");
invitados.add("Ana"); // No passa res, Ana ja està
System.out.println(invitados.size()); // 2, no 3
```

Com sap si un element ja està? Usa `hashCode()` i `equals()`. Si el teu objecte no els sobreescriu bé, `HashSet` farà coses rares.

⚠️ **Advertència:** Si sobreescrius `equals()` en una classe, SOBREESCRIU `hashCode()`. Sempre. Si dos objectes són iguals segons `equals()`, han de tindre el mateix `hashCode()`. Si no, `HashSet` es tornarà boig. Repetisc: **sempre**.

Operacions Típiques amb HashSet


```
HashSet<String> set = new HashSet<>();
set.add("rojo");
set.add("verde");
set.add("azul");
set.remove("rojo");
set.contains("verde"); // true
set.isEmpty();         // false
set.clear();            // ho buida tot
```

TreeSet: L'Organitzat

TreeSet és com un HashSet que s'ordena sol. Internament usa un arbre roig-negre. Tot el que fiques s'ordena automàticament.

```
import java.util.TreeSet;

TreeSet<String> ordenado = new TreeSet<>();
ordenado.add("Zara");
ordenado.add("Ana");
ordenado.add("Bob");
System.out.println(ordenado); // [Ana, Bob, Zara] - ordre alfabètic

// Mètodes extra útils
ordenado.first(); // "Ana"
ordenado.last();  // "Zara"
ordenado.headSet("Bob"); // [Ana] - elements abans de Bob
```

💡 **Consell:** Necessites elements ordenats automàticament? Usa TreeSet. Només necessites eliminar duplicats? Usa HashSet (és més ràpid: $O(1)$ vs $O(\log n)$).

Iterator: El Cambrer que Pren Nota Un a Un

Iterator recorre una col·lecció sense que t'importi com està implementada per dins. És com un cambrer: “Què vol? I vosté? I vosté?”

```
import java.util.Iterator;

ArrayList<String> platos = new ArrayList<>();
platos.add("Tortilla");
platos.add("Paella");
platos.add("Croquetas");

Iterator<String> it = platos.iterator();
while (it.hasNext()) {
    String plato = it.next();
    if (plato.equals("Paella")) {
        it.remove(); // BORRA de la llista ORIGINAL
    }
}
// Ara platos = [Tortilla, Croquetas]
```

⚠ **Advertència:** Mai no facis `llista.remove(element)` mentre uses un `for-each`. Llançaràs `ConcurrentModificationException`. Usa **SEMPRE** `iterator.remove()` si necessites borrar durant el recorregut.

Collections: L'Amic Utilitari

Igual que Arrays per a arrays, Collections és l'amic de les col·leccions:

```
import java.util.Collections;

ArrayList<String> lista = new ArrayList<>();
lista.add("Zara");
lista.add("Ana");
lista.add("Bob");

Collections.sort(lista);           // [Ana, Bob, Zara]
Collections.reverse(lista);        // [Zara, Bob, Ana]
Collections.shuffle(lista);         // ordre aleatori
Collections.max(lista);             // "Zara" (ordre alfabètic)
Collections.min(lista);            // "Ana"
Collections.frequency(lista, "Ana"); // 1
Collections.replaceAll(lista, "Ana", "Ana María");
Collections.rotate(lista, 2);       // roda 2 posicions
```

★ BE THE CODE, MY FRIEND: Collections en Acció

💡 **Don Tip:** Collections.sort() ordena la llista original. Si no vols modificar-la, copia-la abans.

```
import java.util.*;

public class BeTheCollections {
    public static void main(String[] args) {
        ArrayList<Integer> nums = new ArrayList<>();
        nums.add(5);
        nums.add(1);
        nums.add(8);
        nums.add(3);

        Collections.sort(nums);
        Collections.reverse(nums);

        System.out.println(nums.get(1));
    }
}
```

★ BE THE CODE, MY FRIEND

💡 **Don Tip:** Els mètodes estàtics de Collections són molt potents. sort, shuffle, reverse, binarySearch...

Què imprimeix?

- (A) 1
- (B) 3
- (C) 5
- (D) 8

Resposta: (C) 5. Sort → {1,3,5,8}. Reverse → {8,5,3,1}. get(1) = 5.

? No Hi Ha Preguntes Tontes! (Col·leccions)

? **No Hi Ha Preguntes Tontes! P:** Quan use ArrayList i quan LinkedList? **R:** Usa ArrayList per al 95% dels casos. LinkedList només quan necessites afegir/borrar molt al principi o estàs implementant una cua/pila. ArrayList és més simple i més ràpid en accés aleatori.

P: HashSet vs TreeSet? **R:** HashSet és més ràpid ($O(1)$ vs $O(\log n)$), però no ordena. TreeSet ordena automàticament però és més lent. Si no necessites ordre, usa HashSet. Punt.

P: Es poden mesclar tipus en una col·lecció sense genèrics? **R:** Sí, però és com llançar un dau. Si poses `ArrayList llista = new ArrayList();` (sense `<>`) pots ficar qualsevol cosa, però al traure'l has de fer casting i creuar els dits. Amb genèrics, el compilador et protegeix.

P: Què passa si afegisc un null a un HashSet? **R:** HashSet admet un únic null. TreeSet no admet null perquè necessita comparar elements per a ordenar i... com compares null amb alguna cosa?

P: Què és més ràpid, for-each o iterator? **R:** Internament són quasi el mateix. El for-each usa iterator per darrere. Usa el que et resulte més llegible.

Resum Ràpid: Col·leccions

```
ArrayList<String> a = new ArrayList<>();           // Llista dinàmica
LinkedList<String> l = new LinkedList<>();         // Llista doblement enllaçada
HashSet<String> s = new HashSet<>();               // Sense duplicats, sense ordre
TreeSet<String> t = new TreeSet<>();              // Sense duplicats, ordenat

a.add(e);           // afegir
a.get(i);           // obtindre per índex
a.remove(i);        // borrar per índex
a.remove(e);        // borrar objecte
a.size();           // mida
a.contains(e);       // conté?
a.isEmpty();        // està buit?

Collections.sort(lista);
Collections.reverse(lista);
Collections.shuffle(lista);
Collections.min(lista);
Collections.max(lista);
```

Exercicis Proposats

Exercici 1: L'Invers

Escriu un programa que cree un array de 10 enters, els ompliga amb números del 1 al 10, i després els imprimisca en ordre invers.

Exercici 2: Buscamines Simplificat

Crea un array bidimensional de 5x5 que represente un camp de mines. Ompli'l amb 5 mines (representades com a true) en posicions aleatòries. L'usuari introdueix coordenades (fila, columna) i el programa diu si hi ha mina o no. Si encerta una mina, el joc acaba.

Exercici 3: Estadístiques de Classe

Demana a l'usuari les notes de 20 alumnes, guarda-les en un array, i calcula:

- La nota mitjana
- La nota més alta
- La nota més baixa
- Quants alumnes van aprovar (nota ≥ 5)

Exercici 4: Eliminar Duplicats

Escriu un programa que lligca una llista de paraules des del teclat (acaba amb "F") i les mostre sense duplicats, en el mateix ordre en què van aparéixer la primera volta. Pista: usa un `LinkedHashSet` per a mantindre l'ordre d'inserció.

Exercici 5: Llista de la Compra

Crea un programa que gestione una llista de la compra usant `ArrayList<String>`. Ha de permetre: afegir, eliminar, marcar com a comprat i mostrar la llista.

RAs treballats en aquesta unitat:

- **RA6** - Tipus avançats: Arrays i col·leccions



Sergi Garcia Barea — CC BY-SA 4.0

Unidad 9: Genéricos y Mapas

Objectius d'aprenentatge

- Comprendre el concepte de genèrics i type erasure
- Crear classes i mètodes genèrics propis
- Usar wildcards (? extends, ? super) per a flexibilitat
- Gestionar mapes (HashMap, TreeMap) i les seues operacions principals
- Triar entre Map, List i Set segons el problema

Genèrics: El <T> Que Ho Va Canviar Tot

Abans dels genèrics, programar Java era com fer malabars amb ganivets... embenats. Amb genèrics, el compilador és la teua xarxa de seguretat.

Quin Embolic Abans Dels Genèrics!

Imagina que tens una ArrayList a la manera antiga, sense genèrics. Es com una capsa on pots ficar qualsevol cosa: una sabata, una poma, un gat, un número de la sort. El problema és que quan *tires* les coses, Java et torna un Object i tu has de recordar què vas ficar. I si vas ficar un Integer però el tractes com String? **BOOM**. ClassCastException a tota la cara.

```
import java.util.*;

public class InfiernoSinGenericos {
    public static void main(String[] args) {
        ArrayList cajaDeCaos = new ArrayList();
        cajaDeCaos.add(42);
        cajaDeCaos.add("Hola");
        cajaDeCaos.add(3.14);

        // Todo lo que sacas es Object... ¡tú adivina qué es!
        Object cosa = cajaDeCaos.get(0);
        String texto = (String) cosa; // 🌟 ClassCastException en tiempo de ejecución
    }
}
```

Ho veus? El compilador no t'avisà de res. Te n'assabentes quan el programa ja està corrent i explota. Com una granada amb el segur llevat.

⚠ **Advertència:** Sense genèrics, els errors de tipus salten en temps d'EXECUCIÓ (quan l'usuari està usant el teu programa). Amb genèrics, salten en temps de COMPILACIÓ (quan tu estàs programant). Quan prefereixes assabentar-te'n?

Arriben Els Genèrics: El Compilador Es Torna El Teu Amic

A partir de Java 5, els genèrics van canviar les regles del joc. Una `ArrayList<String>` ja no és una capsa de caos: és una màquina expenedora que SOLS dona Coca-Coles. No pots ficar una sabata, i si ho intentes, el compilador et para el braç.

```
import java.util.*;

public class CieloConGenericos {
    public static void main(String[] args) {
        ArrayList<String> maquinaDeCocacolas = new ArrayList<>();
        maquinaDeCocacolas.add("Coca-Cola");
        maquinaDeCocacolas.add("Coca-Cola Light");
        // maquinaDeCocacolas.add(42); // ❌ Error de compilación

        // Al sacar, ya sabes que es String. Sin casting.
        String bebida = maquinaDeCocacolas.get(0);
        System.out.println(bebida.toUpperCase()); // "COCA-COLA" sin miedo
    }
}
```

💡 **Consell:** Els genèrics existeixen per una sola raó: **seguretat de tipus**. Comproven en compilació que no estigues ficant la pota amb els tipus, eliminant la necessitat de castings i els temuts `ClassCastException`.

★ BE THE CODE, MY FRIEND: Ets Una `ArrayList<T>`

👁 **Don Tip:** Els genèrics asseguren que només fiques el tipus correcte. Sense ells, tindries càsting a punta pala.

★ **BE THE CODE, MY FRIEND** 👁 **Don Tip:** `ArrayList<String>` sols accepta Strings. Si intentes ficar un `int`, el compilador et para. Tanca els ulls. Respira profund. Ara ERES una `ArrayList<T>`.

Dins de tu hi ha una `Object[]` que emmagatzema els teus elements. Quan algú fa `add("Hola")`, el compilador ja sap que només acceptes Strings. Quan algú fa `get(0)`, el

compilador sap que tornes un String, no un Object.

Eres com un porter de discoteca que només deixa passar a gent amb el tipus adequat. Sense genèrics, deixaves passar a qualsevol i després la lées. Amb genèrics, eres selectiu, i la festa és molt millor.

La Teua Pròpia Classe Genèrica: Caja<T>

No només les col·leccions poden ser genèriques. Tu també pots crear les teues pròpies classes genèriques! El truc està en el paràmetre de tipus <T> (de “Type”), encara que pots usar qualsevol lletra:

- T → Tipus (Type) — el comodí general
- E → Element (Element) — per a col·leccions
- K / V → Clau / Valor (Key / Value) — per a mapes
- N → Número (Number)

```
// Una caja que guarda UN objeto de cualquier tipo
public class Caja<T> {
    private T contenido;

    public void guardar(T contenido) {
        this.contenido = contenido;
    }

    public T sacar() {
        return contenido;
    }

    public boolean estaVacía() {
        return contenido == null;
    }
}
```

I així s'usa:


```
Caja<String> cajaDeTexto = new Caja<>();
cajaDeTexto.guardar("Mensaje secreto");
String mensaje = cajaDeTexto.sacar(); // Sin casting, directo al pelo

Caja<Integer> cajaDeNumeros = new Caja<>();
cajaDeNumeros.guardar(42);
Integer numero = cajaDeNumeros.sacar();
```

⚠ **Advertència:** No pots usar tipus primitius com a paràmetre de tipus. `Caja<int>` no compila. Usa `Caja<Integer>`, amb la seua classe envoltent (wrapper). L'autoboxing de Java s'encarrega de la resta.

★ BE THE CODE, MY FRIEND: La Capsa Genèrica

💡 **Don Tip:** El tipus `T` és un comodí. Quan instanciïs, es reemplaça pel tipus real.

```
public class BeTheCaja {
    public static void main(String[] args) {
        Caja<Integer> caja = new Caja<>();
        caja.guardar(5);
        caja.guardar(10);
        System.out.println(caja.sacar());
    }
}
```

★ **BE THE CODE, MY FRIEND** 💡 **Don Tip:** `T` pot ser qualsevol tipus. Però no un primitiu: usa `Integer` en lloc de `int`. **Què imprimeix?**

- (A) 5
- (B) 10
- (C) null
- (D) Error de compilació

Resposta: (B) 10. La capsa només guarda un element, el segon `guardar(10)` sobreescriu el 5.

L'Operador Diamant <>: El Peresós Oficial

Des de Java 7, no cal repetir el tipus dos vegades. El compilador l'inferix per tu:

```
// Antes de Java 7 (repetitivo):  
Caja<String> caja1 = new Caja<String>();  
  
// Desde Java 7 (el diamante <> al rescate):  
Caja<String> caja2 = new Caja<>();
```

El <> és com l'“etcètera” dels genèrics: “ja saps de quin tipus estic parlant, no? Doncs això”.

💡 **Consell:** Usa sempre l'operador diamant <>. El teu codi queda més net, més llegible i els teus companys d'equip t'ho agrairan.

Type Erasure: El Mag Es Porta Els Genèrics

Ací va el truc: els genèrics SOLS existeixen en temps de compilació. Quan el teu codi es converteix en bytecode, el compilador borra tota la informació de tipus genèrics. És com si un mag fera desaparèixer els <String> i <Integer>.

```
// En tu código fuente:  
ArrayList<String> nombres = new ArrayList<>();  
ArrayList<Integer> numeros = new ArrayList<>();  
  
// Después de compilar (en bytecode):  
ArrayList nombres = new ArrayList(); // ambos son ArrayList simples  
ArrayList numeros = new ArrayList();
```

A açò se li diu **type erasure**. El compilador:

1. **Verifica** que els tipus siguen correctes (ací no cola un Integer en una llista de Strings)
2. **Borra** la informació genèrica
3. **Afegeix** els castings necessaris on calga

És com un porter que revisa el teu DNI a la porta, però una vegada dins, tu no portes cap identificació.

Mètodes Genèrics: Funcions Per a Tot Tipus

No només les classes poden ser genèriques. Els mètodes també poden declarar els seus propis paràmetres de tipus, independentment de la classe on estiguen.

```

public class Utilidades {

    // Método genérico: declara <T> antes del tipo de retorno
    public static <T> void imprimir(T elemento) {
        System.out.println("Elemento: " + elemento.toString());
    }

    // Método genérico con dos parámetros de tipo
    public static <T, U> boolean sonIguales(T a, U b) {
        return a.equals(b);
    }

    // Invertir un array genérico
    public static <T> T[] invertir(T[] array) {
        for (int i = 0; i < array.length / 2; i++) {
            T temp = array[i];
            array[i] = array[array.length - 1 - i];
            array[array.length - 1 - i] = temp;
        }
        return array;
    }
}

// Uso:
Utilidades.imprimir(42);           // El compilador infiere que T es Integer
Utilidades.imprimir("Hola");       // El compilador infiere que T es String
String[] invertido = Utilidades.invertir(new String[]{"A", "B", "C"});

```

★ BE THE CODE, MY FRIEND: Mètode Genèric

👁️ **Don Tip:** El <T> abans del void declara el tipus del mètode. Sense això, el compilador no sap què és T.

```

public class BeTheGenericMethod {
    public static void main(String[] args) {
        String resultado = Utilidades.<String>obtenerValor("Hola");
        // ¿Qué hace este código? ¿Compila?
    }
}

```

★ **BE THE CODE, MY FRIEND** 🐞 **Don Tip:** Els mètodes genèrics dedueixen el tipus automàticament dels arguments. **Compila i què imprimeix?**

- (A) Sí, imprimeix “Hola”
- (B) No compila, el mètode no existix
- (C) Sí, però imprimeix null
- (D) No compila, la sintaxi és incorrecta

Resposta: (A) Sí, imprimeix “Hola”. La sintaxi `Clase.<Tipo>metodo()` és vàlida per a invocar mètodes genèrics especificant el tipus explícitament, encara que normalment el compilador l'inferix sol.

? No Hi Ha Preguntes Tontes! (Genèrics)

? **No Hi Ha Preguntes Tontes! Q:** Per què no puc fer `new T()` dins d'un mètode genèric? **A:** Perquè en temps de compilació, Java no sap què és `T`. No pot crear una instància d'alguna cosa que no coneix. És com demanar-li a un pastisser que faça “un pastís” però sense dir-li de què.

Q: I `new T[]`? **A:** Tampoc. Els arrays coneixen el seu tipus en temps d'execució, però els genèrics es borren (type erasure). Per això internament s'usa `Object[]` i es casteja.

Q: Els genèrics ralentitzen el meu programa? **A:** No. Java aplica **type erasure**: el compilador borra tota la informació genèrica i la convertix en castings normals. És només sucre sintàctic en compilació. En runtime, no hi ha genèrics.

? No Hi Ha Preguntes Tontes! (Més Genèrics)

? **No Hi Ha Preguntes Tontes! Q:** Què significa `<?>` en els genèrics? **A:** És un wildcard sense restriccions. Significa “qualsevol tipus”. És com dir “m'és igual el tipus, només vull treballar amb la col·lecció”. Però ull: no pots afegir elements (excepte null) perquè el compilador no sap de quin tipus és.

Q: Puc tindre un mètode genèric estàtic en una classe no genèrica? **A:** Sí, totalment. De fet, és molt comú. La classe `Collections` està plena de mètodes estàtics genèrics com `sort()`, `reverse()`, etc.

Q: Què és el “type erasure”? **A:** És el procés pel qual el compilador borra tota la informació de tipus genèrics després de comprovar que tot és correcte. En el bytecode, `ArrayList<String>`

i `ArrayList<Integer>` són tots dos `ArrayList`. És com si el compilador fóra un advocat que revisa el contracte, i després un mag que fa desaparèixer les proves.

Wildcards: `? extends T` vs `? super T`

Els comodins (wildcards) són per a quan vols escriure mètodes que funcionen amb **qualsevol tipus** dins d'una jerarquia.

```

import java.util.*;

public class Wildcards {

    // ? extends T → Covarianza (solo LEER, no escribir)
    // Puedes pasar List<Integer>, List<Double>, List<Number>...
    public static double sumar(ArrayList<? extends Number> numeros) {
        double total = 0.0;
        for (Number n : numeros) {
            total += n.doubleValue();
        }
        // numeros.add(42); // ❌ Error: no puedes añadir nada (excepto null)
        return total;
    }

    // ? super T → Contravarianza (solo ESCRIBIR, leer como Object)
    // Puedes añadir Integers a una lista de Integer, Number, Object...
    public static void rellenar(ArrayList<? super Integer> lista) {
        lista.add(1);
        lista.add(2);
        lista.add(3);
        // Integer n = lista.get(0); // ❌ Error: solo sabes que es Object
        Object obj = lista.get(0); // ✅ Ok
    }
}

// Uso:
ArrayList<Integer> enteros = new ArrayList<>(Arrays.asList(1, 2, 3));
ArrayList<Number> numeros = new ArrayList<>();
ArrayList<Object> objetos = new ArrayList<>();

sumar(enteros); // ✅ ? extends Number funciona con Integer
rellenar(numeros); // ✅ ? super Integer funciona con Number
rellenar(objetos); // ✅ ? super Integer funciona con Object

```

⚠️ Advertència: Mnemotècnia infalible:

- ? extends T → **P**roducer **E**xtends (només produïxes/lleges dades)
- ? super T → **C**onsumer **S**uper (només consumes/escris dades)

El principi PECS de Joshua Bloch: “Producer Extends, Consumer Super”.

★ BE THE CODE, MY FRIEND: Wildcards

💡 **Don Tip:** ? significa 'qualsevol tipus'. ? extends Number limita a Number i les seues subclasses.

```
import java.util.*;

public class BeTheWildcard {
    public static void main(String[] args) {
        List<Integer> enteros = Arrays.asList(1, 2, 3);
        List<Double> dobles = Arrays.asList(1.5, 2.5, 3.5);
        // printNumbers(enteros); // ¿compila?
        // printNumbers(dobles); // ¿compila?
    }

    public static void printNumbers(List<? extends Number> lista) {
        for (Number n : lista) {
            System.out.print(n + " ");
        }
    }
}
```

★ BE THE CODE, MY FRIEND 💡 **Don Tip:** Els wildcards són de només lectura. No pots afegir elements a una List<?> perquè no saps de quin tipus és. **Quantes crides compilen?**

- (A) 0
- (B) 1
- (C) 2
- (D) Error en ambdues

Resposta: (C) 2. List<? extends Number> accepta qualsevol llista el tipus de la qual hereta de Number, tant List<Integer> com List<Double>.

Resum Ràpid: Genèrics

<code>ArrayList<Tipo></code>	← Solo acepta ese <code>Tipo</code> y subclases
<code>Caja<T></code>	← Tu propia clase genérica
<code>new Caja<>()</code>	← El diamante: infiere el tipo
<code><T> void metodo(T)</code>	← Método genérico
<code>? extends T</code>	← Wildcard de lectura (<code>producer</code>)
<code>? super T</code>	← Wildcard de escritura (<code>consumer</code>)

Mapes: La Guia Telefònica

HashMap: La Guia Telefònica

HashMap associa claus amb valors. Com una agenda: busques per nom (clau) i obtens el telèfon (valor).

```
import java.util.HashMap;

HashMap<String, Integer> agenda = new HashMap<>();
agenda.put("Ana", 612345678);
agenda.put("Bob", 698765432);
agenda.put("Ana", 600000000); // Sobrescribe el anterior

int telefono = agenda.get("Ana"); // 600000000
int inexistente = agenda.get("NoExisto"); // null
agenda.containsKey("Bob"); // true
agenda.containsValue(600000000); // true
```



Nota: Les claus d'un HashMap han de ser immutables. Per això String i Integer són perfectes. Si uses un objecte mutable com a clau i després el modifiques, el HashMap no el trobarà mai. És com canviar el pany i esperar que la clau vella continue funcionant.

Recórrer un HashMap


```

HashMap<String, Integer> agenda = new HashMap<>();
agenda.put("Ana", 612345678);
agenda.put("Bob", 698765432);

for (String nombre : agenda.keySet()) {
    System.out.println(nombre + " → " + agenda.get(nombre));
}

for (Integer telefono : agenda.values()) {
    System.out.println("Tel: " + telefono);
}

for (HashMap.Entry<String, Integer> entrada : agenda.entrySet()) {
    System.out.println(entrada.getKey() + " → " + entrada.getValue());
}

```

★ BE THE CODE, MY FRIEND: El HashMap Traïdor

💡 **Don Tip:** HashMap no garanteix ordre. Si necessites ordre, usa TreeMap o LinkedHashMap.

```

import java.util.HashMap;

public class BeTheMap {
    public static void main(String[] args) {
        HashMap<String, String> capitales = new HashMap<>();
        capitales.put("Espanya", "Madrid");
        capitales.put("Francia", "París");
        capitales.put("Italia", "Roma");
        capitales.put("Espanya", "Barcelona");

        System.out.println(capitales.get("Espanya"));
    }
}

```

★ **BE THE CODE, MY FRIEND** 💡 **Don Tip:** Les claus d'un HashMap han de tindre hashCode() i equals() ben implementats. Què imprimeix?

- (A) Madrid
- (B) Barcelona

- (C) null
- (D) Error

Resposta: (B) Barcelona. La clau “España” se sobreescriu amb el nou valor. Madrid ha sigut reemplaçat.

★ BE THE CODE, MY FRIEND: Freqüència de Lletres

💡 **Don Tip:** `getOrDefault()` evita el null check. Usa'l sempre que pugues.

```
import java.util.HashMap;

public class BeTheFrequency {
    public static void main(String[] args) {
        String texto = "banana";
        HashMap<Character, Integer> frec = new HashMap<>();
        for (char c : texto.toCharArray()) {
            frec.put(c, frec.getOrDefault(c, 0) + 1);
        }
        System.out.println(frec.get('a') + " " + frec.get('n'));
    }
}
```

★ **BE THE CODE, MY FRIEND** 💡 **Don Tip:** `merge()` és el teu amic per a freqüències. Combina valor existent amb nou. **Què imprimeix?**

- (A) 1 1
- (B) 3 1
- (C) 3 2
- (D) 2 2

Resposta: (B) 3 1. En “banana”: la ‘a’ apareix 3 vegades, la ‘n’ apareix 1 volta. `getOrDefault` és l’heroi que evita els `NullPointerException`.

TreeMap: L’Ordenat

`TreeMap` és un `HashMap` ordenat per clau. Internament usa un arbre roig-negre.

```
import java.util.TreeMap;

TreeMap<String, Integer> ordenado = new TreeMap<>();
ordenado.put("Zara", 30);
ordenado.put("Ana", 25);
ordenado.put("Bob", 35);
System.out.println(ordenado); // {Ana=25, Bob=35, Zara=30} - orden alfabético

// Mètodos extra útils
ordenado.firstKey(); // "Ana"
ordenado.lastKey(); // "Zara"
ordenado.headMap("Bob"); // {Ana=25} - entrades abans de Bob
```

💡 **Consell:** HashMap per a velocitat ($O(1)$). TreeMap per a ordre ($O(\log n)$). Si pregunten en una entrevista “quin és millor?”, la resposta correcta és “depén”.

? No Hi Ha Preguntes Tontes! (Mapes)

? No Hi Ha Preguntes Tontes! Q: Puc tindre un HashMap amb clau null? **A:** HashMap admet una clau null. TreeMap no. HashMap guarda la clau null en una posició especial.

Q: Què passa si la clau no existix en el mapa? **A:** `get()` torna null. Usa'l amb cura o millor usa `getOrDefault(clau, valorPerDefecte)`.

Q: HashMap o TreeMap? **A:** HashMap per a velocitat. TreeMap per a ordre.

Q: Quin avantatge té un LinkedHashMap? **A:** Manté l'ordre d'inserció. És com un HashMap que recorda en quin ordre vas ficar les coses.

Comparació: Map vs List vs Set

Característica	List	Set	Map
Què guarda?	Elements ordenats	Elements únics	Parells clau→valor
Duplicats?	Sí	No	Claus no, valors sí
Ordre?	D'inserció	Depén (Hash/Tree)	Depén (Hash/Tree)
Accés	Per índex	Per element	Per clau

Característica	List	Set	Map
Nulls?	Sí	HashSet: 1, TreeSet: 0	HashMap: 1 clau, TreeMap: 0
Principal impl.	ArrayList	HashSet	HashMap

Regla pràctica:

- Necessites una llista de coses? → ArrayList
- Coses sense repetir? → HashSet
- Coses ordenades sense repetir? → TreeSet
- Associar una cosa amb una altra? → HashMap

EL RING: HashMap vs TreeMap

Dues implementacions de Map s'enfronten.

HashMap: «Jo soc el rei de la velocitat. $O(1)$ en get i put. No m'importa l'ordre, m'importa la rapidesa.»

TreeMap: «Sí, però jo mantinc les claus ordenades automàticament. Si necessites recórrer-les en ordre alfabètic, soc el teu únic amic.»

HashMap: «¿Ordenat? Això costa. Soc $O(\log n)$ en les teues operacions. Jo soc $O(1)$. ¡Soc imbatible en rendiment!»

TreeMap: «Cert, però puc navegar: `firstKey()`, `lastKey()`, `subMap()`, `headMap()`. Tu per a tot això has de copiar i ordenar.»

HashMap: «Si no necessites ordre, per què pagar el cost? La majoria dels casos usen HashMap.»

TreeMap: «I quan necessiten ordre, ahíestic jo. I no soc tan lent: $O(\log n)$ continua sent molt ràpid per a la majoria de casos.»

HashMap: «Tregua. Cadascú al seu lloc.»

TreeMap: «Fet.»

💡 **Don Tip:** ¿Velocitat? HashMap. ¿Ordre? TreeMap. ¿Ordre d'inserció? LinkedHashMap. Cadascú té el seu superpoder.

Resum Ràpid: Mapes

```
HashMap<K, V> m = new HashMap<>();           // Clave → Valor, rápido
TreeMap<K, V> tm = new TreeMap<>();          // Clave → Valor, ordenado

m.put(clave, valor); // añadir o sobrescribir
m.get(clave);         // obtener (null si no existe)
m.getDefault(c, d);  // obtener o valor por defecto
m.containsKey(c);     // existe la clave?
m.containsValue(v);   // existe el valor?
m.remove(clave);      // borrar entrada
m.size();             // número de entradas
m.keySet();           // conjunto de claves
m.values();           // colección de valores
m.entrySet();         // conjunto de entradas
```

Exercicis Proposats

Exercici 1: Contactes de Tota la Vida

Implementa una mini agenda usant `HashMap<String, String>` que associe nom amb telèfon. El programa ha de permetre afegir contactes, buscar per nom, llistar tots i esborrar. Usa un menú.

Exercici 2: Crea una classe Pareja<T, U>

Emmagatzema dos objectes de tipus possiblement distints. Inclou mètodes `getPrimero()`, `getSegundo()`, `setPrimero(T)`, `setSegundo(U)` i un mètode `intercanviar()` que torne una nova `Pareja<U, T>` amb els valors intercanviats.

Exercici 3: Comptador de Paraules

Escriu un programa que lligga un text i compte quantes vegades apareix cada paraula usant un `HashMap<String, Integer>`.

Exercici 4: Mètode Genèric maximo

Implementa un mètode genèric public static `<T extends Comparable<T>> T maximo(T[] array)` que torne l'element més gran d'un array.

RAs treballats en esta unitat:

- **RA6** - Tipus avançats: Genèrics i mapes
-



Sergi Garcia Barea — CC BY-SA 4.0

Unitat 11: Consola, Fitxers i Expressions Regulars

Objectius d'aprenentatge

- Llegir i escriure en consola amb System.out, printf i Scanner
- Formatar eixides i manejar trampes de Scanner (nextInt + nextLine)
- Llegir i escriure fitxers de text amb File, FileReader, FileWriter, BufferedReader
- Usar try-with-resources, NIO (Files/Paths) i serialització bàsica
- Crear i usar expressions regulars amb Pattern i Matcher
- Validar, buscar, reemplaçar i extraure parts de text amb regex

Comunicació per Consola

System.out

System.out és el megàfon del teu programa:

```
System.out.println("¡Hola, DAM!");    // con salto de línea (ln = line)
System.out.print("Sin salto... ");    // sin salto de línea
System.out.print("...continúa aquí.");
System.out.println();                // salto de línea vacío
```

printf() — Formatat

```
String nombre = "Ana";
int edad = 20;
double nota = 9.5;
System.out.printf("Nombre: %s, Edad: %d, Nota: %.2f%n", nombre, edad, nota);
```

Especificador	Tipus	Exemple
%s	String	"Hola %s" → "Hola Mario"
%d	Enter	"Edad: %d" → "Edad: 25"
%f	Decimal	"%.2f" → "19.99"

Especificador	Tipus	Exemple
%c	Caràcter	"Inicial: %c"
%n	Salt de línia	(independent del SO)
%x	Hexadecimal	"#%06X" → "#FF00AA"

```
double pi = Math.PI;
System.out.printf("2 decimales: %.2f%n", pi);           // 3,14
System.out.printf("4 decimales: %.4f%n", pi);           // 3,1416
System.out.printf("Ancho 10: %10.2f%n", pi);             //          3,14
System.out.printf("Izquierda: %-10.2f%n", pi);           // 3,14
```

String.format() retorna un String sense imprimir-lo:

```
String msg = String.format("Bienvenido, %s.", "Carlos");
System.out.println(msg);
```

Scanner

System.in és l'orella del programa; Scanner és el traductor:

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
System.out.print("Nombre: ");
String nombre = sc.nextLine();
System.out.print("Edad: ");
int edad = sc.nextInt();
System.out.print("Altura: ");
double altura = sc.nextDouble();
System.out.printf("Encantado, %s. %d años, %.2f m.%n", nombre, edad, altura);
sc.close();
```

Mètode	Descripció
nextLine()	Llig una línia completa com a String
next()	Llig una paraula (fins al primer espai)

Mètode	Descripció
nextInt()	Llig un número enter
nextDouble()	Llig un número decimal
nextFloat() / nextLong()	Llig float / long
nextBoolean()	Llig booleà (true/false)
hasNextInt()	Hi ha un int esperant?
hasNextDouble()	Hi ha un double esperant?
close()	Tanca el Scanner

⚠ Compte amb nextInt() + nextLine()!

nextInt() deixa l'Intro al buffer. nextLine() posterior se'l menja i es salta l'entrada.

Solució 1: sc.nextLine() extra després de nextInt()

Solució 2: Usar sempre nextLine() i convertir:

```
int edad = Integer.parseInt(sc.nextLine());
```

Format de números grans

```
import java.text.NumberFormat;
import java.util.Locale;

NumberFormat nf = NumberFormat.getInstance(new Locale("es", "ES"));
System.out.println(nf.format(1234567.89)); // 1.234.567,89

NumberFormat moneda = NumberFormat.getCurrencyInstance(new Locale("es", "ES"));
System.out.println(moneda.format(12345.67)); // 12.345,67 €
```

Errors Comuns amb Scanner

```
// Olvidar import java.util.Scanner;
// No cerrar el Scanner: sc.close();
// Tipo incorrecto: int num = sc.nextDouble(); // Error de compilación
// nextInt() seguido de nextLine() sin consumir el Intro
```

? No Hi Ha Preguntes Tontes! (Consola)

Q: Per què es diu `System.out`?

A: `System` té tres fluxos estàtics: `out` (eixida), `in` (entrada), `err` (errors).

Q: Puc tindre diversos `Scanner` oberts?

A: Tècnicament sí, però usa UN sol `Scanner` per a `System.in`.

Q: Què passa si l'usuari escriu lletres on espere números?

A: `InputMismatchException`. Solució: `try-catch` o `hasNextInt()`.

Q: Puc usar `printf()` per a dates?

A: Sí, amb `%tF` (data ISO: 2026-06-20) i `%tT` (hora: 14:30:00).

Resum Exprés: Consola

- `System.out.println()` / `printf()` → imprimir
- `Scanner sc = new Scanner(System.in)` → llegir teclat
- `sc.nextLine()` / `sc.nextInt()` → llegir text / número
- Sempre `sc.close()` en acabar
- `nextInt()` + `nextLine()` = desastre segur

Fitxers

La Classe File

Abans de llegir o escriure, localitza el fitxer. `java.io.File` és el GPS:

```
import java.io.File;

File f = new File("C:/datos/notas.txt");
System.out.println("¿Existe? " + f.exists());
System.out.println("¿Es archivo? " + f.isFile());
System.out.println("¿Es carpeta? " + f.isDirectory());
System.out.println("Tamaño: " + f.length() + " bytes");
System.out.println("Ruta absoluta: " + f.getAbsolutePath());
System.out.println("Nombre: " + f.getName());
```

💡 **Consell:** Usa `/` o `\\` en Windows. `"C:/datos/notas.txt"` o `"C:\\datos\\notas.txt"`.

Escriure: FileWriter

```
import java.io.FileWriter;
import java.io.IOException;

FileWriter escritor = new FileWriter("salida.txt");
escritor.write("Primera línea.\n");
escritor.write("Segunda línea.\n");
escritor.close();

// Añadir al final (sin borrar):
FileWriter writer = new FileWriter("bitacora.txt", true);
```

Sense `close()` o `flush()` les dades es queden al buffer intern sense escriure's.

Llegir: FileReader + BufferedReader

`BufferedReader` embolica a `FileReader` per a llegir línies senceres d'una volta:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

BufferedReader lector = new BufferedReader(new FileReader("salida.txt"));
String linea = lector.readLine();
while (linea != null) {
    System.out.println(linea);
    linea = lector.readLine();
}
lector.close();
```

★ BE THE CODE, MY FRIEND: El Detectiu de Fitxers

💡 **Don Tip:** `File` representa rutes, no contingut. Per a llegir el contingut usa `Scanner`, `BufferedReader` o `Files.readAllLines()`.

```
import java.io.*;

public class DetectiveDeArchivos {
    public static void main(String[] args) throws IOException {
        File f = new File("misterio.txt");
        if (!f.exists()) {
            System.out.println("Creando archivo...");
            FileWriter w = new FileWriter(f);
            w.write("Tres\npalabras\nmisteriosas\n"); w.close();
        }
        BufferedReader r = new BufferedReader(new FileReader(f));
        String s = "";
        String linea;
        while ((linea = r.readLine()) != null) {
            s = linea + " " + s;
        }
        r.close();
        System.out.println(s);
    }
}
```

★ Què imprimeix la **segona** vegada? (el fitxer ja existix)

Resposta: misteriosas palabras Tres — llig i concatena al revés.

try-with-resources (Java 7+)

Els recursos es tanquen automàticament en eixir del bloc:

```
try (BufferedReader br = new BufferedReader(new FileReader("salida.txt"))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
    }
} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
// No hay br.close() – se cierra solo
```

Scanner + File

```
try (Scanner sc = new Scanner(new File("salida.txt"))) {
    while (sc.hasNextLine()) {
        System.out.println(sc.nextLine());
    }
}
```

Scanner vs BufferedReader: Scanner és millor per a parsejar tokens (nextInt());
BufferedReader és més ràpid per a fitxers grans.

PrintWriter

Igual que System.out però escrivint en fitxers:

```
try (PrintWriter pw = new PrintWriter(new FileWriter("formato.txt"))) {
    pw.println("Línea con salto automático");
    pw.printf("PI vale %.4f%n", Math.PI);
}
```

NIO: Files i Paths

API moderna des de Java 7. Path és com File però amb esteroides:

```
import java.nio.file.*;
import java.util.List;

Path ruta = Paths.get("C:/datos/notas.txt");
List<String> lineas = Files.readAllLines(ruta); // llegir
Files.write(ruta, List.of("Línea 1", "Línea 2")); // escriure
```

★ BE THE CODE, MY FRIEND: NIO en Acció

👁️ **Don Tip:** NIO usa Path i Files. Són més moderns i tenen mètodes útils com Files.walk() per a recórrer arbres.

```
import java.nio.file.*;
import java.util.*;

public class BeTheNIO {
    public static void main(String[] args) throws IOException {
        Path p = Paths.get("nums.txt");
        Files.write(p, List.of("3", "7", "2", "9", "5"));
        List<String> l = Files.readAllLines(p);
        int suma = 0;
        for (String s : l) { suma += Integer.parseInt(s); }
        Files.write(p, List.of("Total: " + suma));
        System.out.println(Files.readString(p));
    }
}
```

★ Què imprimeix? **Resposta:** Total: 26 — suma els números i sobreescriu.

Serialització: ObjectOutputStream

Guardar objectes sencers en fitxers amb Serializable:

```

import java.io.*;

class Persona implements Serializable {
    String nombre; int edad;
    Persona(String n, int e) { this.nombre = n; this.edad = e; }
}

public class GuardandoObjetos {
    public static void main(String[] args) throws Exception {
        Persona p = new Persona("Luis", 25);

        try (ObjectOutputStream oos = new ObjectOutputStream(
            new FileOutputStream("persona.obj"))) {
            oos.writeObject(p);
        }

        try (ObjectInputStream ois = new ObjectInputStream(
            new FileInputStream("persona.obj"))) {
            Persona recuperada = (Persona) ois.readObject();
            System.out.println(recuperada.nombre + " tiene " + recuperada.edad);
        }
    }
}

```

La classe ha d'implementar Serializable. Si té camps no serialitzables → NotSerializableException.

? No Hi Ha Preguntes Tontes! (Fitxers)

Q: File crea el fitxer?

A: No. new File("ruta") sols representa la ruta, no crea res.

Q: Què passa si el fitxer no existix en llegir?

A: FileNotFoundException. Usa f.exists() o try-catch.

Q: Quina diferència hi ha entre File, FileReader i FileWriter?

A: File → l'adreça. FileReader → per a LLEGIR. FileWriter → per a ESCRIURE.

Q: Què passa si no tanque un fitxer?

A: Les dades poden perdre's (buffer sense buidar) i el SO reté recursos.

Expressions Regulars

Una expressió regular (regex) és un **patró de cerca** que descriu un conjunt de cadenes.

⚠ En Java les contrabarras es dupliquen: `\d` → `"\\d"`, `\.` → `"\\."`. És l'infern de les contrabarras.

Pattern i Matcher

- Pattern: l'expressió regular compilada (el motle)
- Matcher: s'aplica a un text concret buscant coincidències

```
import java.util.regex.*;

Pattern patron = Pattern.compile("\\d+"); // un o més dígit
Matcher matcher = patron.matcher("Hay 123 manzanas y 456 peras");

while (matcher.find()) {
    System.out.println("Encontrado: " + matcher.group()
        + " (" + matcher.start() + "-" + matcher.end() + ")");
}

// Encontrado: 123 (4-7), Encontrado: 456 (21-24)
```

Compila el Pattern una sola vegada i reutilitza'l. `Pattern.compile()` és car.

Símbols Regex

Símbol	Significat	Exemple
.	Qualsevol caràcter (ex. salt)	<code>c.sa</code> → "casa", "cose"
<code>\d</code>	Dígit (0-9)	<code>\d{3}</code> → "123"
<code>\D</code>	NO dígit	<code>\D+</code> → "Hola"
<code>\w</code>	Lletra, dígit o _	<code>\w+</code> → "Hola_123"
<code>\W</code>	NO <code>\w</code>	<code>\W</code> → ".", " "
<code>\s</code>	Espai en blanc	<code>\s+</code> → separadors

Símbol	Significat	Exemple
\S	NO espai	\S+ → paraules
*	0 o més vegades	a* → "", "a", "aa"
+	1 o més vegades	a+ → "a", "aa"
?	0 o 1 vegada (opcional)	colou?r → "color", "colour"
{n}	Exactament n	\d{3}
{n,m}	Entre n i m	\d{2,4}
[abc]	Un del conjunt	[aeiou] → vocals
[a-z]	Rang	[a-z] → minúscules
[^abc]	Negació	[^0-9] → no dígit
()	Grup de captura	(\d+)-(\w+)
^	Inici de línia	^Hola
\$	Final de línia	mundo\$
	OR lògic	gato perro
\b	Límit de paraula	\bJava\b ≠ "JavaScript"

String.matches(), replaceAll(), split()

Mètodes de String que accepten regex directament:

```
String texto = "  Hola    mundo  de las  regex  ";

// matches() → true només si TOT el string coincideix
"123".matches("\\d+");           // true
"Hola".matches("\\d+");          // false

// replaceAll() → reemplaça totes les coincidències
String limpio = texto.replaceAll("\\s+", " ").trim();
// "Hola mundo de las regex"

// replaceFirst() → només la primera coincidència
texto.replaceFirst("\\s+", " ").trim();

// split() → dividix pel patró
"a,b,c,d".split(",");           // [a, b, c, d]
"a,b,c,d".split(",", 3);        // [a, b, c,d] (amb límit)
```

`matches()` emparella **tot** el string. Per a buscar subcadenaes usa `find()`.

Validació: Email, Teléfon, DNI

```

public class ValidadorRegex {
    private static final Pattern PATRON_DNI = Pattern.compile("\\d{8}[A-Z]");
    private static final Pattern PATRON_EMAIL =
        Pattern.compile("[\\w.]+@[\\w.]+\\. [a-z]{2,}");
    private static final Pattern PATRON_TELEFONO =
        Pattern.compile("[679]\\d{8}");

    public static boolean esEmailValido(String email) {
        return PATRON_EMAIL.matcher(email.toLowerCase()).matches();
    }
    public static boolean esDNIVálido(String dni) {
        return PATRON_DNI.matcher(dni.toUpperCase()).matches();
    }
    public static boolean esTelefonoValido(String telefono) {
        return PATRON_TELEFONO.matcher(telefono).matches();
    }

    public static void main(String[] args) {
        System.out.println(esEmailValido("user@example.com")); // true
        System.out.println(esEmailValido("user@@example")); // false
        System.out.println(esDNIVálido("12345678Z")); // true
        System.out.println(esDNIVálido("12345678z")); // false
        System.out.println(esTelefonoValido("612345678")); // true
        System.out.println(esTelefonoValido("512345678")); // false
    }
}

```

Per a validar DNI espanyol de veritat necessites comprovar la lletra mòdul 23. La regex sols verifica format.

Grups de Captura

Els parèntesis () capturen allò que coincidix per a extraure-ho després:

```
String texto = "Juan: 28 años, María: 32 años";
Pattern patron = Pattern.compile("(\\w+): (\\d+) años");
Matcher matcher = patron.matcher(texto);

while (matcher.find()) {
    System.out.println(matcher.group(1) + " tiene " + matcher.group(2) + " años");
}

// Juan tiene 28 años, María tiene 32 años
```

Per a agrupar sense capturar (més eficient): (?:patron).

Regex Amb Flags

```
Pattern p1 = Pattern.compile("java", Pattern.CASE_INSENSITIVE);
Pattern p2 = Pattern.compile("^\\d+", Pattern.MULTILINE);
Pattern p3 = Pattern.compile(".*", Pattern.DOTALL);
Pattern p4 = Pattern.compile("java", Pattern.CASE_INSENSITIVE | Pattern.MULTILINE);
Pattern p5 = Pattern.compile("(?i)java"); // flag inline
```

★ BE THE CODE, MY FRIEND: Processant Un Log

💡 **Don Tip:** Les regex es proven amb matches() (tot el string) o find() (subcadena). group() extrau el capturat amb parèntesis.

```

import java.util.regex.*;
import java.util.*;

public class ProcesadorLog {
    public static void main(String[] args) {
        String log = ""
            [ERROR] 2024-03-15 10:30:45 - Usuario no encontrado: id=42
            [INFO] 2024-03-15 10:31:02 - Sesión iniciada: user=admin
            [WARN] 2024-03-15 10:32:10 - Memoria baja: 128MB disponible
            [ERROR] 2024-03-15 10:33:00 - Connection timeout: server=db01
            "";

        Pattern patron = Pattern.compile(
            "\\[(ERROR|INFO|WARN)\\]\\s+" +
            "(\\d{4}-\\d{2}-\\d{2})\\s+" +
            "(\\d{2}:\\d{2}:\\d{2})\\s+-\\s+(.*)");

        Matcher matcher = patron.matcher(log);
        List<String> errores = new ArrayList<>();

        while (matcher.find()) {
            String nivel = matcher.group(1);
            String fecha = matcher.group(2);
            String hora = matcher.group(3);
            String mensaje = matcher.group(4);
            System.out.printf("[%s] %s a las %s: %s%n", nivel, fecha, hora, mensaje);
            if (nivel.equals("ERROR")) errores.add(mensaje);
        }
        System.out.println("\nErrores: " + errores.size());
        for (String e : errores) System.out.println("  ✖ " + e);
    }
}

```

Cada parèntesi captura una part: nivell, data, hora i missatge. Per a un log de 2 GB llegiries línia a línia amb `BufferedReader`.

? No Hi Ha Preguntes Tontes! (Regex)

Q: Per què `"abc123".matches("\\d+")` retorna false?

A: `matches()` emparella **tot** el string. Usa `find()` per a subcadenaes.

Q: Com faig un punt literal?

A: Escapa'l: "\\.".

Q: Puc processar 1M línies amb la mateixa regex?

A: Sí, compila el Pattern una vegada fora del bucle i reutilitza el matcher.

Q: Es poden validar HTML amb regex?

A: No. HTML no és un llenguatge regular. Usa un parser.

Q: \w inclou accents?

A: No per defecte. Usa [a-zA-Záéíóúüñ] o Pattern.UNICODE_CHARACTER_CLASS.

Resum Ràpid: Regex

```
Pattern p = Pattern.compile("regex");    ← compilar
Matcher m = p.matcher(texto);            ← aplicar
m.find() → boolean                       ← buscar coincidència
m.group() / m.group(n) → String           ← obtindre match / grup
texto.matches("regex")                   ← tot coincidix?
texto.replaceAll("regex", "nuevo")       ← reemplaçar totes
texto.split("regex")                     ← dividir per patró
```

```
\d → dígit      \w → lletra/_    \s → espai      . → qualsevol char
* → 0+          + → 1+           ? → 0-1           {n} → exactament n
[abc] → conjunt  () → grup de captura
```

Exercicis Proposats

Exercici 1: L'entrevistador

Pregunta nom, edat, anys d'experiència i llenguatge favorit. Mostra resum amb printf().

Exercici 2: Calculadora de propines

Sol·licita total i percentatge. Mostra propina i total amb 2 decimals amb printf().

Exercici 3: El diari personal

Escriu en `diari.txt` amb data i text. Obri en mode append cada execució sense esborrar l'anterior.

Exercici 4: El comptador de línies

Llig un fitxer amb `BufferedReader` i mostra quantes línies, paraules i caràcters té.

Exercici 5: Validador de contrasenyes

Valida: mínim 8 caràcters, una majúscula, una minúscula, un dígit, un caràcter especial (`@#$$%^&+=`).

Exercici 6: Extractador d'URLs

Extrau URLs (`http/https`) separant protocol, domini i ruta amb grups de captura.

Exercici 7: Formatejador de dates

Convertix `dd/mm/aaaa` a `aaaa-mm-dd` amb `replaceAll()` i grups de captura.

Exercici 8: Xifrat Cèsar

Llig `missatge.txt`, desplaça cada caràcter +3 posicions, escriu en `missatge_xifrat.txt`.

Exercici 9: Analitzador de logs

Donat `[NIVELL] timestamp - missatge: detall`, compta missatges per nivell i mostra els ERROR.

RAs treballats en esta unitat:

- **RA5** - Entrada/Eixida: consola i fitxers
- **RA6** - Tipus avançats: Expressions regulars



Sergi Garcia Barea — CC BY-SA 4.0

Unitat 12: Connexió a Bases de Dades amb JDBC

Objectius d'aprenentatge

- Connectar Java a SQLite usant JDBC
- CRUD: INSERT, SELECT, UPDATE, DELETE amb PreparedStatement
- Entendre SQL Injection i com evitar-la
- Aplicar el patró DAO
- Gestionar transaccions amb commit i rollback

Què és JDBC?

JDBC (Java Database Connectivity) és un conjunt d'interfícies en `java.sql` que permeten a Java parlar amb qualsevol base de dades que tinga un driver JDBC.

 **Consell:** Pensa en JDBC com l'USB de les bases de dades: tant li fa SQLite, MySQL o PostgreSQL. Si tenen driver JDBC, Java es connecta.

Connexió a SQLite

SQLite és ideal per a aprendre: no necessita servidor, és un sol fitxer.

Els 5 Passos

1. Carregar el driver
2. Establir la connexió
3. Crear un Statement
4. Executar la consulta
5. Processar els resultats

Bonus no opcional: Tancar-ho tot.

Dependència Maven


```
<dependency>
  <groupId>org.xerial</groupId>
  <artifactId>sqlite-jdbc</artifactId>
  <version>3.45.1.0</version>
</dependency>
```

Connection

SQLite no necessita usuari ni contrasenya. La URL és un fitxer local:

```
String url = "jdbc:sqlite:instituto.db";
Connection con = DriverManager.getConnection(url);
```



Nota: SQLite crea el fitxer automàticament si no existeix. No necessites crear la BD a part.

Statement i ResultSet

```
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM alumnos");

while (rs.next()) {
    System.out.println(rs.getInt("id") + ": " + rs.getString("nombre"));
}
```

`rs.next()` avança a la següent fila. Torna false quan no queden més. El `ResultSet` comença **abans** de la primera fila.

executeQuery vs executeUpdate

Mètode	Per a	Torna
<code>executeQuery()</code>	SELECT	<code>ResultSet</code>
<code>executeUpdate()</code>	INSERT, UPDATE, DELETE	<code>int</code> (files afectades)



Advertència: Usar `executeQuery()` amb un INSERT llança excepció. Cada cosa al seu lloc.

Try-With-Resources

Des de Java 7, els recursos es tanquen sols:

```
try (Connection con = DriverManager.getConnection(url);
    Statement stmt = con.createStatement();
    ResultSet rs = stmt.executeQuery("SELECT * FROM alumnos")) {

    while (rs.next()) {
        System.out.printf("%d: %s\n", rs.getInt("id"), rs.getString("nombre"));
    }
} catch (SQLException e) {
    System.err.println("Error BD: " + e.getMessage());
}
```



Nota: L'ordre de tancament és invers al d'obertura: ResultSet → Statement → Connection. Try-with-resources ho fa sol. És màgic.

SQLException

És checked, t'obliga a capturar-la:

```
catch (SQLException e) {
    System.err.println("Error: " + e.getMessage());
    System.err.println("Codi: " + e.getErrorCode());
    System.err.println("Estat SQL: " + e.getSQLState());
}
```



Consell: No faces `catch (Exception e) {}` i et quedes tan ample. Això és tapar la llum de “check engine” amb esparadrap.

CRUD amb JDBC

CRUD = Create, Read, Update, Delete.

La Taula

```
CREATE TABLE contactos (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    nombre TEXT NOT NULL,  
    telefono TEXT,  
    email TEXT  
);
```

 **Nota:** SQLite usa INTEGER PRIMARY KEY AUTOINCREMENT en lloc de INT AUTO_INCREMENT.

EL POJO

```
public class Contacto {  
    private int id;  
    private String nombre;  
    private String telefono;  
    private String email;  
  
    public Contacto() {}  
    public Contacto(String nombre, String telefono, String email) {  
        this.nombre = nombre;  
        this.telefono = telefono;  
        this.email = email;  
    }  
  
    public int getId() { return id; }  
    public void setId(int id) { this.id = id; }  
    public String getNombre() { return nombre; }  
    public void setNombre(String nombre) { this.nombre = nombre; }  
    public String getTelefono() { return telefono; }  
    public void setTelefono(String telefono) { this.telefono = telefono; }  
    public String getEmail() { return email; }  
    public void setEmail(String email) { this.email = email; }  
  
    @Override  
    public String toString() {  
        return id + " - " + nombre + " (" + telefono + ")";  
    }  
}
```

INSERT (Create)

```

public void insertarContacto(Contacto c) {
    String sql = "INSERT INTO contactos (nombre, telefono, email) VALUES (?, ?, ?)";
    try (Connection con = DriverManager.getConnection(URL);
        PreparedStatement pstmt = con.prepareStatement(sql)) {
        pstmt.setString(1, c.getNombre());
        pstmt.setString(2, c.getTelefono());
        pstmt.setString(3, c.getEmail());
        pstmt.executeUpdate();
    } catch (SQLException e) {
        System.err.println("Error en inserir: " + e.getMessage());
    }
}

```

SELECT (Read)

```

public List<Contacto> obtenerTodos() {
    List<Contacto> contactos = new ArrayList<>();
    String sql = "SELECT * FROM contactos ORDER BY nombre";
    try (Connection con = DriverManager.getConnection(URL);
        PreparedStatement pstmt = con.prepareStatement(sql);
        ResultSet rs = pstmt.executeQuery()) {
        while (rs.next()) {
            Contacto c = new Contacto(
                rs.getString("nombre"),
                rs.getString("telefono"),
                rs.getString("email")
            );
            c.setId(rs.getInt("id"));
            contactos.add(c);
        }
    } catch (SQLException e) {
        System.err.println("Error en consultar: " + e.getMessage());
    }
    return contactos;
}

```

Cerca amb LIKE

```

public List<Contacto> buscarPorNombre(String nombre) {
    List<Contacto> contactos = new ArrayList<>();
    String sql = "SELECT * FROM contactos WHERE nombre LIKE ?";
    try (Connection con = DriverManager.getConnection(URL);
        PreparedStatement pstmt = con.prepareStatement(sql)) {
        pstmt.setString(1, "%" + nombre + "%");
        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                Contacto c = new Contacto(
                    rs.getString("nombre"),
                    rs.getString("telefono"),
                    rs.getString("email")
                );
                c.setId(rs.getInt("id"));
                contactos.add(c);
            }
        }
    } catch (SQLException e) {
        System.err.println("Error en buscar: " + e.getMessage());
    }
    return contactos;
}

```

💡 **Consell:** Torna List<Contacto> en lloc de ResultSet. El ResultSet està lligat a la connexió. Si la tanques, el ResultSet mor. Millor convertix a objectes Java i tanca-ho tot.

UPDATE (Update)

```

public boolean actualizarContacto(Contacto c) {
    String sql = "UPDATE contactos SET nombre = ?, telefono = ?, email = ? WHERE id = ?";
    try (Connection con = DriverManager.getConnection(URL);
        PreparedStatement pstmt = con.prepareStatement(sql)) {
        pstmt.setString(1, c.getNombre());
        pstmt.setString(2, c.getTelefono());
        pstmt.setString(3, c.getEmail());
        pstmt.setInt(4, c.getId());
        return pstmt.executeUpdate() > 0;
    } catch (SQLException e) {
        System.err.println("Error en actualitzar: " + e.getMessage());
        return false;
    }
}

```

 **Advertència:** Sense WHERE id = ?, actualitzes TOTS els contactes. 500 contactes anomenats “John Doe”. WHERE SEMPRE.

DELETE (Delete)

```

public boolean eliminarContacto(int id) {
    String sql = "DELETE FROM contactos WHERE id = ?";
    try (Connection con = DriverManager.getConnection(URL);
        PreparedStatement pstmt = con.prepareStatement(sql)) {
        pstmt.setInt(1, id);
        return pstmt.executeUpdate() > 0;
    } catch (SQLException e) {
        System.err.println("Error en eliminar: " + e.getMessage());
        return false;
    }
}

```

 **Consell:** Confirma amb l'usuari abans de borrar. El DELETE no té Ctrl+Z. És per a sempre. Com les decisions en una pel·lícula de terror.

PreparedStatement i SQL Injection

Concatenar strings per a construir SQL és PERILLÓS:

```
// PERILL: això és una bomba
String sql = "SELECT * FROM alumnos WHERE nombre = '" + inputUsuario + "'";
```

Si l'usuari escriu Luis'; DROP TABLE alumnos; --, la teua taula alumnos es borra. Açò és **SQL Injection** i passa de veritat.

⚠ **Advertència:** MAI construïscues SQL concatenant strings amb dades de l'usuari. És com deixar les claus posades amb un cartell "PASSE VOSTÉ".

PreparedStatement al Rescat

```
String sql = "SELECT * FROM alumnos WHERE nombre = ?";
PreparedStatement pstmt = conexion.prepareStatement(sql);
pstmt.setString(1, nombre);
ResultSet rs = pstmt.executeQuery();
```

Els ? són placeholders posicionals (comencen en 1):

Tipus	Mètode
String	setString(i, valor)
int	setInt(i, valor)
double	setDouble(i, valor)
boolean	setBoolean(i, valor)
Date	setDate(i, valor) (usa java.sql.Date)
null	setNull(i, Types.TIPO)

💡 **Consell:** Per a moltes consultes iguals, PreparedStatement pot ser MÉS RÀPID perquè la BD compila la consulta una sola vegada.

La Història de Bobby Tables

```
Madre: "He criat al meu fill perquè siga un programador acurat,
       no un hacker que robe dades."
Fill: "Hola, em dic Robert"); DROP TABLE Students; --"
```

No sigues Bobby Tables. Usa PreparedStatement.

Guia de Placeholders

```
String sql = "SELECT * FROM alumnos WHERE curso = ? AND edad > ? ORDER BY nombre";
try (PreparedStatement pstmt = conexion.prepareStatement(sql)) {
    pstmt.setString(1, "DAM");
    pstmt.setInt(2, 18);
    try (ResultSet rs = pstmt.executeQuery()) {
        while (rs.next()) {
            System.out.println(rs.getString("nombre"));
        }
    }
}
```

¿Veu qué net? No hi ha concatenació, no hi ha cometes escapades, no hi ha risc d'injecció. Només ? nets i ordenats.

★ BE THE CODE, MY FRIEND: Tu Eres un PreparedStatement

💡 **Don Tip:** PreparedStatement prevé l'SQL injection perquè separa el codi SQL de les dades. Mai concatenes strings en SQL.

```
Java: Oye, necesito ejecutar "SELECT * FROM productos WHERE precio < ? AND
categoria = ?"
Tú: Dame los valores.
Java: setDouble(1, 50.0); setString(2, "informatica")
Tú: Voy a la BD. Le paso la consulta ya montada y escapada. Sin riesgo.
BD: Aquí tienes los resultados.
Tú: Toma, Java. Un ResultSet limpito.
```

Patrón DAO

El **DAO (Data Access Object)** encapsula l'accés a la base de dades. Separem la lògica de negoci del SQL.

La Interfície


```
public interface ContactoDAO {
    void insertarContacto(Contacto c);
    List<Contacto> obtenerTodos();
    List<Contacto> buscarPorNombre(String nombre);
    boolean actualizarContacto(Contacto c);
    boolean eliminarContacto(int id);
}
```

La Implementació

```
public class ContactoDAOImpl implements ContactoDAO {
    private static final String URL = "jdbc:sqlite:contactos.db";

    @Override
    public void insertarContacto(Contacto c) { ... }

    @Override
    public List<Contacto> obtenerTodos() { ... }

    @Override
    public List<Contacto> buscarPorNombre(String nombre) { ... }

    @Override
    public boolean actualizarContacto(Contacto c) { ... }

    @Override
    public boolean eliminarContacto(int id) { ... }
}
```

Els mètodes són exactament els que vas veure en la secció CRUD. El DAO els unifica davall d'una mateixa interfície.



Nota: DAO vs Repository: són quasi el mateix. DAO està més pegat a la BD (INSERT, SELECT). Repository és més de domini (guardar, buscar). En projectes xicotets s'usen com a sinònims.

Transaccions

De vegades necessites que diverses operacions ocorreguen totes juntes o cap. Com una transferència bancària.

commit i rollback

```
public void transferirContacto(int idOrigen, int idDestino) {
    String quitar = "UPDATE contactos SET grupo = 'vacio' WHERE id = ?";
    String poner = "UPDATE contactos SET grupo = 'destino' WHERE id = ?";

    try (Connection con = DriverManager.getConnection(URL)) {
        con.setAutoCommit(false); // comencem transacció

        try (PreparedStatement q = con.prepareStatement(quitar);
            PreparedStatement p = con.prepareStatement(poner)) {

            q.setInt(1, idOrigen);
            q.executeUpdate();

            if (idOrigen == idDestino)
                throw new SQLException("Destí invàlid");

            p.setInt(1, idDestino);
            p.executeUpdate();

            con.commit(); // tot bé, confirmem
            System.out.println("Transferència OK");

        } catch (SQLException e) {
            con.rollback(); // alguna cosa va fallar, desfem tot
            System.err.println("Va fallar, tot desfet: " + e.getMessage());
        }
    } catch (SQLException e) {
        System.err.println("Error de connexió: " + e.getMessage());
    }
}
```

⚠ Advertència: Sense transaccions, si falla la segona operació, la primera ja es va executar. El sistema queda inconsistent. Usa transaccions per a operacions atòmiques (tot o res). Com un pont que o està sencer o no està.

Savepoints

Pots marcar punts intermedis dins d'una transacció per a no haver de desfer-ho tot si alguna cosa falla:

```
con.setAutoCommit(false);
Savepoint sp = con.setSavepoint("despuesInsert");

// ... més operacions ...

if (algoMal) {
    con.rollback(sp); // torna al savepoint, no desfà tot
} else {
    con.commit();
}
```

★ BE THE CODE, MY FRIEND: Implementa un DAO Complet Des de Zero

💡 **Don Tip:** Un DAO encapsula l'accés a dades. Si canvies de SQLite a MySQL, només canvia el DAO, la resta del codi ni se n'assabenta.

Eres l'arquitecte d'una aplicació de gestió de biblioteca. Tens estes entitats:

```
class Libro {
    String isbn;
    String titulo;
    String autor;
    boolean disponible;
}

class Prestamo {
    int id;
    String isbn;
    String socio;
    LocalDate fechaPrestamo;
    LocalDate fechaDevolucion;
}
```

Preguntes:

1. Quants DAOs crearies? Un per entitat (LibroDAO, PrestamoDAO) o un de sol genèric?
2. Quins mètodes tindria cada DAO?
3. On posaries la lògica de “un llibre no es pot prestar si ja està prestat”?
4. Usaries transaccions per a gestionar préstecs i devolucions?

Usuari: Vull afegir a "Maria".

Tu: PreparedStatement → INSERT → executeUpdate() → 1 fila.

Usuari: Busca'm a Maria.

Tu: PreparedStatement → SELECT LIKE → ResultSet → List<Contacto>.

Usuari: Canvia el telèfon de Maria.

Tu: PreparedStatement → UPDATE WHERE id = ? → 1 fila.

Usuari: Borra a Maria.

Tu: Segur?

Usuari: Sí.

Tu: DELETE WHERE id = ? → 1 fila. Adéu, Maria.

Usuari: Era broma!

Tu: El DELETE no té desfer. Ho sent.



EL RING: Statement vs PreparedStatement

Dos formes d'executar SQL discuteixen.

Statement: «Jo soc l'original. Senzill, directe. `stmt.executeQuery("SELECT * FROM usuarios")`. Per a consultes simples soc perfecte!»

PreparedStatement: «Sí, però concatenar strings en SQL és una bomba de rellotgeria. Injecció SQL, errors de sintaxi amb cometes... Jo separo l'SQL de les dades amb ? i soc segur.»

Statement: «Per a una consulta fixa, una sola vegada, per què he de preparar res?»

PreparedStatement: «A més, jo faig cache del pla d'execució. Si executes la mateixa consulta diverses voltes amb diferents paràmetres, soc més ràpid. I en Java, quasi sempre executes la mateixa consulta amb diferents valors.»

Statement: «Val, però jo servisc per a DDL: CREATE TABLE, ALTER...»

PreparedStatement: «Cert. Per a DDL usa Statement. Per a DML (SELECT, INSERT, UPDATE, DELETE) usa PreparedStatement. Tracte?»

Statement: «Tracte.»

💡 **Don Tip:** Usa sempre PreparedStatement per a consultes amb dades d'usuari. No és només seguretat: és més ràpid en consultes repetitives i més llegible.

¡No Hay Preguntas Tontas!

? ¡No Hay Preguntas Tontas!

Q: Què passa si no tanque la connexió? **A:** Es queda oberta, consumix memòria. Els servidors tenen un límit de connexions simultànies. Si arribes al límit, el pròxim `getConnection()` casca amb "Too many connections". Tanca sempre.

Q: És necessari `Class.forName()` per a carregar el driver? **A:** Des de Java 6 no cal. Però ho veuràs en tutorials antics. Si ho poses no passa res, si no ho poses tampoc.

Q: PreparedStatement és més lent que Statement? **A:** Per a una consulta, igual. Per a moltes iguals, PreparedStatement pot ser MÉS RÀPID perquè la BD compila la consulta una vegada.

Q: Puc usar `?` per a noms de taula o columna? **A:** No. `SELECT * FROM ?` no funcion. Els placeholders només valen per a valors, no per a identificadors.

Q: Statement s'usa encara? **A:** Està zombi. Si veus un Statement en codi real (que no siga una query literal), és bandera roja. PreparedStatement SEMPRE.

Q: DAO i Repository són el mateix? **A:** Quasi. DAO està pegat a la BD (INSERT, SELECT). Repository és més de domini (guardar, buscar). En projectes xicotets s'usen com a sinònims.

Q: He de posar la URL de la BD en el codi? **A:** No. Usa `config.properties` o variables d'entorn. Les credencials no van en el codi. És com portar la contrasenya del banc escrita en el front.

Q: I si canvie de BD, he de reescriure-ho tot? **A:** Si vas usar SQL estàndard i JDBC, només canvis el driver i la URL. Eixa és la màgia de JDBC.

Q: Puc compartir una connexió entre diversos fils? **A:** Tècnicament sí, pràcticament no. Connection no és thread-safe. Usa un pool de connexions (HikariCP) i cada fil que demane la seua.

Exercicis Proposats

1. **Connexió des de zero** — Crea una classe `TestConnexio` que es connecte a `SQLite test.db`, cree una taula `alumnes(id INTEGER PRIMARY KEY, nom TEXT, nota REAL)` i inserisca 3 alumnes.
2. **CRUD d'estudiants** — Implementa un DAO complet per a `Estudiant` amb: insertar, listar, buscarPerId, actualitzar, eliminar. Usa `PreparedStatement`.
3. **Transacció bancària** — Simula una transferència entre dos comptes en una taula `comptes(id, titular, saldo)`. La transferència ha de ser atòmica: si falla, fes `rollback()`.
4. **Consulta amb JOIN** — Crea dos taules relacionades (`comandes` i `clients`). Escriu una consulta `JOIN` que mostre el nom del client i el total de les seues comandes.
5. **Exportar a CSV** — Afig un mètode `exportarCSV(String archivo)` al teu DAO que llegeixca tots els registres i els escriu en CSV amb `try-with-resources`.
6. **DAO genèric** — Refactoritza el teu DAO per a que siga genèric: `public abstract class DAO<T>`. Implementa `EstudiantDAO extends DAO<Estudiant>`.

Bones Pràctiques: El Decàleg del JDBC

1. **PreparedStatement sempre.** Mai concatenes SQL.
2. **Try-with-resources.** `Connection`, `Statement`, `ResultSet` es tanquen sols.
3. **No tornes ResultSet.** Torna llistes d'objectes.
4. **Captura SQLException amb missatge descriptiu.** No atrapes `Exception` a la babalà.
5. **Usa transaccions per a operacions múltiples.** Tot o res.
6. **No exposes credencials en el codi.** Usa fitxers de configuració.
7. **Comprova files afectades.** `executeUpdate()` et diu si va funcionar.
8. **No faces consultes dins de bucles.** Lentíssim. Una sola consulta basta.
9. **WHERE sempre en UPDATE i DELETE.** O pagues les conseqüències.
10. **Confirma abans de borrar.** L'usuari sempre s'equivoca.

L'ENIGMA

Tens dues taules en `SQLite`: `usuarios(id, nombre, email)` i `pedidos(id, usuario_id, total)`.

Vols obtenir tots els usuaris que han fet algun pedido amb total superior a 100€. Quantes consultes SQL necessites com a mínim? I si uses JOIN?

Pista: hi ha una diferència entre “diverses consultes en bucle” i “una consulta amb JOIN”.

☛ **Don Tip:** Una consulta amb JOIN és UNA sola crida a la base de dades. Fer una consulta per cada usuari en un bucle és N+1 consultes, que és molt més lent.

Resum Exprés

Concepte	Analogia
JDBC	Traductor universal Java ↔ BD
Driver	L'endoll específic per a cada BD
Connection	El cable telefònic
PreparedStatement	El missatger segur
ResultSet	La resposta (taula virtual)
executeQuery	Per a SELECT
executeUpdate	Per a INSERT/UPDATE/DELETE
commit / rollback	Transaccions (tot o res)
DAO	Patró que separa SQL del negoci

Sempre:

1. Obri connexió
2. Crea PreparedStatement
3. Executa consulta
4. Processa resultats
5. Tanca-ho tot (try-with-resources)

RAs treballats en esta unitat:

- **RA9** - Bases de dades relacionals



Sergi Garcia Barea — CC BY-SA 4.0



Unitat 13: Servir i Consumir APIs amb Web

Objectius d'aprenentatge

- Entendre el model petició-resposta HTTP
- Crear un servidor web mínim amb `HttpServer`
- Servir pàgines HTML des de Java
- Processar formularis i paràmetres GET/POST
- Tornar JSON i consumir-lo amb JavaScript
- Consumir APIs externes amb `java.net.http.HttpClient`
- Parsejar JSON i gestionar errors HTTP
- Conèixer frameworks moderns (Javalin, Spring Boot)

Hola, Món Web

“Fins ara els teus programes vivien en la terminal. És hora que isquen a Internet. Sense JavaScript frameworks. Sense servidors d'aplicacions. Només Java i HTTP.”

Hem vist consola, fitxers i fins bases de dades. Ara toca allò que hui en dia fa quasi qualsevol aplicació: **parlar per HTTP**. I sí, Java pot ser servidor web sense instal·lar Tomcat ni Spring. Però a més, també pot **consumir** APIs externes com un client més.

1. El Protocol HTTP en 30 Segons

Quan escrius `https://google.com` en el teu navegador, ocorre això:



HTTP és un protocol de **petició-resposta**. Tu demanes una URL i el servidor et torna un recurs (HTML, JSON, imatge, etc.). Això és tot. La resta (cookies, sessions, APIs) són capes que es construeixen damunt.

2. Servidor Web Mínim amb HttpServer

Java inclou des de la versió 6 un servidor HTTP bàsic en el paquet `com.sun.net.httpserver`. No necessites res més:

```
import com.sun.net.httpserver.HttpServer;
import com.sun.net.httpserver.HttpExchange;
import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;

public class ServidorMinimo {
    public static void main(String[] args) throws IOException {
        HttpServer server = HttpServer.create(
            new InetSocketAddress(8080), 0
        );
        server.createContext("/", intercambio -> {
            String resp = "Hola, Mundo Web!";
            intercambio.sendResponseHeaders(200, resp.length());
            OutputStream os = intercambio.getResponseBody();
            os.write(resp.getBytes());
            os.close();
        });
        server.setExecutor(null);
        server.start();
        System.out.println("Servidor en http://localhost:8080");
    }
}
```

Compila i executa. Obri `http://localhost:8080` en el teu navegador. Acabes de crear el teu primer servidor web en Java.

3. Servint HTML

Tornar text pla està bé per a provar, però volem HTML. Anem a servir una pàgina completa:

```

server.createContext("/", intercambio -> {
    String html = ""
        <!DOCTYPE html>
        <html>
        <head><title>Mi App Java</title>
        <meta charset="UTF-8">
        </head>
        <body>
            <h1>Bienvenido a mi App</h1>
            <p>Esto se sirve desde Java.</p>
        </body>
        </html>
        "";
    intercambio.getResponseHeaders()
        .set("Content-Type", "text/html; charset=UTF-8");
    intercambio.sendResponseHeaders(200, html.getBytes().length);
    intercambio.getResponseBody().write(html.getBytes());
    intercambio.getResponseBody().close();
});

```

També pots llegir HTML des d'un fitxer amb `Files.readString()` i servir-lo igual. Així tens l'HTML separat del codi Java.

4. Paràmetres GET: El Client Pregunta

Quan un formulari s'envia per GET, les dades van en la URL: `http://localhost:8080/saludo?nombre=Ana`. Així es llig:

```

server.createContext("/saludo", intercambio -> {
    String query = intercambio.getRequestURI().getQuery();
    String nombre = "Mundo";
    if (query != null && query.startsWith("nombre=")) {
        nombre = query.split("=")[1];
    }
    String html = "<h1>Hola, " + nombre + "!</h1>";
    intercambio.getResponseHeaders()
        .set("Content-Type", "text/html");
    intercambio.sendResponseHeaders(200, html.getBytes().length);
    intercambio.getResponseBody().write(html.getBytes());
    intercambio.getResponseBody().close();
});

```

⚠ WARNING: Això és fràgil. Si el nom conté & o %20, petarà. En producció usa `URLDecoder.decode()` i un parser de query params de veritat. Això és un exemple didàctic, no codi de producció.

5. Formularis POST: El Client Envia Dades

Per a rebre dades d'un formulari POST necessites llegir el cos de la petició:

```

server.createContext("/procesar", intercambio -> {
    if ("POST".equals(intercambio.getRequestMethod())) {
        byte[] body = intercambio.getRequestBody().readAllBytes();
        String datos = new String(body);
        // datos = "nombre=Ana&edad=25"
        String nombre = extraerParametro(datos, "nombre");
        String respuesta = "<h1>Recibido: " + nombre + "</h1>";
        intercambio.getResponseHeaders()
            .set("Content-Type", "text/html");
        intercambio.sendResponseHeaders(200,
            respuesta.getBytes().length);
        intercambio.getResponseBody()
            .write(respuesta.getBytes());
        intercambio.getResponseBody().close();
    }
});

```

I l'HTML del formulari:

```
<form action="/procesar" method="POST">
  <input name="nombre" placeholder="Tu nombre">
  <button>Enviar</button>
</form>
```



Consell: Per què `"POST".equals(intercambio.getRequestMethod())` i no `intercambio.getRequestMethod().equals("POST")`? Perquè si `getRequestMethod()` torna null, la segona forma petà amb `NullPointerException`. La primera forma és **null-safe**. És una mania que t'estalviarà maldecaps.

6. Tornant JSON (Com una API de Veritat)

Les aplicacions modernes no tornen HTML directament. El frontend (JavaScript) demana dades i el backend li les dona en JSON. Ací un exemple:

```
server.createContext("/api/usuarios", intercambio -> {
    String json = ""
    [
        {"id":1,"nombre":"Ana","edad":25},
        {"id":2,"nombre":"Luis","edad":30}
    ]
    """;
    intercambio.getResponseHeaders()
        .set("Content-Type", "application/json");
    intercambio.sendResponseHeaders(200, json.getBytes().length);
    intercambio.getResponseBody().write(json.getBytes());
    intercambio.getResponseBody().close();
});
```

I des del HTML pots cridar-ho amb JavaScript:

```
<script>
fetch('/api/usuarios')
  .then(r => r.json())
  .then(data => console.log(data));
</script>
```

7. Mini Projecte: Gestor de Tasques (API REST)

Combinant-ho tot, pots construir una API REST completa. El patró és:

Mètode	Ruta	Acción
GET	/api/taques	Llistar totes
POST	/api/taques	Crear una
PUT	/api/taques/1	Actualitzar
DELETE	/api/taques/1	Esborrar

Cada ruta s'implementa com un context en el servidor, i les dades es guarden en un `ArrayList` en memòria (més avant, en base de dades).

8. Consumir APIs Externes amb `HttpClient`

Fins ara has sigut el **servidor**. Però en el món real, els teus programes també seran **clients** que criden a APIs de tercers: GitHub, OpenAI, el temps, la teua xarxa social preferida...

Java 11 portà `java.net.http.HttpClient` — un client HTTP modern, sense dependències externes, que suporta HTTP/2, peticions síncrones i asíncrones, i gestió de capçaleres.



Nota: Abans de Java 11 havies d'usar `HttpURLConnection` (lleig, verbós, un càstig) o llibreries externes com Apache `HttpClient` o `OkHttp`. `java.net.http.HttpClient` arribà per a salvar la humanitat.

8.1 GET: Demanar Dades a una API

Començem per lo bàsic: fer una petició GET a una API pública i llegir la resposta com a text.

```

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class ClienteGET {
    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newHttpClient();

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://api.github.com/users/octocat"))
            .GET()
            .build();

        HttpResponse<String> response = client.send(
            request, HttpResponse.BodyHandlers.ofString()
        );

        System.out.println("Código: " + response.statusCode());
        System.out.println("Cuerpo: " + response.body());
    }
}

```



Nota: `HttpResponse.BodyHandlers.ofString()` li diu a Java “convertix-me el cos de la resposta en un String”. Hi ha altres handlers: `ofByteArray()`, `ofInputStream()`, `ofFile(Path)`... segons el que necessites.

8.2 Parsejar JSON amb la Llibreria que Calga

El cos que torna GitHub és JSON. En Java no hi ha un parser JSON natiu (sí, és trist, però és així). Necessites una llibreria. Les més usades:

Llibreria	Grup Maven	Ideal per a
Gson (Google)	<code>com.google.code.gson:gson</code>	Senzilla, mapeig a classes
Jackson	<code>com.fasterxml.jackson.core:jackson-databind</code>	Potent, ràpida, estàndard industrial

Llibreria	Grup Maven	Ideal per a
org.json	org.json:json	La més simple, sense mapeig

En els exemples usarem **Gson**, que és la més intuitiva:

```
import com.google.gson.Gson;
import com.google.gson.JsonObject;

// ... después de obtener response.body()
Gson gson = new Gson();
JsonObject json = gson.fromJson(response.body(), JsonObject.class);

String login = json.get("login").getAsString();
String nombre = json.get("name").getAsString();
System.out.println("Usuario: " + login + " - " + nombre);
```

O si prefereixes mapejar a una classe:

```
record UsuarioGitHub(String login, String name, int public_repos) {}

UsuarioGitHub usuario = gson.fromJson(response.body(), UsuarioGitHub.class);
System.out.println(usuario.name() + " tiene " + usuario.public_repos() + " repos
públicos");
```

⚠ WARNING: Si l'API torna camps que no existixen en el teu record, Gson per defecte els ignora. Si el teu record té camps que no estan en el JSON, es queden amb valor null. Amb Jackson pots configurar-ho amb `@JsonIgnoreProperties(ignoreUnknown = true)`.

8.3 POST: Enviar Dades a una API

Per a enviar dades (crear un recurs, fer login, etc.) usem POST amb un cos JSON:


```

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class ClientePOST {
    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newHttpClient();

        String json = """
            {"title": "Aprender HttpClient",
            "body": "Ya casi lo tengo",
            "userId": 1}
            """;

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://jsonplaceholder.typicode.com/posts"))
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(json))
            .build();

        HttpResponse<String> response = client.send(
            request, HttpResponse.BodyHandlers.ofString()
        );

        System.out.println("Código: " + response.statusCode());
        System.out.println("Creado: " + response.body());
    }
}

```



Consell: `HttpRequest.BodyPublishers` té mètodes per a enviar `String`, `byte[]`, fitxers, etc. El més comú és `ofString()` per a JSON. Si no poses la capçalera `Content-Type: application/json`, el servidor pot rebutjar la teua petició o interpretar mal el cos.

8.4 Capçaleres, Timeouts i Gestió d'Errors

Les APIs no sempre es comporten bé. De vegades tarden, de vegades cauen, de vegades et tornen 401, 403, 404 o 500. El teu codi ha d'estar preparat.

```

import java.net.http.HttpConnectTimeoutException;
import java.time.Duration;

HttpClient client = HttpClient.newBuilder()
    .connectTimeout(Duration.ofSeconds(10)) // tiempo máximo para conectar
    .build();

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.github.com/users/someone"))
    .header("Accept", "application/vnd.github+json")
    .header("User-Agent", "MiAppJava/1.0")
    .timeout(Duration.ofSeconds(15)) // tiempo máximo para recibir respuesta
    .GET()
    .build();

try {
    HttpResponse<String> response = client.send(request,
        HttpResponse.BodyHandlers.ofString());

    int codigo = response.statusCode();
    if (codigo >= 200 && codigo < 300) {
        System.out.println("Éxito: " + response.body());
    } else if (codigo == 404) {
        System.out.println("El recurso no existe.");
    } else if (codigo == 403) {
        System.out.println("Prohibido. ¿Token necesario?");
    } else if (codigo >= 500) {
        System.out.println("Error del servidor. Vuelve a intentarlo.");
    }
} catch (HttpConnectTimeoutException e) {
    System.out.println("El servidor no responde (timeout de conexión).");
} catch (java.net.http.HttpTimeoutException e) {
    System.out.println("La respuesta tardó demasiado.");
} catch (java.io.IOException e) {
    System.out.println("Error de red: " + e.getMessage());
} catch (InterruptedException e) {
    System.out.println("La petición fue interrumpida.");
}

```

⚠ WARNING: Moltes APIs públiques (GitHub, Twitter, etc.) requereixen un **token d'autenticació** en la capçalera Authorization. Sense ell, tens quotes molt baixes (rate limiting) o accés denegat. Si veus un 403, probablement necessites registrar una aplicació i obtindre un token.

Ací tens les capçaleres més comunes que enviaràs:

Capçalera	Propòsit
Content-Type	Tipus de dades que envies (application/json)
Accept	Tipus de dades que esperes rebre
Authorization	Token Bearer, Basic Auth, etc.
User-Agent	Identifica la teua aplicació (moltes APIs ho exigixen)
Cache-Control	Control de memòria cau

8.5 Exemple Complet: Consultar Repos de GitHub

Anem a juntar-ho tot: consumir l'API de GitHub per a llistar els repositoris d'un usuari.

```

import com.google.gson.Gson;
import com.google.gson.JsonArray;
import com.google.gson.JsonObject;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;

public class GitHubRepos {
    public static void main(String[] args) throws Exception {
        String usuario = args.length > 0 ? args[0] : "octocat";

        HttpClient client = HttpClient.newBuilder()
            .connectTimeout(Duration.ofSeconds(10))
            .build();

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://api.github.com/users/" + usuario + "/repos"))
            .header("Accept", "application/vnd.github+json")
            .header("User-Agent", "JavaCliente/1.0")
            .timeout(Duration.ofSeconds(15))
            .GET()
            .build();

        HttpResponse<String> response = client.send(request,
            HttpResponse.BodyHandlers.ofString());

        if (response.statusCode() == 200) {
            Gson gson = new Gson();
            JsonArray repos = gson.fromJson(response.body(), JsonArray.class);

            System.out.println("Repositorios de " + usuario + ":");
            System.out.println("_____");
            for (int i = 0; i < repos.size(); i++) {
                JsonObject repo = repos.get(i).getAsJsonObject();
                String nombre = repo.get("name").getString();
                String lenguaje = repo.has("language")
                    && !repo.get("language").isJsonNull()
                    ? repo.get("language").getString()
                    : "?";
                int estrellas = repo.get("stargazers_count").getAsInt();
            }
        }
    }
}

```

```

        System.out.printf("★ %s (%s) ★ %d%n", nombre, lenguaje, estrellas);
    }
} else if (response.statusCode() == 403) {
    System.out.println("Rate limit alcanzado. Espera un minuto.");
} else {
    System.out.println("Error " + response.statusCode()
        + " - ¿existe el usuario " + usuario + "?");
}
}
}
}

```

 **Nota:** Fixa't en `repo.has("language") && !repo.get("language").isJsonNull()`. En JSON un camp pot existir però valdre null. Si intentes `getAsString()` sobre un null, Gson et llança una excepció. Este patró és molt comú al parsejar APIs reals.

Exemple d'eixida:

Repositoris de octocat:

★ Hello-World (Java) ★ 2727
 ★ Spoon-Knife (HTML) ★ 13291
 ★ Octocat (?) ★ 5

8.6 Crides Asíncrones amb `sendAsync`

Si necessites fer diverses peticions sense bloquejar el programa, `HttpClient` també suporta mode asíncron:

```
CompletableFuture<HttpResponse<String>> futuro = client.sendAsync(  
    request, HttpResponse.BodyHandlers.ofString()  
);  
  
// Mientras tanto, puedes hacer otras cosas...  
System.out.println("Esperando respuesta...");  
  
// Cuando llegue:  
futuro.thenAccept(response -> {  
    System.out.println("Código: " + response.statusCode());  
    System.out.println(response.body());  
});  
  
// O bloquear hasta que llegue (no recomendado si vas justo de tiempo):  
HttpResponse<String> resp = futuro.get();
```

💡 **Consell:** `sendAsync` torna un `CompletableFuture`, que és la forma elegant de Java per a treballar amb codi asíncron sense liar-te amb fils manuals. Pots encadenar diverses crides amb `thenApply`, `thenCompose`, etc.

8.7 PUT, DELETE i Altres Mètodes

`HttpClient` suporta tots els mètodes HTTP. La diferència és la crida al builder:

```
// PUT
HttpRequest putRequest = HttpRequest.newBuilder()
    .uri(URI.create("https://api.ejemplo.com/recursos/1"))
    .header("Content-Type", "application/json")
    .PUT(HttpRequest.BodyPublishers.ofString(
        "{\"nombre\": \"Actualizado\"}"))
    .build();

// DELETE
HttpRequest deleteRequest = HttpRequest.newBuilder()
    .uri(URI.create("https://api.ejemplo.com/recursos/1"))
    .DELETE()
    .build();

// Con método personalizado (PATCH, etc.)
HttpRequest patchRequest = HttpRequest.newBuilder()
    .uri(URI.create("https://api.ejemplo.com/recursos/1"))
    .method("PATCH", HttpRequest.BodyPublishers.ofString(
        "{\"nombre\": \"Parcial\"}"))
    .build();
```

9. Frameworks Moderns

El `HttpServer` bàsic està bé per a aprendre, però en el món real s'usen frameworks:

- **Javalin:** Minimalista, modern, ideal per a APIs. Amb 3 línies tens un servidor.
- **Spring Boot:** L'estàndard industrial. Té de tot, però és més pesat.
- **Spark:** Semblant a Javalin, molt lleuger.

Exemple amb Javalin:

```
import io.javalin.Javalin;

public class App {
    public static void main(String[] args) {
        Javalin app = Javalin.create().start(8080);
        app.get("/", ctx -> ctx.result("Hola desde Javalin"));
        app.get("/api/usuarios", ctx ->
            ctx.json(List.of(new Usuario(1, "Ana"))));
    }
}
```

 **Consell:** Per als butlletins d'esta unitat usarem el `HttpServer` natiu de Java com a servidor i `HttpClient` per a consumir APIs. No necessites instal·lar res més que Gson (o la llibreria JSON que preferisques). Els frameworks els veuràs en el mòdul de “Desenrotllament d'Interfícies” i en el projecte final.

★ BE THE CODE, MY FRIEND: Seguiu la Pista d'una Petició HTTP

 **Don Tip:** El mètode HTTP el determina el client. GET per a obtenir, POST per a enviar. El `exchange.getRequestMethod()` et diu quin és.

Obre el `HttpServer` que has creat. Afegiu això just abans de `server.start()`:

```
server.createContext("/track", exchange -> {
    System.out.println("Mètode: " + exchange.getRequestMethod());
    System.out.println("URI: " + exchange.getRequestURI());
    System.out.println("Capçaleres: " + exchange.getRequestHeaders().entrySet());
    exchange.getResponseHeaders().add("Content-Type", "text/plain");
    exchange.sendResponseHeaders(200, 0);
    try (var os = exchange.getResponseBody()) {
        os.write("Tot bé, gràcies per preguntar.".getBytes());
    }
});
```

Preguntes:

1. Obri `http://localhost:8080/track?nombre=Ana&edad=25` — Què veus en la consola del servidor?

2. Quin mètode HTTP has usat sense especificar-lo? (Pista: el navegador fa GET per defecte)
3. Què passa si envies una petició POST des de curl o des de JavaScript?
4. **Repte:** Modifica el `System.out` per a que escriba en un fitxer `access.log`. Cada petició en una línia: `[MÈTODE] URI → 200 OK`

? No Hi Ha Preguntes Tontes!

HTTP i HTTPS són el mateix? — Quasi. HTTPS és HTTP amb xifrat (SSL/TLS). Per a producció sempre HTTPS; per a desenvolupament local, HTTP val.

Quin port usar? — 8080 és l'estàndard per a desenvolupament. 80 és el port HTTP per defecte (requerix permisos d'administrador). 443 és per a HTTPS.

Què és CORS i per què em dona error? — Cross-Origin Resource Sharing. El navegador bloqueja peticions d'un domini a un altre per seguretat. Per a desenvolupament, afeg: `exchange.getResponseHeaders().add("Access-Control-Allow-Origin", "*")`.

Gson o Jackson? — Gson és més senzill per a començar. Jackson és més ràpid i potent. Per a este curs, Gson és suficient.

El servidor no arranca: "Address already in use" — Un altre procés té el port ocupat. Canvia el port o mata el procés: `netstat -ano | findstr 8080 & taskkill /PID <numero> /F`.

Puc servir pàgines web normals (HTML+CSS+JS)? — Sí. Posant els fitxers en una carpeta `web/` dins de `resources/` i servint-los amb `Files.readString()`.

Cal una base de dades per a l'API? — No. Pots tindre una llista en memòria (`ArrayList`). Al reiniciar el servidor les dades es perden, però és perfecte per a aprendre.

Resum

Concepte	Analogia
HTTP	L'idioma que parlen client i servidor
GET	Demandar informació
POST	Enviar informació

Concepte	Analogia
PUT	Reemplaçar informació
DELETE	Esborrar informació
Status 200	Tot bé
Status 404	No trobat
Status 500	Error intern
JSON	Format de dades universal
HttpServer	La teua porta al món web
HttpClient	La teua finestra a altres APIs

Flux bàsic d'una API REST:

1. Client fa una petició HTTP
2. Servidor processa la petició
3. Servidor torna JSON (o HTML, o text)
4. Client parseja la resposta i fa alguna cosa

Exercicis Proposats

1. **API de frases cèlebres** — Crea un servidor amb un endpoint `/frase` que torne una frase aleatòria d'una llista en memòria.
2. **Comptador de visites** — Cada visita a `/comptador` incrementa un comptador i torna JSON: `{"visites": 42}`.
3. **API de tasques (CRUD complet)** — Implementa GET/POST/PUT/DELETE per a una llista de tasques.
4. **Consumir i transformar** — Usa `HttpClient` per a consultar l'API de GitHub i mostra els noms dels repositoris. Guarda'ls en `repos.txt`.
5. **Proxy simple** — Crea `/proxy?url=...` que faci una petició GET a eixa URL i torne el contingut.

6. **Mini xat en temps real (simulat)** — Usa /enviar (POST) i /missatges (GET) per a simular un xat en memòria.

RAs treballats en esta unitat:

- **RA5** — Desenvolupa interfícies gràfiques (ara web) seguint el model vista-controlador.



Sergi Garcia Barea — CC BY-SA 4.0

Butlletí 1 – Inicial Resolt: Introducció

Ací tens les solucions del butlletí inicial. No les mires fins que ho hagues intentat pel teu compte. Vinga, dona't una oportunitat.

Exercici 1: Què imprimeix este programa?

```
public class MensajeSecreto {  
    public static void main(String[] args) {  
        System.out.println("Java mola");  
        System.out.println("mucho");  
        System.out.print("¿O no?");  
    }  
}
```

Eixida:

```
Java mola  
mucho  
¿O no?
```

 **Explicació:** println imprimeix i salta a la línia següent. print imprimeix i es queda en la mateixa línia. Fixa't que “¿O no?” apareix just després de “mucho”, sense espai ni salt. Els println anteriors sí que van afegir salts de línia després de cada text. La diferència entre print i println és com parlar amb algú: println solta la frase i espera resposta; print solta la frase i es queda mirant-te esperant que continues.

Exercici 2: Troba els 3 errors

```
Public class MiPrograma  
    public static void main(String[] args) {  
        System.out.println("Hola, mundo")  
        System.out.prinltn("Esto es DAM")  
    }  
}
```

Versió corregida:

```
public class MiPrograma {  
    public static void main(String[] args) {  
        System.out.println("Hola, mundo");  
        System.out.println("Esto es DAM");  
    }  
}
```

Errors:

1. Public ha de ser public (minúscula). Java és sensible a majúscules, com el teu ex.
2. Falta { després de MiPrograma. La classe necessita les claus d'obertura.
3. Falten els ; al final de cada println. En Java, el punt i coma és el punt final de cada frase. Sense ell, el compilador es queda esperant més.

A més, println en lloc de println en la segona línia (error 4 si volem ser generosos). Ho vas veure? Ets un bon detectiu.

 **Explicació:** Cada instrucció en Java acaba amb ;. Sense ell, el compilador es confon i pensa que la línia continua. És com si parlaves sense respirar. La classe necessita claus {} per a delimitar el seu cos. I els noms de les coses (com public o println) s'han d'escriure exactament igual que els va definir Java, amb les seues majúscules i minúscules exactes.

Exercici 3: Completa el mètode

```
public class Saludo {  
    public static void main(String[] args) {  
        System.out.println("Bienvenidos al curso DAM");  
    }  
}
```

 **Explicació:** El mètode main és la porta d'entrada de qualsevol programa Java. Sense la línia `System.out.println(...)`, el programa s'executa però no diu res. És com un presentador que ix a l'escenari i es queda callat. Incòmode. `System.out.println()` és la veu del teu programa: tot allò que poses entre parèntesis (entre cometes dobles si és text) s'imprimirà per consola.

Exercici 4: Escriu el teu primer programa

```
public class Presentacion {  
    public static void main(String[] args) {  
        System.out.println("Me llamo Sergi");  
        System.out.println("Tengo 30 años");  
        System.out.println("Me gusta la programación");  
    }  
}
```

Eixida:

```
Me llamo Sergi  
Tengo 30 años  
Me gusta la programación
```

💡 **Explicació:** Cada `println` imprimeix una línia. És com escriure en un quadern: un rengló per cada `println`. Pots posar qualsevol text entre les cometes dobles. El programa executa les línies en ordre, de dalt a baix, com quan lliges una recepta de cuina. No se salta cap, no inventa res. És un robot obedient i avorrit.

Exercici 5: Què fa este programa?

```
public class SumaRara {  
    public static void main(String[] args) {  
        System.out.println("Resultado: " + (3 + 4));  
        System.out.println("Resultado: " + 3 + 4);  
    }  
}
```

Eixida:

```
Resultado: 7  
Resultado: 34
```

💡 **Explicació:** Has vist quina cosa més rara? El primer `println` suma `3 + 4` dins del parèntesi i dona 7. El segon, al no tindre parèntesis, el `+` es converteix en concatenació de text. És a dir, "Resultado: " + 3 dona "Resultado: 3", i després + 4 dona "Resultado: 34". Java, quan veu

un String amb un +, diu “ah, estem ajuntant text” i converteix tot la resta a text també. Els parèntesis trenquen eixa lògica i forcen la suma matemàtica primer. És com si en una conversació digueren “tinc 3” i després “4” i algú entendra “tinc 34”. Els parèntesis aclarien: “¡NO, és una suma, 3+4=7!”

Exercici 6: AceptaElReto.com — 116 ¡Hola mundo!

```
public class Problema116 {  
    public static void main(String[] args) {  
        System.out.println("Hola mundo.");  
    }  
}
```

💡 **Explicació:** El problema més fàcil d’AceptaElReto. Imprimeix exactament “Hola mundo.” amb la H majúscula, tot junt, i amb el punt al final. No és “Hola Mundo”, no és “hola mundo”, no és “Hola mundo”. És “Hola mundo.”. Este problema existix perquè aprengues a usar la web: pujar codi, vore el veredict, i sentir eixa primera vegada que veus “AC” (Accepted). Gaudix-la.

Exercici 7: CodeWars — Multiply

```
public class Multiply {  
    public static double multiply(double a, double b) {  
        return a * b;  
    }  
}
```

💡 **Explicació:** CodeWars et dona l’estructura de la classe i el mètode; tu només has d’escriure `return a * b;`. És la primera kata per alguna cosa: és tan simple que fins la teua àvia podria fer-la (si la teua àvia sabera Java). Però té la seua gràcia: t’ensenya que en CodeWars els mètodes tenen una signatura exacta que has de respectar, i que `return` torna un valor. Si no poses `return`, el mètode torna `void` i fallen els tests. És com si et demanaren un cafè i els donares un got buit.

Butlletí 1 – Inicial: Introducció

Sense solucions. Sense presses. Amb un editor de text i moltes ganes de compilar. Açò tot just comença.

Exercici 1: Desordena això

Les línies d'este programa estan desordenades. Ordena-les per a formar un programa Java vàlid que compile i s'execute.

```
}  
  
    public static void main(String[] args) {  
        System.out.println("Mi primer programa ordenado");  
    public class Ordenado {  
        }
```

Exercici 2: Què imprimeix?

Sense executar, escriu l'eixida exacta d'este programa:

```
public class Escapista {  
    public static void main(String[] args) {  
        System.out.print("Dijo: \"Java mola\\");  
        System.out.println(" y siguió: \\tprogramando.");  
    }  
}
```

Pista: \" imprimeix una cometa literal, \\t és un tabulador.

Exercici 3: Caçador d'errors

Este codi té **4 errors**. Troba'ls i corregeix-los.


```
Public class ErrorFinder {  
    public static void main(string[] args) {  
        System.out.println("Hola, "Mundo");  
        System.out.println("Esto funciona");  
    }  
}
```

Exercici 4: La teua fitxa personal

Escriu un programa anomenat `FichaPersonal` que mostre:

```
Nombre: [Tu nombre]  
Edad: [Tu edad]  
Lenguaje favorito: Java  
¿Emocionado?: true
```

Usa una línia `println` per a cada camp i assegura't que els booleans no porten cometes.

Exercici 5: Completa el programa

Falta una línia crucial i un parell de caràcters. Afegeix-los perquè compile i mostre “Aprobat, açò funciona”.

```
public class Completame {  
    public static void main(String[] args) {  
        System.out.println("Aprobado, esto funciona")  
    }  
}
```

Exercici 6: Aparella conceptes

Relaciona cada concepte de l'esquerra amb la seua definició de la dreta:

Concepte

Definició

1. `class`

A. Punt d'entrada del programa

Concepte	Definició
2. main	B. Imprimeix text i salta de línia
3. System.out.println	C. Defineix un nou tipus de dades
4. //	D. Comentari d'una línia
5. args	E. Conté els arguments de línia de comandaments

Escriu les respostes com "1→C, 2→A, ..."

Exercici 7: CodeWars — Square(n) Sum

Resol la kata "Square(n) Sum" (8 kyu) en [CodeWars](#).

Donat un array de números, eleva cada un al quadrat i suma els resultats. Per exemple, [1, 2, 2] → 1 + 4 + 4 = 9.

Exercici 8: El detectiu d'errors

El següent codi té 2 errors que impedeixen que compile. Troba'ls i corregeix-los:

```
public class Detective {  
    public static void main(String[] args) {  
        System.out.println("Soy un detective")  
        System.out.println("y resuelvo errores");  
    }  
}
```

Escriu la versió corregida. Després, executa-la i comprova que funciona.

Exercici 9: La teua biografia

Escriu un programa anomenat Biografia que mostre:

- El teu nom

- La teua edat
- El teu llenguatge de programació favorit
- Una frase que et motive

Cada cosa en una línia. Usa **una sola** instrucció `System.out.println` amb `\n` per als salts.

Butlletí 1 – Result: Introducció

Exercicis de dificultat progressiva. Els ★ són per a escalfar, ★★ per a pensar, ★★★ per a concursar.

★ Exercici 1: El programa que saluda tres vegades

Escriu un programa que imprimeixca “Hola” tres vegades en tres línies separades, però només usant UNA línia de `System.out.println` (pista: usa `\n`).

💡 **Explicació:** `\n` és un caràcter especial que significa “nova línia”. És com polsar Enter dins del text. Java interpreta `\n` com un salt de línia, encara que estiga dins de les cometes. Així que amb un sol `println` pots imprimir diverses línies. És com escriure un poema en una sola línia del quadern: els `\n` són els punts i a part invisibles.

★ Exercici 2: Arguments en l'arena

Escriu un programa que reba arguments des de la línia de comandaments i els imprimeixca en ordre invers.

💡 **Explicació:** `args` és un array que conté tot allò que escrigues després de `java NombreClasse`. Si poses `java ArgumentosInversos hola mundo cruel`, `args[0]` és “hola”, `args[1]` és “mundo”, `args[2]` és “cruel”. El bucle `for` comença per l'últim índex (`args.length - 1`) i va cap arrere. Si no passes arguments, `args.length` és 0 i el bucle no s'executa. El programa no diu res. Com un concert sense públic.

★★ Exercici 3: Comentari o no comentari

Sense executar, què imprimeix este programa?

```
public class Comentarios {
    public static void main(String[] args) {
        // System.out.println("Uno");
        System.out.println("Dos");
        /* System.out.println("Tres"); */
        System.out.println(/* "Cuatro" */ "Cinco");
    }
}
```

Solució:

```
Dos
Cinco
```

💡 **Explicació:** Les línies amb `//` s'ignoren completament. `System.out.println("Dos")` s'executa normal. El bloc `/* ... */` també s'ignora. Però l'última línia és un parany: el comentari està DINS de la línia. `System.out.println(/* "Cuatro" */ "Cinco")` — el `/* "Cuatro" */` s'ignora, però "Cinco" roman com a argument del `println`. Així que imprimeix "Cinco". Els comentaris no són comandaments, són notes adhesives: pots posar-los on vulgues, fins i tot enmig d'una línia, i el compilador els ignorarà com si mai hagen existit.

★ ★ Exercici 4: La festa d'acceptaelreto.com

Resol el problema 117 — La fiesta de [AcceptaElReto.com](https://www.acepta-el-reto.com/).

Diu: donat un número N, imprimeix "Hola mundo." N vegades.

💡 **Explicació:** El problema et dona un número N en la primera línia. Has de llegir-lo amb `Scanner` (sí, ja sé que `Scanner` no l'hem vist oficialment, però `AcceptaElReto` exigix llegir de teclat). Després un `for` que es repetix N vegades, imprimint "Hola mundo." cada vegada. Si et fixes, el `for` és com una gravació en bucle: "digues-ho una altra volta, digues-ho una altra volta, digues-ho una altra volta...". El `sc.close()` és opcional però educat: tanques el `Scanner` perquè eres una persona neta.

★ ★ Exercici 5: Calculadora sense Scanner

Declara dos variables `int a` i `int b` amb valors fixes (15 i 4). Mostra: suma, resta, multiplicació, divisió entera, divisió real i residu. Tot amb `System.out.println`.

Eixida:

```
a = 15, b = 4
Suma: 19
Resta: 11
Multiplicació: 60
Divisió entera: 3
Divisió real: 3.75
Residu: 3
```

💡 **Explicació:** La divisió entera ($15 / 4$) dona 3 perquè tots dos són `int`: Java tritura els decimals i es queda amb la part entera. Per a la divisió real, convertim `a` a `double` amb `(double) a`. Així Java entén que volem decimals. El `%` et dona el residu: $15 \% 4 = 3$ perquè 4 cap 3 vegades en 15 ($4 \times 3 = 12$) i sobren 3. Açò és la base de TOT: saber quan Java talla decimals i quan no et salvarà de molts maldecaps.

☆☆☆ Exercici 6: Javadoc de la teua vida

Crea una classe `SobreMi` amb:

- Comentari Javadoc en la classe explicant qui eres
- Comentari Javadoc en el mètode `main`
- Un `println` que mostre el teu nom i motivació

💡 **Explicació:** Javadoc són comentaris especials que comencen amb `/**` i permeten generar documentació automàtica amb l'eina `javadoc`. Es col·loquen just abans de la classe o mètode que documenten. Les etiquetes `@author` i `@version` són per a la classe; `@param` per als paràmetres del mètode. No és obligatori, però quan treballes en equip i et demanen documentar el teu codi, donaràs les gràcies. A més, en els exàmens sol caure. I sí, en la vida real quasi ningú documenta amb Javadoc... però els que ho fan dormen millor.

☆☆☆ Exercici 7: El primer depurador

Escriu un programa amb un bucle que sume els números de l'1 al 10. Pos un breakpoint en la línia de la suma i executa pas a pas. Anota:

1. Quantes vegades es para el breakpoint?
2. Quin valor té la variable suma en cada parada?
3. Quin és el valor final?

Respostes:

1. Es para **10 vegades** (una per cada iteració del for, quan i val 1, 2, 3... fins a 10).
2. Valors de suma: 1, 3, 6, 10, 15, 21, 28, 36, 45, 55.
3. Valor final: **55**.

💡 **Explicació:** El depurador és com tindre una màquina del temps per al teu codi. Pos un breakpoint (punt de ruptura) en una línia i el programa es deté just allí. Pots vore el valor de totes les variables. Després avances pas a pas i veus com canvien. En este cas, suma comença en 0, i en cada volta se li suma el valor d'i. i va d'1 a 10. La suma total $1+2+3+\dots+10 = 55$. Si no sabbies esta fórmula, ara sí: la suma d'1 a N és $N*(N+1)/2$. Per a $N=10$: $10*11/2 = 55$. El depurador et permet confirmar que és cert, pas a pas, com un científic boig verificant la seua teoria.

★ ★ ★ Exercici 8: CodeWars — Return Negative

Resol la kata "**Return Negative**" (8 kyu) en CodeWars.

Donat un número, torna'l negatiu. Si ja és negatiu, torna'l tal qual. Si és 0, torna 0.

💡 **Explicació:** Si el número ja és negatiu (o zero), el tornem tal qual. Si és positiu, li posem un - davant. També es pot fer amb l'operador ternari: `return x <= 0 ? x : -x;` en una sola línia. La kata t'ensenya que a voltes el més simple funciona. No necessites una funció de 20 línies per a això. Una línia (o un if) basten. És com quan et pregunten "com estàs?" i respons "bé". No necessites un discurs de 5 minuts.

Referències

Plataforma	Problema	Dificultat
AceptaElReto	116 — ¡Hola mundo!	Principiant
AceptaElReto	117 — La fiesta	Fàcil
AceptaElReto	119 — Futbolistas	Fàcil
CodeWars	Multiply (8 kyu)	Principiant
CodeWars	Return Negative (8 kyu)	Principiant

Butlletí 1 – Intermedi: Introducció

Exercicis de dificultat progressiva. Els ★ són per a escalfar, ★★ per a pensar, ★★★ per a concursar.

★ Exercici 1: ASCII art amb prints

Escriu un programa que dibuixi esta figura usant combinacions de `System.out.print` i `System.out.println`:

```
*
***
*****
***
*
```

Has d'usar exactament **5 línies de codi** (una per cada fila), combinant `print` i `println` sense usar bucles. Pensa quan necessites salt de línia i quan no.

★ Exercici 2: Sense executar — seqüències d'escapament

Què imprimeix exactament este programa? Escriu l'eixida caràcter per caràcter.

```
public class EscapeRoom {
    public static void main(String[] args) {
        System.out.println("Java\n\tmola\n\"mucho\"");
        System.out.println("C:\\carpeta\\archivo.java");
    }
}
```

Les seqüències `\n` (nova línia), `\t` (tabulador), `\"` (cometa literal) i `\\` (barra invertida literal) són les protagonistes.

★★ Exercici 3: El rellotge de mil·lisegons

`System.currentTimeMillis()` torna el nombre de mil·lisegons des de l'1 de gener de 1970 (l'“epoch” d'Unix). Escriu un programa que:

1. Capture el moment actual amb `long inicio = System.currentTimeMillis();`
2. Faça una pausa artificial (un bucle que conte fins a 100.000.000 per a perdre temps)
3. Capture el moment després amb `long fin = System.currentTimeMillis();`
4. Mostre quants mil·lisegons han passat

No et preocupes si el temps varia cada vegada que l'executes: depén de la velocitat del teu ordinador.

★ ★ Exercici 4: Comptador d'arguments

Escriu un programa anomenat `ContadorArgs` que reba arguments des de la línia de comandaments i mostre:

- Quants arguments es van rebre
- El primer argument (si existix)
- L'últim argument (si existix)

Si no es reben arguments, ha de mostrar: “No es van rebre arguments. Programa cancel·lat per falta de dades.”

Exemple d'execució:

```
> java ContadorArgs hola mundo cruel
Argumentos recibidos: 3
Primer argumento: hola
Último argumento: cruel
```

★ ★ ★ Exercici 5: L'edat còsmica

La Terra tarda 365.25 dies a orbitar el Sol. Mercuri tarda 87.97 dies. Escriu un programa que, usant constants final:

1. Declare `final double DIAS_TIERRA = 365.25;`
2. Declare `final double DIAS_MERCURIO = 87.97;`

3. Emmagatzeme en una variable `int edadTerrestre = 20` (la teua edat en anys terrestres)
4. Calcule els anys que tindries en Mercuri (dividix els dies terrestres viscuts entre els dies de Mercuri)
5. Mostre: "A la Terra tinc X anys. A Mercuri tindria Y anys."

Per a calcular els dies viscuts a la Terra: `diasVividos = edadTerrestre * DIAS_TIERRA`.

Exercici 6: CodeWars — Grasshopper - Summation

Resol la kata "Grasshopper - Summation" (8 kyu) en [CodeWars](#).

Suma tots els números de l'1 fins a n. Si $n = 4$, torna $1+2+3+4 = 10$. Hi ha una fórmula matemàtica, però també pots fer-ho amb un bucle (encara que no l'hagem vist oficialment, segur que te'n ixques).

Exercici 7: AceptaElReto — 119 Futbolistes

Resol el problema 119 — Futbolistes en [AceptaElReto.com](#).

Llig els minuts que juga cada futbolista i determina quants partits complets (90 minuts) ha jugat cada un. Pista: cada cas de prova acaba quan apareix un -1. Necessitaràs llegir números fins a trobar el marcador de fi.



Referències

Plataforma	Problema	Dificultat
AceptaElReto	116 — ¡Hola mundo!	Principiant
AceptaElReto	119 — Futbolistas	Fàcil
AceptaElReto	114 — Último dígito del factorial	Mitjà
CodeWars	Square(n) Sum (8 kyu)	Principiant

Plataforma	Problema	Dificultat
CodeWars	Grasshopper - Summation (8 kyu)	Principiant

Butlletí 01 — Extres: Introducció a Java

★ **Extres** — CodeWars i AceptaElReto comencen a partir de la unitat 3, quan tinguem suficients conceptes per afrontar-los amb garanties. De moment, repassa els exercicis inicials i afirma el que has après. Pronte arribaran els reptes de veritat!

Butlletí 2 – Inicial Resolt: Variables i Operadors

Ací estan les solucions. Però només mires si ja ho has intentat. La memòria muscular es construeix equivocant-se.

Exercici 1: Declara el tipus correcte

1. `int` — l'edat cap en un `int` (2.147 milions és el límit, 150 no fa por)
2. `double` (o `float`) — els preus tenen decimals
3. `long` — 8.000.000.000 no cap en un `int` (2.147.483.647 és el topall)
4. `boolean` — només dos valors: `true` o `false`
5. `char` — una sola lletra amb cometes simples
6. `double` — decimals per a la temperatura

 **Explicació:** Cada tipus té una grandària i un propòsit. Usar `int` per a tot és com portar sempre un camió per si de cas t'has de mudar. `byte` per a l'edat, `short` per a la població d'un poble, `int` per a quasi tot, `long` per a coses astronòmiques, `double` per a decimals, `boolean` per a sí/no, `char` per a una lletra. La clau és triar el tipus adequat: ni un microbús per a dues persones, ni un Smart per a una família de 5.

Exercici 2: Què imprimeix?

```
public class Operaciones {  
    public static void main(String[] args) {  
        int a = 10;  
        int b = 3;  
        System.out.println(a / b);  
        System.out.println(a % b);  
        System.out.println((double) a / b);  
    }  
}
```

Eixida:

```
3
1
3.3333333333333335
```

💡 **Explicació:** $10 / 3$ amb enters dona 3 (es truncen els decimals). $10 \% 3$ dona 1 (el residu de la divisió). (double) a / b converteix a a double (10.0) i aleshores la divisió dona 3.333... El (double) davant de a és un *casting*: li dius a Java “confia en mi, tracta això com a decimal encara que siga enter”. És com posar-li ulleres a un miop: de sobte veu els decimals.

Exercici 3: Troba l'error

```
public class Errores {
    public static void main(String[] args) {
        int 1numero = 10;           // ERROR 1
        float precio = 19.99;       // ERROR 2
        System.out.println(1numero + precio);
    }
}
```

Correcció:

```
public class Errores {
    public static void main(String[] args) {
        int numero1 = 10;
        float precio = 19.99f;
        System.out.println(numero1 + precio);
    }
}
```

💡 **Explicació:** Error 1: les variables no poden començar amb número. 1numero és il·legal. numero1 és legal. És com les matrícules dels cotxes: poden acabar en número però no començar amb ell. Error 2: els decimals en Java són double per defecte. Per a assignar-los a un float necessites afegir f al final: 19.99f. Sense la f, Java es queixa perquè estàs ficant un double en una caixa float i es podria perdre precisió. És com intentar ficar una botella gran en un got xicotet: alguna cosa es derramarà.

Exercici 4: L'intercanvi

```
public class Intercambio {  
    public static void main(String[] args) {  
        int a = 5;  
        int b = 10;  
  
        System.out.println("Abans: a = " + a + ", b = " + b);  
  
        int temp = a;  
        a = b;  
        b = temp;  
  
        System.out.println("Després: a = " + a + ", b = " + b);  
    }  
}
```

Eixida:

```
Abans: a = 5, b = 10  
Després: a = 10, b = 5
```

💡 **Explicació:** Necessites una variable temporal `temp` per a no perdre el valor de `a`. Si fas `a = b` directament, perds el 5. És com quan vols intercanviar el contingut de dos gots: necessites un tercer got buit. Si abocares el de vi en el d'aigua sense buidar abans el d'aigua, tindries vi amb aigua. La variable temporal és eixe tercer got.

Exercici 5: Escriu aquest programa


```
public class AreaCirculo {  
    public static void main(String[] args) {  
        final double PI = 3.1416;  
        double radio = 7.5;  
  
        double area = PI * radio * radio;  
        double perimetro = 2 * PI * radio;  
  
        System.out.println("Àrea: " + area);  
        System.out.println("Perímetre: " + perimetro);  
    }  
}
```

Eixida:

```
Àrea: 176.715  
Perímetre: 47.124
```

💡 **Explicació:** `final` fa que `PI` siga constant. No podràs canviar-la després. Intenta posar `PI = 4;` després i el compilador et saltarà al coll. L'àrea del cercle és `PI` per radi al quadrat. El perímetre (o circumferència) és `2` per `PI` per radi. `Math.PI` existeix com a constant més precisa (3.141592653589793), però ací usem la nostra. La gràcia de `final` és que li dius a Java i a altres programadors: “açò no es toca, ni que vinga el mismíssim Bill Gates a demanar-t’ho”.

Exercici 6: AceptaElReto 149 — San Fermine

```
import java.util.Scanner;

public class Problema149 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        while (sc.hasNextInt()) {
            int n = sc.nextInt();
            int max = 0;
            for (int i = 0; i < n; i++) {
                int vel = sc.nextInt();
                if (vel > max) max = vel;
            }
            System.out.println(max);
        }
    }
}
```

💡 **Explicació:** El problema de San Fermín et dona diverses línies, cadascuna amb un número N seguit de N velocitats. Has d'imprimir la velocitat màxima de cada cas. Usem `while (sc.hasNextInt())` perquè no sabem quantes línies hi ha. Llegim N, després N números amb un bucle, i ens quedem amb el màxim. El truc és que `max` comença en 0 (les velocitats són positives). Si hi haguera velocitats negatives, caldria començar amb `Integer.MIN_VALUE`. Però en San Fermín tots corren cap avant, no com alguns que corren cap arrere.

Exercici 7: CodeWars — Even or Odd

```
public class EvenOrOdd {
    public static String evenOrOdd(int number) {
        return number % 2 == 0 ? "Even" : "Odd";
    }
}
```

💡 **Explicació:** L'operador `%` et dona el residu de la divisió. Si un número és parell, `numero % 2` dona 0. Si és senar, dona 1. El ternari `? :` és un if-else en una línia: `condició ? valorSiTrue : valorSiFalse`. Torna "Even" si el residu és 0, "Odd" si no. Una línia. Elegant. Com un esmòquing per al teu codi. També ho podries fer amb if-else, però el ternari queda més net. CodeWars premia l'elegància, no les 15 línies per a una tonteria.

Butlletí 2 – Inicial: Variables i Operadors

Sense solucions. Obri l'IDE, declara variables i embruta't les mans de bits. Ningun no naix sabent què és un double.

Exercici 1: Conversor de temperatures

Escriu un programa anomenat `ConversorTemperatura` que convertisca 30 graus Celsius a Fahrenheit. Usa la fórmula:

$$F = C * 9/5 + 32$$

Declara `int celsius = 30` i una variable `double fahrenheit` per al resultat. Mostra les dues temperatures.

Exercici 2: Què imprimeix?

Sense executar, escriu l'eixida exacta:

```
public class Incrementos {
    public static void main(String[] args) {
        int x = 5;
        System.out.println(x++);
        System.out.println(++x);
        System.out.println(x--);
        System.out.println(--x);
        System.out.println(x);
    }
}
```

Exercici 3: Calculadora de descomptes

Declara una constant `final double DESCUENTO = 0.15` i declara `double precioOriginal = 120.0`. Calcula:

1. El descompte (`precioOriginal * DESCUENTO`)

2. El preu final (precioOriginal - descuento)

Mostra-ho tot amb println. T'animes a declarar el descompte com a final? Si després intentes canviar-lo, el compilador s'enfadarà.

Exercici 4: Precedència de malson

Sense executar, quin valor té resultado?

```
int resultado = 2 + 3 * 4 - 8 / 2 + (6 - 1) * 2;
```

Escriu el pas a pas i el valor final. No hi ha trampa, només matemàtiques i ordre d'operacions.

Exercici 5: El tipus perfecte

Indica quin tipus de dada primitiu (int, double, boolean, char, long) utilitzaries per a cada cas:

1. El nombre d'habitants de la teua ciutat (~500.000)
 2. La distància en quilòmetres fins a la lluna (~384.400)
 3. La inicial del teu segon cognom
 4. La nota mitjana d'un examen (3.7)
 5. Si has aprovat o no l'examen anterior
 6. El preu d'un café en cèntims (enter)
-

Exercici 6: L'intercanvi màgic

Donades dues variables `int a = 7;` i `int b = 3;`, intercanvia els seus valors **sense usar una variable temporal**. Pista: usa suma i resta:

```
a = a + b;  
b = a - b;  
a = a - b;
```

Mostra els valors abans i després per a comprovar que va funcionar.

Exercici 7: CodeWars — Will you make it?

Resol la kata “Will you make it?” (8 kyu) en [CodeWars](#).

Et donen la distància fins a una gasolinera, els litres que té el teu cotxe i els quilòmetres per litre. Determina si arribes o no. Torna `true` si hi arribes, `false` si et quedes tirat.

Butlletí 2 - Intermedi Result: Variables i Operadors

De menys a més. De ★ a ★★ ★. De “açò és pa sucats amb oli” a “açò ja no és un entrepà”.

★ Exercici 1: Parell o senar?

Demana un número a l'usuari i determina si és parell o senar usant l'operador %.

💡 **Explicació:** `num % 2` et dona el residu de dividir el número entre 2. Si és 0, és parell. Si és 1, és senar. No hi ha misteri. És l'operació més usada en programació després de sumar. El % ix a tot arreu: per a saber si alguna cosa és parell, per a cicles, per a jocs, per a rellotges. Sense ell, no hi hauria videojocs (bé, sí, però serien més difícils de programar).

★ Exercici 2: Constant del mal

Declara `final double PRECIO_BASE = 100;` i `final double IVA = 0.21`. Calcula el preu final. Després intenta modificar IVA després. Quin error dona?

💡 **Explicació:** En descomentar `IVA = 0.10`, el compilador diu: “No puc assignar un valor a una variable final”. `final` és com un contracte firmat: no pots canviar d'opinió. És útil per a valors que no han de canviar mai: l'IVA, el nom de la teua app, el nombre d'intents màxims. Si algú intenta canviar-los, el compilador es planta. Millor que discutir amb un programador que canvia l'IVA a mitat de facturació.

★★ Exercici 3: Dau trucat

Genera 10 números aleatoris entre 1 i 6. Compta quants 6 han eixit i mostra el resultat.

💡 **Explicació:** `Math.random()` torna un número entre 0.0 i 0.999999... Multipliques per 6, obtens entre 0.0 i 5.999999. El `(int)` trunca els decimals i et dona 0-5. Sumes 1 i obtens 1-6. El bucle es repeteix 10 vegades. Cada volta que ix un 6, increments el comptador. És com si llançares un dau físic 10 vegades i anotares quants 6 traus. La probabilitat és baixa (1/6 per tirada), així que el normal és que isquen entre 1 i 2 sisos. Si etixen 7, el dau està trucat (o tens una sort increïble).

☆☆ Exercici 4: Any de traspàs

Demana un any a l'usuari. Determina si és de traspàs: divisible entre 4 I (no entre 100 O sí entre 400).

💡 **Explicació:** La regla de l'any de traspàs és: divisible entre 4, però si és divisible entre 100 NO és de traspàs, a menys que també siga divisible entre 400. 1900 no va ser de traspàs (divisible entre 100 però no entre 400). 2000 sí (divisible entre 400). 2024 sí (divisible entre 4 però no entre 100). Els operadors lògics && (AND) i || (OR) combinats amb % permeten expressar esta regla en una sola línia. És com la lija fina de la programació: condicions precises per a resultats exactes. Els parèntesis són importants: sense ells, la precedència podria donar un resultat incorrecte.

☆☆ Exercici 5: Nom al revés

Demana a l'usuari el seu nom. Mostra: longitud, versió en majúscules, primera lletra i última lletra.

💡 **Explicació:** `length()` torna quants caràcters té el String. `toUpperCase()` el converteix a majúscules. `charAt(0)` torna el primer caràcter (els Strings comencen en 0, com els arrays). `charAt(nombre.length() - 1)` torna l'últim. Fixat que els mètodes de String es criden SOBRE l'objecte, no són estàtics. L'objecte és nombre, i li preguntes: "oi, tu, quant mesura?". String és una classe amb molts mètodes útils. Aprén a usar-los i t'estalviaràs reinventar la roda cada dos per tres.

☆☆☆ Exercici 6: AceptaElReto 152 — Números de parells

Resol el problema 152 — Números de parells (també conegut com "Va de modes...") en AceptaElReto.

Donada una llista de números, determina si la quantitat de parells és major que la de senars.

💡 **Explicació:** El problema va llegint casos fins a trobar un 0 (que marca el final). Cada cas comença amb N (quants números venen). Després llegim N números. Per cada un, mirem si és parell o senar amb `% 2` i comptem. Al final comparem els comptadors. És una versió ampliada de l'exercici 1, però amb lectura de dades i múltiples casos. La gràcia està en

manejar correctament la lectura quan no saps quants casos hi ha. `while (sc.hasNextInt())` i `break` quan veus un 0.

☆☆☆ Exercici 7: L'enigma del ++

Sense executar, determina el resultat de:

```
int a = 2;
int b = a++ * 3 + --a;
System.out.println("a = " + a + ", b = " + b);
```

Solució:

```
a = 2, b = 7
```

Pas a pas:

1. `a = 2`
2. `a++` → POST: usa `a` (2), després `a = 3`. L'expressió `a++` val **2**.
3. `--a` → PRE: `a` val 3, decrementa `a` **2**, després usa `a` (2). Val **2**.
4. `b = 2 * 3 + 2 = 6 + 2 = 7`
5. `a` va quedar en 2 (va pujar a 3 amb `a++`, va baixar a 2 amb `--a`)

💡 **Explicació:** `a++` (post-increment) primer USA el valor i després incrementa. `--a` (pre-decrement) primer decrementa i després USA. Per això el valor final de `a` és 2 (va pujar i baixar com un io-io). I `b` és 7. Estos enigmes són el terror dels exàmens i l'alegria dels professors malvats. El meu consell: en la vida real, no mescles `++` i `--` en expressions complicades. Usa'ls en línies separades. El teu jo del futur t'ho agrairà.

☆☆☆ Exercici 8: AceptaElReto 140 — Suma de dígit

Resol el problema **140 — Suma de dígit** en AceptaElReto.

Donat un número, suma els seus dígit. Després suma els dígit del resultat, i així fins que quede un sol dígit. Mostra el procés.

💡 **Explicació:** Per a extraure els dígit d'un número, usem $n \% 10$ (obtenim l'últim dígit) i $n / 10$ (llevem l'últim dígit). Repetim fins que n siga 0. Sumem tots els dígit. Si la suma té més d'un dígit (≥ 10), repetim el procés. És com quan doblegues un paper una vegada i una altra fins que no pots més: el número es va reduint fins a un sol dígit. Exemple: $123 \rightarrow 1+2+3 = 6$. $987 \rightarrow 9+8+7 = 24 \rightarrow 2+4 = 6$. És l'“arrel digital” d'un número. Molt usat en numerologia i en exàmens de programació.



Referències

Plataforma	Problema	Dificultat
AceptaElReto	149 — San Fermín	Fàcil
AceptaElReto	152 — Números de parells	Fàcil
AceptaElReto	140 — Suma de dígit	Mitjà
CodeWars	Even or Odd (8 kyu)	Principiant
CodeWars	Opposite number (8 kyu)	Principiant

Butlletí 2 – Intermedi: Variables i Operadors

De menys a més. De ★ a ★★ ★. De variables senzilles a operacions que et faran esbotzar.

★ Exercici 1: Calculadora de propines

Escriu un programa que calcule quant deixar de propina en un restaurant. Declara:

- `double totalCuenta = 45.50;`
- `int porcentajePropina = 15;` (percentatge, sense el símbol)

Calcula la propina (`totalCuenta * porcentajePropina / 100`) i el total final (`totalCuenta + propina`). Mostra-ho tot amb 2 decimals aproximats.

Eixida esperada:

```
Total compte: 45.5€
Propina (15%): 6.825€
Total a pagar: 52.325€
```

★ Exercici 2: Conversor dòlar-euro

Declara final `double TASA_CAMBIO = 0.92;` (1 dòlar = 0.92 euros). Declara `double dolares = 100.0;` i calcula el seu equivalent en euros. També fes la conversió inversa: donat `double euros = 50.0;`, calcula quants dòlars són.

Mostra:

```
100.0$ són 92.0€
50.0€ són 54.34782608695652$
```

★★ Exercici 3: Què imprimeix? — el casting traïdor

Sense executar, escriu l'eixida exacta:

```

public class CastingTraidor {
    public static void main(String[] args) {
        int a = 7;
        int b = 2;
        double resultado1 = a / b;
        double resultado2 = (double) a / b;
        double resultado3 = a / (double) b;

        System.out.println(resultado1);
        System.out.println(resultado2);
        System.out.println(resultado3);
        System.out.println(3 + 4 * 2.0);
        System.out.println((int) (3.7 + 2.3));
    }
}

```

Fixat bé en on està el casting i en quin moment s'aplica la divisió entera.

☆☆ Exercici 4: Interés compost (sense bucle)

Declara final double CAPITAL_INICIAL = 1000.0; , final double TASA = 0.05; (5% anual) i int años = 3;. Calcula el capital final després de 3 anys usant la fórmula de l'interés compost SENSE bucles:

```
capitalFinal = capitalInicial * (1 + tasa)^años
```

Per a la potència, usa `Math.pow(base, exponente)`. Mostra el capital any a any:

```

Any 0: 1000.0€
Any 1: 1050.0€
Any 2: 1102.5€
Any 3: 1157.625€

```

Per a mostrar cada any, necessitaràs calcular `Math.pow(1 + TASA, i)` per a cada valor de `i` de 1 a 3... però sense bucle. Crea tres variables diferents.

☆☆☆ Exercici 5: L'enigma del post-increment

Sense executar, determina el valor de cada variable després d'executar aquest codi. Escriu el pas a pas.

```
public class EnigmaIncremento {  
    public static void main(String[] args) {  
        int x = 3;  
        int y = x++ + ++x;  
        int z = --y + y-- + x++;  
        System.out.println("x = " + x);  
        System.out.println("y = " + y);  
        System.out.println("z = " + z);  
    }  
}
```

Pista: fes una taula amb els valors de x, y després de cada operació. x++ usa x i després incrementa; ++x incrementa i després usa x.

☆☆☆ Exercici 6: CodeWars — Convert boolean values to strings 'Yes' or 'No'

Resol la kata "Convert boolean values to strings 'Yes' or 'No'" (8 kyu) en [CodeWars](https://www.codewars.com/kata/convert-boolean-values-to-strings-yes-or-no).

Completa el mètode `public static String boolToWord(boolean b)` que torne "Yes" si rep true i "No" si rep false. Es pot fer en una línia amb l'operador ternari.

☆☆☆ Exercici 7: AceptaElReto — 114 Últim dígit del factorial

Resol el problema 114 — Últim dígit del factorial en [AceptaElReto.com](https://www.acepta-el-reto.com/problems/114).

Donat un número N ($0 \leq N \leq 1.000.000$), calcula l'últim dígit de N! (factorial de N). Pista: no necessites calcular el factorial sencer. Què passa amb l'últim dígit quan multipliques per 10? I quan $N \geq 5$? Observa que $5! = 120$, $6! = 720$... a partir de 5, el factorial sempre acaba en 0.



Referències

Plataforma	Problema	Dificultat
AceptaElReto	114 — Últim dígit del factorial	Fàcil
AceptaElReto	140 — Suma de dígit	Mitjà
AceptaElReto	152 — Números de parells	Fàcil
CodeWars	Even or Odd (8 kyu)	Principiant
CodeWars	Opposite number (8 kyu)	Principiant
CodeWars	Convert boolean to Yes/No (8 kyu)	Principiant

Butlletí 02 — Extres: Variables, Tipus de Dades i Operadors

★ **Extres** — CodeWars i AceptaElReto comencen a partir de la unitat 3, quan tinguem suficients conceptes per afrontar-los amb garanties. De moment, repassa els exercicis inicials i afirma el que has après. Pronte arribaran els reptes de veritat!

Butlletí 3 – Inicial Resolt: Estructures de Control

Les solucions estan aquí. Però només les necessites si ja has sudat un poc. La suor programa millor que les solucions.

Exercici 1: Què imprimeix este if-else?

```
public class QueImprime {  
    public static void main(String[] args) {  
        int edad = 17;  
        boolean conPadres = true;  
  
        if (edad >= 18) {  
            System.out.println("Entrada libre");  
        } else if (conPadres) {  
            System.out.println("Pasas con tus viejos");  
        } else {  
            System.out.println("A casa");  
        }  
    }  
}
```

Resposta: “Pasas con tus viejos”

💡 **Explicació:** `edad >= 18` és false (en té 17), així que no entra al primer if. Després avalua `conPadres` que és true, per tant entra al `else if` i imprimeix “Pasas con tus viejos”. El `else` final no s’executa mai perquè ja va entrar al `else if`. Java avalua les condicions en ordre i es queda amb la PRIMERA que siga true. Com un semàfor: si el primer està verd, passes; si no, mires el següent. No te’n saltes cap, però tampoc mires més enllà del que ja està verd.

Exercici 2: Completa el switch

```

int dia = 3;
String nombreDia;

switch (dia) {
    case 1:
        nombreDia = "Lunes";
        break;
    case 2:
        nombreDia = "Martes";
        break;
    case 3:
        nombreDia = "Miércoles";
        break;
    default:
        nombreDia = "Desconocido";
}
System.out.println(nombreDia);

```

Amb breaks: Imprimeix “Miércoles”.

Sense breaks (fall-through): Imprimeix “Miércoles”. Però si `dia = 1`, sense breaks imprimiria “LunesMartesMiércoles” (tot seguit!). Amb `dia = 2` sense breaks: “MartesMiércoles”.

💡 **Explicació:** `break` és la instrucció de “eixir del switch”. Si no el poses, el codi “cau” al següent case (fall-through). És com si les escales no tingueren descansets: baixes d’una tirada fins al final. En el nostre exemple, amb `dia = 3`, el `break` del case 3 frena la caiguda. Però en case 1 i case 2, sense `break`, el programa continuaria executant els case següents fins a trobar un `break` o arribar al default.

Exercici 3: Troba l’error (bucle infinit)

```

public class BucleInfinito {
    public static void main(String[] args) {
        int i = 1;
        while (i <= 5) {
            System.out.println("Vuelta " + i);
        }
    }
}

```


Error: Falta `i++` dins del bucle. `i` sempre val 1, la condició `i <= 5` sempre és true.

Corregit:

```
public class BucleCorregido {  
    public static void main(String[] args) {  
        int i = 1;  
        while (i <= 5) {  
            System.out.println("Vuelta " + i);  
            i++;  
        }  
    }  
}
```

💡 **Explicació:** Sense `i++`, la variable de control no canvia mai. És com un gos perseguint-se la cua: no para mai. El bucle `while` només comprova la condició. Si sempre es compleix, el bucle és etern. `i++` és el que fa que `i` augmente fins a 6, moment en què `i <= 5` és fals i el bucle acaba. Sempre, SEMPRE, assegura't que la variable de control s'actualitzi dins del bucle. És la primera regla dels bucles: “no oblidis actualitzar la condició”.

Exercici 4: Escriu este programa (while)

```

import java.util.Scanner;

public class PositivoNegativo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int numero;

        System.out.println("Introduce números (0 para salir:");
        numero = sc.nextInt();
        while (numero != 0) {
            if (numero > 0) {
                System.out.println(numero + " es positivo");
            } else {
                System.out.println(numero + " es negativo");
            }
            numero = sc.nextInt();
        }
        System.out.println("Fin del programa");
        sc.close();
    }
}

```

💡 **Explicació:** Llegim el primer nombre abans del bucle (açò es diu “lectura anticipada”). Després, mentre no siga 0, avaluem si és positiu o negatiu, i tornem a llegir. Quan l'usuari introduïx 0, el bucle acaba. Podríem usar un do-while per a no repetir la lectura, però així és més clar. És com un porter que pregunta “quants anys tens?” fins que li dius 0 i aleshores et deixa passar... espera, això no té sentit. Però el codi sí.

Exercici 5: El for de tota la vida

```

public class SumaFor {
    public static void main(String[] args) {
        int suma = 0;
        for (int i = 1; i <= 100; i++) {
            suma += i;
        }
        System.out.println("Suma del 1 al 100: " + suma);
    }
}

```

💡 **Explicació:** El bucle for és perfecte quan saps exactament quantes vegades repetir. Ací sabem que anem d'1 a 100. La variable i comença en 1, s'incrementa d'1 en 1, i en cada volta se suma a suma. Al final, suma conté 5050. Hi ha una fórmula matemàtica: $n*(n+1)/2 = 100*101/2 = 5050$. Però en programació, el bucle està bé. La història conta que Gauss va descobrir esta fórmula de xiquet per a fer el càlcul més ràpid. Tu estàs aprenent a programar, no a ser Gauss. Encara que si vols ser Gauss, també val.

Exercici 6: AceptaElReto 200 — Aburrimiento en las aulas

```
import java.util.Scanner;

public class Problema200 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num = sc.nextInt();
        while (num != 0) {
            int a = 5 * num * num + 4;
            int b = 5 * num * num - 4;
            int raizA = (int) Math.sqrt(a);
            int raizB = (int) Math.sqrt(b);

            if (raizA * raizA == a || raizB * raizB == b) {
                System.out.println("Fibonacci");
            } else {
                System.out.println("No fibonacci");
            }
            num = sc.nextInt();
        }
    }
}
```

💡 **Explicació:** La propietat matemàtica diu: un nombre és part de la seqüència de Fibonacci si i només si $(5n^2 + 4)$ o $(5n^2 - 4)$ és un quadrat perfecte. Per a comprovar si alguna cosa és quadrat perfecte, calculem l'arrel quadrada, la truncuem a enter, i elevem al quadrat. Si coincidix amb l'original, és quadrat perfecte. Exemple: per a $n=5$: $5*25+4=129$ (no és quadrat), $5*25-4=121=11^2 \rightarrow$ Fibonacci! Efectivament, 5 està en la seqüència. Per a $n=4$: $5*16+4=84$ (no),

$516-4=76$ (no) → No fibonacci. El 4 no està. El while segueix llegint nombres fins que s'introdueix un 0.

Exercici 7: CodeWars — Century From Year

```
public class CenturyFromYear {  
    public static int century(int number) {  
        return (number + 99) / 100;  
    }  
}
```

💡 **Explicació:** Per a calcular el segle d'un any, sumem 99 i dividim entre 100. Any 1: $(1+99)/100 = 1$. Any 100: $(100+99)/100 = 1$. Any 101: $(101+99)/100 = 2$. Any 2000: $(2000+99)/100 = 20$. Any 2001: $(2001+99)/100 = 21$. La lògica: el segle 1 abasta de l'any 1 al 100. En sumar 99, desplaçem el rang perquè la divisió entera funcione. És un truc matemàtic que evita usar if o `Math.ceil()`. Simple, elegant, i et fa quedar bé en els sopars de Nadal quan algú pregunta "en quin segle estem?".

Butlletí 3 – Inicial: Estructures de Control

Sense solucions. Respira. Els bucles no mosseguen. Les excepcions tampoc. L'únic perill eres tu sense dormir.

Exercici 1: Què imprimeix este if?

Sense executar, escriu l'eixida exacta:

```
public class Clasificador {  
    public static void main(String[] args) {  
        int nota = 65;  
  
        if (nota >= 90) {  
            System.out.println("Sobresaliente");  
        } else if (nota >= 70) {  
            System.out.println("Notable");  
        } else if (nota >= 50) {  
            System.out.println("Aprobado");  
        } else {  
            System.out.println("Suspenso");  
        }  
    }  
}
```

Exercici 2: Troba l'error en el for

El següent bucle hauria de comptar de l'1 al 5, però no funciona. Per què? Corregeix-lo.

```
public class ForError {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 10; i--) {  
            System.out.println("Cuenta: " + i);  
        }  
    }  
}
```

Pista: mira la condició i l'actualització. Una de les dos no quadra.

Exercici 3: Completa el programa (do-while)

Falta la condició del do-while. Completa-la perquè el programa demane nombres fins que l'usuari introduïska un nombre negatiu.

```
import java.util.Scanner;

public class DoWhile {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int numero;

        do {
            System.out.print("Introduce un número (negativo para salir): ");
            numero = sc.nextInt();
            System.out.println("Has escrito: " + numero);
        } while (/* tu condición aquí */);

        System.out.println("¡Hasta luego!");
        sc.close();
    }
}
```

Exercici 4: Compte arrere

Escriu un programa que mostre un compte arrere des de 10 fins a 0 usant un for. Després del 0, ha d'imprimir "¡Enlairament!".

Eixida esperada:

```
10
9
8
...
1
0
¡Despegue!
```

Exercici 5: Menú amb switch

Escriu un programa que mostre un menú de begudes i demane a l'usuari que trie una opció (1-4). Usa switch per a mostrar el nom de la beguda seleccionada.

```
=== BAR JAVA ===  
1. Café solo  
2. Café con leche  
3. Té  
4. Refresco  
Elige una opción:
```

Si l'usuari tria 2, ha de mostrar “Has elegido Café con leche”. Si tria un nombre invàlid, mostra “Opción no válida”.

Exercici 6: Taula de multiplicar del 7

Escriu un programa que mostre la taula de multiplicar del 7 usant un for. Ha d'imprimir:

```
7 x 1 = 7  
7 x 2 = 14  
...  
7 x 10 = 70
```

Exercici 7: CodeWars — Grasshopper - Grade book

Resol la kata “Grasshopper - Grade book” (8 kyu) en [CodeWars](#).

Reps tres notes (0-100). Torna la lletra corresponent segons la mitjana:

Mitjana	Nota
>= 90	'A'
>= 80	'B'

Mitjana	Nota
≥ 70	'C'
≥ 60	'D'
< 60	'F'

Butlletí 3 – Intermedi Result: Estructures de Control

Dificultat progressiva. Els ★ t'escalfen, els ★★ et fan pensar, els ★★★ et preparen per al món real (i per a ProgramaMe).

★ Exercici 1: El classificador de notes sarcàstic

Demana una nota (0-100) i classifica-la: 90+ “Sobresaliente”, 70+ “Notable”, 50+ “Aprobado”, menys “Suspenso”. Captura que la nota siga vàlida (0-100).

💡 **Explicació:** Primer validem que la nota estiga entre 0 i 100. Després, amb `else if`, avaluem de major a menor. Si la nota és 85, no entra en `>= 90` però sí en `>= 70`. L'ordre és crucial: si posares `>= 50` abans que `>= 90`, un 95 també entraria en `>= 50`. Ves sempre del més específic al més general. Com en la vida: primer l'urgent, després l'important, després el que siga.

★ Exercici 2: Taula de multiplicar

Demana un nombre i mostra la seua taula de multiplicar de l'1 al 10 amb un `for`.

💡 **Explicació:** El `for` va d'1 a 10. En cada volta, multiplica `n` per `i`. Simple, directe. És com quan eres xicotet i recitaves la taula del 7: “7 per 1 és 7, 7 per 2 és 14...”. La diferència és que ara l'ordinador ho fa per tu i no s'equivoca (a menys que tu t'equivoques en programar-ho, clar).

★★ Exercici 3: Divisió segura amb try-catch

Demana dos nombres enters. Dividix el primer entre el segon. Captura `ArithmeticException` (divisió per zero) i `InputMismatchException` (no escriure un nombre).

💡 **Explicació:** El bloc `try` conté el codi perillós. Si ocorre un `ArithmeticException` (divisió per zero) o un `InputMismatchException` (l'usuari escriu lletres), s'executa el `catch` corresponent. El `finally` s'executa sempre, hi haja error o no. És com l'assegurança de vida del teu programa: passe el que passe, es tanca el Scanner i es diu alguna cosa. Sense `try-catch`, si

l'usuari dividix per zero, el programa mor amb un missatge roig d'error. Amb try-catch, el programa sobreviu i segueix avant com un heroi d'acció.

★ ★ Exercici 4: El triangle constructor

Dibuixa un triangle d'asteriscs amb bucles niats. L'usuari introduïx l'altura.

```
*  
**  
***  
****  
*****
```

💡 **Explicació:** El bucle exterior (i) controla les files (d'1 a altura). El bucle interior (j) controla les columnes de cada fila: en la fila 1 imprimeix 1 asterisc, en la fila 2 imprimeix 2... Usem print (sense \n) perquè els asteriscs isquen en la mateixa línia. En acabar cada fila, fem un println buit per a saltar a la següent línia. Els bucles niats són como nines russes: un bucle dins d'un altre. L'interior dona voltes completes per cada volta de l'exterior.

★ ★ Exercici 5: Fibonacci amb for

Mostra els primers N nombres de Fibonacci. La seqüència comença: 0, 1, 1, 2, 3, 5, 8, 13...

💡 **Explicació:** Fibonacci és la seqüència on cada nombre és la suma dels dos anteriors. Comencem amb a=0, b=1. En cada iteració, calculem c = a+b, mostrem c, i desplacem: a passa a ser b, b passa a ser c. És com una cursa de relleus: el valor es passa d'una variable a una altra. El bucle comença en 3 perquè ja hem imprès els dos primers (0 i 1). Este és l'exercici clàssic d'entrevista tècnica. Si el resols en la primera entrevista, l'entrevistador assenteix amb el cap. Si no el resols, et pregunten "i què tal se't dona treballar en equip?"

★ ★ ★ Exercici 6: AceptaElReto 340 — Juegos de naipes

Resol el problema 340 — Juegos de naipes en AceptaElReto.

Donada una seqüència de cartes representades per nombres (1 a 10), determina quantes vegades cal ordenar-les perquè queden en ordre ascendent segons un algoritme específic de “col·locar la primera al final”.

💡 **Explicació:** El problema simula un joc de cartes on busques la carta 1, després la 2, etc. Vas passant cartes de la primera a l'última posició (com si passares d'una punta a una altra) fins que trobes la que busques. El $% n$ fa que l'índex done la volta quan arriba al final (com una ruleta). És un exercici de lògica amb arrays i bucles. La dificultat està en entendre l'algoritme de cerca circular. No et preocupes si no ix a la primera: els jocs de naipes sempre han sigut complicats, fins i tot per als programadors.

☆☆☆ Exercici 7: Excepció personalitzada

Crea `EdadInvalidaException` (excepció checked). Llança-la si l'edat és menor que 0 o major que 150. Crea un programa que la prove.

Eixida:

```
Error: Edad negativa: -5. ¿Eres viajero en el tiempo?  
Error: 200 años? O eres inmortal o me tomas el pelo  
Edad válida: 25
```

💡 **Explicació:** Creem una excepció personalitzada heretant d'`Exception`. Li posem un constructor que cride a `super(mensaje)` perquè el missatge es guardi. Després, en `validarEdad`, llancem l'excepció amb `throw` quan l'edat és invàlida. Com és checked, qui cride al mètode ha de capturar-la amb `try-catch` o declarar-la amb `throws`. És com crear el teu propi tipus d'error: “açò no és un error qualsevol, és EL MEU error”.

☆☆☆ Exercici 8: AceptaElReto 100 — Kaprekar

Resol el problema 100 — Kaprekar en AceptaElReto.

Donat un nombre de 4 dígit (no tots iguals), ordena'l ascendent i descendentment, resta, i repetix. Simple arribes a 6174 (la constant de Kaprekar). Compta quantes iteracions es necessiten.

💡 **Explicació:** Kaprekar va descobrir que, per a qualsevol nombre de 4 dígit (no tots iguals), si ordenes els dígit de major a menor, restes l'ordre invers, i repetixes, sempre arribes a 6174 en pocs passos. El programa extrau els dígit, els ordena amb `Arrays.sort()`, construeix el nombre ascendent i descendent, resta, i repetix fins a arribar a 6174. És com un embut matemàtic: tots els camins porten a 6174. Este problema és un clàssic de les olimpíades de programació (ProgramaMe). Si el resols, ja pots posar “expert en Kaprekar” al teu LinkedIn.



Referències

Plataforma	Problema	Dificultat
AceptaElReto	200 — Aburrimiento en las aulas	Mitjà
AceptaElReto	340 — Juegos de naipes	Mitjà
AceptaElReto	100 — Kaprekar	Difícil
AceptaElReto	151 — ¿Es matriz identidad?	Mitjà
CodeWars	Century From Year (8 kyu)	Principiant
CodeWars	Cat years, Dog years (7 kyu)	Fàcil

Butlletí 3 – Intermedi: Estructures de Control

Dificultat progressiva. Els ★ t'escalfen, els ★★ et fan pensar, els ★★★ et preparen per al món real (i perquè el switch no se t'agarrote).

★ Exercici 1: Nombres parells i senars

Escriu un programa que recorregui els nombres de l'1 al 20 i mostri al costat si cada un és parell o senar. Usa un for i l'operador % dins d'un if-else.

Eixida esperada:

```
1 es impar
2 es par
3 es impar
...
20 es par
```

★ Exercici 2: Suma acumulativa

Demana nombres a l'usuari (amb Scanner) fins que introduïska un 0. Mostra la suma acumulada després de cada nombre. Usa un bucle while.

Exemple d'execució:

```
Introduce un número (0 para salir): 5
Suma acumulada: 5
Introduce un número (0 para salir): 3
Suma acumulada: 8
Introduce un número (0 para salir): 0
Suma acumulada final: 8
```

★★ Exercici 3: Nombre perfecte

Un nombre perfecte és aquell que és igual a la suma dels seus divisors propis (excloent-se a si mateix). Per exemple, $6 = 1 + 2 + 3$. Escriu un programa que demane un nombre N i determine si és perfecte o no.

Pista: usa un for que vaja d'1 a $N/2$, comprovant si N és divisible entre i ($N \% i == 0$) i sumant els divisors.

★ ★ Exercici 4: Menú amb validació

Crea un programa que mostre un menú d'operacions i valide que l'opció siga correcta. Usa un `do-while` per al menú principal i un `switch` per a les opcions.

```
=== CALCULADORA RUPESTRE ===  
1. Sumar  
2. Restar  
3. Multiplicar  
4. Salir  
Elige opción (1-4):
```

Si l'usuari tria una opció invàlida (menor que 1 o major que 4), mostra "Opción no válida, intenta de nuevo" i repetix el menú. Les opcions 1-3 han de demanar dos nombres i mostrar el resultat. L'opció 4 acaba el programa.

★ ★ ★ Exercici 5: Garbell d'Eratòstenes (nombres primers)

Escriu un programa que demane un nombre N i mostre tots els nombres primers des de 2 fins a N usant el **Garbell d'Eratòstenes**:

1. Crea un array de booleans amb grandària $N+1$, inicialitzat a `true` (tots són primers en teoria)
2. Des de 2 fins a $\sqrt{N}$, si el nombre és primer, marca com a `false` tots els seus múltiples
3. Al final, imprimeix els nombres que continuen sent `true`

Exemple per a $N = 30$:

★ ★ ★ Exercici 6: CodeWars — Vowel Count

Resol la kata "Vowel Count" (7 kyu) en [CodeWars](#).

Torna el nombre de vocals (a, e, i, o, u) en un String donat. L'entrada serà només minúscules i espais. Per exemple, "ho1a mundo" té 4 vocals.

★ ★ ★ Exercici 7: AceptaElReto — 151 ¿Es matriz identidad?

Resol el problema 151 — ¿Es matriz identidad? en [AceptaElReto.com](#).

Donada una matriu quadrada, determina si és la matriu identitat: tots els 1 en la diagonal principal i 0 en la resta. El programa ha de llegir casos de prova, cada un amb una grandària N seguit de N×N nombres.

Pista: per a recórrer la matriu, necessitaràs bucles niats. La diagonal principal són les posicions on fila == columna.

Referències

Plataforma	Problema	Dificultat
AceptaElReto	151 — ¿Es matriz identidad?	Mitjà
AceptaElReto	200 — Aburrimiento en las aulas	Mitjà
AceptaElReto	340 — Juegos de naipes	Mitjà
CodeWars	Century From Year (8 kyu)	Principiant
CodeWars	Vowel Count (7 kyu)	Fàcil
CodeWars	Cat years, Dog years (7 kyu)	Fàcil

Butlletí 03 — Extres: Estructures de Control i Excepcions

★ **Extres — Converteix-te en un programador de molt alt nivell** Aquests exercicis són completament opcionals, però si aspirem a ser un programador o programadora d'elit, ací tens el teu camp d'entrenament. CodeWars i AceptaElReto són plataformes on els millors programadors del món es repte cada dia. Cada problema que resolguis et farà més fort, més ràpid i més creatiu. No els subestimis: són la diferència entre saber programar i ser un vertader professional.

CodeWars:

- [Even or Odd](#) (8 kyu)
Pista: Utilitza l'operador mòdul (%) — si el reste al dividir entre 2 és 0, és parell.
- [Count the divisors of a number](#) (7 kyu)
Pista: Només necessites iterar fins $\sqrt{n}$. Per cada divisor i , el parell n/i també ho és.
- [Multiples of 3 or 5](#) (6 kyu)
Pista: Aplica el principi d'inclusió-exclusió: suma els de 3 i 5, resta els de 15.

AceptaElReto:

- [117 - La festa dels nombres](#) (★ ★)
Pista: Porta el compte dels números que ja han aparegut amb un conjunt (HashSet) o un array de booleans.
- [149 - Punts de cadira](#) (★ ★)
Pista: Calcula primer el mínim de cada fila i el màxim de cada columna per separat, després busca coincidències.

Butlletí 4 – Inicial Resolt: Algorítmica I

Solucions del butlletí inicial. Intenta-ho abans de mirar.

★ Exercici 1: Cerca lineal

```
public static int buscarLineal(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++)
        if (arr[i] == target) return i;
    return -1;
}
```

★ Exercici 2: Cerca binària

```
public static int buscarBinaria(int[] arr, int target) {
    int izq = 0, der = arr.length - 1;
    while (izq <= der) {
        int medio = (izq + der) / 2;
        if (arr[medio] == target) return medio;
        if (arr[medio] < target) izq = medio + 1;
        else der = medio - 1;
    }
    return -1;
}
```

★ ★ Exercici 3: Bombolla

```
public static void burbuja(int[] arr) {
    for (int i = 0; i < arr.length - 1; i++)
        for (int j = 0; j < arr.length - 1 - i; j++)
            if (arr[j] > arr[j + 1]) {
                int t = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = t;
            }
}
```

☆☆ Exercici 4: Inserció

```
public static void insercion(int[] arr) {
    for (int i = 1; i < arr.length; i++) {
        int key = arr[i], j = i - 1;
        while (j >= 0 && arr[j] > key) { arr[j + 1] = arr[j]; j--; }
        arr[j + 1] = key;
    }
}
```

☆☆ Exercici 5: Comptar passos bombolla

```
public static int contarPasos(int[] arr) {
    int pasos = 0;
    for (int i = 0; i < arr.length - 1; i++)
        for (int j = 0; j < arr.length - 1 - i; j++) {
            pasos++;
            if (arr[j] > arr[j + 1]) {
                int t = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = t;
            }
        }
    return pasos;
}
```

☆☆☆ Exercici 6: Buscar en matriu ordenada

```
public static boolean buscarMatriz(int[][] mat, int target) {
    int fila = 0, col = mat[0].length - 1;
    while (fila < mat.length && col >= 0) {
        if (mat[fila][col] == target) return true;
        if (mat[fila][col] > target) col--;
        else fila++;
    }
    return false;
}
```

Butlletí 4 – Inicial: Algorítmica I

Sense solucions. Agafa un array, un bucle i molta paciència. Els algorismes són com les receptes de cuina: si et saltes un pas, el pastís explota.

★ Exercici 1: El més gran i el més menut

Escriu un mètode `public static int[] mayorYMenor(int[] arr)` que torne un array de 2 enters: el primer és el valor més gran de l'array, el segon el més menut.

Exemple: `mayorYMenor([3, 7, 2, 9, 4]) → {9, 2}`

★ Exercici 2: Suma total

Escriu un mètode `public static int sumarElementos(int[] arr)` que torne la suma de tots els elements de l'array.

Exemple: `sumarElementos([1, 2, 3, 4, 5]) → 15`

Si l'array està buit, torna 0.

★ ★ Exercici 3: Cercador elemental

Escriu un mètode `public static boolean contiene(int[] arr, int valor)` que torne true si valor està a l'array, o false si no.

No val usar `Arrays.asList(arr).contains()`. Fes-ho amb un bucle.

★ ★ Exercici 4: Invertir array

Escriu un mètode `public static void invertir(int[] arr)` que modifiqui l'array original invertint l'ordre dels seus elements. No val crear un array nou.

Exemple: `[1, 2, 3, 4, 5] → [5, 4, 3, 2, 1]`

Pista: intercanvia el primer amb l'últim, el segon amb el penúltim...

★ ★ Exercici 5: Mitjana de doubles

Escriu un mètode public static double calcularMedia(double[] arr) que torne la mitjana aritmètica dels valors.

Exemple: calcularMedia([2.5, 3.5, 4.0]) → 3.333...

Si l'array està buit, torna 0.0.

★ ★ ★ Exercici 6: CodeWars — Sum without highest and lowest number

Resol la kata “Sum without highest and lowest number” (8 kyu) en [CodeWars](#).

Suma tots els números d'un array excepte el més alt i el més baix. Si l'array està buit o té un sol element, torna 0.

★ ★ ★ Exercici 7: CodeWars — You're a square!

Resol la kata “You're a square!” (7 kyu) en [CodeWars](#).

Donat un número enter, determina si és un quadrat perfecte. Torna true si ho és, false si no. Per exemple, 25 és quadrat perfecte (5×5), 26 no.

Butlletí 4 – Intermedi Result: Algorítmica I

Exercicis resolts de dificultat progressiva.

☆☆ Exercici 1: Parells amb suma objectiu

```
public static List<int[]> paresConSuma(int[] arr, int suma) {
    List<int[]> res = new ArrayList<>();
    Set<Integer> vistos = new HashSet<>();
    for (int n : arr) {
        if (vistos.contains(suma - n))
            res.add(new int[]{suma - n, n});
        vistos.add(n);
    }
    return res;
}
```

☆☆ Exercici 2: Eliminar duplicats (in-place)

```
public static int eliminarDuplicados(int[] arr) {
    if (arr.length == 0) return 0;
    int i = 0;
    for (int j = 1; j < arr.length; j++)
        if (arr[j] != arr[i]) arr[++i] = arr[j];
    return i + 1;
}
```

☆☆☆ Exercici 3: Major suma de subarray (Kadane)

```

public static int maxSubarraySum(int[] arr) {
    int maxG = arr[0], maxA = arr[0];
    for (int i = 1; i < arr.length; i++) {
        maxA = Math.max(arr[i], maxA + arr[i]);
        maxG = Math.max(maxG, maxA);
    }
    return maxG;
}

```

☆☆☆ Exercici 4: Rotar array k voltes

```

public static void rotar(int[] arr, int k) {
    k %= arr.length;
    invertir(arr, 0, arr.length - 1);
    invertir(arr, 0, k - 1);
    invertir(arr, k, arr.length - 1);
}

private static void invertir(int[] a, int l, int r) {
    while (l < r) { int t = a[l]; a[l] = a[r]; a[r] = t; l++; r--; }
}

```

☆☆☆ Exercici 5: Comptar ocurrencies (binària optimitzada)

```
public static int contar(int[] arr, int target) {  
    int izq = buscar(arr, target, true);  
    if (izq == -1) return 0;  
    return buscar(arr, target, false) - izq + 1;  
}  
  
private static int buscar(int[] a, int t, boolean primero) {  
    int l = 0, r = a.length - 1, res = -1;  
    while (l <= r) {  
        int m = (l + r) / 2;  
        if (a[m] == t) { res = m; if (primero) r = m - 1; else l = m + 1; }  
        else if (a[m] < t) l = m + 1;  
        else r = m - 1;  
    }  
    return res;  
}
```

Butlletí 4 – Intermedi: Algorítmica I

Exercicis de dificultat progressiva. Els ★ són per a escalfar, ★★ per a pensar, ★★★ per a competir.

★★ Exercici 1: Moure zeros al final

Escriu un mètode `public static void moverCeros(int[] arr)` que moga tots els zeros de l'array al final, mantenint l'ordre relatiu dels no zeros. Fes-ho **in-place**, sense crear un array nou.

Exemple: `[0, 1, 0, 3, 12] → [1, 3, 12, 0, 0]`

Pista: usa un índex `posicionNoCero` que avanci només quan trobes un número distint de zero. Intercanvia o desplaça segons el valor.

★★ Exercici 2: El número que falta

Donat un array de N enters únics que conté números del 0 al N (és a dir, té N+1 números possibles però només N elements), troba el número que falta.

Exemple: `[3, 0, 1]` falta el 2. `[9,6,4,2,3,5,7,0,1]` falta el 8.

Escriu el mètode `public static int numeroFaltante(int[] arr)`.

Pista: suma tots els números de l'array. Després suma tots els números de 0 a N (que és `arr.length`). La diferència és el número que falta.

★★ Exercici 3: Primer caràcter no repetit

Escriu un mètode `public static char primerNoRepetido(String s)` que torne el primer caràcter de la cadena que no es repeteix en cap altra posició. Si tots es repeteixen o la cadena està buida, torna ' ' (espai).

Exemple: "amor a roma" — la 'a' es repeteix, la 'm' es repeteix, la 'o' es repeteix, la 'r' es repeteix... l'espai es repeteix. En este cas, no hi ha caràcters no repetits, torna ' '.

Exemple: "programacion" — la 'p' no es repeteix → torna 'p'.

Pista: pots usar un array de 26 enters (un per lletra) per a comptar freqüències, o un array de 256 per a tot l'ASCII.

☆☆☆ Exercici 4: Anagrames

Dos strings són anagrames si contenen els mateixos caràcters amb la mateixa freqüència. Escriu `public static boolean sonAnagramas(String a, String b)` que torne `true` si ho són.

Exemples:

- "listen" i "silent" → `true`
- "java" i "avaj" → `true`
- "hola" i "halo" → `true`
- "hola" i "adios" → `false`

Pista: converteix-los a arrays de `char`, ordena'ls amb `Arrays.sort()`, i compara'ls. O compta freqüències amb un array de 26 enters.

☆☆☆ Exercici 5: Ordenar 0s, 1s i 2s (Dutch National Flag)

Donat un array que conté només 0, 1 i 2 ordena'l **in-place** en una sola passada (complexitat $O(n)$). Escriu `public static void ordenarColores(int[] arr)`.

Exemple: `[0, 2, 1, 2, 0, 1, 0] → [0, 0, 0, 1, 1, 2, 2]`

Pista: usa tres punters: `izquierdo` (per a 0s), `derecho` (per a 2s) i `actual` (que recorre l'array). Intercanvia segons el valor trobat. Este algorisme es diu "Dutch National Flag" i el va inventar Edsger Dijkstra. Si el resols, tens nivell d'entrevista tècnica a Google.

☆☆☆ Exercici 6: CodeWars — Disemvowel Trolls

Resol la kata "Disemvowel Trolls" (7 kyu) en [CodeWars](#).

Elimina totes les vocals (a, e, i, o, u) d'un string donat. El input pot tindre majúscules i minúscules. Per exemple, "This website is for losers LOL!" → "Ths wbst s fr lsrs LL!".

☆☆☆ Exercici 7: AceptaElReto — 219 La loteria

Resol el problema 219 — La loteria en [AceptaElReto.com](https://www.acepta-el-reto.com/problems/219).

Donat un array de números de loteria, compta quantes “inversions” hi ha: parells de posicions (i, j) on $i < j$ però el número en i és major que el de j. En altres paraules, compta quants parells estan desordenats.

Pista: pots fer-ho amb dos bucles niats (complexitat $O(n^2)$), però per a valors grans necessaries merge sort ($O(n \log n)$). Comença per la versió simple.



Referències

Plataforma	Problema	Dificultat
AceptaElReto	151 — ¿Es matriz identidad?	Mitjà
AceptaElReto	219 — La loteria	Mitjà
AceptaElReto	340 — Jocs de naips	Mitjà
CodeWars	Vowel Count (7 kyu)	Fàcil
CodeWars	Disemvowel Trolls (7 kyu)	Fàcil
CodeWars	You're a square! (7 kyu)	Fàcil

Butlletí 04 — Extres: Algorísmica I: Fonaments

★ **Extres — Converteix-te en un programador de molt alt nivell** Aquests exercicis són completament opcionals, però si aspirem a ser un programador o programadora d'elit, ací tens el teu camp d'entrenament. CodeWars i AceptaElReto són plataformes on els millors programadors del món es repte cada dia. Cada problema que resolguis et farà més fort, més ràpid i més creatiu. No els subestimis: són la diferència entre saber programar i ser un vertader professional.

CodeWars:

- [Find the odd int](#) (6 kyu)

Pista: Aplica XOR (^) a tots els números del array — els parells es cancel·len i el que queda és el solitari.

- [Playing with digits](#) (6 kyu)

Pista: Converteix el número a cadena, itera per cada dígit elevat-lo a la potència corresponent i suma.

- [Maximum subarray sum](#) (5 kyu)

Pista: Aplica Kadane: mantén la suma actual i la màxima, reinicia la suma actual si es torna negativa.

AceptaElReto:

- [219 - La loteria](#) (★ ★)

Pista: Compta les inversions amb un bucle niuat o amb una estratègia de divideix i venceràs més eficient.

- [340 - Suma de dígit](#) (★)

Pista: Trau l'últim dígit amb % 10, suma i divideix entre 10 fins que no queden dígit. Repeteix fins que la suma siga d'un sol dígit (arrel digital).

Boletín 5 – Inicial Resuelto: Algorítmica II

Soluciones del boletín inicial.

★ Ejercicio 1: Factorial recursivo

```
public static int factorial(int n) {  
    if (n <= 1) return 1;  
    return n * factorial(n - 1);  
}
```

★ Ejercicio 2: Fibonacci recursivo

```
public static int fibo(int n) {  
    if (n <= 1) return n;  
    return fibo(n - 1) + fibo(n - 2);  
}
```

★ ★ Ejercicio 3: Suma recursiva de array

```
public static int sumaArray(int[] arr, int i) {  
    if (i == arr.length) return 0;  
    return arr[i] + sumaArray(arr, i + 1);  
}
```

★ ★ Ejercicio 4: Invertir cadena

```
public static String invertir(String s) {  
    if (s.isEmpty()) return "";  
    return invertir(s.substring(1)) + s.charAt(0);  
}
```

★ ★ ★ Ejercicio 5: Fibonacci con memoización

```

public static int fiboMemo(int n, int[] memo) {
    if (n <= 1) return n;
    if (memo[n] != 0) return memo[n];
    memo[n] = fiboMemo(n - 1, memo) + fiboMemo(n - 2, memo);
    return memo[n];
}
// Uso: fiboMemo(10, new int[11])

```

Ejercicio 6: Quicksort

```

public static void quicksort(int[] a, int l, int r) {
    if (l >= r) return;
    int p = partition(a, l, r);
    quicksort(a, l, p - 1);
    quicksort(a, p + 1, r);
}

private static int partition(int[] a, int l, int r) {
    int p = a[r], i = l - 1;
    for (int j = l; j < r; j++)
        if (a[j] <= p) { i++; int t = a[i]; a[i] = a[j]; a[j] = t; }
    int t = a[i + 1]; a[i + 1] = a[r]; a[r] = t;
    return i + 1;
}

```

Boletí 5 – Inicial: Algorísmica II

Sense solucions. La recursivitat és com caure per un pou, però el pou es repeteix a si mateix.

Exercici 1: Potència recursiva

Implementa una funció recursiva que calcule a elevat a n (amb $n \geq 0$). Sense usar `Math.pow()` ni bucles.

```
public static int potencia(int a, int n) {  
    // el teu codi aquí  
}
```

Exemple: `potencia(2, 3) → 8`, `potencia(5, 0) → 1`.

Exercici 2: Comptar dígitos recursiu

Implementa una funció recursiva que compte quants dígitos té un nombre enter positiu.

```
public static int contarDigits(int n) {  
    // el teu codi aquí  
}
```

Exemple: `contarDigits(12345) → 5`, `contarDigits(7) → 1`, `contarDigits(0) → 1`.

Exercici 3: Què imprimeix?

Sense executar, digues què imprimeix este codi malvat:

```
public class Misteri {  
    public static int enigma(int x) {  
        if (x < 10) return x;  
        return enigma(x / 10) + x % 10;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(enigma(1234));  
        System.out.println(enigma(999));  
        System.out.println(enigma(5));  
    }  
}
```

Exercici 4: Palíndrom recursiu

Implementa una funció recursiva que determine si un String és un palíndrom (es llig igual cap endavant i cap arrere).

```
public static boolean esPalindrom(String s) {  
    // el teu codi aquí  
}
```

Exemples: esPalindrom("ana") → true, esPalindrom("reconéixer") → true,
esPalindrom("hola") → false.

Exercici 5: Troba l'error

Què li passa a aquest codi? Compila, però no funciona com esperem. Explica per què:

```
public static int factorial(int n) {  
    return n * factorial(n - 1);  
}
```

Què ocorre si cridem factorial(5)?

Exercici 6: Sumatori recursiu

Implementa una funció recursiva que sume tots els nombres des de 1 fins a n.

```
public static int sumatori(int n) {  
    // el teu codi ací  
}
```

Exemple: `sumatori(5)` → 15 (1+2+3+4+5).

Exercici 7: CodeWars — Reversed Strings

Resol la kata “**Reversed Strings**” (8 kyu) en CodeWars.

Completa la funció `solution` que rep un `String` i torna el `String` al revés. Fàcil, oi? Doncs fes-ho **recursivament**, no amb `StringBuilder.reverse()`.

```
public class StringReverser {  
    public static String solution(String s) {  
        // el teu codi recursiu ací  
    }  
}
```

Exercici 8: AceptaElReto — 165 Número hyperpar

Resol el problema **165 — Número hyperpar** en [AceptaElReto.com](https://www.aceptaelreto.com/).

Un nombre és “hyperpar” si tots els seus dígitos són parells. Donada una seqüència de nombres fins que s'introdueixi -1, indica si cada un és hyperpar o no.

Exemple: 246 → SI, 2486 → SI, 123 → NO, 0 → SI.

Boletín 5 - Intermedio Resuelto: Algorítmica II

Ejercicios resueltos de nivel intermedio.

☆☆ Ejercicio 1: Torres de Hanoi

```
public static void hanoi(int n, char origen, char destino, char aux) {
    if (n == 1) System.out.println(origen + " -> " + destino);
    else {
        hanoi(n - 1, origen, aux, destino);
        System.out.println(origen + " -> " + destino);
        hanoi(n - 1, aux, destino, origen);
    }
}
```

☆☆ Ejercicio 2: Mergesort

```
public static void mergesort(int[] a, int l, int r) {
    if (l >= r) return;
    int m = (l + r) / 2;
    mergesort(a, l, m);
    mergesort(a, m + 1, r);
    merge(a, l, m, r);
}

private static void merge(int[] a, int l, int m, int r) {
    int[] t = new int[r - l + 1];
    int i = l, j = m + 1, k = 0;
    while (i <= m && j <= r) t[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i <= m) t[k++] = a[i++];
    while (j <= r) t[k++] = a[j++];
    System.arraycopy(t, 0, a, l, t.length);
}
```

☆☆☆ Ejercicio 3: Búsqueda binaria recursiva

```
public static int binariaRec(int[] a, int t, int l, int r) {
    if (l > r) return -1;
    int m = (l + r) / 2;
    if (a[m] == t) return m;
    return (a[m] < t) ? binariaRec(a, t, m + 1, r) : binariaRec(a, t, l, m - 1);
}
```

☆☆☆ Ejercicio 4: Potencia rápida recursiva ($O(\log n)$)

```
public static long potencia(long base, long exp) {
    if (exp == 0) return 1;
    long mitad = potencia(base, exp / 2);
    return (exp % 2 == 0) ? mitad * mitad : mitad * mitad * base;
}
```

☆☆☆ Ejercicio 5: Subconjuntos (backtracking)

```
public static List<List<Integer>> subconjuntos(int[] nums) {
    List<List<Integer>> res = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), res);
    return res;
}
private static void backtrack(int[] n, int i, List<Integer> cur, List<List<Integer>> r)
{
    r.add(new ArrayList<>(cur));
    for (int j = i; j < n.length; j++) {
        cur.add(n[j]);
        backtrack(n, j + 1, cur, r);
        cur.remove(cur.size() - 1);
    }
}
```

Boletí 5 – Intermedi: Algorísmica II: Tècniques Avançades

Exercicis de dificultat progressiva. Els ★ són per a escalfar, ★★ per a pensar, ★★★ per a concursar. Si la recursivitat et sembla un bucle sense fi, espera a fer aquests exercicis. Bé, en realitat SÍ que és un bucle sense fi, però controlat.

★ Exercici 1: MCD per Euclides recursiu

Implementa l'algoritme d'Euclides per a calcular el màxim comú divisor de dos nombres enters **de forma recursiva**. Sense trampes: res de bucles, res de `Math.min()` anant cap enrere. Hi ha d'haver una crida a si mateix.

L'algoritme d'Euclides diu:

- Si $b == 0$, el MCD és a .
- Si no, el MCD de a i b és el MCD de b i $a \% b$.

```
public static int mcd(int a, int b) {  
    // el teu codi aquí  
}
```

Exemple: `mcd(48, 18) → 6`. `mcd(100, 25) → 25`.

★ Exercici 2: Comptar ocurrències recursiu

Donat un array d'enters i un nombre objectiu, compta quantes vegades apareix eixe nombre en l'array **usant recursivitat**. Sense bucles, sense streams, sense trampes.

```
public static int contarOcurrències(int[] arr, int objectiu, int index) {  
    // el teu codi aquí  
}
```

Exemple: `contarOcurrències(new int[]{1, 2, 3, 2, 4, 2, 5}, 2, 0) → 3`.

☆☆ Exercici 3: Invertir nombre recursiu

Donat un nombre enter positiu, torna el seu invers. Es a dir, 1234 es converteix en 4321. **Sense convertir a String**. Tot amb operacions matemàtiques i recursivitat.

```
public static int invertirNombre(int n) {  
    // el teu codi aquí  
}
```

Exemple: `invertirNombre(1234)` → 4321. `invertirNombre(700)` → 7.

☆☆ Exercici 4: Combinacions d'un array (backtracking)

Donat un array d'enters i un nombre k, genera **totes les combinacions possibles** de grandària k dels elements de l'array. L'ordre dels elements dins de cada combinació no importa, i no hi ha d'haver combinacions repetides.

```
public static List<List<Integer>> combinacions(int[] arr, int k) {  
    // el teu codi aquí  
}
```

Exemple: `combinacions(new int[]{1, 2, 3}, 2)` torna `[[1,2], [1,3], [2,3]]`.

☆☆☆ Exercici 5: N-Reines (backtracking)

El clàssic dels clàssics. Col·loca N reines en un tauler de N×N de manera que cap s'amenace entre si. Dos reines s'amenacen si estan en la mateixa fila, columna o diagonal.

```
public static boolean resoldre(int[][] tauler, int col) {  
    // el teu codi aquí  
}
```



Exercici 6: CodeWars — Tribonacci Sequence

Resol la kata “Tribonacci Sequence” (6 kyu) en CodeWars.

El Fibonacci s’ha quedat obsolet. Ara toca el Tribonacci: igual però sumant els tres últims en lloc de dos. Reps un array signature de 3 nombres i un n que indica quants elements ha de tindre la seqüència resultant.

```
public double[] tribonacci(double[] s, int n) {  
    // el teu codi ací  
}
```



Exercici 7: AceptaElReto — 140 Suma de dígit

Resol el problema 140 — Suma de dígit en [AceptaElReto.com](https://www.acepta-el-reto.com/).

Donat un nombre, suma els seus dígit. Si el resultat té més d’un dígit, torna a sumar. Repeteix fins a obtenir un sol dígit. Es la “rel digital” o “suma de dígit recursiva”.



Referències

Plataforma	Problema	Dificultat
CodeWars	Tribonacci Sequence	6 kyu
CodeWars	Sum of Digits / Digital Root	6 kyu
AceptaElReto	140 — Suma de dígit	Fàcil
AceptaElReto	162 — Suma de dígit	Fàcil

Butlletí 05 — Extres: Algorísmica II: Tècniques Avançades

★ **Extres — Converteix-te en un programador de molt alt nivell** Aquests exercicis són completament opcionals, però si aspirem a ser un programador o programadora d'elit, ací tens el teu camp d'entrenament. CodeWars i AceptaElReto són plataformes on els millors programadors del món es repte cada dia. Cada problema que resolguis et farà més fort, més ràpid i més creatiu. No els subestimis: són la diferència entre saber programar i ser un vertader professional.

CodeWars:

- [Sum of Intervals](#) (4 kyu)

Pista: Ordena els intervals per inici. Fusiona els solapants i suma les longituds dels no solapants.

- [Snail Sort](#) (4 kyu)

Pista: Recorre la matriu per capes concèntriques: dreta, avall, esquerra, amunt; reduïx els límits en cada volta.

- [Range Extraction](#) (4 kyu)

Pista: Recorre el array ordenat i agrupa els números consecutius en rangs [inici-fi].

AceptaElReto:

- [375 - El linx ibèric](#) (★★★★)

Pista: Modela el mapa com un graf i aplica BFS per trobar la distància mínima entre dos punts.

- [465 - Nombres afortunats](#) (★★★★)

Pista: Simula el procés amb una llista booleana (garbell) i ves ratllant posicions segons el nombre afortunat actual.

Boletín 4 – Inicial Resuelto: POO – Clases y Objetos

Las soluciones están aquí, esperándote. Pero primero inténtalo. La POO se aprende creando objetos, no leyendo sobre ellos.

Ejercicio 1: Completa la clase Perro

```
public class Perro {  
    String nombre;  
    int edad;  
  
    public Perro(String nombre, int edad) {  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
  
    public void ladrar() {  
        System.out.println("Guau, soy " + nombre);  
    }  
  
    public void cumplirAnios() {  
        edad++;  
    }  
  
    public static void main(String[] args) {  
        Perro rex = new Perro("Rex", 3);  
        rex.ladrar();  
        System.out.println("Edad: " + rex.edad);  
        rex.cumplirAnios();  
        System.out.println("Después de cumple: " + rex.edad);  
    }  
}
```



Explicación: El constructor `public Perro(String nombre, int edad)` asigna los parámetros a los atributos usando `this` para distinguir entre el parámetro `nombre` y el atributo `this.nombre`. Sin `this`, Java pensaría que son la misma variable (la del parámetro) y no asignaría nada al atributo. El método `cumplirAnios()` simplemente hace `edad++` igual que

harías con cualquier variable. Al crear el objeto con `new Perro("Rex", 3)`, se ejecuta el constructor y el perro nace con 3 años. Luego ladra y cumple años como cualquier ser vivo (bueno, los perros no dicen “Guau, soy Rex”, pero si lo hicieran, sería exactamente así).

Ejercicio 2: ¿Qué imprime?

```
public class Coche {
    String marca;
    int velocidad;

    public Coche(String marca) {
        this.marca = marca;
        this.velocidad = 0;
    }

    public void acelerar(int kmh) {
        velocidad += kmh;
    }

    public static void main(String[] args) {
        Coche c = new Coche("Seat");
        c.acelerar(50);
        c.acelerar(30);
        c.acelerar(-10);
        System.out.println(c.marca + " va a " + c.velocidad + " km/h");
    }
}
```

Salida: Seat va a 70 km/h

 **Explicación:** Creamos un Seat con velocidad 0. Aceleramos +50 → 50. Aceleramos +30 → 80. Aceleramos -10 → 70. El método `acelerar` acepta valores negativos (frenar en realidad). No hay validación que impida ir a velocidad negativa porque no la programamos. Esto es un ejemplo de por qué la encapsulación es importante: cualquiera podría pasar un -500 y el coche iría a -420 km/h... ¿marcha atrás? En el siguiente boletín veremos cómo evitarlo con setters y validación.

Ejercicio 3: Encuentra el error


```
public class Main {  
    public static void main(String[] args) {  
        Rectangulo r = Rectangulo(5, 3); // ERROR: falta 'new'  
        System.out.println(r.area());  
    }  
}
```

Error: Falta new. Debe ser new Rectangulo(5, 3).

💡 **Explicación:** En Java, los objetos se crean con new. Sin new, estás tratando de llamar a un método que no existe (como si Rectangulo fuera un método estático). Java se queja: “no sé qué es Rectangulo(5, 3)”. Es como si dijeras “coche()” en lugar de “coche nuevo()”. Sin new, el objeto no se crea en memoria. Solo declaras una referencia a nada (null). **Regla de oro:** si quieres un objeto, necesitas new. Siempre.

Ejercicio 4: Escribe la clase Persona

```

public class Persona {
    String nombre;
    int edad;
    String ciudad;

    public Persona(String nombre, int edad, String ciudad) {
        this.nombre = nombre;
        this.edad = edad;
        this.ciudad = ciudad;
    }

    public void saludar() {
        System.out.println("Hola, soy " + nombre + " de " + ciudad);
    }

    public boolean esMayorEdad() {
        return edad >= 18;
    }

    public static void main(String[] args) {
        Persona p1 = new Persona("Ana", 25, "Madrid");
        Persona p2 = new Persona("Luis", 17, "Barcelona");

        p1.saludar();
        System.out.println(p1.nombre + " es mayor? " + p1.esMayorEdad());

        p2.saludar();
        System.out.println(p2.nombre + " es mayor? " + p2.esMayorEdad());
    }
}

```

💡 **Explicación:** La clase Persona encapsula tres datos (nombre, edad, ciudad) y dos comportamientos (saludar, esMayorEdad). Cada objeto Persona tiene su propia copia de estos atributos. p1 y p2 son independientes: Ana tiene 25 años y es mayor de edad; Luis tiene 17 y no lo es. La POO permite agrupar datos y comportamiento en una misma entidad. Es como tener fichas de personas: cada ficha tiene sus datos y sus acciones. Sin POO, tendrías arrays separados para nombres, edades y ciudades, y funciones sueltas. Con POO, todo está junto y ordenado.

Ejercicio 5: ¿Constructor por defecto?

```
public class Prueba {  
    int x;  
    boolean activo;  
    String texto;  
  
    public static void main(String[] args) {  
        Prueba p = new Prueba();  
        System.out.println("x = " + p.x);  
        System.out.println("activo = " + p.activo);  
        System.out.println("texto = " + p.texto);  
    }  
}
```

Salida:

```
x = 0  
activo = false  
texto = null
```



Explicación: Cuando no defines ningún constructor, Java te regala uno por defecto (sin parámetros). Ese constructor inicializa los atributos a sus valores por defecto: números a 0, booleanos a false, objetos (como String) a null. Es como cuando abres una caja nueva: está vacía (null), no hay nadie dentro (false) y el contador está a 0. Por eso x es 0, activo es false y texto es null. Si hubieras creado un constructor propio, el constructor por defecto desaparece. Es como si al personalizar tu casa, perdieras la versión básica.

Ejercicio 6: AceptaElReto 291 — Números afortunados

```

import java.util.Scanner;

public class Problema291 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int casos = sc.nextInt();
        for (int c = 0; c < casos; c++) {
            int n = sc.nextInt();
            int suma = 0;
            while (n > 0) {
                suma += n % 10;
                n /= 10;
            }
            if (suma == 7 || suma % 7 == 0) {
                System.out.println("afortunado");
            } else {
                System.out.println("desafortunado");
            }
        }
    }
}

```

💡 **Explicación:** Extraemos los dígitos de n con $n \% 10$ y $n / 10$, los sumamos, y comprobamos si la suma es 7 o múltiplo de 7. Por ejemplo: $16 \rightarrow 1+6=7 \rightarrow$ afortunado. $25 \rightarrow 2+5=7 \rightarrow$ afortunado. $34 \rightarrow 3+4=7 \rightarrow$ afortunado. $10 \rightarrow 1+0=1 \rightarrow$ desafortunado. $7 \rightarrow 7 \rightarrow$ afortunado. Es un problema sencillo de AceptaElReto que combina el bucle de extracción de dígitos con condiciones. Si tu número favorito es el 7, estás de suerte.

Ejercicio 7: CodeWars — Grasshopper - Summation

```
public class Grasshopper {  
    public static int summation(int n) {  
        int suma = 0;  
        for (int i = 1; i <= n; i++) {  
            suma += i;  
        }  
        return suma;  
    }  
}
```

💡 **Explicación:** CodeWars espera el método exacto `public static int summation(int n)`. El método suma del 1 a n con un bucle. También se puede hacer con la fórmula matemática $n * (n + 1) / 2$, que es más eficiente. Pero para un 8 kyu, el bucle está bien. La kata te enseña que CodeWars tiene una firma concreta que debes seguir al pie de la letra. Si el método se llama `summation` y devuelve `int`, no puedes llamarlo `sumar` ni devolver `double`. CodeWars es muy puntilloso con las firmas, como un profesor de lengua con las tildes.

Boletí 4 – Inicial: POO: Classes i Objectes

Sense solucions. Les classes no es crearan soles. I si es crearen soles, probablement serien classes abstractes, però això és per a un altre butlletí.

Exercici 1: Completa la classe Alumne

Falten algunes parts. Completa-les perquè funcione:

```
public class Alumne {  
    String nom;  
    String cognoms;  
    int edat;  
  
    // Completa el constructor: ha de rebre nom, cognoms i edat  
    public Alumne(String nom, String cognoms, int edat) {  
        // Què va ací?  
    }  
  
    public void presentarSe() {  
        System.out.println("Hola, soc " + nom + " " + cognoms);  
    }  
  
    public void cumpleAnys() {  
        // Incrementa l'edat en 1  
    }  
}
```

Exercici 2: Què imprimeix?

Sense executar, digues què ix:

```
public class Comptador {  
    int valor;  
  
    public Comptador(int valorInicial) {  
        valor = valorInicial;  
    }  
  
    public void incrementar() {  
        valor++;  
    }  
  
    public void decrementar() {  
        valor--;  
    }  
  
    public static void main(String[] args) {  
        Comptador c = new Comptador(10);  
        c.incrementar();  
        c.incrementar();  
        c.decrementar();  
        c.incrementar();  
        System.out.println("Valor final: " + c.valor);  
    }  
}
```

Exercici 3: Troba l'error

```
public class Producte {  
    String nom;  
    double preu;  
  
    public Producte(String nom, double preu) {  
        nom = nom;  
        preu = preu;  
    }  
  
    public void mostrar() {  
        System.out.println(nom + " costa " + preu + "€");  
    }  
}
```

Hi ha 1 error. Troba'l.

Exercici 4: Escriu la classe Pel·lícula

Crea una classe Pelicula amb:

- Atributs: String titol, String genere, int durada (en minuts)
 - Constructor que reba els tres paràmetres
 - Mètode void reproduir() que imprimeixca “Reproduint [títol] ([gènere]) - [durada] min”
 - Mètode boolean esLlarga() que torne true si la durada és major a 120 minuts
-

Exercici 5: Constructor buit i setters

Què imprimeix aquest codi?


```
public class Estudiant {  
    String nom;  
    double notaMitjana;  
  
    public Estudiant() {  
        this.nom = "Desconegut";  
        this.notaMitjana = 5.0;  
    }  
  
    public Estudiant(String nom, double notaMitjana) {  
        this.nom = nom;  
        this.notaMitjana = notaMitjana;  
    }  
  
    public static void main(String[] args) {  
        Estudiant e1 = new Estudiant();  
        Estudiant e2 = new Estudiant("Carles", 8.5);  
        System.out.println(e1.nom + " - " + e1.notaMitjana);  
        System.out.println(e2.nom + " - " + e2.notaMitjana);  
    }  
}
```

Exercici 6: AcceptaElReto — 417 Nombres binomials

Resol el problema **417 — Nombres binomials** en [AcceptaElReto.com](https://www.acepta-el-reto.com/).

Calcula el coeficient binomial “n sobre k”. Tradicionalment es defineix com $n! / (k! * (n-k)!)$.

Exercici 7: CodeWars — Return the day

Resol la kata “Return the day” (8 kyu) en CodeWars.

Crea una classe DayOfWeek amb un mètode estàtic `getDay(int n)` que torne el dia de la setmana corresponent.

Boletín 4 – Resuelto: POO – Clases y Objetos

Ejercicios progresivos. De crear tu primera clase a competir en AceptaElReto con objetos. Ponte el cinturón que la POO acelera.

★ Ejercicio 1: Clase Rectángulo

Crea una clase `Rectangulo` con `ancho` y `alto` (`double`). Constructor parametrizado, `calcularArea()`, `calcularPerimetro()`. Sobrescribe `toString()`.

💡 **Explicación:** El constructor asigna `ancho` y `alto`. Los métodos calculan área y perímetro con fórmulas básicas. `toString()` devuelve una representación legible del objeto. Sin `toString()`, Java imprimiría algo como `Rectangulo@2f92e0f4` (la clase y una dirección de memoria). Con `toString()`, imprime `Rectángulo [5 x 3]`. Sobrescribir `toString()` es una de las mejores prácticas de Java: tu yo del futuro (y tu profesor) te lo agradecerán cuando depures.

★ Ejercicio 2: Clase CuentaBancaria

`CuentaBancaria` con `titular`, `saldo`, `numeroCuenta`. Dos constructores: solo `titular` (`saldo 0`) o `titular + saldo`. Métodos `ingresar(double)` y `retirar(double)` (valida `saldo` suficiente). `toString()`.

💡 **Explicación:** Usamos sobrecarga de constructores: uno llama al otro con `this(titular, 0)`. El número de cuenta se genera automáticamente con un contador estático (cada cuenta nueva tiene un número distinto). `ingresar` solo acepta cantidades positivas. `retirar` valida que haya `saldo` suficiente. Es como un cajero automático de verdad, pero sin colas ni comisiones. La validación evita que te quedes en números rojos. Bueno, o al menos te avisa.

★★ Ejercicio 3: Clase Hora con validación

`Hora` con `hora` (0-23), `minuto` (0-59), `segundo` (0-59). Constructor con validación (lanza `IllegalArgumentException`). `incrementarSegundo()` que maneje desbordamientos.

💡 **Explicación:** El constructor valida que hora, minuto y segundo estén en rangos válidos. Si no, lanza una `IllegalArgumentException` (una excepción unchecked que no necesita try-catch, pero mata el programa si no la capturas). `incrementarSegundo()` suma 1 segundo y maneja los desbordamientos: si segundo llega a 60, pasa a 0 y aumenta minuto; igual con minuto→hora→día. Ejemplo: 23:59:59 + 1 segundo → 00:00:00. Es como un reloj digital, pero sin pilas.

☆☆ Ejercicio 4: equals() en Persona

Sobrescribe `equals()` en la clase `Persona` para que dos personas sean iguales si tienen el mismo nombre y edad.

💡 **Explicación:** Sin `equals()`, comparar objetos con `equals` es lo mismo que con `==` (comparan referencias, no contenido). Aquí sobrescribimos `equals` para que compare nombre y edad. `Objects.equals(nombre, persona.nombre)` es seguro porque maneja nulls. `hashCode()` debe sobrescribirse siempre que sobrescribas `equals` (contrato de Java). Si no, las colecciones como `HashSet` o `HashMap` se comportarán de forma extraña. Es como el jamón y el queso: `equals` y `hashCode` van juntos.

☆☆ Ejercicio 5: Clase Punto y distancia euclídea

`Punto` con `x` e `y` (int). Constructor, getters, `distancia(Punto otro)` (distancia euclídea), `toString()`. Calcula el perímetro de un triángulo dados 3 puntos.

💡 **Explicación:** La distancia euclídea entre dos puntos es la raíz cuadrada de $(dx^2 + dy^2)$. Con `Math.sqrt()` lo calculamos. El perímetro del triángulo es la suma de las distancias entre cada par de puntos. El triángulo (0,0)-(3,0)-(0,4) es un triángulo rectángulo clásico de lados 3, 4 y 5 (hipotenusa). Perímetro = 3 + 4 + 5 = 12. La POO permite tratar los puntos como objetos con comportamientos. En lugar de tener funciones sueltas `distancia(x1,y1,x2,y2)`, tenemos `a.distancia(b)`. Mucho más limpio.

☆☆☆ Ejercicio 6: AceptaElReto 417 — Números binomiales

Resuelve el problema **417 — Números binomiales** en AceptaElReto.

Dados dos números n y k , calcula el coeficiente binomial “ n sobre k ”. Usa una función recursiva o iterativa.

💡 **Explicación:** El coeficiente binomial “ n sobre k ” cuenta cuántas formas hay de elegir k elementos de un conjunto de n . La fórmula iterativa es más eficiente que la recursiva (evita calcular factoriales enormes). Usamos la propiedad de simetría: si $k > n-k$, usamos $n-k$ (reduce iteraciones). El bucle va acumulando el resultado multiplicando y dividiendo para evitar números muy grandes. El problema lee pares (n,k) hasta encontrar $(0,0)$. Es un problema clásico de combinatoria, muy común en olimpiadas de programación.

☆☆☆ Ejercicio 7: AceptaElReto 458 — El espejo

Resuelve el problema **458 — El espejo** en AceptaElReto.

Dada una hora en formato “HH:MM”, calcula cuántos minutos faltan para la siguiente hora que sea un “espejo” (palíndromo). Ejemplo: 10:01 es espejo, 23:32 también.

💡 **Explicación:** Un “espejo” significa que la hora y los minutos se reflejan: las decenas de la hora son las unidades de los minutos, y viceversa. 10:01 $\rightarrow 1=1$ y $0=0 \rightarrow$ es espejo. 12:21 $\rightarrow 1=1$ y $2=2 \rightarrow$ es espejo. El bucle incrementa minutos de 1 en 1 hasta encontrar una hora espejo, manejando el desbordamiento de minutos y horas. Es una búsqueda exhaustiva pero con máximo $60 \cdot 24 = 1440$ iteraciones, que es trivial. El espejo más cercano a 23:32 (que ya es espejo) es 0 minutos. Para 23:33, el siguiente espejo es 0:00 (27 minutos después). Mírate al espejo y verás la solución.

☆☆☆ Ejercicio 8: Simulación de batalla Pokémon (lite)

Crea una clase Pokemon con nombre, vida, ataque. Método atacar(Pokemon otro) que resta `this.ataque` a la vida del otro. Crea dos pokémon y simula una batalla por turnos hasta que uno se quede sin vida.

💡 **Explicación:** Cada Pokémon tiene nombre, vida y ataque. El método atacar resta el ataque del atacante a la vida del defensor. La batalla alterna turnos. El bucle continúa

mientras ambos estén vivos. Cuando uno llega a 0 de vida, `estaVivo()` devuelve `false` y el bucle termina. Es una versión muy simplificada de un juego de lucha. Podrías añadir defensa, tipos, habilidades especiales... pero eso ya es cosa tuya. Este ejercicio demuestra cómo la POO permite modelar entidades del mundo real (o de mundos ficticios) de forma natural. Cada Pokémon es un objeto independiente con sus propios datos y comportamientos.



Referencias

Plataforma	Problema	Dificultad
AceptaElReto	291 — Números afortunados	Fácil
AceptaElReto	417 — Números binomiales	Medio
AceptaElReto	458 — El espejo	Medio
CodeWars	Grasshopper - Summation (8 kyu)	Principiante
CodeWars	Basic variable assignment (8 kyu)	Principiante

Boletí 6 – Intermedi: POO: Classes i Objectes

Exercicis de dificultat progressiva. Els ★ són per a escalfar, ★★ per a pensar, ★★★ per a concursar. De crear la teua primera classe a construir objectes que farien plorar d'enveja al teu professor de FP Bàsica.

★ Exercici 1: Classe Llibre

Crea una classe Llibre amb:

- Atributs privats: `String titol`, `String autor`, `int pàgines`
- Constructor que reba els tres paràmetres
- Getters per a tots els atributs
- Mètode `toString()` que torne "El llibre '[títol]' de [autor] té [pàgines] pàgines."

```
public class Llibre {  
    // atributs  
    // constructor  
    // getters  
    // toString()  
}
```

★ Exercici 2: Classe Termòmetre

Crea una classe Termometre que emmagatzeme la temperatura en graus Celsius (privat). Ha de tindre:

- Constructor que reba els Celsius inicials
- Getter `getCelsius()`
- Mètode `getFahrenheit()` que torne la temperatura en Fahrenheit: $^{\circ}\text{F} = ^{\circ}\text{C} * 9/5 + 32$
- Mètode `getKelvin()` que torne la temperatura en Kelvin: $\text{K} = ^{\circ}\text{C} + 273.15$

```
public class Termometre {  
    private double celsius;  
    // constructors, getters, setters  
}
```

★ ★ Exercici 3: Classe Data amb validació

Crea una classe Data amb dia, mes, any (int, privats). El constructor ha de validar que la data siga vàlida: dies correctes per mes, tenint en compte anys de traspàs.

```
public class Data {  
    private int dia, mes, any;  
  
    public Data(int dia, int mes, int any) {  
        // validar  
    }  
  
    public boolean esTraspas() {  
        // (any % 4 == 0 && any % 100 != 0) || any % 400 == 0  
    }  
}
```

★ ★ Exercici 4: Classe Relotge amb avanç

Crea una classe Relotge amb hora (0-23), minut (0-59), segon (0-59), tots privats. Constructor amb validació.

```
public class Rellotge {  
    private int hora, minut, segon;  
  
    public Rellotge(int hora, int minut, int segon) {  
        // validar  
    }  
  
    public void avançar(int segons) {  
        // suma segons, gestiona desbordaments  
    }  
  
    public String toString() {  
        // format HH:MM:SS  
    }  
}
```

☆☆☆ Exercici 5: Classe Matriu 2D

Crea una classe Matriu que represente una matriu bidimensional d'enters.

```
public class Matriu {  
    private int[][] dades;  
    private int files, columnes;  
    // constructors, sumar, restar, multiplicar, toString()  
}
```

☆☆☆ Exercici 6: CodeWars — Build Tower

Resol la kata “Build Tower” (6 kyu) en CodeWars.

Crea una classe TowerBuilder amb un mètode estàtic build(int nFloors) que torne un array de Strings representant una torre amb nFloors pisos feta d'asteriscs *.

☆☆☆ Exercici 7: AceptaElReto — 246 Buscant el pin



Referències

Plataforma	Problema	Dificultat
CodeWars	Build Tower	6 kyu
CodeWars	Persistent Bugger	6 kyu
AceptaElReto	246 — Buscant el pin	Mitjà
AceptaElReto	367 — Ascensors	Mitjà

Butlletí 06 — Extres: POO: Classes i Objectes

★ **Extres — Converteix-te en un programador de molt alt nivell** Aquests exercicis són completament opcionals, però si aspirem a ser un programador o programadora d'elit, ací tens el teu camp d'entrenament. CodeWars i AceptaElReto són plataformes on els millors programadors del món es repte cada dia. Cada problema que resolguis et farà més fort, més ràpid i més creatiu. No els subestimis: són la diferència entre saber programar i ser un vertader professional.

CodeWars:

- [Build a pile of Cubes](#) (6 kyu)
Pista: Resta cubs n^3 des de $n=1$ fins que el resultat siga 0 (o torna -1 si te'n passes).
- [Valid Braces](#) (6 kyu)
Pista: Utilitza una pila (Stack). En obrir, apila; en tancar, comprova que coincidisca amb el cim.
- [Simple Pig Latin](#) (5 kyu)
Pista: Dividix per paraules. Si la paraula comença per vocal, afig "way"; si no, mou la primera consonant al final i afig "ay".

AceptaElReto:

- [246 - Buscant el pin](#) (★ ★)
Pista: Construeix un graf d'adjacències entre les caselles del pinball i aplica DFS/BFS per trobar el camí.
- [367 - Ascensors](#) (★ ★)
Pista: Simula els ascensors volta a volta. En cada parada, puja i baixa persones fins a arribar al límit de capacitat.

Boletín 5 – Inicial Resuelto: Visibilidad, Encapsulación y Static

Las soluciones explicadas con humor. Porque la encapsulación no tiene por qué ser aburrida.

Ejercicio 1: Encuentra el error de visibilidad

```
public class Casa {
    public String direccion;
    protected String telefono;
    int numeroHabitaciones;
    private String contrasenaWifi;

    public void mostrarTodo() {
        System.out.println(direccion);           // OK: misma clase
        System.out.println(telefono);            // OK: misma clase
        System.out.println(numeroHabitaciones);  // OK: misma clase
        System.out.println(contrasenaWifi);      // OK: misma clase
    }
}

public class Vecino {
    public void espionar() {
        Casa c = new Casa();
        System.out.println(c.direccion);          // OK: public
        System.out.println(c.telefono);           // OK: protected (mismo paquete)
        System.out.println(c.numeroHabitaciones); // OK: package-private (mismo
paquete)
        System.out.println(c.contrasenaWifi);    // ERROR: private
    }
}
```

💡 **Explicación:** `contrasenaWifi` es `private`. Solo la clase `Casa` puede acceder a él. Ni su vecino (`Vecino`), ni su madre, ni el perro. `private` es el equivalente a tu diario secreto con candado. Los otros niveles (`public`, `protected`, `package-private`) permiten acceso según el contexto. La propia clase (`mostrarTodo`) sí puede acceder a todo porque está dentro de la misma clase. Es como si tú mismo pudieras leer tu propio diario, pero los vecinos no.

Ejercicio 2: Completa los getters y setters

```
public class CuentaBancaria {
    private double saldo;

    public double getSaldo() {
        return saldo;
    }

    public void setSaldo(double saldo) {
        if (saldo >= 0) {
            this.saldo = saldo;
        } else {
            System.out.println("El saldo no puede ser negativo");
        }
    }

    public void ingresar(double cantidad) {
        if (cantidad > 0) {
            saldo += cantidad;
        } else {
            System.out.println("Cantidad inválida");
        }
    }

    public boolean retirar(double cantidad) {
        if (cantidad > 0 && cantidad <= saldo) {
            saldo -= cantidad;
            return true;
        }
        System.out.println("No se puede retirar esa cantidad");
        return false;
    }
}
```



Explicación: La encapsulación consiste en: atributos private, acceso controlado por métodos públicos. El getter solo devuelve el valor. El setter valida que el saldo no sea negativo. ingresar solo acepta cantidades positivas. retirar comprueba que haya saldo suficiente. Todo el control está en manos de la clase, no del usuario externo. Es como un cajero automático: tú no puedes abrir la máquina y coger billetes; solo puedes usar los

botones que la máquina te permite. La encapsulación es el “cajero automático” de tus objetos.

Ejercicio 3: ¿Qué imprime? Static vs instancia

```
public class Gato {  
    public static int totalGatos = 0;  
    public String nombre;  
  
    public Gato(String nombre) {  
        this.nombre = nombre;  
        totalGatos++;  
    }  
  
    public static void main(String[] args) {  
        Gato g1 = new Gato("Misi");  
        Gato g2 = new Gato("Garfield");  
        Gato g3 = new Gato("Tom");  
  
        System.out.println("Total: " + Gato.totalGatos);  
        System.out.println("Nombre: " + g2.nombre);  
    }  
}
```

Salida:

```
Total: 3  
Nombre: Garfield
```

💡 **Explicación:** `totalGatos` es `static`, lo que significa que es compartido por todos los objetos de la clase. Cada vez que se crea un `Gato`, el constructor incrementa `totalGatos`. Al crear 3 gatos, `totalGatos` es 3. En cambio, `nombre` es de instancia: cada gato tiene su propio nombre. `g2.nombre` es “Garfield” porque es el nombre del segundo gato. La diferencia es como el grupo de WhatsApp de la clase (static) vs los mensajes privados (instancia). Todos ven el mensaje del grupo, pero cada uno tiene sus propios DMs.

Ejercicio 4: Escribe la clase utilitaria

```

public class StringUtils {
    private StringUtils() {} // Constructor privado: no se puede instanciar

    public static boolean esVacio(String s) {
        return s == null || s.trim().isEmpty();
    }

    public static String invertir(String s) {
        if (s == null) return null;
        return new StringBuilder(s).reverse().toString();
    }

    public static void main(String[] args) {
        System.out.println("¿Vacío? " + StringUtils.esVacio(" ")); // true
        System.out.println("¿Vacío? " + StringUtils.esVacio("Hola")); // false
        System.out.println("Invertir: " + StringUtils.invertir("Hola")); // aloH
    }
}

```

💡 **Explicación:** Las clases utilitarias tienen constructor privado para que nadie pueda instanciarlas. Todos sus métodos son estáticos y se llaman con `NombreClase.metodo()`. Es como `Math`: no haces `new Math()`, usas `Math.random()` directamente. `esVacio` comprueba si un `String` es `null` o solo espacios. `invertir` usa `StringBuilder` que tiene un método `reverse()` incorporado. Podrías hacerlo a mano con un bucle, pero `StringBuilder` es más eficiente. La clase `StringUtils` es tu navaja suiza para `Strings`.

Ejercicio 5: Escribe la clase Config

```

public class Config {
    private Config() {} // No se puede instanciar

    public static final String NOMBRE_APP = "Gestión DAM";
    public static final String VERSION = "1.0.0";
    public static final int MAX_USUARIOS = 100;

    private static int contadorAccesos = 0;

    public static void incrementarAcceso() {
        contadorAccesos++;
    }

    public static int getContadorAccesos() {
        return contadorAccesos;
    }

    public static void main(String[] args) {
        System.out.println("App: " + Config.NOMBRE_APP);
        System.out.println("Versión: " + Config.VERSION);
        System.out.println("Max usuarios: " + Config.MAX_USUARIOS);

        Config.incrementarAcceso();
        Config.incrementarAcceso();
        Config.incrementarAcceso();
        System.out.println("Accesos: " + Config.getContadorAccesos());
    }
}

```

💡 **Explicación:** Las constantes `static final` son públicas e inmutables. Se escriben en MAYÚSCULAS por convención. El constructor privado evita instanciación. `contadorAccesos` es privado y estático: solo la clase puede modificarlo, pero existe a nivel de clase (no de objeto). Se usa como un contador global de accesos a la aplicación. Es como el marcador de visitas de una web: todos ven el número, pero solo el servidor puede incrementarlo.

Ejercicio 6: AceptaElReto 106 — Código de barras

```

import java.util.Scanner;

public class Problema106 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String codigo = sc.nextLine();
        while (!codigo.equals("0")) {
            int longitud = codigo.length();
            boolean esEan13 = longitud == 13;
            int sumaPar = 0, sumaImpar = 0;

            int limite = esEan13 ? 12 : 7;
            for (int i = 0; i < limite; i++) {
                int digito = codigo.charAt(i) - '0';
                if (i % 2 == 0) sumaImpar += digito;
                else sumaPar += digito;
            }

            int total = esEan13 ? sumaImpar * 3 + sumaPar : sumaImpar + sumaPar * 3;
            int digitoControl = (10 - total % 10) % 10;
            int ultimoDigito = codigo.charAt(longitud - 1) - '0';

            String pais = "";
            int prefijo = Integer.parseInt(codigo.substring(0, 3));
            if (longitud == 8) {
                pais = "EEUU";
            } else if (prefijo == 0 || prefijo == 1) {
                pais = "EEUU";
            } else if (prefijo >= 380 && prefijo <= 379) { /* improbable */
            } else if (prefijo >= 380 && prefijo <= 389) { /* no */
            } else if (prefijo >= 380 && prefijo <= 379) { /* error */
            } else if (prefijo >= 380 && prefijo < 400) {
                pais = "Bulgaria";
            } else { /* simplificación */
                pais = "Desconocido";
            }

            // Simplificación: solo mostramos validez
            if (digitoControl == ultimoDigito) {
                System.out.println("SI");
            } else {
                System.out.println("NO");
            }
        }
    }
}

```



```

    }
    codigo = sc.nextLine();
}
}
}

```

💡 **Explicación:** Esta es una versión simplificada del problema 106. El algoritmo del código de barras: para EAN-13, sumas los dígitos impares (x3) + pares; para EAN-8, al revés. Calculas el dígito de control con $(10 - \text{total} \% 10) \% 10$ y lo comparas con el último dígito. El código completo en AceptaElReto también debe identificar el país según el prefijo. La encapsulación aquí está en la lógica: cada cálculo está separado y es fácil de entender. Es un ejemplo de cómo validar datos de entrada con un algoritmo estándar.

Ejercicio 7: CodeWars — Convert boolean to string

```

public class YesOrNo {
    public static String boolToWord(boolean b) {
        return b ? "Yes" : "No";
    }
}

```

💡 **Explicación:** El ternario `? :` es perfecto aquí. Si `b` es `true`, devuelve “Yes”. Si no, “No”. Una línea. Elegante. También podrías hacerlo con `if-else`, pero el ternario queda más limpio. CodeWars premia la simplicidad. Es como cuando te preguntan “¿quieres café?” y respondes “Sí” o “No”. No necesitas un párrafo explicando tu relación con la cafeína. La clase es utilitaria (método estático), así que la llamas con `YesOrNo.boolToWord(true)`.

Boletí 5 – Inicial: Visibilitat, Encapsulació i Static

Sense solucions. `private` no vol dir que sigues tímit, vol dir que protegeixes les teues dades. I `static` no és electricitat estàtica, és memòria compartida.

Exercici 1: Troba l'error de visibilitat (la nevera)

Quines línies d'este codi donen error de compilació i per què?

```
public class Nevera {
    public String marca;
    protected int capacitatLitres;
    double temperatura;
    private String codiDesbloqueig;

    public void mostrarInfo() {
        System.out.println(marca);
        System.out.println(capacitatLitres);
        System.out.println(temperatura);
        System.out.println(codiDesbloqueig);
    }
}

public class Amic {
    public void obrirNevera() {
        Nevera n = new Nevera();
        System.out.println(n.marca);
        System.out.println(n.capacitatLitres);
        System.out.println(n.temperatura);
        System.out.println(n.codiDesbloqueig);
    }
}
```

Exercici 2: Completa els getters i setters de la classe Producte

Completa la classe Producte amb encapsulació real:

```
public class Producte {  
    private String nom;  
    private double preu;  
    private int stock;  
  
    public String getNom() { // què va aquí? }  
    public void setNom(String nom) { // què va aquí? }  
    public double getPreu() { // què va aquí? }  
    public void setPreu(double preu) { // què va aquí? }  
    public int getStock() { // què va aquí? }  
    public void setStock(int stock) { // què va aquí? }  
    public boolean vendre(int quantitat) { // què va aquí? }  
}
```

Exercici 3: Què imprimeix? Static vs instància II

Sense executar, digues què imprimeix. Pista: té truc amb el static.

```
public class Universitat {  
    public static String nomUniversitat = "UAX";  
    public String nomAlumne;  
  
    public Universitat(String nomAlumne) {  
        this.nomAlumne = nomAlumne;  
    }  
  
    public static void main(String[] args) {  
        Universitat u1 = new Universitat("Anna");  
        Universitat u2 = new Universitat("Lluís");  
        u1.mostrar();  
        u2.mostrar();  
        Universitat.nomUniversitat = "UPM";  
        u1.mostrar();  
        u2.mostrar();  
    }  
}
```

Exercici 4: Escriu la classe utilitària MathUtils

Crea una classe `MathUtils` amb constructor privat i mètodes estàtics:

- `int maxim(int a, int b)` — torna el major de dos nombres
 - `int minim(int a, int b)` — torna el menor de dos nombres
 - `boolean esParell(int n)` — torna true si n és parell
-

Exercici 5: Escriu la classe AppConfig

Crea una classe `AppConfig` amb constants `static final`:

- `NOM_APP = "MiAplicacio"`
- `VERSIO = "2.0.0"`
- `MAX_INTENTS = 3`

A més, un atribut `private static int usuarisConnectats` amb getter i mètodes per a incrementar/decrementar. No es pot instanciar la classe.

Exercici 6: AceptaElReto — 117 Trobar el major

Resol el problema 117 — Trobar el major en [AceptaElReto.com](https://www.acepta-el-reto.com/).

Llig nombres fins que s'introduïska un nombre igual a l'anterior. En eixe moment, imprimeix quants nombres es van llegir en total.

Exercici 7: CodeWars — Opposite number

Resol la kata “Opposite number” (8 kyu) en CodeWars.

Completa el mètode `opposite` que rep un nombre i torna el seu oposat (negatiu si és positiu, positiu si és negatiu).

```
public class OppositeNumber {  
    public static int opposite(int number) {  
        // el teu codi ací  
    }  
}
```

Boletín 5 – Resuelto: Visibilidad, Encapsulación y Static

De ★ a ★★☆☆. De “hago getters y setters” a “construyo una API con clases utilitarias”. La encapsulación te espera.

★ Ejercicio 1: Clase Empleado con validación

Empleado con nombre, salarioBase (double) y departamento privados. El salario no puede ser negativo. El setter lanza `IllegalArgumentException`. Método `calcularSalarioAnual()`.

💡 **Explicación:** El setter `setSalarioBase` valida que el salario no sea negativo. Si lo es, lanza una `IllegalArgumentException`. Esto es mejor que simplemente ignorar el valor inválido: el programa se detiene y te dice qué pasa. El constructor usa `setSalarioBase` para aprovechar la validación. `calcularSalarioAnual()` multiplica por 12 (suponiendo 12 pagas). Este es el patrón estándar de encapsulación en Java: atributos privados, getters/setters públicos con validación. Así como en la vida real no puedes tener un salario negativo (aunque a veces lo parezca), en tu código tampoco.

★ Ejercicio 2: Círculo encapsulado

Círculo con radio privado. Setter valida que el radio sea positivo. `getArea()` y `getPerimetro()`. ¿Qué pasa si desde otra clase accedes a radio?

Desde otra clase: `circulo.radio` daría ERROR de compilación. Solo se puede acceder con `circulo.getRadio()`.

💡 **Explicación:** Si intentas `circulo.radio` desde otra clase, el compilador dice: “El campo `Circulo.radio` es invisible”. `private` es eso: invisible para todos excepto para la propia clase. Ni siquiera puedes verlo, y mucho menos modificarlo. Para obtener el radio, usas `getRadio()`. Para cambiarlo, `setRadio(...)`. Y el setter valida que sea positivo. Es como el botón de “no pasar” de una puerta: solo el dueño (la clase) puede decidir quién entra.

★★ Ejercicio 3: JavaBean Alumno

Clase Alumno JavaBean: nombre, edad, curso, notaMedia (double), matriculado (boolean). Constructor sin args. Nota media entre 0 y 10.

💡 **Explicación:** JavaBean es una convención: clase pública, constructor sin argumentos, atributos privados, getters y setters públicos. Para boolean, el getter se llama isMatriculado() en lugar de getMatriculado(). El constructor sin args inicializa todo a valores por defecto. El setter de notaMedia valida que esté entre 0 y 10. Los JavaBeans se usan en frameworks como Spring, Hibernate, JSF... Son el estándar de la industria. Aprende el patrón y lo usarás en el 90% de tus clases.

★ ★ Ejercicio 4: Contador de objetos con static

Clase Usuario con contador estático de objetos creados, id autoincremental, constante DOMINIO_EMAIL, y método generarEmail().

💡 **Explicación:** contador (static) se incrementa en cada constructor. Es compartido por todos los objetos. id es de instancia: cada usuario recibe el valor actual del contador en el momento de su creación. DOMINIO_EMAIL es una constante estática. generarEmail() combina el nombre (en minúsculas) con el dominio. Es como cuando te creas un email en el instituto: ana@dam.com. El contador estático te dice cuántos usuarios se han creado en total, independientemente de qué objetos sigan vivos.

★ ★ Ejercicio 5: Conversor de unidades (estáticos)

Clase Conversor con constantes y métodos estáticos para convertir entre km y millas, Celsius y Fahrenheit, kg y libras.

💡 **Explicación:** Constructor privado, solo métodos estáticos. Cada método usa una constante o una fórmula. Las constantes son static final y se escriben en mayúsculas. Es como tener una calculadora de conversiones siempre disponible, sin necesidad de crear objetos. La fórmula Celsius a Fahrenheit es $^{\circ}\text{C} \times 9/5 + 32$. Fahrenheit a Celsius es $(^{\circ}\text{F} - 32) \times 5/9$. Las conversiones son todas lineales: multiplicas por un factor. Simple, útil, y demuestra el patrón de clase utilitaria.



Ejercicio 6: AceptaElReto 364 —

Spiderman

Resuelve el problema **364 — Spiderman** en AceptaElReto.

Spiderman lanza telarañas para salvar ciudadanos. En cada lanzamiento, la telaraña recorre una distancia. El problema pide algo con espías y distancias. (Nota: léelo en la web, es un problema de espías enemigos y distancias).

 **Explicación:** El problema 364 (SpiderMan / Espías) te da una serie de alturas de edificios. Debes contar cuántos edificios “ven” el lado derecho sin ser bloqueados por uno más alto. La solución recorre de derecha a izquierda, manteniendo la altura máxima vista. Si el edificio actual es más alto que el máximo visto, se cuenta. Es un problema clásico de “visibilidad” que encaja perfectamente con el tema de visibilidad de esta unidad. Los edificios más altos bloquean la vista de los más bajos, como los modificadores `private` bloquean el acceso desde fuera.



Ejercicio 7: AceptaElReto 462 — Tres dedos

Resuelve el problema **462 — Tres dedos** en AceptaElReto.

Dado un número en base 10, conviértelo a base -2 (base negativa). Los dígitos resultantes serán 0 o 1.

 **Explicación:** Las bases negativas son un concepto matemático curioso. En base -2, los números se representan con dígitos 0 y 1, pero con pesos que alternan signo: 1, -2, 4, -8, 16... El algoritmo es similar a la conversión a binario, pero con un ajuste: si el resto es negativo, le sumamos 2 y aumentamos el cociente en 1. Ejemplo: 6 en base -2 es 11010 ($1 \cdot 16 + 1 \cdot -8 + 0 \cdot 4 + 1 \cdot -2 + 0 \cdot 1 = 16 - 8 - 2 = 6$). Es un problema de concurso (ProgramaMe) que requiere pensar fuera de la caja. Literalmente, fuera de la base positiva.



Ejercicio 8: Simulación estática

Clase `Simulacion` con método `main` que lanza dos dados 1000 veces usando `Math.random()`. Usa una clase `Dado` con método estático `lanzar()` (1-6). Cuenta cuántas veces sale cada suma (2-12).

Salida esperada (aproximada):

Resultados de 1000 lanzamientos:

Suma | Frecuencia

-----|-----

2 | 28

3 | 56

4 | 83

5 | 111

6 | 139

7 | 167

8 | 139

9 | 111

10 | 83

11 | 56

12 | 28

💡 **Explicación:** La clase Dado es utilitaria: constructor privado y método lanzar() estático. No necesitas crear dados individuales porque todos son iguales. La simulación usa un array de 13 posiciones (0-12) donde sumas[7] cuenta cuántas veces ha salido suma 7. El 7 es el más probable porque hay más combinaciones de dados que suman 7 (1+6, 2+5, 3+4, 4+3, 5+2, 6+1) que cualquier otra suma. La ley de los grandes números hace que la distribución se acerque a una campana (distribución normal). Es la misma razón por la que en el casino el 7 es la apuesta más popular en los dados. Pero no apuestes, que la casa siempre gana. Bueno, tú ganas con este código.



Referencias

Plataforma	Problema	Dificultad
AceptaElReto	106 — Código de barras	Medio
AceptaElReto	364 — Spiderman	Medio
AceptaElReto	462 — Tres dedos	Difícil
CodeWars	Convert boolean to string (8 kyu)	Principiante
CodeWars	Return the day (8 kyu)	Principiante

Boletí 5 – Intermedi: Visibilitat, Encapsulació i Static

Exercicis de dificultat progressiva. Els ★ són per a escalfar, ★★ per a pensar, ★★★ per a concursar. L'encapsulació no és un encanteri de Harry Potter, és sentit comú. Però per a estos exercicis, igual necessites un encanteri o dos.

★ Exercici 1: Classe CompteEstalvis encapsulada

Crea una classe `CompteEstalvis` amb:

- Atribut privat `double saldo`
 - Atribut privat `double interesAnual` (percentatge, ex: 2.5 significa 2.5%)
 - Constructor que reba el saldo inicial i l'interés anual. Si el saldo és negatiu, s'establix a 0.
 - Getter per a saldo
 - Mètode void `aplicarInteres()` que augmenta el saldo segons l'interés anual
 - Mètode boolean `retirar(double quantitat)` que només permeta retirar si hi ha saldo suficient
-

★ Exercici 2: Classe Persona amb validació estricta

Crea una classe `Persona` amb atributs privats `String nom` i `int edat`. El constructor i els setters han de validar que:

- `nom` no siga null ni estiga buit (ni només espais)
 - `edat` estiga entre 0 i 120 (inclòs)
-

★★ Exercici 3: Classe TicketCompra amb ID autoincremental

Crea una classe `TicketCompra` que modele un ticket de supermercat:

- Atribut estàtic `private static int contadorTickets = 0`

- Atribut d'instància `private int id` (autoincremental)
 - Atributs `private String[] productes` i `private double[] preus`
 - Mètodes per a calcular total, IVA i `toString()`
-

☆☆ Exercici 4: Classe Biblioteca amb comptador estàtic

Crea una classe `Biblioteca` que gestione préstecs de llibres:

- Comptador estàtic de llibres prestats
 - Mètodes per a prestar i devolre llibres
 - Getter estàtic del total de préstecs globals
-

☆☆☆ Exercici 5: Classe `CalculadoraEstadística` utilitària

Crea una classe `CalculadoraEstadística` que siga una classe utilitària (constructor privat, només mètodes estàtics):

- `media(double[] dades)`
 - `mediana(double[] dades)`
 - `moda(double[] dades)`
 - `desviacioTipica(double[] dades)`
-

☆☆☆ Exercici 6: CodeWars — Find the odd int

Resol la kata “Find the odd int” (6 kyu) en CodeWars.

Crea una classe utilitària `FindOdd` amb un mètode estàtic `findIt(int[] arr)` que rep un array d'enters i torna l'enter que apareix un nombre impar de vegades.

☆☆☆ Exercici 7: AceptaElReto — 120 Nombre de parells i senars

Resol el problema 120 — Nombre de parells i senars en [AceptaElReto.com](https://www.acepta-el-reto.com/).

Donat un nombre enter positiu, compta quants dels seus dígit són parells i quants són senars.



Referències

Plataforma	Problema	Dificultat
CodeWars	Find the odd int	6 kyu
CodeWars	Regex validate PIN code	7 kyu
AceptaElReto	120 — Nombre de parells i senars	Fàcil
AceptaElReto	157 — ¿Son iguales?	Mitjà

Butlletí 07 — Extres: Visibilitat, Encapsulació i Static

★ **Extres — Converteix-te en un programador de molt alt nivell** Aquests exercicis són completament opcionals, però si aspirem a ser un programador o programadora d'elit, ací tens el teu camp d'entrenament. CodeWars i AceptaElReto són plataformes on els millors programadors del món es repte cada dia. Cada problema que resolguis et farà més fort, més ràpid i més creatiu. No els subestimis: són la diferència entre saber programar i ser un vertader professional.

CodeWars:

- [Regex validate PIN code](#) (7 kyu)
Pista: Utilitza l'expressió regular `^\d{4}(\d{2})?$` o `^[0-9]{4}([0-9]{2})?$` per validar PINs de 4 o 6 dígit.
- [Sum of two lowest positive integers](#) (7 kyu)
Pista: Ordena el array amb `Arrays.sort()` i suma els dos primers elements.
- [Growth of a Population](#) (6 kyu)
Pista: Bucle while: any a any, aplica el creixement $p = p + p * (\text{percent} / 100) + \text{aug}$ fins a superar l'objectiu.

AceptaElReto:

- [120 - Nombre de parells i senars](#) (★)
Pista: Itera pels dígit del número i compta quants són parells i quants senars.
- [157 - Són iguals?](#) (★ ★)
Pista: Dos números són "iguals" segons el problema si en intercanviar un parell de dígit coincideixen. Compara dígit a dígit.

Boletín 6 – Inicial Resuelto: Herencia y Polimorfismo

Aquí con soluciones y explicaciones. Sin trampas.

Ejercicio 1: ¿Qué imprime? — La llamada a la familia

```
class Abuelo {
    void saludar() { System.out.println("Soy el abuelo"); }
}
class Padre extends Abuelo {
    void saludar() { System.out.println("Soy el padre"); }
}
class Hijo extends Padre {
    void saludar() { System.out.println("Soy el hijo"); }
}

public class Test {
    public static void main(String[] args) {
        Hijo h = new Hijo();
        h.saludar();
    }
}
```

Solución: Imprime "Soy el hijo".

💡 **Explicación:** Java busca el método `saludar()` empezando por la clase más específica (Hijo). La encuentra ahí mismo y la ejecuta. Nunca sube a Padre ni a Abuelo. Esto se llama *dynamic dispatch*: el método se resuelve en tiempo de ejecución según el tipo real del objeto, no según el tipo de la referencia. Y como `h` es un Hijo de verdad (de los que no llaman a los abuelos), ejecuta su propia versión.

Ejercicio 2: Encuentra el error — extends mal usado

```

public class Vehiculo {
    private String matricula;
    public Vehiculo(String matricula) {
        this.matricula = matricula;
    }
}

public class Coche extends Vehiculo {
    private int numPuertas;
    public Coche(int numPuertas) {
        this.numPuertas = numPuertas;
    }
}

```

Solución: El error es que Coche no llama al constructor de Vehiculo. Cuando una clase hija no pone `super()` explícitamente, Java intenta llamar a `super()` sin parámetros. Pero Vehiculo no tiene constructor sin parámetros — solo tiene `Vehiculo(String)`. Por tanto: **error de compilación**.

```

public class Coche extends Vehiculo {
    private int numPuertas;
    public Coche(String matricula, int numPuertas) {
        super(matricula); // ¡Aquí está la clave!
        this.numPuertas = numPuertas;
    }
}

```

💡 **Explicación:** Piensa en `super()` como llamar a papá para que configure su parte antes de que tú configures la tuya. Si papá necesita una matrícula para construirse, tú tienes que pasársela. No puedes escaquearte. Es como intentar construir una casa sin cimientos: el constructor del padre es la base.

Ejercicio 3: Completa el código — the instanceof

```
class Animal {
    public void hacerSonido() {
        System.out.println("...");
    }
}

class Perro extends Animal {
    public void ladrar() {
        System.out.println("¡Guau!");
    }
}

// En main:
Animal a = new Perro();
if (a instanceof Perro) {
    Perro p = (Perro) a;
    p.ladrar();
}
```

💡 **Explicación:** instanceof pregunta: “¿eres realmente un Perro o solo te disfrazas de Animal?”. Como el objeto real es un new Perro(), la respuesta es true. Luego hacemos downcasting con (Perro) a para tener una referencia de tipo Perro y poder llamar a ladrar(). Sin el casting, el compilador no te deja: “Oye, Animal no tiene ladrar(), no me engañas”.

Ejercicio 4: Escribe este programa — la jerarquía de animales


```

class SerVivo {
    protected int edad;

    public SerVivo(int edad) {
        this.edad = edad;
    }
}

class Animal extends SerVivo {
    protected String nombre;

    public Animal(String nombre, int edad) {
        super(edad);
        this.nombre = nombre;
    }
}

class Perro extends Animal {
    public Perro(String nombre, int edad) {
        super(nombre, edad);
    }

    public void ladrar() {
        System.out.println(nombre + " dice: ¡Guau! Edad: " + edad);
    }
}

public class Test {
    public static void main(String[] args) {
        Perro p = new Perro("Firulais", 3);
        p.ladrar();
    }
}

```

Salida: Firulais dice: ¡Guau! Edad: 3



Explicación: La herencia en cadena: Perro → Animal → SerVivo. Cada constructor llama a su padre con `super()`. Al final, Perro tiene acceso a nombre (de Animal) y edad (de SerVivo) porque están declarados como `protected`. Si fueran `private`, ni Perro los vería. Es como la herencia familiar: lo que es privado en casa de los abuelos, no lo ven ni los nietos.

Ejercicio 5: ¿Qué imprime? — Referencias polimórficas

```
class A {  
    void foo() { System.out.println("A"); }  
}  
class B extends A {  
    void foo() { System.out.println("B"); }  
}  
class C extends B { }  
  
public class Test {  
    public static void main(String[] args) {  
        A ref1 = new B();  
        A ref2 = new C();  
        B ref3 = new C();  
  
        ref1.foo(); // "B"  
        ref2.foo(); // "C"  
        ref3.foo(); // "C"  
    }  
}
```

💡 **Explicación:** El tipo de la REFERENCIA (A, A, B) no importa. Lo que importa es el tipo REAL del objeto (B, C, C). Java siempre ejecuta el método más específico del objeto real. Es como si llevaras una chaqueta de tu padre: por fuera pareces tu padre (la referencia), pero por dentro eres tú (el objeto). Cuando hablas, se oye tu voz, no la de tu padre. Dynamic binding en todo su esplendor.

Ejercicio 6: Encuentra el error — polimorfismo y casting

```
Animal a = new Gato();  
Perro p = (Perro) a; // ✨ ClassCastException en runtime  
p.ladRAR();
```

Solución: El error es que estás intentando disfrazar un Gato de Perro. Java no se deja engañar: en tiempo de ejecución, el objeto real es un Gato, no un Perro, y lanza `ClassCastException`.

Para arreglarlo:

```
Animal a = new Gato();  
if (a instanceof Perro) {  
    Perro p = (Perro) a;  
    p.ladRAR();  
} else {  
    System.out.println("No es un perro, no puedo hacerlo ladRAR.");  
}
```

💡 **Explicación:** instanceof es tu red de seguridad. Siempre úsalo antes de hacer downcasting. Es como mirar por la mirilla antes de abrir la puerta: si es un repartidor de pizzas, abre; si es un león, mejor no. En este caso, es un Gato, así que el instanceof da false y te ahorras el ClassCastException.

Ejercicio 7: Escribe este programa — el zoo polimórfico

```

import java.util.ArrayList;

abstract class Animal {
    public abstract void hacerSonido();
}

class Perro extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("¡Guau!");
    }
}

class Gato extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("¡Miau!");
    }
}

public class Zoo {
    public static void main(String[] args) {
        ArrayList<Animal> animales = new ArrayList<>();
        animales.add(new Perro());
        animales.add(new Gato());

        for (Animal a : animales) {
            a.hacerSonido();
        }
    }
}

```

Salida:

```

¡Guau!
¡Miau!

```



Explicación: Aquí pasa la magia del polimorfismo. Declaramos `ArrayList<Animal>`, que acepta cualquier subclase de `Animal`. Cuando recorremos la lista con un for-each, cada `Animal a` apunta a un objeto diferente (primero `Perro`, luego `Gato`). Pero al llamar a `hacerSonido()`, Java ejecuta la versión del objeto real, no la de `Animal`. Un solo bucle,

múltiples comportamientos. Esto es polimorfismo en acción. Sin él, tendrías que tener listas separadas para cada tipo de animal, como en la edad de piedra de la programación.

Referencias para seguir practicando

- CodeWars: [Is this a triangle?](#) (7 kyu)
- CodeWars: [Basic subclasses - Adam and Eve](#) (8 kyu)
- AceptaElReto.com: [364 - Spiderman](#)
- AceptaElReto.com: [458 - El espejo](#)

Boletí 6 – Inicial: Herència i Polimorfisme

Sense solucions. L'herència en Java és com la de veritat: de vegades et portes bé amb les subclasses, de vegades vols renegar de tot.

Exercici 1: Què imprimeix? — La família musical

```
class Music {
    void tocar() { System.out.println("El músic toca un instrument"); }
}
class Guitarista extends Music {
    void tocar() { System.out.println("El guitarrista toca la guitarra"); }
}
class Baixista extends Guitarista {
    void tocar() { System.out.println("El baixista toca el baix"); }
}

public class Banda {
    public static void main(String[] args) {
        Baixista b = new Baixista();
        b.tocar();
    }
}
```

Què imprimeix? Per què?

Exercici 2: Troba l'error — extends mal usat II

```
public class Animal {
    private String especie;
    public Animal(String especie) {
        this.especie = especie;
    }
}

public class Gos extends Animal {
    private String raça;
    public Gos(String raça) {
        this.raça = raça;
    }
}
```

Este codi **no compila**. Per què?

Exercici 3: Completa el codi — instanceof amb downcasting

Completa el següent programa perquè funcione correctament:

```
Vehicle v = new Cotxe();
if (v _____ Cotxe) {
    _____ c = (_____) v;
    c.conduir();
}
```

Declara les classes Vehicle (amb moure()) i Cotxe (amb conduir()).

Exercici 4: Escriu este programa — l'herència de vehicles

Crea una jerarquia de 3 nivells usant extends:

- Vehicle (atribut: String marca)
- Cotxe (atribut: int numPortes)
- Esportiu (atribut: int velocitatMaxima)

En `main()`, crea un `Esportiu` de marca “Ferrari”, 2 portes i 340 km/h.

Exercici 5: Què imprimeix? — Polimorfisme amb referències

```
class X {
    void missatge() { System.out.println("X"); }
}
class Y extends X {
    void missatge() { System.out.println("Y"); }
}
class Z extends Y { }

public class Test {
    public static void main(String[] args) {
        X ref1 = new Y();
        X ref2 = new Z();
        Y ref3 = new Z();
        ref1.missatge();
        ref2.missatge();
        ref3.missatge();
    }
}
```

Exercici 6: Troba l'error — `ClassCastException`

```
Animal a = new Gos();
Gat g = (Gat) a;
g.maullar();
```

Què ocorre en temps d'execució? Com ho arreglaries?

Exercici 7: Escriu este programa — la granja polimòrfica

Crea una classe `Animal` amb mètode `ferSo()`. Crea `Vaca`, `Ovella` i `Gallina` que el sobreescriguen. En `main()`, crea un `ArrayList<Animal>` i recorre'l polimòrficament.

Referències per a seguir practicant

- CodeWars: [Is this a triangle?](#) (7 kyu)
- CodeWars: [Basic subclasses - Adam and Eve](#) (8 kyu)
- AceptaElReto.com: [120 - Número de pares y nones](#)
- AceptaElReto.com: [154 - L'ascensor](#)

Boletín 6 – Resuelto: Herencia y Polimorfismo

Dificultad progresiva. De ★ a ★★★★★.

★ Ejercicio 1: La orquesta polimórfica

Crea una clase `Instrumento` con método `tocar()`. Crea 3 subclases: `Guitarra`, `Piano`, `Bateria` que sobrescriban `tocar()`. En `main()`, crea un array de `Instrumento` con una instancia de cada uno y recórrelo llamando a `tocar()`.

★ Ejercicio 2: Herencia de constructores — la cadena alimenticia

Crea: `Animal` (constructor con `String nombre`), `Mamifero` (constructor con `String nombre`, `boolean tienePelo`), `Perro` (constructor con `String nombre`, `boolean tienePelo`, `String raza`). Cada constructor debe llamar al de su padre. Demuestra que funciona.

★ Ejercicio 3: El problema del jarrón (Fragile Base Class)

Crea `Base` con métodos `a()` y `b()` donde `b()` llama internamente a `a()`. Crea `Derivada` que sobrescribe `a()`. Crea una instancia de `Derivada` y llama a `b()`. ¿Qué ocurre?

★ ★ Ejercicio 4: Batalla de personajes

Crea `Personaje` (abstracta) con `nombre`, `vida`, `atacar(Personaje p)` y `recibirDano(int dmg)`. Crea `Guerrero` (daño fijo 20), `Mago` (daño 30 pero solo cada 2 turnos) y `Arquero` (daño 15 pero siempre acierta). Simula una batalla.

★ ★ Ejercicio 5: La calculadora de figuras

Crea `Figura` con método abstracto `calcularArea()`. Implementa `Circulo`, `Rectangulo` y `Triangulo`. En `main()`, crea un `ArrayList<Figura>`, añade varias figuras y calcula el área total.

☆☆ Ejercicio 6: Downcasting seguro

Jerarquía `Empleado` → `Programador`, `Diseñador`. `Programador` tiene `escribirCodigo()`, `Diseñador` tiene `diseñar()`. Crea un `ArrayList<Empleado>` con varios empleados y recórrelo usando `instanceof` para llamar a los métodos específicos.

☆☆☆ Ejercicio 7 (ProgramaMe): El simulador de ecosistema

Diseña un simulador de ecosistema con la siguiente jerarquía:

```
SerVivo (abstracta)
├─ Animal (abstracta)
│   ├── Herbivoro
│   └── Carnivoro
└─ Planta
```

Cada `SerVivo` tiene `int energia`. Los `Animal` pueden `mover()`, `comer(SerVivo objetivo)`. Los `Herbivoro` solo comen `Planta`. Los `Carnivoro` solo comen `Animal`. Cada `Planta` hace `fotosintesis()` que le da +10 de energía.

Simula un ecosistema con 3 plantas, 2 herbívoros y 1 carnívoro durante 10 turnos. Cada turno: las plantas hacen fotosíntesis, los herbívoros comen plantas si pueden, el carnívoro come herbívoros si puede. Si un ser vivo llega a 0 de energía, muere.

Al final, muestra quién sobrevivió.

☆☆☆ Ejercicio 8 (ProgramaMe): Vehículos con combustible

Crea una jerarquía de vehículos:

- `Vehiculo (abstracta)`: `String matricula`, `int combustible`, `abstract void mover()`

- Coche: gasta 5 de combustible por movimiento
- Moto: gasta 3 de combustible por movimiento
- Camion: gasta 10 de combustible por movimiento, pero puede llevar int carga

Cada vehículo tiene un método `mover()` que reduce el combustible. Si no hay suficiente, imprime “Sin combustible”.

En `main()`, crea un `ArrayList<Vehiculo>` con varios vehículos. Cada vehículo se mueve repetidamente hasta que se queda sin combustible. Lleva la cuenta de cuántos movimientos hizo cada uno.

Referencias para seguir practicando

- CodeWars: [Thinkful - Logic Drills: Traffic light](#) (7 kyu)
- CodeWars: [Fun with ES6 Classes #1 - People](#) (7 kyu)
- CodeWars: [Fight Club](#) (7 kyu)
- AceptaElReto.com: [364 - Spiderman](#)
- AceptaElReto.com: [462 - Tres dedos](#)
- AceptaElReto.com: [458 - El espejo](#)

Boletí 6 – Intermedi: Herència, Polimorfisme i Interfícies

Exercicis de dificultat progressiva. Els ★ són per a escalfar, ★★ per a pensar, ★★★ per a concursar. L'herència és com la família: de vegades heretes coses bones, de vegades et toca la col·lecció de segells del teu tio avi. Però amb interfícies, almenys tries què implementar.

★ Exercici 1: Interfície FiguraGeometrica

Crea una interfície `FiguraGeometrica` amb dos mètodes:

- `double calcularArea()`
- `double calcularPerimetre()`

Implementa-la en `Cercle`, `Rectangle` i `TriangleRectangle`.

★ Exercici 2: Jerarquia d'Empleats

Crea una classe base `Empleat` i dos subclasses `Gerent` i `Venedor` que sobreescriguen `calcularSalari()`.

★★ Exercici 3: Sistema de pagaments amb interfície

Crea una interfície `Pagable` amb `boolean procesarPagament(double quantitat)`. Implementa-la en `TarjetaCredito`, `PayPal` i `TransferenciaBancaria`.

★★ Exercici 4: Interfícies múltiples: Volador i Nedador

Crea les interfícies `Volador` i `Nedador`. Implementa-les en `Ànec` (ambdues), `Avió` (sols `Volador`) i `Peix` (sols `Nedador`).



Exercici 5: Sistema de notificaciones polimòrfic

Crea una interfície `Notificable` amb `void enviar(String missatge)` i `String getEstat()`. Implementa-la en `EmailNotificacio`, `SMSNotificacio` i `PushNotificacio`.



Exercici 6: CodeWars — Is this a triangle?

Resol la kata “Is this a triangle?” (7 kyu) en CodeWars.

Implementa una classe `TriangleValidator` amb un mètode estàtic `isTriangle(int a, int b, int c)`.



Exercici 7: AceptaElReto — 154 L'ascensor

Resol el problema 154 — L'ascensor en [AceptaElReto.com](https://www.acepta-el-reto.com/).

Modela un ascensor que recorre diverses plantes i calcula quantes vegades canvia de direcció.



Referències

Plataforma	Problema	Dificultat
CodeWars	Is this a triangle?	7 kyu
CodeWars	Thinkful - Logic Drills: Traffic light	7 kyu
AceptaElReto	154 — L'ascensor	Mitjà
AceptaElReto	369 — Navegació en Google Maps	Difícil

Butlletí 08 — Extres: Herència, Polimorfisme i Interfícies

★ Extres — Converteix-te en un programador de molt alt nivell

Aquests exercicis són completament opcionals, però si aspirem a ser un programador o programadora d'elit, ací tens el teu camp d'entrenament. CodeWars i AceptaElReto són plataformes on els millors programadors del món es repte cada dia. Cada problema que resolguis et farà més fort, més ràpid i més creatiu. No els subestimis: són la diferència entre saber programar i ser un vertader professional.

CodeWars:

- [Convert string to camel case](#) (6 kyu)

Pista: Divideix el string per guions o guions baixos, capitalitza la primera lletra de cada paraula excepte la primera.

- [Counting Duplicates](#) (6 kyu)

Pista: Converteix a minúscules, compta freqüències amb un `Map<Character, Long>` i filtra les que apareixen més d'una vegada.

- [Human Readable Time](#) (5 kyu)

Pista: Fes servir divisió entera i mòdul: `hores = segons / 3600`, `minuts = (segons % 3600) / 60`, `segons = segons % 60`. Formata amb `String.format("%02d:%02d:%02d")`.

AceptaElReto:

- [154 - L'ascensor](#) (★ ★)

Pista: Simula planta per planta; compta només les vegades que canvia la direcció de l'ascensor.

- [369 - Navegació en Google Maps](#) (★ ★ ★)

Pista: Modela les interseccions com a nodes d'un graf i aplica Dijkstra amb cua de prioritat.

Boletín 8 – Inicial Resuelto: Arrays y Colecciones

Con soluciones. A aprender.

Ejercicio 1: ¿Qué imprime? — Array básico

```
int[] nums = {1, 2, 3, 4, 5};
for (int i = 0; i < nums.length; i++) {
    nums[i] = nums[i] * 2;
}
System.out.println(nums[3]);
```

Solución: Imprime 8.

💡 **Explicación:** El array original es {1, 2, 3, 4, 5}. El bucle multiplica cada elemento por 2: {2, 4, 6, 8, 10}. `nums[3]` es el cuarto elemento (recuerda: los índices empiezan en 0). Así que `nums[3] = 8`.

Ejercicio 2: Encuentra el error — `ArrayIndexOutOfBoundsException`

```
String[] nombres = new String[3];
nombres[0] = "Ana";
nombres[1] = "Bob";
nombres[2] = "Carlos";
nombres[3] = "Diana";
```

Solución: La última línea lanza `ArrayIndexOutOfBoundsException`. El array tiene tamaño 3, los índices válidos son 0, 1 y 2. `nombres[3]` está fuera de los límites.

💡 **Explicación:** Los arrays en Java tienen tamaño fijo. Si declaras `new String[3]`, las plazas son 0, 1 y 2. Intentar acceder a la 3 es como intentar aparcar en una plaza que no existe: el parking solo tiene 3 plazas (índices 0, 1, 2) y tú buscas la 3. Crash asegurado.

Ejercicio 3: Completa el código — for-each

```
int[] numeros = {4, 7, 2, 9, 5};
int suma = 0;

for (int n : numeros) {
    suma += n;
}

System.out.println("Suma: " + suma);
```

Solución: `int n : numeros` y `suma += n`. La suma total es 27.

💡 **Explicación:** El for-each (o “enhanced for”) recorre cada elemento del array sin necesidad de índice. `int n` toma el valor de cada elemento en cada iteración. `suma += n` es equivalente a `suma = suma + n`. Es más limpio que el for tradicional cuando solo necesitas leer y no modificar.

Ejercicio 4: Escribe este programa — Array de 10 enteros al revés

```
public class ArrayInverso {
    public static void main(String[] args) {
        int[] numeros = new int[10];
        for (int i = 0; i < numeros.length; i++) {
            numeros[i] = i + 1;
        }
        for (int i = numeros.length - 1; i >= 0; i--) {
            System.out.print(numeros[i] + " ");
        }
    }
}
```

Salida: 10 9 8 7 6 5 4 3 2 1

💡 **Explicación:** Dos bucles: uno para rellenar (del 1 al 10) y otro para imprimir al revés (del índice 9 al 0). El truco está en `numeros.length - 1` como índice inicial (último elemento) y `i >= 0` como condición (incluir el primero). Es como leer una lista del final al principio.

Ejercicio 5: ¿Qué imprime? — ArrayList misterioso

```
import java.util.ArrayList;

public class Test {
    public static void main(String[] args) {
        ArrayList<Integer> lista = new ArrayList<>();
        lista.add(10);
        lista.add(20);
        lista.add(30);
        lista.add(1, 15);          // inserta 15 en índice 1
        lista.remove(Integer.valueOf(20)); // borra el objeto 20

        for (Integer n : lista) {
            System.out.print(n + " ");
        }
    }
}
```

Solución: Imprime 10 15 30.

💡 **Explicación:** Paso a paso: add(10) → [10], add(20) → [10, 20], add(30) → [10, 20, 30], add(1, 15) → [10, **15**, 20, 30] (inserta, no reemplaza), remove(Integer.valueOf(20)) → [10, 15, 30] (borra la primera ocurrencia del valor 20). Fíjate que remove(1) (con int) borraría por índice, pero remove(Integer.valueOf(1)) (con Integer) borra por valor.

Ejercicio 6: Encuentra el error — colección sin genérico

```
ArrayList lista = new ArrayList();
lista.add("Hola");
lista.add(42);
lista.add(3.14);

String texto = (String) lista.get(1); // 🌟 ClassCastException
```

Solución: lista.get(1) devuelve el Integer 42 (por el autoboxing), pero intentas castearlo a String. Como un Integer no es un String, salta ClassCastException en tiempo de ejecución.

💡 **Explicación:** Sin genéricos, ArrayList es una caja de caos donde todo es Object. El compilador no te avisa. Te enteras cuando el programa ya está ejecutándose y explota. Con genéricos: ArrayList<String> solo acepta Strings, y el error saltaría en compilación. Los genéricos convierten errores de runtime en errores de compilación, que es donde queremos que estén.

Ejercicio 7: Escribe este programa — HashSet sin duplicados

```
import java.util.HashSet;
import java.util.Scanner;

public class SinDuplicados {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        HashSet<String> palabras = new HashSet<>();

        System.out.println("Introduce 5 palabras:");
        for (int i = 0; i < 5; i++) {
            System.out.print("Palabra " + (i + 1) + ": ");
            String p = sc.nextLine();
            palabras.add(p);
        }

        System.out.println("Palabras distintas: " + palabras.size());
        System.out.println("Contenido: " + palabras);
        sc.close();
    }
}
```

💡 **Explicación:** HashSet no permite duplicados. Si el usuario introduce “hola” dos veces, la segunda llamada a add(“hola”) simplemente no hace nada y devuelve false. Al final, size() te dice cuántas palabras distintas hay. Es el mecanismo más sencillo para eliminar duplicados en Java: deja que HashSet haga el trabajo sucio.

Referencias para seguir practicando

- CodeWars: [Convert number to reversed array of digits](#) (8 kyu)
- CodeWars: [Find the smallest integer in the array](#) (7 kyu)
- AceptaElReto.com: [100 - Kaprekar](#)
- AceptaElReto.com: [152 - Suma pares e impares](#)

Boletín 8 – Inicial: Arrays i Col·leccions

Sense solucions. A programar.

Ejercicio 1: ¿Qué imprime? — Array de booleanos

```
boolean[] flags = new boolean[3];
flags[1] = true;
System.out.println(flags[0] + " " + flags[1] + " " + flags[2]);
```

¿Qué imprime? ¿Cuál es el valor por defecto de un boolean en un array?

Ejercicio 2: Encuentra el error — NullPointerException

```
String[] nombres = new String[3];
nombres[0] = "Ana";
nombres[1] = "Bob";
System.out.println(nombres[2].toUpperCase());
```

¿Qué ocurre al ejecutar este código? ¿Por qué?

Ejercicio 3: Completa el código — for básico para buscar el mayor

Completa el siguiente programa para que encuentre e imprima el número más grande del array:

```
int[] numeros = {12, 45, 7, 34, 89, 23};
int mayor = numeros[0];

for (int i = 1; i < _____; i++) {    // ¿hasta dónde llega el bucle?
    if (numeros[i] _____ mayor) {    // ¿qué operador?
        _____ = numeros[i];        // ¿qué asignamos?
    }
}

System.out.println("El mayor es: " + mayor);
```

Ejercicio 4: Escribe este programa — contar números pares

Crea un array de 10 enteros con valores que tú elijas. Recórrelo con un bucle for y cuenta cuántos de ellos son pares. Al final, imprime el total de pares y el array original.

Ejemplo de salida:

```
Array: [3, 8, 12, 5, 7, 10, 2, 9, 6, 1]
Pares: 5
```

Ejercicio 5: ¿Qué imprime? — ArrayList remove por índice vs valor

```
import java.util.ArrayList;

public class Test {
    public static void main(String[] args) {
        ArrayList<String> lista = new ArrayList<>();
        lista.add("A");
        lista.add("B");
        lista.add("C");
        lista.add("B");
        lista.add("D");

        lista.remove(1);           // remove por índice
        lista.remove("B");         // remove por objeto

        System.out.println(lista);
    }
}
```

¿Qué imprime? ¿Por qué el segundo remove("B") no borra el mismo que el primero?

Ejercicio 6: Encuentra el error — length vs length()

```
int[] numeros = {10, 20, 30};
String texto = "Hola";

System.out.println(numeros.length());
System.out.println(texto.length);
```

¿Qué líneas tienen error? Explica la diferencia entre length (sin paréntesis) y length() (con paréntesis).

Ejercicio 7: Escribe este programa — búsqueda lineal

Crea un array de enteros llamado edades con 8 valores. Pide al usuario un número por teclado (con Scanner) y busca si ese número está en el array. Imprime «Encontrado» o «No encontrado».

Introduce edad a buscar: 25

Encontrado en posición 3

Referències per seguir practicant

- CodeWars: [Convert number to reversed array of digits](#) (8 kyu)
- CodeWars: [Find the smallest integer in the array](#) (7 kyu)
- AceptaElReto.com: [152 - Suma pares e impares](#)
- AceptaElReto.com: [100 - Kaprekar](#)

Boletín 8 – Resuelto: Arrays y Colecciones

Dificultad progresiva. De ★ a ★★★★★.

★ Ejercicio 1: El inversor de arrays

Escribe un método `invertirArray(int[] arr)` que devuelva un nuevo array con los elementos en orden inverso. Pruébalo en `main()`.

★ Ejercicio 2: ArrayList de números pares

Crea un programa que genere los primeros 20 números pares (2, 4, 6, ..., 40) y los guarde en un `ArrayList<Integer>`. Luego, recorre la lista y suma solo los que sean múltiplos de 5.

★ Ejercicio 3: Estadísticas con array

Pide al usuario las notas de 10 alumnos (usa `Scanner`), guárdalas en un array `double[]`, y calcula: nota media, nota máxima, nota mínima y cuántos aprobaron (≥ 5).

★ ★ Ejercicio 4: Buscaminas simplificado

Crea un array bidimensional `boolean[5][5]` que represente un campo de minas. Coloca 5 minas (`true`) en posiciones aleatorias. El usuario introduce coordenadas (fila, columna) y el programa dice si hay mina. Si acierta una mina, el juego termina.

★ ★ Ejercicio 5: Lista de la compra con ArrayList

Crea un programa que gestione una lista de la compra usando `ArrayList<String>`. Menú con opciones: 1) Añadir, 2) Eliminar, 3) Marcar como comprado (añadir “[OK]” al final), 4) Mostrar lista, 0) Salir.

☆☆ Ejercicio 6: HashSet y TreeSet — eliminando duplicados

Crea un programa que lea una frase del usuario, separe las palabras (con `split(" ")`), las guarde en un `TreeSet<String>` y luego las muestre ordenadas alfabéticamente y sin duplicados.

☆☆☆ Ejercicio 7 (ProgramaMe): El juego de la memoria

Crea un juego de memoria usando un array bidimensional `int[4][4]`. Coloca 8 pares de números (1-8) en posiciones aleatorias. El jugador destapa dos posiciones. Si son iguales, permanecen visibles. Si no, se ocultan de nuevo. El juego termina cuando todos los pares están descubiertos.

☆☆☆ Ejercicio 8 (ProgramaMe): Estadísticas de texto con colecciones

Crea un programa que analice un texto largo (puedes hardcodearlo) y muestre:

1. Número total de palabras
2. Las 5 palabras más frecuentes
3. La palabra más larga
4. Porcentaje de palabras únicas

Usa `HashMap<String, Integer>` para frecuencias, `TreeSet` para orden, y `ArrayList` para manipulación.

Referencias para seguir practicando

- CodeWars: [Convert number to reversed array of digits](#) (8 kyu)
- CodeWars: [Find the smallest integer in the array](#) (7 kyu)
- CodeWars: [Sorting arrays](#) (7 kyu)
- AceptaElReto.com: [100 - Kaprekar](#)

- AceptaElReto.com: [340 - Juegos de naipes](#)
- AceptaElReto.com: [101 - Cuadrado Sator](#)

Boletín 8 – Intermedi: Arrays i Col·leccions

Exercicis de dificultat progressiva. De ★ a ★★ ★★ ★★★★★.

★ Ejercicio 1: La fusión de arrays ordenados

Escribe un método `fusionarArrays(int[] a, int[] b)` que reciba dos arrays ordenados de menor a mayor y devuelva un **nuevo array** también ordenado con todos los elementos de ambos. No uses `Arrays.sort()` ni colecciones. Hazlo con el algoritmo de fusión (merge) tipo «dos punteros».

Pista: Avanza con dos índices, uno por array, comparando en cada paso cuál elemento es menor.

★ Ejercicio 2: Rotación circular a la derecha

Implementa un método `rotarDerecha(int[] arr, int k)` que desplace cada elemento del array k posiciones hacia la derecha. Los elementos que «salen» por el final vuelven a entrar por el principio. No uses estructuras auxiliares grandes (vale un array temporal del tamaño de k , pero no copies todo).

Ejemplo: $\{1, 2, 3, 4, 5\}$ con $k = 2 \rightarrow \{4, 5, 1, 2, 3\}$.

★ Ejercicio 3: Suma de diagonales (matriz cuadrada)

Crea un programa que genere una matriz cuadrada `int[N][N]` con valores aleatorios entre 1 y 100, y calcule:

1. Suma de la **diagonal principal** (de arriba-izquierda a abajo-derecha).
2. Suma de la **diagonal secundaria** (de arriba-derecha a abajo-izquierda).
3. Diferencia absoluta entre ambas sumas.

Usa $N = 5$ para las pruebas.

☆☆ Ejercicio 4: Cola del supermercado con LinkedList

Simula una cola de supermercado usando `LinkedList<String>`. El programa debe mostrar un menú:

1. **Llega cliente** → Añade un nombre al final de la cola.
2. **Atender cliente** → Elimina y muestra el primero de la cola.
3. **¿Quién sigue?** → Muestra el primero sin eliminarlo.
4. **Estado de la cola** → Muestra todos los clientes en orden.
5. **Salir**

Usa los métodos `addLast()`, `removeFirst()`, `getFirst()` de `LinkedList`.

☆☆ Ejercicio 5: Intersección y unión de conjuntos

Crea dos `HashSet<Integer>` con números aleatorios (entre 1 y 20, 8 elementos cada uno). Calcula y muestra:

- **Intersección:** elementos que están en ambos conjuntos.
- **Unión:** todos los elementos sin repetir.
- **Diferencia simétrica:** elementos que están en uno u otro, pero no en ambos.

Pista: Usa `retainAll()`, `addAll()`, `removeAll()` de la interfaz `Set`.

☆☆ Ejercicio 6: Eliminar duplicados manteniendo orden

Crea un `ArrayList<Integer>` con elementos repetidos (`[3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]`). Escribe un método que devuelva un nuevo `ArrayList<Integer>` **sin duplicados pero manteniendo el orden de primera aparición**. No puedes usar `HashSet` directamente porque pierdes el orden. La solución es usar un `LinkedHashSet` o recorrer manualmente.

☆☆☆ Ejercicio 7 (CodeWars): Find the odd int

Dado un array de enteros, encuentra el que aparece un número impar de veces.

```
public static int findIt(int[] arr) {  
    // tu código aquí  
}
```

Ejemplo: {1, 2, 2, 3, 3, 3, 4, 3, 3, 3, 2, 2, 1} → devuelve 4.

Pista: Puedes usar un `HashMap<Integer, Integer>` para contar frecuencias, o el truco del XOR (operador ^).

 **CodeWars:** [Find the odd int](#) (6 kyu)

☆☆☆ Ejercicio 8 (AceptaElReto): La lotería de la peña

En una peña de lotería, cada socio aporta una cantidad y compra participaciones. Implementa un programa que:

1. Lea un array con las aportaciones de cada socio (en euros).
2. Calcule el total recaudado.
3. Si toca un premio de X euros, calcule cuánto le corresponde a cada socio proporcionalmente a su aportación.
4. Muestre los resultados ordenados de mayor a menor ganancia.

Usa arrays para almacenar nombres y aportaciones. El premio se pide por teclado.

 **AceptaElReto.com:** [340 - Juegos de naipes](#)

Referències

- **CodeWars:** [Find the odd int](#) (6 kyu)
- **CodeWars:** [Array.diff](#) (6 kyu)
- **CodeWars:** [Sort the odd](#) (6 kyu)
- **AceptaElReto.com:** [340 - Juegos de naipes](#)

- AceptaElReto.com: [101 - Cuadrado Sator](#)
- AceptaElReto.com: [241 - Buscando el nivel](#)

Butlletí 09 — Extres: Arrays i Col·leccions

★ Extres — Converteix-te en un programador de molt alt nivell

Aquests exercicis són completament opcionals, però si aspirem a ser un programador o programadora d'elit, aquí tens el teu camp d'entrenament. CodeWars i AceptaElReto són plataformes on els millors programadors del món es repte cada dia. Cada problema que resolguis et farà més fort, més ràpid i més creatiu. No els subestimis: són la diferència entre saber programar i ser un vertader professional.

CodeWars:

- [Mexican Wave](#) (6 kyu)

Pista: Itera índex a índex, construeix cada string posant en majúscula el caràcter en la posició i.

- [Delete occurrences of an element](#) (6 kyu)

Pista: Usa un HashMap per portar el compte de cada element; només afegeix-lo al resultat si el seu comptador no ha superat el límit.

- [Roman Numerals Encoder](#) (6 kyu)

Pista: Recorre els valors en ordre descendent (1000→1), resta el valor i afegeix el símbol romà corresponent.

AceptaElReto:

- [102 - Encriptació de missatges](#) (★ ★)

Pista: Aplica un desplaçament fix (xifrat Cèsar) a cada caràcter; vigila el wrap-around al final de l'alfabet.

- [341 - Matriu identitat](#) (★ ★)

Pista: Comprova que tots els elements de la diagonal són 1 i la resta són 0.

Boletín 9 – Inicial Resuelto: Genéricos y Mapas

Con soluciones. A aprender.

Ejercicio 1: Completa el código — clase genérica

```
public class Caja<T> {  
    private T contenido;  
  
    public void guardar(T contenido) {  
        this.contenido = contenido;  
    }  
  
    public T sacar() {  
        return contenido;  
    }  
}
```

La declaración correcta es `public class Caja<T>`.

💡 **Explicación:** `<T>` declara un parámetro de tipo. `T` es una convención (de “Type”), pero podrías usar cualquier letra. A partir de ahí, puedes usar `T` como un tipo cualquiera dentro de la clase. Cuando alguien haga `Caja<String>`, todas las `T` se convierten en `String`. Es como una plantilla: dejas huecos (`T`) que se rellenan cuando alguien usa la clase.

Ejercicio 2: ¿Qué imprime? — HashMap básico

```
import java.util.HashMap;

public class Test {
    public static void main(String[] args) {
        HashMap<String, String> capitales = new HashMap<>();
        capitales.put("España", "Madrid");
        capitales.put("Francia", "París");
        capitales.put("Italia", "Roma");
        capitales.put("España", "Barcelona");

        System.out.println(capitales.get("España"));
    }
}
```

Solución: Imprime Barcelona.



Explicación: En un HashMap, las claves son únicas. Cuando haces `put("España", "Madrid")` y luego `put("España", "Barcelona")`, la segunda llamada SOBREScribe el valor anterior. “Madrid” se pierde para siempre. Es como apuntar un número en tu agenda y luego tacharlo para poner otro: solo queda el último. Si quisieras mantener ambos, tendrías que usar una estructura diferente (como `HashMap<String, List<String>>`).

Ejercicio 3: Encuentra el error — tipo primitivo en genérico

```
ArrayList<int> numeros = new ArrayList<>(); // ERROR
numeros.add(1);
numeros.add(2);
numeros.add(3);
```

Solución: No puedes usar tipos primitivos como parámetros de tipo en genéricos. `int` debe ser `Integer`.

```
ArrayList<Integer> numeros = new ArrayList<>();
numeros.add(1); // autoboxing: int → Integer
numeros.add(2);
numeros.add(3);
```

💡 **Explicación:** Los genéricos solo funcionan con tipos referencia (objetos). `int`, `double`, `boolean` son tipos primitivos y no pueden ser parámetros de tipo. Por eso existen las clases wrapper: `Integer`, `Double`, `Boolean`. El autoboxing de Java convierte automáticamente `int` a `Integer` al añadir y `Integer` a `int` al obtener, por lo que apenas notas la diferencia en el código.

Ejercicio 4: Escribe este programa — mini agenda HashMap

```
import java.util.HashMap;
import java.util.Scanner;

public class MiniAgenda {
    public static void main(String[] args) {
        HashMap<String, String> agenda = new HashMap<>();
        agenda.put("Ana", "612345678");
        agenda.put("Bob", "698765432");
        agenda.put("Carlos", "655111222");

        Scanner sc = new Scanner(System.in);
        System.out.print("Buscar teléfono de: ");
        String nombre = sc.nextLine();

        String telefono = agenda.get(nombre);
        if (telefono != null) {
            System.out.println(nombre + " → " + telefono);
        } else {
            System.out.println(nombre + " no está en la agenda.");
        }
        sc.close();
    }
}
```

💡 **Explicación:** `HashMap` es perfecto para asociar nombres con teléfonos. La clave es el nombre (único) y el valor es el teléfono. `get(nombre)` devuelve `null` si la clave no existe, así que comprobamos antes de usarlo. Es la estructura de datos perfecta para una agenda: búsqueda rápida por clave ($O(1)$).

Ejercicio 5: Completa el código — getOrDefault

```
HashMap<String, Integer> edades = new HashMap<>();
edades.put("Ana", 25);
edades.put("Bob", 30);

int edadAna = edades.get("Ana");           // 25
int edadCarlos = edades.get("Carlos");     // null → NullPointerException (en
realidad: error de compilación si es int, NPE si Integer)
int edadCarlosSeguro = edades.getOrDefault("Carlos", 0); // 0
```

Solución: edadAna = 25. edad.get("Carlos") devuelve null (la clave no existe). Si la variable es int, no compila o da error porque no puedes asignar null a un primitivo. getOrDefault("Carlos", 0) devuelve 0 (el valor por defecto).

💡 **Explicación:** getOrDefault() es el salvavidas de los HashMap. En lugar de hacer if (map.get(clave) != null), haces map.getOrDefault(clave, valorDefecto) y te ahorras líneas y posibles NullPointerException. Es como tener un plan B: “si no encuentras a Carlos, devuelve 0”. Siempre que trabajes con mapas, úsalo.

Ejercicio 6: ¿Qué imprime? — método genérico

```
public class Util {
    public static <T> void imprimir(T elemento) {
        System.out.println("Valor: " + elemento);
    }

    public static void main(String[] args) {
        Util.imprimir(42);
        Util.imprimir("Hola");
        Util.imprimir(3.14);
    }
}
```

Solución:

Valor: 42
Valor: Hola
Valor: 3.14

💡 **Explicación:** El método imprimir es genérico: <T> antes del tipo de retorno. El compilador INFIERE el tipo T a partir del argumento. En la primera llamada, T es Integer. En la segunda, T es String. En la tercera, T es Double. No necesitas especificarlo: Java lo deduce solo. Es como un profesor que se adapta al alumno: da la misma clase pero adaptada a cada uno.

Ejercicio 7: Encuentra el error — la clave mutable

```
HashMap<ArrayList<Integer>, String> mapa = new HashMap<>();  
ArrayList<Integer> lista = new ArrayList<>();  
lista.add(1);  
lista.add(2);  
mapa.put(lista, "valor");  
lista.add(3); // modificamos la clave después de usarla  
System.out.println(mapa.get(lista)); // posiblemente null
```

Solución: El problema es que las claves de un HashMap deben ser INMUTABLES. Al modificar lista después de usarla como clave, su hashCode() cambia. El HashMap busca en el bucket antiguo, pero la clave tiene un hash diferente ahora, así que get() puede devolver null aunque la clave esté en el mapa.

💡 **Explicación:** Las claves de un HashMap deben ser inmutables (como String o Integer). Si usas un objeto mutable y lo modificas, el HashMap se vuelve impredecible. Es como cambiar la cerradura de tu casa y esperar que tu llave vieja siga funcionando. Por eso String es la clave perfecta: es inmutable. Nunca uses ArrayList, arrays, o tus propias clases mutables como claves de un HashMap a menos que sepas muy bien lo que haces (spoiler: no lo sabes).

Referencias para seguir practicando

- CodeWars: [Grasshopper - Grade book](#) (7 kyu)
- CodeWars: [Word Count](#) (7 kyu)
- AceptaElReto.com: [416 - Casillas](#)
- AceptaElReto.com: [462 - Tres dedos](#)

Boletín 9 – Inicial: Genèrics i Mapes

Sense solucions. A donar-li.

Ejercicio 1: Completa el código — clase con dos tipos genéricos

```
public class Par<_____, _____> {    // ¿qué dos tipos faltan?
    private T primero;
    private U segundo;

    public Par(T primero, U segundo) {
        this.primero = primero;
        this.segundo = segundo;
    }

    public T getPrimero() { return primero; }
    public U getSegundo() { return segundo; }
}
```

Completa la declaración para que Par acepte dos tipos genéricos distintos.

Ejercicio 2: ¿Qué imprime? — HashMap con merge

```
import java.util.HashMap;

public class Test {
    public static void main(String[] args) {
        HashMap<String, Integer> mapa = new HashMap<>();
        mapa.put("Ana", 10);
        mapa.put("Bob", 20);
        mapa.put("Ana", 30);

        System.out.println(mapa.get("Ana"));
        System.out.println(mapa.size());
    }
}
```

¿Qué imprime? ¿Por qué size() no es 3?

Ejercicio 3: Encuentra el error — TreeSet sin Comparable

```
import java.util.TreeSet;

public class Test {
    public static void main(String[] args) {
        TreeSet<Persona> conjunto = new TreeSet<>();
        conjunto.add(new Persona("Ana"));
        conjunto.add(new Persona("Bob"));
        System.out.println(conjunto);
    }
}

class Persona {
    String nombre;
    Persona(String nombre) { this.nombre = nombre; }
}
```

¿Qué ocurre al ejecutar? ¿Por qué TreeSet necesita que Persona implemente una interfaz concreta?

Ejercicio 4: Escribe este programa — contador de palabras con HashMap

Crea un programa que tenga un array de palabras (hardcodeado) como este:

```
String[] palabras = {"hola", "mundo", "hola", "java", "mundo", "hola", "adios"};
```

Usa un HashMap<String, Integer> para contar cuántas veces aparece cada palabra. Al final, recorre el mapa con un bucle for-each sobre entrySet() y muestra cada palabra con su cuenta.

Ejercicio 5: ¿Qué imprime? — método genérico con límite

```
public class Util {  
    public static <T extends Comparable<T>> T maximo(T a, T b) {  
        return a.compareTo(b) > 0 ? a : b;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(maximo(5, 8));  
        System.out.println(maximo("gato", "perro"));  
    }  
}
```

¿Qué imprime? ¿Qué pasaría si T no tuviera el límite Comparable<T>?

Ejercicio 6: Encuentra el error — clave duplicada el primero se pierde

```
HashMap<Integer, String> mapa = new HashMap<>();  
mapa.put(1, "uno");  
mapa.put(2, "dos");  
mapa.put(1, "Uno otra vez");  
  
System.out.println(mapa.get(1));
```

¿Qué imprime? ¿Se pierde el primer valor asociado a la clave 1?

Ejercicio 7: Escribe este programa — TreeMap con valores por defecto

Crea un `TreeMap<String, Integer>` para almacenar las edades de 5 personas. Rellénalo con nombres y edades. Luego, pide al usuario un nombre por teclado y muestra su edad. Si el nombre no existe, muestra un mensaje de error.

Usa `getOrDefault()` para evitar el `null`.

Referències per seguir practicant

- CodeWars: [Grasshopper - Grade book](#) (7 kyu)
- CodeWars: [Word Count](#) (7 kyu)
- AceptaElReto.com: [416 - Casillas](#)
- AceptaElReto.com: [462 - Tres dedos](#)

Boletín 9 – Resuelto: Genéricos y Mapas

Dificultad progresiva. De ★ a ★★★★★.

★ Ejercicio 1: Clase genérica Pareja<T, U>

Crea una clase genérica `Pareja<T, U>` que almacene dos objetos de tipos posiblemente distintos. Incluye métodos `getPrimero()`, `getSegundo()`, `setPrimero(T)`, `setSegundo(U)` y un método `intercambiar()` que devuelva una nueva `Pareja<U, T>`.

★ Ejercicio 2: Contador de palabras con HashMap

Escribe un programa que lea un texto (hardcodeado o por `Scanner`) y cuente cuántas veces aparece cada palabra usando `HashMap<String, Integer>`.

★ Ejercicio 3: Método genérico maximo

Implementa un método genérico `public static <T extends Comparable<T>> T maximo(T[] array)` que devuelva el elemento más grande de un array usando el método `compareTo()`.

★ ★ Ejercicio 4: Agenda completa con menú

Implementa una agenda usando `HashMap<String, String>` con menú interactivo: añadir contacto, buscar por nombre, listar todos, borrar contacto y salir.

★ ★ Ejercicio 5: TreeMap — ordenando por clave

Crea un programa que lea 5 países y sus capitales, los guarde en un `TreeMap<String, String>` y los muestre ordenados alfabéticamente por país. Luego muestra el primer y último país alfabéticamente.

☆☆☆ Ejercicio 6 (ProgramaMe): Wildcards — el método de comparación

Crea un método `compararMedias(List<? extends Number> a, List<? extends Number> b)` que compare la media de dos listas de números. Devuelve -1, 0 o 1 según si la media de a es menor, igual o mayor que la de b. Usa wildcards para que acepte `List<Integer>`, `List<Double>`, etc.

☆☆☆ Ejercicio 7 (ProgramaMe): Sistema de votaciones

Crea un sistema de votaciones donde:

- Cada votante puede votar por un candidato (`String`)
- Usa un `HashMap<String, Integer>` para los votos
- Usa un `TreeMap<String, Integer>` para mostrar el ranking ordenado

Crea un método genérico `public static <T> T obtenerGanador(Map<T, Integer> votos)` que devuelva la clave con más votos.

☆☆☆ Ejercicio 8 (ProgramaMe): Cache simple con genéricos

Implementa una clase `Cache<K, V>` que almacene hasta `maxElementos` pares clave-valor usando un `LinkedHashMap<K, V>`. Cuando se añade un elemento y la cache está llena, elimina el elemento más antiguo (LRU - Least Recently Used). Incluye `get(K clave)` y `put(K clave, V valor)`.

Referencias para seguir practicando

- CodeWars: [Word Count](#) (7 kyu)
- CodeWars: [Sort arrays - 1](#) (6 kyu)
- CodeWars: [Counting duplicates](#) (6 kyu)
- AceptaElReto.com: [416 - Casillas](#)
- AceptaElReto.com: [417 - Binomiales](#)

- AceptaElReto.com: [462 - Tres dedos](#)

Boletín 9 – Intermedi: Genèrics i Mapes

Exercicis de dificultat progressiva. De ★ a ★★ ★★.

★ Ejercicio 1: Pila genérica <T>

Implementa una clase genérica `Pila<T>` que funcione como una pila (LIFO). Debe tener los métodos:

- `void push(T elemento)` — apila un elemento.
- `T pop()` — desapila y devuelve el elemento superior (lanza `EmptyStackException` si está vacía).
- `T peek()` — devuelve el elemento superior sin desapilarlo.
- `boolean isEmpty()` — indica si está vacía.
- `int size()` — número de elementos.

Internamente, usa un `ArrayList<T>` como almacenamiento. Pruébala con `Pila<Integer>`, `Pila<String>` y `Pila<Double>`.

★ Ejercicio 2: HashMap inverso

Escribe un método genérico estático:

```
public static <K, V> HashMap<V, K> invertirMapa(HashMap<K, V> original)
```

Que devuelva un nuevo `HashMap` intercambiando claves y valores. Si hay valores duplicados en el mapa original, el último encontrado sobrescribe al anterior.

Prueba con un mapa de `String → Integer` y otro de `String → String`.

★ Ejercicio 3: Método genérico — filtrar por condición

Implementa un método genérico:

```
public static <T> List<T> filtrar(List<T> lista, Predicate<T> condicion)
```

Que devuelva una nueva lista con los elementos que cumplen la condición. Usa la interfaz `Predicate<T>` de Java.

Pruébalo filtrando números pares de una `List<Integer>` y palabras que empiecen por «A» de una `List<String>`.

★ ★ Ejercicio 4: Caché LRU con LinkedHashMap

Crea una clase `CacheLRU<K, V>` que extienda o use internamente un `LinkedHashMap<K, V>` con capacidad máxima de 5 elementos. Cuando se añade un elemento y ya hay 5, se elimina el **menos recientemente usado** (acceso, no inserción).

Pista: `LinkedHashMap` tiene el constructor con `accessOrder=true` y el método `removeEldestEntry()`.

★ ★ Ejercicio 5: TreeMap — frecuencia de letras

Escribe un programa que lea un texto por teclado (o use uno hardcodeado) y cuente cuántas veces aparece cada **letra** (ignorando espacios, números y signos). Usa un `TreeMap<Character, Integer>` para que las letras se muestren automáticamente ordenadas alfabéticamente.

Ejemplo de salida para «Hola mundo»:

```
a: 1, d: 1, h: 1, l: 1, m: 1, n: 1, o: 2, u: 1
```

★ ★ Ejercicio 6 (Wildcards): Suma de números genérica

Implementa un método que sume todos los números de una lista, aceptando cualquier subtipo de `Number`:

```
public static double sumar(List<? extends Number> lista)
```

Pruébalo con `List<Integer>`, `List<Double>` y `List<Float>`. ¿Qué ocurre si intentas pasar una `List<String>`?

Crea también un segundo método que **mezcle** dos listas de números de tipos distintos en una sola `List<Double>`:

```
public static List<Double> mezclar(List<? extends Number> a, List<? extends Number> b)
```

☆☆☆ Ejercicio 7 (ProgramaMe): Sistema de inventario genérico

Crea un sistema de inventario genérico para una tienda:

1. Clase `Producto<T>` con `String` nombre, `double` precio, `T` categoria (el tipo de categoría puede ser `String` o un `Enum`).
2. Clase `Inventario<T>` que almacene `Producto<T>` en un `HashMap<String, Producto<T>>` (clave = nombre del producto).
3. Métodos: `agregar`, `eliminar`, `buscar`, `listar`, `valorTotal()`.
4. Crea un inventario de productos con categoría `String` y otro con categoría `enum`.

☆☆☆ Ejercicio 8 (CodeWars + AceptaElReto)

Resuelve los siguientes problemas aplicando mapas y genéricos:

CodeWars: [Counting Duplicates](#) (6 kyu) — Cuenta cuántos caracteres aparecen más de una vez en una cadena. Ideal para `HashMap<Character, Integer>`.

AceptaElReto: [416 - Casillas de corrección](#) — Gestiona casillas de corrección usando un mapa de errores por alumno.



Referències

- CodeWars: [Counting Duplicates](#) (6 kyu)
- CodeWars: [Sort arrays - 1](#) (6 kyu)

- CodeWars: [Merging sorted integer arrays](#) (6 kyu)
- AceptaElReto.com: [416 - Casillas](#)
- AceptaElReto.com: [462 - Tres dedos](#)
- AceptaElReto.com: [417 - Binomiales](#)

Butlletí 10 — Extres: Genèrics i Mapes

★ Extres — Converteix-te en un programador de molt alt nivell

Aquests exercicis són completament opcionals, però si aspirem a ser un programador o programadora d'elit, ací tens el teu camp d'entrenament. CodeWars i AceptaElReto són plataformes on els millors programadors del món es repte cada dia. Cada problema que resolguis et farà més fort, més ràpid i més creatiu. No els subestimis: són la diferència entre saber programar i ser un vertader professional.

CodeWars:

- [Counting sheep](#) (8 kyu)

Pista: Filtra els valors null i compta els true amb `stream().filter(b -> b).count()`.

- [Find the unique number](#) (6 kyu)

Pista: Divideix els tres primers nombres per saber quin es repeteix; el que no coincideix és l'únic. O usa XOR.

- [Array.diff](#) (6 kyu)

Pista: Converteix el segon array en un `Set<Integer>` i filtra el primer amb `removeIf` o un `stream`.

AceptaElReto:

- [302 - Quadrat màgic](#) (★ ★)

Pista: Calcula la suma objectiu = total / n; verifica files, columnes i diagonals contra eixe valor.

- [432 - Nombre de vegades que apareix](#) (★)

Pista: Usa un `HashMap<Character, Integer>` per a comptar freqüències de cada caràcter.

CodeWars:

- [Who likes it?](#) (6 kyu)

Pista: Usa un `Map<Integer, String>` amb els patrons de resposta. La clau és la mida del array, el valor és la plantilla.

AceptaElReto:

- [217 - Quin costat del carrer?](#) (★ ★)

Pista: Llig el número com String, no com int. Només t'interessa l'últim dígit per saber si és parell o senar.

Boletín 10 – Inicial Resuelto: Consola y Ficheros

Con soluciones. A aprender.

Ejercicio 1: ¿Qué imprime? — printf()

```
String nombre = "Ana";
int edad = 20;
double nota = 9.5;

System.out.printf("Nombre: %s, Edad: %d, Nota: %.2f%n", nombre, edad, nota);
```

Solución: Imprime Nombre: Ana, Edad: 20, Nota: 9.50 y un salto de línea.

💡 **Explicación:** printf() no es println(): no añade salto de línea automático. Por eso se usa %n al final, que es el salto de línea independiente de plataforma. %s para String, %d para entero, %.2f para decimal con 2 dígitos. El %n es como un \n pero más educado: sabe si estás en Windows (salto con \r\n) o Linux (solo \n).

Ejercicio 2: Encuentra el error — nextInt() + nextLine()

```
Scanner sc = new Scanner(System.in);
System.out.print("Edad: ");
int edad = sc.nextInt();           // lee "25" y deja el "\n" sin consumir
System.out.print("Nombre: ");
String nombre = sc.nextLine();     // se traga el "\n" que quedó
System.out.println(nombre + " tiene " + edad + " años.");
sc.close();
```

Solución: nextInt() NO consume el salto de línea. Cuando el usuario escribe 25 y pulsa Enter, en el buffer queda 25\n. nextInt() consume 25 y deja \n. Luego nextLine() consume ese \n y devuelve una cadena vacía. El programa imprime tiene 25 años. (nombre vacío).

Para arreglarlo: añade un `sc.nextLine()` extra después del `nextInt()` para consumir el salto de línea.

```
int edad = sc.nextInt();
sc.nextLine(); // consume el \n pendiente
String nombre = sc.nextLine();
```

💡 **Explicación:** Es el error más famoso de Java con Scanner. Siempre que uses `nextInt()`, `nextDouble()` o `next()` y luego `nextLine()`, necesitas un `nextLine()` extra para limpiar el buffer. O mejor: usa siempre `nextLine()` y convierte con `Integer.parseInt()`, que es más seguro.

Ejercicio 3: Completa el código — leer archivo

```
import java.io.*;
import java.nio.file.*;
import java.util.List; // ¡este import falta!

public class Test {
    public static void main(String[] args) throws IOException {
        Path ruta = Paths.get("datos.txt");
        List<String> lineas = Files.readAllLines(ruta);
        for (String linea : lineas) {
            System.out.println(linea);
        }
    }
}
```

Falta línea en el `println` y también el `import java.util.List;`.

💡 **Explicación:** `Files.readAllLines()` devuelve un `List<String>`, que necesita `import`. El `for-each` recorre cada línea y `linea` es la variable que contiene cada línea. Sin `import java.util.List;`, el compilador no sabe qué es `List` y se queja.

Ejercicio 4: Escribe este programa — Hola mundo archivo

```

import java.io.*;

public class HolaArchivo {
    public static void main(String[] args) {
        // Escribir
        try {
            FileWriter writer = new FileWriter("saludo.txt");
            writer.write(";Hola, archivo!\n");
            writer.write("Esto es una segunda línea.\n");
            writer.close();
            System.out.println("Archivo escrito.");
        } catch (IOException e) {
            System.out.println("Error al escribir: " + e.getMessage());
        }

        // Leer
        try {
            BufferedReader reader = new BufferedReader(new FileReader("saludo.txt"));
            String linea = reader.readLine();
            while (linea != null) {
                System.out.println(linea);
                linea = reader.readLine();
            }
            reader.close();
        } catch (IOException e) {
            System.out.println("Error al leer: " + e.getMessage());
        }
    }
}

```

💡 **Explicación:** `FileWriter` escribe caracteres en un archivo. Si el archivo no existe, lo crea. Si existe, lo sobrescribe. `BufferedReader` envuelve a `FileReader` para leer por líneas (más rápido). El bucle `while (linea != null)` es el estándar para leer archivos línea por línea. Siempre cierra los recursos con `close()` para evitar pérdidas de datos.

Ejercicio 5: ¿Qué imprime? — el contador de líneas

```
import java.io.*;

public class Test {
    public static void main(String[] args) throws IOException {
        File f = new File("datos.txt");
        FileWriter w = new FileWriter(f);
        w.write("linea1\nlinea2\nlinea3\n");
        w.close();

        BufferedReader r = new BufferedReader(new FileReader(f));
        int contador = 0;
        while (r.readLine() != null) {
            contador++;
        }
        r.close();
        System.out.println(contador);
    }
}
```

Solución: Imprime 3.

💡 **Explicación:** El archivo tiene tres líneas: “linea1”, “linea2”, “linea3” (el último \n crea una línea adicional vacía, pero `readLine()` no la cuenta como línea porque devuelve `null` cuando no hay más). `readLine()` devuelve cada línea hasta que no quedan más, momento en que devuelve `null`. El contador se incrementa 3 veces. Nota: en realidad, si el archivo termina con \n, `readLine()` leería “linea3” y luego devolvería `null`, por lo que contaríamos 3 líneas. Si no hubiera \n al final, también serían 3.

Ejercicio 6: Encuentra el error — archivo no cerrado

```
FileWriter writer = new FileWriter("notas.txt");
writer.write("Esto es una nota importante.");
// falta: writer.close();
```

Solución: Falta `writer.close()`. Sin cerrar el archivo, los datos pueden no escribirse físicamente en el disco porque `FileWriter` usa un buffer interno.

💡 **Explicación:** `FileWriter` no escribe directamente en el disco. Usa un buffer. Cuando haces `write()`, los datos se almacenan en el buffer. Si no llamas a `close()` o `flush()`, parte de los datos pueden perderse cuando el programa termina. Es como echar una carta al buzón pero no cerrar la puerta: el cartero puede no recogerla. Además, el sistema operativo mantiene el archivo bloqueado hasta que se cierre. **Siempre cierra los archivos** o usa `try-with-resources` (Java 7+).

Ejercicio 7: Escribe este programa — Scanner que suma números

```
import java.util.Scanner;

public class SumaNumeros {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int suma = 0;
        int num;

        System.out.println("Introduce números enteros (0 para terminar):");
        do {
            System.out.print("Número: ");
            num = sc.nextInt();
            suma += num;
        } while (num != 0);

        System.out.printf("La suma total es: %d\n", suma);
        sc.close();
    }
}
```

💡 **Explicación:** El bucle `do-while` asegura que se pida al menos un número. Cuando el usuario introduce 0, el bucle termina. `suma += num` acumula todos los números, incluido el 0 final (que no afecta a la suma). `printf()` con `%d` formatea el entero. Es un programa simple pero que combina todas las piezas: `Scanner`, bucle, acumulación y salida formateada.

Referencias para seguir practicando

- CodeWars: [Get the Middle Character](#) (7 kyu)
- CodeWars: [String repeat](#) (8 kyu)
- AceptaElReto.com: [140 - Suma de dígitos](#)
- AceptaElReto.com: [149 - San Fermines](#)

Boletín 10 – Inicial: Consola, Fitxers i Regex

Sense solucions. A donar-li al teclat.

Ejercicio 1: ¿Qué imprime? — printf con conversiones

```
int entero = 42;
double decimal = 3.1416;
String texto = "Java";

System.out.printf("%d %f %s %n", entero, decimal, texto);
```

¿Qué imprime? ¿Qué hace %n al final?

Ejercicio 2: Encuentra el error — IOException sin capturar

```
import java.io.*;

public class Test {
    public static void main(String[] args) {
        FileWriter writer = new FileWriter("salida.txt");
        writer.write("Hola mundo");
        writer.close();
    }
}
```

Este código no compila. ¿Por qué? ¿Qué dos formas hay de solucionarlo?

Ejercicio 3: Completa el código — try-with-resources

Completa el siguiente programa para que lea un archivo y muestre su contenido:

```
import java.io.*;
import java.nio.file.*;

public class Lector {
    public static void main(String[] args) {
        Path ruta = Paths.get("datos.txt");

        try (_____ reader = Files.newBufferedReader(ruta)) { // ¿qué tipo?
            String linea;
            while ((linea = reader.readLine()) != null) {
                System.out.println(_____); // ¿qué va aquí?
            }
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Ejercicio 4: Escribe este programa — guardar array en archivo

Crea un array de cadenas con 5 nombres de ciudades. Escribe cada nombre en una línea de un archivo llamado `ciudades.txt`, usando `BufferedWriter` y `try-with-resources`. No olvides el salto de línea.

Ejercicio 5: ¿Qué imprime? — Scanner next vs nextLine

```
import java.util.Scanner;

public class Test {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Palabra: ");
        String palabra = sc.next();
        System.out.print("Frase: ");
        String frase = sc.nextLine();
        System.out.println "[" + palabra + "] [" + frase + "]");
    }
}
```

Si el usuario introduce `Hola mundo` y pulsa `Enter`, luego `Esto es una frase` y pulsa `Enter`, ¿qué imprime exactamente? ¿Por qué `next()` y `nextLine()` se comportan distinto?

Ejercicio 6: Encuentra el error — `File.createNewFile` sin comprobar

```
File f = new File("documento.txt");
f.createNewFile();
FileWriter w = new FileWriter(f);
w.write("Contenido importante");
w.close();
```

¿Qué pasa si el archivo `documento.txt` ya existe? ¿Qué devuelve `createNewFile()`?

Ejercicio 7: Escribe este programa — menú con `Scanner` y `switch`

Crea un programa que muestre un menú con estas opciones:

1. **Saludar** → imprime «¡Hola, programador!»
2. **Despedirse** → imprime «¡Hasta luego!»
3. **Cuenta atrás** → pide un número y cuenta desde ese número hasta 0
4. **Salir**

El menú debe repetirse hasta que el usuario elija 0. Usa Scanner, switch y printf() para formatear la salida.

Referències per seguir practicant

- CodeWars: [Get the Middle Character](#) (7 kyu)
- CodeWars: [String repeat](#) (8 kyu)
- AceptaElReto.com: [140 - Suma de dígitos](#)
- AceptaElReto.com: [149 - San Fermín](#)

Boletín 10 – Resuelto: Consola y Ficheros

Dificultad progresiva. De ★ a ★★★★★.

★ Ejercicio 1: La entrevista de trabajo

Crea un programa que haga una entrevista de trabajo simulada. Pregunta nombre, edad, años de experiencia y lenguaje favorito. Al final, muestra un resumen con `printf()` formateado en forma de tarjeta.

★ Ejercicio 2: Calculadora de propinas

Solicita el total de la cuenta y el porcentaje de propina. Calcula la propina y el total final. Muestra los resultados con 2 decimales usando `printf()`.

★ Ejercicio 3: El diario personal (append)

Crea un programa que escriba en un archivo `diario.txt` una línea con la fecha y el texto que el usuario introduzca. Cada ejecución debe añadir una nueva entrada al final sin borrar las anteriores. Usa `FileWriter` en modo `append`.

★ ★ Ejercicio 4: El contador de líneas, palabras y caracteres

Crea un programa que lea un archivo de texto y muestre cuántas líneas, palabras y caracteres tiene. Usa `BufferedReader` para leer.

★ ★ Ejercicio 5: El gestor de contactos (archivo)

Crea un programa con menú que permita: añadir un contacto (nombre, teléfono) a `contactos.txt`, listar todos los contactos, buscar un contacto por nombre. Usa `BufferedReader` y `PrintWriter`. Formatea con `printf()`.

★ ★ Ejercicio 6: Lectura con NIO (Files y Paths)

Reescribe el ejercicio del contador de líneas usando la API NIO (`Files.readAllLines()` y `Path`). Compara la diferencia.

★ ★ ★ Ejercicio 7 (ProgramaMe): Cifrado César con archivos

Crea un programa que lea un archivo `mensaje.txt`, desplace cada carácter 3 posiciones en el alfabeto (cifrado César) y escriba el resultado en `mensaje_cifrado.txt`. Luego, otro programa (o el mismo con una opción) que lo descifre. Usa `try-with-resources` y `BufferedReader/PrintWriter`.

★ ★ ★ Ejercicio 8 (ProgramaMe): Serialización de objetos

Crea una clase `Estudiante` que implemente `Serializable` con `String nombre`, `int edad`, `double notaMedia`. Crea un programa que guarde un `ArrayList<Estudiante>` en un archivo `estudiantes.dat` usando `ObjectOutputStream`. Luego, otro programa (o el mismo con opción) que lo lea con `ObjectInputStream` y muestre los datos formateados.

Referencias para seguir practicando

- CodeWars: [Get the Middle Character](#) (7 kyu)
- CodeWars: [String repeat](#) (8 kyu)
- CodeWars: [Exes and Ohs](#) (7 kyu)
- AceptaElReto.com: [140 - Suma de dígitos](#)
- AceptaElReto.com: [149 - San Fermín](#)
- AceptaElReto.com: [152 - Suma pares e impares](#)

Boletín 10 – Intermedi: Consola, Fitxers i Regex

Exercicis de dificultat progressiva. De ★ a ★★ ★★.

★ Ejercicio 1: Formateador de ticket de compra

Crea un programa que simule un ticket de compra. Usa un array de productos (nombre, precio, cantidad) y muestra un ticket formateado con `printf()`:

```
=====
      TICKET DE COMPRA
=====
Pan           2 x 1.20€ =  2.40€
Leche         3 x 0.95€ =  2.85€
Huevos        1 x 3.50€ =  3.50€
-----
TOTAL                        =  8.75€
=====
```

Requisitos: alinear nombres a la izquierda, precios a la derecha, 2 decimales, ancho fijo de columnas.

★ Ejercicio 2: Buscador de archivos por extensión

Crea un programa que pida una ruta de directorio y una extensión (ej: `.txt`, `.java`) y liste **recursivamente** todos los archivos con esa extensión. Usa la clase `File` y su método `listFiles()`.

Pista: Si el archivo es un directorio, llama al método de nuevo (recursión).

★ Ejercicio 3: Lector de CSV con Scanner

Dado un archivo `datos.csv` con el siguiente formato (sin cabecera):

```
Ana;25;DAM
Bob;22;DAW
Carlos;30;DAM
```

Usa `Scanner` con `useDelimiter()` para leer el archivo y mostrar los datos en formato tabla alineada. Usa `printf()` para formatear.

Asegúrate de que el `Scanner` maneje correctamente tanto el delimitador `;` como el salto de línea.

☆☆ Ejercicio 4: Filtro de líneas por palabra clave

Crea un programa que lea un archivo de texto (`origen.txt`) y escriba en `destino.txt` solo las líneas que contienen una palabra clave (pedida al usuario). Usa `BufferedReader` y `PrintWriter`.

Muestra al final cuántas líneas coincidieron y cuántas se descartaron.

☆☆ Ejercicio 5: Separador de líneas pares e impares

Crea un programa que lea un archivo `entrada.txt` y genere dos archivos:

- `pares.txt` → contiene las líneas en posición par (0, 2, 4...).
- `impares.txt` → contiene las líneas en posición impar (1, 3, 5...).

Usa `try-with-resources` con **tres** recursos (un `BufferedReader` y dos `PrintWriter`).

☆☆ Ejercicio 6: Split con regex — analizador de frases

Escribe un programa que lea una frase del usuario y use `split()` con una expresión regular para:

1. Separar las palabras (ignorando espacios, comas, puntos, signos).

2. Mostrar cuántas palabras hay.
3. Mostrar la palabra más larga.
4. Mostrar las palabras que empiezan por vocal.

Ejemplo: "Hola, mundo. Esto es Java: ¿mola?" →

```
Palabras: 6
Más larga: "mundo"
Empiezan por vocal: ["Esto"]
```

☆☆☆ Ejercicio 7 (ProgramaMe): Validador de datos con regex

Crea un programa que lea un archivo `datos.txt` donde cada línea contiene un dato y su tipo (separados por `;`):

```
ana@email.com;email
12345678Z;dni
+34 612345678;telefono
91 123 45 67;telefono
esto-no-es-email;email
```

Valida cada línea según el tipo usando expresiones regulares:

- **Email:** formato básico `xxx@xxx.xxx`
- **DNI:** 8 dígitos + letra mayúscula (la letra debe ser válida según el algoritmo)
- **Teléfono:** opcional `+34` seguido de 9 dígitos, con o sin espacios

Muestra un resumen: cuántos válidos, cuántos inválidos, y lista los inválidos.

☆☆☆ Ejercicio 8 (CodeWars + AceptaElReto)

Resuelve estos problemas que combinan ficheros, regex y consola:

CodeWars: [Regex validate PIN code](#) (7 kyu) — Valida que un String sea un PIN de 4 o 6 dígitos exactos.

CodeWars: [Exes and Ohs](#) (7 kyu) — Cuenta si el número de X y O es el mismo en un String (puedes resolverlo con o sin regex).

AceptaElReto: [149 - San Fermine](#)s — Lectura de múltiples casos de prueba desde consola.



Referències

- CodeWars: [Regex validate PIN code](#) (7 kyu)
- CodeWars: [Exes and Ohs](#) (7 kyu)
- CodeWars: [String repeat](#) (8 kyu)
- AceptaElReto.com: [149 - San Fermine](#)s
- AceptaElReto.com: [140 - Suma de dígitos](#)
- AceptaElReto.com: [152 - Suma pares e impares](#)

Butlletí 11 — Extres: Consola, Fitxers i Expressions Regulars

★ Extres — Converteix-te en un programador de molt alt nivell

Aquests exercicis són completament opcionals, però si aspirem a ser un programador o programadora d'elit, ací tens el teu camp d'entrenament. CodeWars i AceptaElReto són plataformes on els millors programadors del món es repte cada dia. Cada problema que resolguis et farà més fort, més ràpid i més creatiu. No els subestimis: són la diferència entre saber programar i ser un vertader professional.

CodeWars:

- [Categorize New Member](#) (7 kyu)

Pista: Edat ≥ 55 i hándicap $> 7 \rightarrow$ "Senior", si no $\rightarrow$ "Open". Simple lògica booleana.

- [Two to One](#) (7 kyu)

Pista: Concatena els dos strings, converteix-los a `char[]`, ordena'ls, i usa un `StringBuilder` per eliminar duplicats.

- [Primes in numbers](#) (5 kyu)

Pista: Dividix per 2, després per imparells fins a $\sqrt{n}$; compta les repeticions de cada divisor.

AceptaElReto:

- [108 - Formigues](#) (★ ★)

Pista: Pens en posicions relatives: quan dos formigues es creuen, és com si s'ignoraren.

- [380 - Quin gran és el cine!](#) (★ ★)

Pista: Simula l'ompliment de butaques fila per fila; busca forats consecutius suficients per a cada grup.

Boletín 14 – Inicial Resuelto: JDBC – Conexión y Consultas

💡 Los 5 pasos, bien explicados y con humor.

Ejercicio 1: Los 5 pasos

1. **Cargar el driver** → `Class.forName("com.mysql.cj.jdbc.Driver")` (opcional desde Java 6)
2. **Establecer conexión** → `DriverManager.getConnection(url, user, pass)` devuelve `Connection`
3. **Crear Statement** → `con.createStatement()` devuelve `Statement` o `con.prepareStatement(sql)` devuelve `PreparedStatement`
4. **Ejecutar consulta** → `stmt.executeQuery(sql)` para `SELECT` o `stmt.executeUpdate(sql)` para `INSERT/UPDATE/DELETE`
5. **Procesar resultados** → `rs.next() + rs.getXxx("columna")`
6. **(Bonus) Cerrar todo** → `con.close(), stmt.close(), rs.close()` (o `try-with-resources`)

💡 **Explicación:** El paso 1 es opcional desde Java 6 (los drivers JDBC 4.0 se cargan automáticamente si están en el classpath). Pero verás `Class.forName()` en mucho código legacy. No pasa nada si lo pones. El paso 6 es obligatorio: conexiones abiertas = servidor saturado.

Ejercicio 2: Completa la conexión

```

String url = "jdbc:mysql://localhost:3306/instituto";
String user = "root";
String pass = "admin123";

Connection con = DriverManager.getConnection(url, user, pass);
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM alumnos");

while (rs.next()) {
    System.out.println(rs.getString("nombre"));
}

// ¡Falta cerrar!
con.close();
stmt.close();
rs.close();

```

💡 **Explicación:** Connection, Statement, ResultSet. Al final hay que cerrar todo. Mejor con try-with-resources para que se cierre automáticamente. Si se te olvida cerrar, la conexión queda abierta hasta que el garbage collector decida visitarte (y no sabe cuándo venir).

Ejercicio 3: ¿Qué imprime?

```

try (Connection con = DriverManager.getConnection(url, user, pass);
     Statement stmt = con.createStatement();
     ResultSet rs = stmt.executeQuery("SELECT COUNT(*) FROM alumnos")) {

    if (rs.next()) {
        System.out.println(rs.getInt(1)); // número total de alumnos
    }
}

```

💡 **Explicación:** Imprime el número total de filas en la tabla alumnos. COUNT(*) devuelve una sola fila con una columna. Como no tiene nombre (o el nombre depende de la BD), accedemos por índice 1 (la primera columna). También funciona rs.getInt("COUNT(*)") pero es más feo. rs.getInt(1) es la forma estándar cuando hay una columna sin alias.

Ejercicio 4: Encuentra el error

```
public void buscarAlumno(String nombre) {  
    String sql = "SELECT * FROM alumnos WHERE nombre = '" + nombre + "'";  
    // ¡SQL INJECTION!  
    try (Statement stmt = con.createStatement();  
        ResultSet rs = stmt.executeQuery(sql)) { ... }  
}
```

💡 **Explicación:** ¡SQL Injection! Si nombre vale Luis'; DROP TABLE alumnos; --, la consulta se convierte en:

```
SELECT * FROM alumnos WHERE nombre = 'Luis'; DROP TABLE alumnos; --'
```

Adiós, tabla alumnos. Solución: usar PreparedStatement:

```
String sql = "SELECT * FROM alumnos WHERE nombre = ?";  
PreparedStatement pstmt = con.prepareStatement(sql);  
pstmt.setString(1, nombre);  
ResultSet rs = pstmt.executeQuery();
```

Nunca, jamás, concatenes SQL con datos de usuario. Es la regla de oro número 1 de JDBC.

Ejercicio 5: INSERT con PreparedStatement

```
String sql = "INSERT INTO alumnos (nombre, edad, curso) VALUES (?, ?, ?)";  
  
try (PreparedStatement pstmt = con.prepareStatement(sql)) {  
    pstmt.setString(1, "María");  
    pstmt.setInt(2, 22);  
    pstmt.setString(3, "DAM");  
    int filas = pstmt.executeUpdate();  
    System.out.println("Insertadas " + filas + " filas");  
}
```

💡 **Explicación:** setString, setInt, etc. reemplazan los ? en orden (empezando en 1). executeUpdate() devuelve el número de filas afectadas. Si es 1, todo bien. Si es 0, algo falló (o el INSERT tenía una condición WHERE que no coincidió — cosa rara en INSERT).

Ejercicio 6: ¿Qué hace executeUpdate()?

```
int filas = stmt.executeUpdate("DELETE FROM alumnos WHERE id = 10");
System.out.println(filas);
```

💡 **Explicación:** executeUpdate() devuelve el **número de filas afectadas**. Si el alumno con id=10 existe y se borra, imprime 1. Si no existe, imprime 0. No lanza excepción por no encontrar filas — solo devuelve 0. Siempre comprueba el valor devuelto para saber si la operación tuvo efecto.

Ejercicio 7: Diferencia entre executeQuery y executeUpdate

1. Mostrar todos los productos → executeQuery() (SELECT)
2. Borrar un cliente por ID → executeUpdate() (DELETE)
3. Actualizar el precio de un producto → executeUpdate() (UPDATE)
4. Contar cuántos usuarios hay → executeQuery() (SELECT COUNT)
5. Insertar un nuevo pedido → executeUpdate() (INSERT)

💡 **Explicación:** Regla mnemotécnica: si esperas datos de vuelta (filas con columnas), usa executeQuery(). Si solo quieres saber cuántas filas se modificaron, usa executeUpdate(). Mezclarlos da SQLException. Como meter un tenedor en el microondas.

🔗 **CodeWars:** [SQL with Street Fighter](#) (6kyu)

🔗 **AceptaElReto.com:** [200 - Aburrimiento en las aulas](#)

Boletín 14 – Inicial: Connexió a BD amb JDBC

Sense solucions. 7 passos per no perdre la connexió.

Ejercicio 1: ¿Qué necesitas para usar JDBC?

Responde brevemente:

1. ¿Qué archivo .jar necesitas incluir en tu proyecto para conectar con MySQL?
 2. ¿Qué clase de Java proporciona el método getConnection()?
 3. ¿Qué interfaz representa una conexión a la base de datos?
 4. ¿Qué excepción checked tienes que manejar siempre al trabajar con JDBC?
-

Ejercicio 2: Completa el código — try-with-resources

Completa los tipos que faltan:

```
String url = "jdbc:mysql://localhost:3306/instituto";
String user = "root";
String pass = "admin123";

String sql = "SELECT * FROM alumnos";

try (_____ con = DriverManager.getConnection(url, user, pass);
     _____ stmt = con.createStatement();
     _____ rs = stmt.executeQuery(sql)) {

    while (rs.next()) {
        System.out.println(rs.getString("nombre"));
    }

} catch (_____ e) { // ¿qué excepción?
    System.err.println("Error: " + e.getMessage());
}
```


Ejercicio 3: ¿Qué imprime? — ResultSet vacío

```
try (Connection con = DriverManager.getConnection(url, user, pass);
    Statement stmt = con.createStatement();
    ResultSet rs = stmt.executeQuery("SELECT * FROM alumnos WHERE id = 9999")) {

    if (rs.next()) {
        System.out.println("Encontrado: " + rs.getString("nombre"));
    } else {
        System.out.println("No encontrado");
    }
}
```

Si no hay ningún alumno con `id = 9999`, ¿qué imprime? ¿Qué devuelve `rs.next()` la primera vez que se llama?

Ejercicio 4: Encuentra el error — SQLException sin manejo

```
public class Test {
    public static void main(String[] args) {
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/instituto", "root", "admin123");
        Statement stmt = con.createStatement();
        ResultSet rs = stmt.executeQuery("SELECT * FROM alumnos");
        while (rs.next()) {
            System.out.println(rs.getString("nombre"));
        }
    }
}
```

Este código no compila. ¿Por qué? ¿Qué dos cosas faltan para que funcione?

Ejercicio 5: Escribe este programa — mostrar nombres de columnas

Escribe un programa que se conecte a la base de datos y, usando `ResultSetMetaData`, muestre:

1. El número de columnas de la tabla alumnos.
2. El nombre de cada columna.
3. El tipo de dato de cada columna (usando `getColumnTypeName()`).

Ejemplo de salida:

```
La tabla alumnos tiene 4 columnas:  
id (INT)  
nombre (VARCHAR)  
edad (INT)  
curso (VARCHAR)
```

Ejercicio 6: ¿Qué imprime? — `executeQuery` en `UPDATE`

```
Statement stmt = con.createStatement();  
ResultSet rs = stmt.executeQuery("UPDATE alumnos SET edad = 25 WHERE id = 1");
```

¿Qué ocurre al ejecutar esta línea? ¿Por qué no debes usar `executeQuery()` para un `UPDATE`?

Ejercicio 7: Encuentra el error — `PreparedStatement` con índices incorrectos

```
String sql = "INSERT INTO alumnos (nombre, edad, curso) VALUES (?, ?, ?)";  
PreparedStatement pstmt = con.prepareStatement(sql);  
pstmt.setString(0, "Ana");    // ¿índice correcto?  
pstmt.setInt(1, 22);         // ¿índice correcto?  
pstmt.setString(2, "DAM");    // ¿índice correcto?  
pstmt.executeUpdate();
```

¿Qué índices son correctos para los `?` en un `PreparedStatement`? ¿En qué número empiezan?

Ejercicio 8: Completa el código — cerrar recursos

```
public void listarAlumnos() {  
    // Escribe el código para:  
    // 1. Conectarte a la BD usando try-with-resources  
    // 2. Ejecutar un SELECT * FROM alumnos  
    // 3. Mostrar cada alumno con printf("%d - %s (%d)", id, nombre, edad)  
    // 4. Manejar SQLException  
}
```

Completa el método. Recuerda que try-with-resources cierra automáticamente Connection, Statement y ResultSet.

Referències per seguir practicant

- CodeWars: [SQL with Street Fighter](#) (6 kyu)
- CodeWars: [SQL Basics: Simple JOIN](#) (6 kyu)
- AceptaElReto.com: [200 - Aburrimiento en las aulas](#)

Boletín 14 – Resuelto: JDBC – Conexión y Consultas

★ Ejercicio 2: INSERT con PreparedStatement

Crea un método que inserte un nuevo alumno. Los parámetros deben pasarse como argumentos. Usa PreparedStatement.

```
import java.sql.*;

public class InsertarAlumno {
    public static void insertar(String nombre, int edad, String curso) {
        String sql = "INSERT INTO alumnos (nombre, edad, curso) VALUES (?, ?, ?)";

        try (Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/instituto", "root", "admin123");
            PreparedStatement pstmt = con.prepareStatement(sql)) {

            pstmt.setString(1, nombre);
            pstmt.setInt(2, edad);
            pstmt.setString(3, curso);

            int filas = pstmt.executeUpdate();
            System.out.println("Insertado: " + filas + " fila(s)");

        } catch (SQLException e) {
            System.err.println("Error al insertar: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        insertar("Luis García", 22, "DAM");
    }
}
```

💡 **Explicación:** Los ? son placeholders. `setString(1, ...)` asigna al primer ?. El índice empieza en 1 (no en 0 como los arrays — trampa clásica). `executeUpdate()` devuelve 1 si todo fue bien. Si el nombre tiene una comilla simple como “Luis'O'Connell”, PreparedStatement la escapa automáticamente. Sin riesgo de SQL Injection.

★ Ejercicio 3: UPDATE con PreparedStatement

Actualiza la edad de un alumno buscándolo por nombre. Muestra cuántas filas se actualizaron.

```
import java.sql.*;

public class ActualizarAlumno {
    public static void main(String[] args) {
        String sql = "UPDATE alumnos SET edad = ? WHERE nombre = ?";

        try (Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/instituto", "root", "admin123");
            PreparedStatement pstmt = con.prepareStatement(sql)) {

            pstmt.setInt(1, 23);
            pstmt.setString(2, "Luis García");

            int filas = pstmt.executeUpdate();
            if (filas > 0) {
                System.out.println("Actualizados " + filas + " alumno(s)");
            } else {
                System.out.println("No se encontró al alumno");
            }

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

💡 **Explicación:** `executeUpdate()` devuelve filas afectadas. Si el `WHERE` no encuentra nada, devuelve 0 — no es error. Siempre comprueba el valor. Si olvidas el `WHERE`, actualizas TODOS los alumnos con la misma edad. No preguntes cómo lo sé.

★ Ejercicio 4: DELETE con PreparedStatement

Borra un alumno por ID. Pide confirmación antes de borrar.

```

import java.sql.*;
import java.util.Scanner;

public class EliminarAlumno {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("ID del alumno a borrar: ");
        int id = sc.nextInt();
        sc.nextLine();
        System.out.print("¿Seguro? (s/n): ");
        String conf = sc.nextLine();

        if (!conf.equalsIgnoreCase("s")) {
            System.out.println("Cancelado");
            return;
        }

        String sql = "DELETE FROM alumnos WHERE id = ?";

        try (Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/instituto", "root", "admin123");
            PreparedStatement pstmt = con.prepareStatement(sql)) {

            pstmt.setInt(1, id);
            int filas = pstmt.executeUpdate();
            System.out.println(filas > 0 ? "Eliminado" : "No encontrado");

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}

```

💡 **Explicación:** Confirmación antes de borrar: buena práctica. El DELETE sin WHERE es el beso de la muerte para tus datos. `executeUpdate()` devuelve filas afectadas. Si devuelve 0, el ID no existe. Y recuerda: en SQL no hay papelera de reciclaje.

★ ★ Ejercicio 5: Búsqueda por nombre con LIKE

Implementa una búsqueda de alumnos por nombre usando LIKE y PreparedStatement. El usuario escribe parte del nombre y se muestran todos los que coinciden.

```
import java.sql.*;

public class BuscarAlumnos {
    public static void buscar(String nombreParcial) {
        String sql = "SELECT * FROM alumnos WHERE nombre LIKE ?";

        try (Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/instituto", "root", "admin123");
            PreparedStatement pstmt = con.prepareStatement(sql)) {

            pstmt.setString(1, "%" + nombreParcial + "%");

            try (ResultSet rs = pstmt.executeQuery()) {
                boolean encontrado = false;
                while (rs.next()) {
                    System.out.printf("%d - %s (%d)%n",
                        rs.getInt("id"),
                        rs.getString("nombre"),
                        rs.getInt("edad"));
                    encontrado = true;
                }
                if (!encontrado) System.out.println("Sin resultados");
            }

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        buscar("Luis"); // Busca cualquier nombre que contenga "Luis"
    }
}
```

💡 **Explicación:** LIKE ? con `setString(1, "%" + nombre + "%")`. Los % son comodines SQL: %Luis% busca "Luis" en cualquier posición. La concatenación aquí es segura porque los % son parte del valor, no de la SQL. El ResultSet está anidado en otro try-with-resources para que se cierre automáticamente.

★ ★ Ejercicio 6: Contar alumnos por curso

Escribe un programa que muestre cuántos alumnos hay en cada curso usando GROUP BY.

```
import java.sql.*;

public class ContarPorCurso {
    public static void main(String[] args) {
        String sql = "SELECT curso, COUNT(*) AS total FROM alumnos GROUP BY curso ORDER BY total DESC";

        try (Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/instituto", "root", "admin123");
            Statement stmt = con.createStatement();
            ResultSet rs = stmt.executeQuery(sql)) {

            System.out.println("Alumnos por curso:");
            while (rs.next()) {
                System.out.printf("  %s: %d alumno(s)%n",
                    rs.getString("curso"),
                    rs.getInt("total"));
            }

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

💡 **Explicación:** GROUP BY agrupa por curso. COUNT(*) cuenta filas de cada grupo. AS total da nombre a la columna calculada. ORDER BY total DESC ordena de mayor a menor. Aquí usamos Statement porque no hay parámetros variables — la SQL es fija. PreparedStatement también valdría, pero no es necesario.

★ ★ Ejercicio 7: Transacciones básicas

Simula una transferencia de puntos entre dos alumnos. Usa transacciones: si alguna operación falla, todo se deshace.


```

import java.sql.*;

public class TransferenciaPuntos {
    public static void main(String[] args) {
        String sqlQuitar = "UPDATE alumnos SET puntos = puntos - ? WHERE id = ?";
        String sqlPoner = "UPDATE alumnos SET puntos = puntos + ? WHERE id = ?";

        try (Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/instituto", "root", "admin123")) {

            con.setAutoCommit(false);

            try (PreparedStatement q = con.prepareStatement(sqlQuitar);
                PreparedStatement p = con.prepareStatement(sqlPoner)) {

                // Quitar 10 puntos al alumno 1
                q.setInt(1, 10);
                q.setInt(2, 1);
                q.executeUpdate();

                // Poner 10 puntos al alumno 2
                p.setInt(1, 10);
                p.setInt(2, 2);
                p.executeUpdate();

                con.commit();
                System.out.println("Transferencia OK ✅");

            } catch (SQLException e) {
                con.rollback();
                System.err.println("Error, todo deshecho: " + e.getMessage());
            }

        } catch (SQLException e) {
            System.err.println("Error de conexión: " + e.getMessage());
        }
    }
}

```

💡 **Explicación:** `setAutoCommit(false)` desactiva el modo “cada sentencia es una transacción”. Ahora tú controlas: `commit()` confirma todo, `rollback()` deshace todo. Si falla

la segunda sentencia, la primera se deshace con `rollback()`. Las transacciones son para operaciones que deben ser atómicas: “todo o nada”.

☆☆☆ Ejercicio 8: CRUD completo con menú

Crea un programa con menú textual que permita: Listar, Insertar, Actualizar, Eliminar y Buscar alumnos. Cada operación en su propio método. Usa `PreparedStatement` siempre.

```

import java.sql.*;
import java.util.*;

public class CrudAlumnosCompleto {
    private static final String URL = "jdbc:mysql://localhost:3306/instituto";
    private static final String USER = "root";
    private static final String PASS = "admin123";

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int op;
        do {
            System.out.println("\n=== CRUD ALUMNOS ===");
            System.out.println("1. Listar  2. Buscar  3. Insertar");
            System.out.println("4. Actualizar  5. Eliminar  6. Salir");
            System.out.print("Opción: ");
            op = sc.nextInt(); sc.nextLine();

            switch (op) {
                case 1 -> listar();
                case 2 -> buscar(sc);
                case 3 -> insertar(sc);
                case 4 -> actualizar(sc);
                case 5 -> eliminar(sc);
            }
        } while (op != 6);
    }

    static void listar() {
        try (Connection c = DriverManager.getConnection(URL, USER, PASS);
            Statement st = c.createStatement();
            ResultSet rs = st.executeQuery("SELECT * FROM alumnos ORDER BY nombre")) {
            while (rs.next())
                System.out.printf("%d - %s (%d) %s\n",
                    rs.getInt("id"), rs.getString("nombre"),
                    rs.getInt("edad"), rs.getString("curso"));
        } catch (SQLException e) { System.err.println("Error: " + e.getMessage()); }
    }

    static void buscar(Scanner sc) {
        System.out.print("Nombre a buscar: ");
        try (Connection c = DriverManager.getConnection(URL, USER, PASS);

```

```

        PreparedStatement ps = c.prepareStatement("SELECT * FROM alumnos WHERE
nombre LIKE ?")) {
    ps.setString(1, "%" + sc.nextLine() + "%");
    ResultSet rs = ps.executeQuery();
    boolean ok = false;
    while (rs.next()) { ok = true;
        System.out.printf("%d - %s (%d)%n",
            rs.getInt("id"), rs.getString("nombre"), rs.getInt("edad")); }
    if (!ok) System.out.println("Sin resultados");
    rs.close();
} catch (SQLException e) { System.err.println("Error: " + e.getMessage()); }
}

static void insertar(Scanner sc) {
    System.out.print("Nombre: "); String n = sc.nextLine();
    System.out.print("Edad: "); int e = sc.nextInt(); sc.nextLine();
    System.out.print("Curso: "); String c = sc.nextLine();
    try (Connection con = DriverManager.getConnection(URL, USER, PASS);
        PreparedStatement ps = con.prepareStatement(
            "INSERT INTO alumnos (nombre, edad, curso) VALUES (?, ?, ?)")) {
        ps.setString(1, n); ps.setInt(2, e); ps.setString(3, c);
        System.out.println(ps.executeUpdate() > 0 ? "OK" : "Error");
    } catch (SQLException ex) { System.err.println("Error: " + ex.getMessage()); }
}

static void actualizar(Scanner sc) {
    System.out.print("ID: "); int id = sc.nextInt(); sc.nextLine();
    System.out.print("Nueva edad: "); int edad = sc.nextInt(); sc.nextLine();
    try (Connection con = DriverManager.getConnection(URL, USER, PASS);
        PreparedStatement ps = con.prepareStatement(
            "UPDATE alumnos SET edad = ? WHERE id = ?")) {
        ps.setInt(1, edad); ps.setInt(2, id);
        System.out.println(ps.executeUpdate() > 0 ? "Actualizado" : "No
encontrado");
    } catch (SQLException e) { System.err.println("Error: " + e.getMessage()); }
}

static void eliminar(Scanner sc) {
    System.out.print("ID: "); int id = sc.nextInt(); sc.nextLine();
    System.out.print("¿Seguro? (s/n): ");
    if (!sc.nextLine().equalsIgnoreCase("s")) { System.out.println("Cancelado");
return; }
    try (Connection con = DriverManager.getConnection(URL, USER, PASS);

```

```

        PreparedStatement ps = con.prepareStatement("DELETE FROM alumnos WHERE id
= ?")) {
    ps.setInt(1, id);
    System.out.println(ps.executeUpdate() > 0 ? "Eliminado" : "No encontrado");
} catch (SQLException e) { System.err.println("Error: " + e.getMessage()); }
    }
}

```

💡 **Explicación:** CRUD completo en ~130 líneas. Cada operación abre y cierra su propia conexión (no óptimo para producción, pero sí para aprender). PreparedStatement en todas las consultas con parámetros. Statement solo en listar (consulta fija sin parámetros). Fíjate que ResultSet también se cierra automáticamente cuando está dentro de try-with-resources.

Ejercicio 9: Paginación con LIMIT y OFFSET

Crea un método que muestre alumnos por páginas de 5 en 5. El usuario elige la página.

```

import java.sql.*;
import java.util.Scanner;

public class PaginacionAlumnos {
    private static final int TAM_PAGINA = 5;

    public static void mostrarPagina(int pagina) {
        String sql = "SELECT * FROM alumnos ORDER BY id LIMIT ? OFFSET ?";

        try (Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/instituto", "root", "admin123");
            PreparedStatement pstmt = con.prepareStatement(sql)) {

            int offset = (pagina - 1) * TAM_PAGINA;
            pstmt.setInt(1, TAM_PAGINA);
            pstmt.setInt(2, offset);

            try (ResultSet rs = pstmt.executeQuery()) {
                System.out.println("--- Página " + pagina + " ---");
                while (rs.next()) {
                    System.out.printf("%d - %s (%d) %s\n",
                        rs.getInt("id"), rs.getString("nombre"),
                        rs.getInt("edad"), rs.getString("curso"));
                }
            }

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int pag = 1;
        while (true) {
            mostrarPagina(pag);
            System.out.print("Siguiente (s) / Anterior (a) / Salir (x): ");
            String cmd = sc.nextLine();
            if (cmd.equals("s")) pag++;
            else if (cmd.equals("a") && pag > 1) pag--;
            else if (cmd.equals("x")) break;
        }
    }
}

```

```
}  
  
}
```

💡 **Explicación:** LIMIT ? OFFSET ?: LIMIT=5 (filas por página), OFFSET=(pag-1)*5 (desplazamiento). OFFSET permite navegar por páginas sin cargar todos los datos. Esto es eficiente para tablas grandes: la BD solo devuelve las 5 filas de la página. En PostgreSQL se usa LIMIT ? OFFSET ?, en SQL Server OFFSET ... FETCH NEXT.

☆☆☆ Ejercicio 10: Exportar a CSV con metadatos

Exporta todos los alumnos a un archivo CSV. Además de los datos, incluye una fila de metadatos: nombre de las columnas, número de filas exportadas y fecha de exportación.

```

import java.sql.*;
import java.io.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class ExportarCSV {
    public static void main(String[] args) {
        String csvFile = "alumnos_export.csv";

        try (Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/instituto", "root", "admin123");
            Statement stmt = con.createStatement();
            ResultSet rs = stmt.executeQuery("SELECT * FROM alumnos ORDER BY id");
            BufferedWriter writer = new BufferedWriter(new FileWriter(csvFile))) {

            // Metadatos
            writer.write("# Exportación generada el " +
                LocalDateTime.now().format(DateTimeFormatter.ISO_LOCAL_DATE_TIME));
            writer.newLine();

            // Cabecera desde ResultSetMetaData
            ResultSetMetaData meta = rs.getMetaData();
            int numCols = meta.getColumnCount();
            for (int i = 1; i <= numCols; i++) {
                if (i > 1) writer.write(";");
                writer.write(meta.getColumnName(i));
            }
            writer.newLine();

            // Datos
            int filas = 0;
            while (rs.next()) {
                for (int i = 1; i <= numCols; i++) {
                    if (i > 1) writer.write(";");
                    String valor = rs.getString(i);
                    if (valor != null) writer.write(valor);
                }
                writer.newLine();
                filas++;
            }

            // Pie con total

```



```

        writer.write("# Total: " + filas + " filas exportadas");
        writer.newLine();

        System.out.println("Exportadas " + filas + " filas a " + csvFile);

    } catch (SQLException | IOException e) {
        System.err.println("Error: " + e.getMessage());
    }
}
}

```

💡 **Explicación:** ResultSetMetaData da información sobre las columnas: nombre, tipo, tamaño. getColumnCount() + getColumnName(i) genera la cabecera dinámicamente — funciona para cualquier consulta, no solo SELECT *. El CSV usa ; como separador (estilo español). La fecha de exportación y el total de filas se escriben como comentarios con #. Esto es exportación profesional.

🔗 **CodeWars:** [SQL with Street Fighter](#) (6kyu) • [SQL Basics: Simple JOIN](#) (6kyu)

🔗 **AceptaElReto.com:** [200 - Aburrimiento en las aulas](#) • [340 - Juegos de naipes](#) • [100 - Kaprekar](#)

Boletín 14 – Intermedi: Connexió a BD amb JDBC

Exercicis de dificultat progressiva. De ★ a ★★★★★.

★ Ejercicio 1: Conexión desde archivo de propiedades

Crea un archivo `db.properties` con los datos de conexión:

```
url=jdbc:mysql://localhost:3306/instituto
user=root
password=admin123
```

Escribe un programa que lea este archivo usando `Properties` y establezca la conexión. Si el archivo no existe o falta alguna propiedad, muestra un mensaje de error claro.

★ Ejercicio 2: INSERT con clave autogenerada

Inserta un nuevo alumno en la tabla `alumnos` y **recupera el ID** que la base de datos le ha asignado automáticamente. Usa `PreparedStatement` con la opción `Statement.RETURN_GENERATED_KEYS` y el método `getGeneratedKeys()`.

Pista:

```
PreparedStatement pstmt = con.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS);
```

★ Ejercicio 3: UPDATE condicional

Actualiza el curso de todos los alumnos que tengan una edad superior a un valor dado. Por ejemplo:

```
¿Edad mínima? 25
¿Nuevo curso? DAM2
```

Todos los alumnos mayores de 25 años pasan al curso «DAM2». Muestra cuántas filas se actualizaron.

★ ★ Ejercicio 4: INNER JOIN con PreparedStatement

Dada una tabla `matriculas` con `id_alumno`, `asignatura`, `nota`, escribe un programa que reciba un nombre de alumno y muestre todas sus asignaturas y notas. Usa un `INNER JOIN` entre `alumnos` y `matriculas`.

Ejemplo de salida:

```
Alumno: Ana García
Matemáticas: 8.5
Programación: 9.0
Bases de Datos: 7.5
```

★ ★ Ejercicio 5: Fechas en JDBC

Añade una columna `fecha_nacimiento` `DATE` a la tabla `alumnos` (asume que ya existe). Crea un programa que:

1. Pida nombre, edad, curso y fecha de nacimiento (formato `YYYY-MM-DD`).
2. Inserte el alumno usando `PreparedStatement` con `java.sql.Date.valueOf()`.
3. Liste todos los alumnos mostrando también su fecha de nacimiento formateada con `DateTimeFormatter`.

★ ★ Ejercicio 6: Batch INSERT — 100 alumnos de prueba

Crea un programa que inserte **100 alumnos de prueba** en la tabla `alumnos` usando lotes (batch). Los nombres pueden ser genéricos: `Alumno1`, `Alumno2`, etc.

Usa `addBatch()` y `executeBatch()` de `PreparedStatement`. Mide el tiempo que tarda con `System.currentTimeMillis()`.

Compara: ¿cuánto tardaría si hicieras 100 `executeUpdate()` individuales?

☆☆☆ Ejercicio 7 (ProgramaMe): Patrón DAO

Implementa el patrón **Data Access Object (DAO)** para la tabla `alumnos`. Crea las siguientes clases:

1. `Alumno` — clase modelo con `id`, `nombre`, `edad`, `curso`.
 2. `AlumnoDAO` — interfaz con métodos:
 - `List<Alumno> listar()`
 - `Alumno buscarPorId(int id)`
 - `List<Alumno> buscarPorNombre(String nombre)`
 - `boolean insertar(Alumno a)`
 - `boolean actualizar(Alumno a)`
 - `boolean eliminar(int id)`
 3. `AlumnoDAOImpl` — implementación concreta con JDBC.
 4. `Main` — programa con menú que use el DAO.
-

☆☆☆ Ejercicio 8 (CodeWars + AceptaElReto)

Resuelve estos problemas que refuerzan conceptos de Bases de Datos y SQL:

CodeWars: [SQL Basics: Simple JOIN](#) (6 kyu) — Practica JOINS en SQL.

CodeWars: [SQL with Street Fighter](#) (6 kyu) — Consultas con LIKE y ordenación.

AceptaElReto: [200 - Aburrimiento en las aulas](#) — Problema de estructura de datos que puedes resolver con JDBC.



Referències

- CodeWars: [SQL Basics: Simple JOIN](#) (6 kyu)
- CodeWars: [SQL with Street Fighter](#) (6 kyu)
- CodeWars: [SQL Basics: Simple HAVING](#) (6 kyu)
- AceptaElReto.com: [200 - Aburrimiento en las aulas](#)
- AceptaElReto.com: [340 - Juegos de naipes](#)
- AceptaElReto.com: [100 - Kaprekar](#)

Butlletí 12 — Extres: Connexió a Bases de Dades amb JDBC

★ Extres — Converteix-te en un programador de molt alt nivell

Aquests exercicis són completament opcionals, però si aspirem a ser un programador o programadora d'elit, ací tens el teu camp d'entrenament. CodeWars i AceptaElReto són plataformes on els millors programadors del món es repte cada dia. Cada problema que resolguis et farà més fort, més ràpid i més creatiu. No els subestimis: són la diferència entre saber programar i ser un vertader professional.

CodeWars:

- [SQL with Street Fighter](#) (6 kyu)

Pista: Usa `INNER JOIN` amb `LIKE '%K%'` per a filtrar per nom i `ORDER BY` per a ordenar.

- [SQL Basics: Simple JOIN](#) (6 kyu)

Pista: `INNER JOIN` sobre la clau forana; selecciona les columnes que et demane el problema.

- [SQL Basics: Simple HAVING](#) (6 kyu)

Pista: `GROUP BY` la columna indicada i filtra amb `HAVING COUNT(*) > algun valor`.

AceptaElReto:

- [245 - Qui guanya la partida?](#) (★ ★ ★)

Pista: Simula els torns amb una `Queue<Integer>`; cada ronda mou el primer al final si no encerta.

- [424 - Bitllets d'autobús](#) (★ ★)

Pista: Algorisme voraç: ordena per hora d'eixida, tria sempre la següent ruta que acabe abans.

Boletín 11 – Inicial resuelto: Interfaz Web

Los mismos ejercicios con soluciones paso a paso.

Ejercicio 1: Hola Mundo Web

```
import com.sun.net.httpserver.HttpServer;
import java.io.OutputStream;
import java.net.InetSocketAddress;

public class HolaMundoWeb {
    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);
        server.createContext("/", e -> {
            String resp = "Hola Mundo";
            e.sendResponseHeaders(200, resp.getBytes().length);
            OutputStream os = e.getResponseBody();
            os.write(resp.getBytes());
            os.close();
        });
        server.setExecutor(null);
        server.start();
        System.out.println("Servidor en http://localhost:8080");
    }
}
```

Ejercicio 2: Hora del servidor

```
server.createContext("/hora", e -> {
    String resp = java.time.LocalDateTime.now().toString();
    e.sendResponseHeaders(200, resp.getBytes().length);
    e.getResponseBody().write(resp.getBytes());
    e.getResponseBody().close();
});
```

Ejercicio 3: HTML bonito

```

server.createContext("/", e -> {
    String html = ""
        <html><head><meta charset="UTF-8"><title>Mi Web</title>
        <style>body{background:#e0f7e0;font-family:sans-serif;padding:2em}</style>
        </head><body><h1>Bienvenido</h1><p>Esto es HTML servido desde Java.</p>
        </body></html>
        "";
    e.getResponseHeaders().set("Content-Type", "text/html; charset=UTF-8");
    e.sendResponseHeaders(200, html.getBytes().length);
    e.getResponseBody().write(html.getBytes());
    e.getResponseBody().close();
});

```

Ejercicio 4: Saludo personalizado

```

server.createContext("/saludo", e -> {
    String query = e.getRequestURI().getQuery();
    String nombre = "Mundo";
    if (query != null && query.startsWith("nombre="))
        nombre = java.net.URLDecoder.decode(query.split("=")[1], "UTF-8");
    String html = "<h1>Hola, " + nombre + "!</h1>";
    e.getResponseHeaders().set("Content-Type", "text/html; charset=UTF-8");
    e.sendResponseHeaders(200, html.getBytes().length);
    e.getResponseBody().write(html.getBytes());
    e.getResponseBody().close();
});

```

Ejercicio 5: Contador de visitas


```

public class ContadorVisitas {
    static int visitas = 0;
    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);
        server.createContext("/", e -> {
            visitas++;
            String resp = "Visita #" + visitas;
            e.sendResponseHeaders(200, resp.getBytes().length);
            e.getResponseBody().write(resp.getBytes());
            e.getResponseBody().close();
        });
        server.start();
        System.out.println("http://localhost:8080");
    }
}

```

Ejercicio 6: Tabla HTML

```

server.createContext("/tabla", e -> {
    StringBuilder sb = new StringBuilder("<table border=1>");
    sb.append("<tr><th>N</th><th>N²</th></tr>");
    for (int i = 1; i <= 10; i++)
        sb.append("<tr><td>").append(i).append("</td><td>").append(i*i).append("</td></tr>");
    sb.append("</table>");
    String html = "<html><body>" + sb + "</body></html>";
    e.getResponseHeaders().set("Content-Type", "text/html; charset=UTF-8");
    e.sendResponseHeaders(200, html.getBytes().length);
    e.getResponseBody().write(html.getBytes());
    e.getResponseBody().close();
});

```

Butlletí 13 – Inicial: Servir i Consumir APIs amb Web

Escalfa motors amb `HttpServer`. Promet que cap bit eixirà ferit.

Exercici 1: Servidor de l'hora

Crea un servidor HTTP al port 8080 amb una ruta `/hora` que torne l'hora actual en text pla amb format `HH:mm:ss`.

Cada vegada que recarregues el navegador, l'hora canvia. Màgia negra amb `LocalTime.now()`.

```
// Pista:  
String hora = java.time.LocalTime.now().format(  
    java.time.format.DateTimeFormatter.ofPattern("HH:mm:ss")  
);
```

Exercici 2: Pàgina que diu el teu nom

Afig una ruta `/saludo?nombre=Pepe` que torne una pàgina HTML amb `<h1>¡Hola, Pepe!</h1>`.

Si no es passa nom, que diga `¡Hola, desconocido!`.

Pista: `e.getRequestURI().getQuery()` et dona `nombre=Pepe`. Divideix per `=` i llest.

Exercici 3: Comptador de visites global

Usa una variable `static int visites = 0`. Cada vegada que algú visita `/`, incrementa el comptador i mostra:

```
Eres el visitante número 47
```

Què passa si dos persones recarreguen alhora? (spoiler: problemes — però de moment no et preocupes).

Exercici 4: Generador d'excuses per a lliuraments tardans

Crea rutes /excusa que torne una excusa generada aleatòriament combinant elements de tres arrays:

```
String[] subjectes = {"El meu gos", "GitHub", "El plugin d'IntelliJ", "La connexió"};
String[] verbs     = {"es va menjar", "va esborrar", "va corrompre", "va perdre"};
String[] objectes  = {"l'examen", "la pràctica", "els apunts", "la meua paciència"};
```

Exemple: *"GitHub va esborrar la pràctica"*. Torna-ho en HTML amb bona lletra.

Exercici 5: Taula de multiplicar personalitzada

Ruta /tabla?num=7 que genere una taula HTML completa amb la taula de multiplicar del 7 (del 7×1 al 7×10).

Si no es passa número, usa el 5 per defecte. Formata la taula amb border=1 i colors alterns en les files.

Exercici 6: Pàgina d'estat del servidor

Ruta /estado que torne un JSON amb esta pinta:

```
{
  "servidor": "ok",
  "hora": "14:30:01",
  "visites": 47,
  "versio": "1.0",
  "autor": "El teu nom ací"
}
```

No oblidés el Content-Type: application/json o el navegador es tornarà tonto.

Exercici 7: Convertidor d'euros a pessetes (sí, pessetes)

Ruta /conversor?euros=50 que torne HTML amb el resultat: "50 euros son 8319.3 pesetas".

1 € = 166.386 pts. Mostra dos decimals.

Pista: `String.format("%.2f", valor)` per als decimals. Sí, pessetes. Si eres jove, pregunta-li a un vell què era això.



Referències

- CodeWars: [Decode the Morse code](#) (6 kyu)
- AceptaElReto: [396 - ¿Cuántos días faltan?](#) (★ ★)
- Documentació Oracle: [HttpServer](#)

Boletín 11 – Resuelto: Interfaz Web

Ejercicios de nivel creciente. Sin soluciones, solo pistas.

★ Ejercicio 1: Servidor de archivos estáticos

Sirve cualquier fichero de la carpeta `web/` (HTML, CSS, JS, imágenes) usando `Files.probeContentType()` para el Content-Type.

Pista: usa `Files.readAllBytes(Path.of("web", ruta))` y captura `NoSuchFileException` para devolver 404.

★ Ejercicio 2: Formulario de contacto

Crea una ruta `/contacto` que sirva un formulario HTML (GET) y otra ruta `/enviar` que reciba los datos por POST y los muestre en una página de confirmación.

Pista: en el handler de `/enviar` comprueba `e.getRequestMethod().equals("POST")` y lee el cuerpo con `e.getRequestBody().readAllBytes()`.

★ ★ Ejercicio 3: API JSON de productos

Implementa una API REST:

- GET `/api/productos` → lista todos los productos
- POST `/api/productos` → añade uno (recibe JSON)
- DELETE `/api/productos/{id}` → borra uno

Usa un `ArrayList<Producto>` como almacén en memoria.

Pista: para parsear JSON a mano, usa `String.split()` o la clase `java.util.Scanner`. Para algo más serio, `org.json` o `Gson`.

★ ★ Ejercicio 4: Chat en memoria

Crea un chat simple donde los mensajes se guardan en un `ArrayList`. El frontend pregunta cada 2 segundos con `setInterval` si hay mensajes nuevos.

- GET `/api/mensajes?ultimoId=5` → mensajes nuevos desde el ID 5
- POST `/api/mensajes` → body: `{usuario:"Ana", texto:"Hola"}`

Pista: El frontend usa `fetch()` dentro de un `setInterval`. El backend filtra mensajes por ID mayor que `ultimoId`.

☆☆☆ Ejercicio 5: Mini framework MVC

Implementa un mini enrutador que permita definir rutas con esta sintaxis:

```
get("/usuarios", (req, res) -> res.html("<h1>Usuarios</h1>"));
post("/usuarios", (req, res) -> res.json(nuevoUsuario));
```

Pista: Crea una clase Router con mapas de `HashMap<String, Handler>` para GET y POST. Cada handler recibe `HttpExchange` y tiene métodos auxiliares `html()` y `json()`.

☆☆☆ Ejercicio 6: Subida de archivos

Permite al usuario seleccionar un archivo desde el navegador y subirlo al servidor. Guárdalo en `uploads/` y muestra el nombre y tamaño en una tabla.

- Frontend: `<input type="file">` + `FormData` + `fetch POST`
- Backend: leer `multipart/form-data` a mano o con `Scanner`

Pista: El Content-Type `multipart/form-data` incluye un `boundary`. Separa el cuerpo por ese `boundary` para extraer el nombre y contenido del archivo.

Butlletí 13 – Intermedi: Servir i Consumir APIs amb Web

Exercicis per a dominar l'art de servir JSON, processar formularis i fer que frontend i backend es parlen sense barallar-se.

★ Exercici 1: API de frases motivacionals

Crea un endpoint GET `/api/frase` que torne un JSON amb una frase aleatòria d'un array precarregat i el seu autor.

```
{"frase": "El código limpio es como un buen chiste: si tienes que explicarlo, es malo", "autor": "Alguien que sabe"}
```

El frontend és un HTML amb un botó “Nova frase” que al fer clic fa `fetch('/api/frase')` i mostra la frase en pantalla.

Pista: usa `Math.random()` per a triar un índex aleatori de l'array.

★ Exercici 2: Traductor xungo (però funcional)

Implementa un endpoint POST `/api/traducir` que reba:

```
{"texto": "hola", "idioma": "en"}
```

I torne:

```
{"traduccion": "hello"}
```

Usa un `HashMap<String, HashMap<String, String>>` com a diccionari. Fica almenys 10 paraules en espanyol traduïdes a anglés i francès.

Pista: Inicialitza el diccionari amb blocs static. `diccionario.get("hola").get("en")` et dona “hello”.

☆☆ Exercici 3: API REST de tasques amb prioritat

Implementa un CRUD complet de tasques on cada tasca té: id, títol, prioritat ("ALTA", "MITJA", "BAIXA").

Mètode	Ruta	Descripció
GET	/api/tareas	Llista totes
POST	/api/tareas	Crea una (JSON: {"titulo": "...", "prioridad": "ALTA"})
PUT	/api/tareas/{id}	Canvia prioritat (JSON: {"prioridad": "BAJA"})
DELETE	/api/tareas/{id}	Borra una

Frontend: taula amb colors de fons segons prioritat (roig ALTA, groc MITJA, verd BAIXA). Botons per a crear, canviar prioritat i borrar.

Pista: guarda les tasques en un `ConcurrentHashMap<Integer, Tarea>` amb un `AtomicInteger` per a IDs.

☆☆ Exercici 4: El temps que NO fa

Crea GET `/api/clima?ciudad=Madrid` que torne un JSON amb dades meteorològiques **aleatòries** (generades cada vegada):

```
{"ciudad": "Madrid", "temperatura": 28, "humedad": 45, "estado": "soleado"}
```

Estats possibles: "soleado", "nublado", "lluvia", "tormenta". Frontend amb emojis i temperatures de colors.

Pista: usa `String[] estados = {"soleado", "nublado", "lluvia", "tormenta"}` i tria aleatòriament. La temperatura pot ser `random.nextInt(40) - 5`.

☆☆ Exercici 5: Catàleg de pel·lícules amb filtres

Precarrega un array de 10-15 pel·lícules (amb títol, genere, any, puntuacio). Implementa:

- GET /api/peliculas → llista totes
- GET /api/peliculas?genero=comedia → filtra per gènere
- GET /api/peliculas?genero=comedia&anyo=1994 → filtra per gènere i any
- GET /api/peliculas/3 → detall de la pel·lícula amb ID 3

Frontend: selectors de gènere i any, que al canviar actualitzen la llista via fetch.

Pista: per a filtrar usa `stream().filter(p -> p.getGenero().equals(genero)).toList()`. Per al path param, parseja la URL.

☆☆☆ Exercici 6: Middleware de logging

Crea una classe `LoggerMiddleware` que embolique qualsevol `HttpHandler` i registre en consola:

```
[2026-06-21 14:30:01] GET /api/peliculas → 200 (15ms)
[2026-06-21 14:30:05] POST /api/tareas → 201 (3ms)
```

Ha de poder aplicar-se a qualsevol handler així:

```
server.createContext("/api", new LoggerMiddleware(new TareasHandler()));
```

Pista: guarda `System.currentTimeMillis()` abans i després de cridar al handler original. Usa `e.getRequestMethod()` i `e.getResponseCode()` (després d'enviar capçaleres).

☆☆☆ Exercici 7: Server-Sent Events — El rellotge del servidor

Implementa un endpoint GET /api/eventos que use Server-Sent Events (SSE). Cada 5 segons, el servidor envia un esdeveniment amb l'hora actual:

```
data: {"hora": "14:30:05", "timestamp": 1718975405}
```

Frontend amb `EventSource`:

```
const source = new EventSource('/api/eventos');
source.onmessage = (e) => {
    document.getElementById('reloj').textContent = JSON.parse(e.data).hora;
};
```

Pista: en el handler, posa `e.getResponseHeaders().set("Content-Type", "text/event-stream")` i NO tanques la connexió. Usa `e.getResponseBody().write()` en un bucle amb `Thread.sleep(5000)`.

★ Exercici 8: Pedra, paper, tisora online

Endpoint POST `/api/jugar` que rep:

```
{"jugada": "piedra"}
```

I torna:

```
{"jugadaPC": "tijera", "resultado": "ganaste"}
```

Regles clàssiques: pedra > tisora, tisora > paper, paper > pedra.

Frontend: tres botons amb emojis 🪨 📄 ✂️. Al fer clic, envia la jugada i mostra el resultat. Porta un comptador de victòries/derrotes/empats.

Pista: la jugada del PC es tria amb Random. Les regles es poden implementar amb un `Map<String, String>` on la clau venç el valor: `{"piedra": "tijera", "tijera": "papel", "papel": "piedra"}`.



Referències

- CodeWars: [IP Validation](#) (6 kyu)
- CodeWars: [Simple URL parser](#) (6 kyu)
- AceptaElReto: [462 - Día de la semana](#) (★ ★)
- Documentació Oracle: [HttpExchange](#)
- MDN: [Server-Sent Events](#)

Butlletí 13 — Extres: Servir i Consumir APIs amb Web

★ Extres — Converteix-te en un programador de molt alt nivell

Aquests exercicis són completament opcionals, però si aspireu a ser un programador o programadora d'elit, ací tens el teu camp d'entrenament. CodeWars i AceptaElReto són plataformes on els millors programadors del món es repte cada dia. Cada problema que resolguis et farà més fort, més ràpid i més creatiu. No els subestimis: són la diferència entre saber programar i ser un vertader professional.

CodeWars:

- [IP Validation](#) (6 kyu)

Pista: Dividix per '.'; han de ser exactament 4 parts, cada una entre 0 i 255, sense zeros a l'esquerra.

- [Decode the Morse code](#) (6 kyu)

Pista: Crea un Map amb cada símbol Morse → lletra; separa paraules per tres espais i lletres per un.

- [Simple URL parser](#) (6 kyu)

Pista: Extrau primer el protocol (abans de ://), després el domini (següent segment) i la resta com a ruta.

AceptaElReto:

- [396 - Quants dies falten?](#) (★ ★)

Pista: Converteix cada data a dia de l'any (nombre de dies des de l'1 de gener) i resta.

- [462 - Dia de la setmana](#) (★ ★)

Pista: Usa la congruència de Zeller o pren un dia de referència conegut per a calcular el residu.